

Claude Code from Source

Architecture, Patterns & Internals of Anthropic's AI Coding Agent

Alejandro Balderas

2024-12-31

Table of contents

Preface	1
I Part I: Foundations	3
1 Chapter 1: The Architecture of an AI Agent	5
1.1 What You're Looking At	5
1.2 The Six Key Abstractions	6
1.3 The Golden Path: From Keystroke to Output	8
1.4 The Permission System	9
1.5 Multi-Provider Architecture	11
1.6 The Build System	11
1.7 How the Pieces Connect	12
1.8 Apply This	13
2 Chapter 2: Starting Fast – The Bootstrap Pipeline	15
2.1 The Shape of the Pipeline	16
2.2 Phase 0: Fast-Path Dispatch (cli.tsx)	17
2.3 Phase 1: Module-Level I/O (main.tsx)	18
2.4 Phase 2: Parse and Trust (init.ts)	19
2.5 Phase 3: Setup (setup.ts)	21
2.6 Phase 4: Launch (replLauncher.ts)	22
2.7 The Startup Timeline	22
2.8 The Migration System	23
2.9 What Startup Teaches About System Design	24
2.10 Apply This	24
3 Chapter 3: State – The Two-Tier Architecture	27
3.1 3.1 Bootstrap State – The Process Singleton	28

3.2	3.2 AppState – The Reactive Store	32
3.3	3.3 How the Two Tiers Relate	34
3.4	3.4 Side Effects: onChangeAppState	36
3.5	3.5 Context Building	36
3.6	3.6 Cost Tracking	37
3.7	3.7 What We Learned	37
3.8	3.8 State Architecture Summary	38
3.9	3.9 Apply This	38
4	Chapter 4: Talking to Claude – The API Layer	41
4.1	The Multi-Provider Client Factory	42
4.2	System Prompt Construction	43
4.3	Streaming	45
4.4	Prompt Cache System	45
4.5	The queryModel Generator	46
4.6	Apply This	47
4.7	Looking Ahead	48
II	Part II: The Core Loop	49
5	Chapter 5: The Agent Loop	51
5.1	The Beating Heart	51
5.2	Why an Async Generator	52
5.3	What Callers Provide	53
5.4	The Two-Layer Entry Point	54
5.5	The State Object	54
5.6	The Loop Body	56
5.7	Context Management: Four Compression Layers	57
5.8	Model Streaming	59
5.9	Error Recovery: The Escalation Ladder	61
5.10	Worked Example: “Fix the Bug in auth.ts”	62
5.11	Token Budgets	63
5.12	Stop Hooks: Forcing the Model to Keep Working	64
5.13	State Transitions: The Complete Catalog	64
5.14	Orphaned Tool Results: The Protocol Safety Net	65
5.15	Abort Handling: Two Paths	66
5.16	The Thinking Rules	66
5.17	Dependency Injection	66
5.18	Apply This: Building Your Own Agent Loop	67

5.19 Summary	68
6 Chapter 6: Tools – From Definition to Execution	69
6.1 The Nervous System	69
6.2 The Tool Interface	70
6.3 ToolUseContext: The God Object	72
6.4 The Registry	73
6.5 The 14-Step Execution Pipeline	73
6.6 The Permission System	75
6.7 Tool Deferred Loading	77
6.8 Result Budgeting	78
6.9 Individual Tool Highlights	79
6.10 How Tools Interact with the Message History	81
6.11 Apply This: Designing a Tool System	81
6.12 What Comes Next	82
7 Chapter 7: Concurrent Tool Execution	83
7.1 The Cost of Waiting	83
7.2 The Partition Algorithm	84
7.3 Batch Execution	86
7.4 The Streaming Tool Executor	88
7.5 Tool Concurrency Properties	94
7.6 The Interrupt Behavior Contract	96
7.7 Context Modifiers: The Serial-Only Contract	97
7.8 Apply This	97
7.9 Summary	98
III Part III: Multi-Agent Orchestration	101
8 Chapter 8: Spawning Sub-Agents	103
8.1 The Multiplication of Intelligence	103
8.2 The AgentTool Definition	104
8.3 The runAgent Lifecycle	110
8.4 Built-In Agent Types	124
8.5 Fork Agents	127
8.6 Agent Definitions from Frontmatter	128
8.7 Apply This: Designing Agent Types	130
9 Chapter 9: Fork Agents and the Prompt Cache	135

9.1	The Ninety-Five Percent Insight	135
9.2	What a Fork Child Inherits	136
9.3	The Byte-Identical Prefix Trick	137
9.4	The Fork Boilerplate Tag	139
9.5	Recursive Fork Prevention	139
9.6	The Sync-to-Async Transition	141
9.7	Auto-Backgrounding	142
9.8	When Fork Is NOT Used	142
9.9	The Economics	143
9.10	Design Tensions	144
9.11	Apply This: Designing for Prompt Cache Efficiency	145
10	Chapter 10: Tasks, Coordination, and Swarms	147
10.1	The Limits of a Single Thread	147
10.2	The Task State Machine	148
10.3	Communication Patterns	153
10.4	Coordinator Mode	157
10.5	The Swarm System	162
10.6	Inter-Agent Communication: SendMessage	166
10.7	TaskStop: The Kill Switch	170
10.8	Choosing Between Patterns	171
10.9	The Cost of Orchestration	174
10.10	What the Orchestration Layer Reveals	174
IV	Part IV: Persistence and Intelligence	177
11	Chapter 11: Memory – Learning Across Conversations	179
11.1	The Stateless Problem	179
11.2	The Four-Type Taxonomy	182
11.3	The Write Path	183
11.4	The Recall Path	184
11.5	Staleness	186
11.6	MEMORY.md as the Always-Loaded Index	187
11.7	Team Memory	188
11.8	KAIROS Mode: Append-Only Daily Logs	189
11.9	Background Extraction	191
11.10	Path Resolution and Security	192
11.11	Apply This: Designing Agent Memory	193

12 Chapter 12: Extensibility – Skills and Hooks	195
12.1 Two Dimensions of Extension	195
12.2 Skills: Teaching the Model New Tricks	196
12.3 Hooks: Controlling When Things Happen	198
12.4 The Snapshot Security Model	202
12.5 Execution Flow	204
12.6 Integration: Where Skills Meet Hooks	205
12.7 Apply This: Designing an Extensibility System	206
 V Part V: The Interface	 209
13 Chapter 13: The Terminal UI	211
13.1 Why Build a Custom Renderer?	211
13.2 The Custom DOM	212
13.3 The React Fiber Container	214
13.4 The Rendering Pipeline	215
13.5 Double-Buffer Rendering and Frame Scheduling	218
13.6 Pool-Based Memory: Why Interning Matters	220
13.7 The REPL Component	223
13.8 Selection and Search Highlighting	224
13.9 Streaming Markdown	226
13.10 Apply This: Rendering Streaming Output Efficiently	227
 14 Chapter 14: Input and Interaction	 231
14.1 Raw Bytes, Meaningful Actions	231
14.2 The Key Parsing Pipeline	232
14.3 Multi-Protocol Support	235
14.4 The Keybinding System	240
14.5 Chord Support	243
14.6 Vim Mode	244
14.7 Virtual Scrolling	248
14.8 Apply This: Building a Context-Aware Keybinding System	250
14.9 Summary: Two Systems, One Design Philosophy	252
 VI Part VI: Connectivity	 255
15 Chapter 15: MCP – The Universal Tool Protocol	257
15.1 Why MCP Matters Beyond Claude Code	257

15.2	Eight Transport Types	258
15.3	Configuration Loading and Scoping	258
15.4	Tool Wrapping: From MCP to Claude Code	259
15.5	OAuth for MCP Servers	260
15.6	In-Process Transport	262
15.7	Connection Management	263
15.8	Claude.ai Proxy Transport	264
15.9	Timeout Architecture	264
15.10	Apply This: Integrating MCP Into Your Own Agent	264
16	Chapter 16: Remote Control and Cloud Execution	267
16.1	The Agent Reaches Beyond Localhost	267
16.2	Bridge v1: Poll, Dispatch, Spawn	269
16.3	Bridge v2: Direct Sessions and SSE	269
16.4	Message Routing and Echo Deduplication	270
16.5	The Asymmetric Design: Persistent Reads, HTTP POST Writes	271
16.6	Remote Session Management	272
16.7	Direct Connect: The Local Server	272
16.8	Upstream Proxy: Credential Injection in Containers	272
16.9	Apply This: Designing Remote Agent Execution	274
VII	Part VII: Performance Engineering	275
17	Chapter 17: Performance – Every Millisecond and Token Counts	277
17.1	The Senior Engineer’s Playbook	277
17.2	Saving Milliseconds at Startup	278
17.3	Saving Tokens in the Context Window	279
17.4	Saving Money on API Calls	280
17.5	Saving CPU in Rendering	281
17.6	Saving Memory and Time in Search	282
17.7	The Memory Relevance Side-Query	283
17.8	Speculative Tool Execution	283
17.9	Streaming and the Raw API	283
17.10	Apply This: Performance for Agentic Systems	284
18	Chapter 18: What We Learned	285
18.1	Five Architectural Bets	285
18.2	What Transfers, What Does Not	288

TABLE OF CONTENTS

ix

18.3 The Cost of Complexity	289
18.4 Where Agentic Systems Are Heading	290
18.5 Closing	292

Preface

This book is the result of studying the architecture of Anthropic’s Claude Code and distilling the patterns, trade-offs, and design decisions into a technical narrative that any engineer can learn from.

When Anthropic shipped Claude Code on npm, the `.js.map` source maps contained a `sourcesContent` field with the full original TypeScript. 36 AI agents analyzed nearly two thousand TypeScript files across four phases — exploration, analysis, writing, and review — producing this 18-chapter book in approximately six hours.

What You Will Find Here

Each chapter has layered depth: a narrative flow for technical leaders, deep-dive sections for implementers, and an *Apply This* closing that extracts transferable patterns you can use in your own systems.

The 10 Patterns That Make It Work

If you read nothing else:

1. **AsyncGenerator as agent loop** — yields Messages, typed Terminal return, natural backpressure and cancellation
2. **Speculative tool execution** — start read-only tools during model streaming, before the response completes
3. **Concurrent-safe batching** — partition tools by safety, run reads in parallel, serialize writes
4. **Fork agents for cache sharing** — parallel children share byte-identical prompt prefixes, saving ~95% input tokens
5. **4-layer context compression** — snip, microcompact, collapse, auto-compact — each lighter than the next
6. **File-based memory with LLM recall** — Sonnet side-query selects relevant memories, not keyword matching

7. **Two-phase skill loading** — frontmatter only at startup, full content on invocation
8. **Sticky latches for cache stability** — once a beta header is sent, never unset mid-session
9. **Slot reservation** — 8K default output cap, escalate to 64K on hit (saves context in 99% of requests)
10. **Hook config snapshot** — freeze at startup to prevent runtime injection attacks

i Note

This book does not contain any source code from Claude Code. All code blocks are original pseudocode using different variable names, written to illustrate architectural patterns. No proprietary prompt text, internal constants, or exact function implementations are included. This project exists purely for educational purposes.

Part I

Part I: Foundations

Chapter 1

Chapter 1: The Architecture of an AI Agent

1.1 What You’re Looking At

A traditional CLI is a function. It takes arguments, does work, and exits. `grep` does not decide to also run `sed`. `curl` does not open a file and patch it based on what it downloaded. The contract is simple: one command, one action, deterministic output.

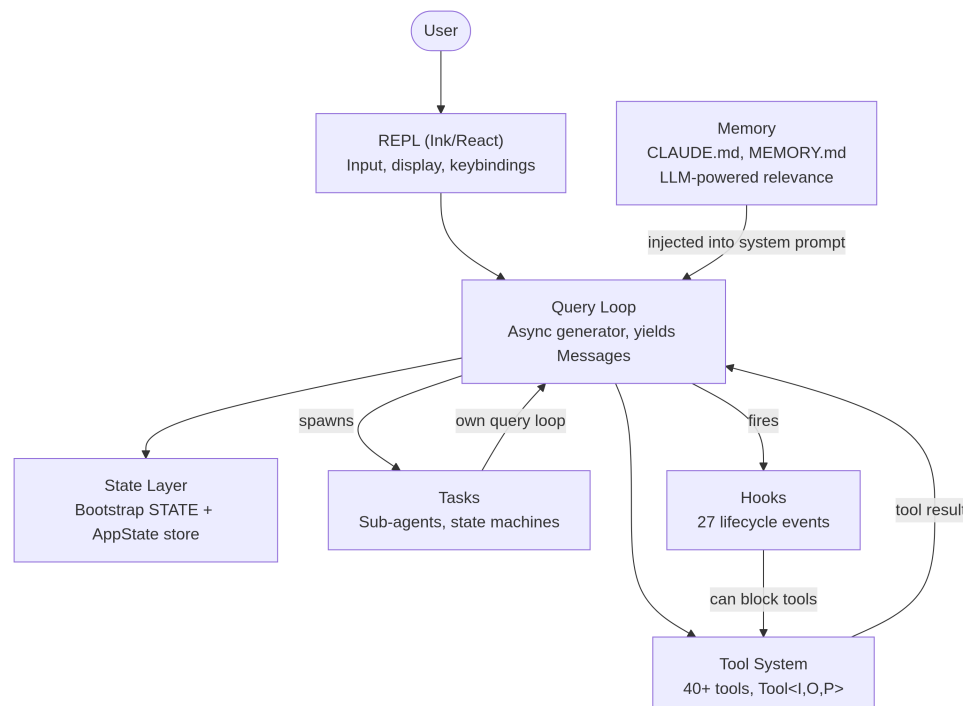
An agentic CLI breaks every part of that contract. It takes a natural language prompt, decides what tools to use, executes them in whatever order the situation demands, evaluates the results, and loops until the task is done or the user stops it. The “program” is not a fixed sequence of instructions – it is a loop around a language model that generates its own instruction sequence at runtime. The tool calls are the side effects. The model’s reasoning is the control flow.

Claude Code is Anthropic’s production implementation of this idea: a TypeScript monolith of nearly two thousand files that turns a terminal into a full development environment powered by Claude. It shipped to hundreds of thousands of developers, which means every architectural decision carries real-world consequences. This chapter gives you the mental model. Six abstractions define the entire system. A single data flow connects them. Once you internalize the golden path from keystroke to final output, every subsequent chapter is a zoom into one segment of that path.

What follows is a retrospective decomposition – these six abstractions were not designed upfront on a whiteboard. They emerged from the pressures of shipping a production agent to a large user base. Understanding them as they are, not as they were planned, sets the right expectations for the rest of the book.

1.2 The Six Key Abstractions

Claude Code is built on six core abstractions. Everything else – the 400+ utility files, the forked terminal renderer, the vim emulation, the cost tracker – exists to support these six.



Here is what each one does and why it exists.

1. The Query Loop (`query.ts`, ~1,700 lines). An async generator that is the heartbeat of the entire system. It streams a model response, collects tool calls, executes them, appends results to the message history, and loops. Every interaction – REPL, SDK, sub-agent, headless `--print` – flows through this

single function. It yields `Message` objects that the UI consumes. Its return type is a discriminated union called `Terminal` that encodes exactly why the loop stopped: normal completion, user abort, token budget exhaustion, stop hook intervention, max turns, or unrecoverable error. The generator pattern – rather than callbacks or event emitters – gives natural backpressure, clean cancellation, and typed terminal states. Chapter 5 covers the loop’s internals in full.

2. The Tool System (`Tool.ts`, `tools.ts`, `services/tools/`). A tool is anything the agent can do in the world: read a file, run a shell command, edit code, search the web. That simplicity of purpose hides significant machinery. Each tool implements a rich interface covering identity, schema, execution, permissions, and rendering. Tools are not just functions – they carry their own permission logic, concurrency declarations, progress reporting, and UI rendering. The system partitions tool calls into concurrent and serial batches, and a streaming executor starts concurrency-safe tools before the model even finishes its response. Chapter 6 covers the full tool interface and execution pipeline.

3. Tasks (`Task.ts`, `tasks/`). Tasks are background work units – primarily sub-agents. They follow a state machine: `pending` -> `running` -> `completed` | `failed` | `killed`. The `AgentTool` spawns a new `query()` generator with its own message history, tool set, and permission mode. Tasks give Claude Code its recursive capability: an agent can delegate to sub-agents, which can delegate further.

4. State (two layers). The system maintains state at two levels. A mutable singleton (`STATE`) holds ~80 fields of session-level infrastructure: working directory, model configuration, cost tracking, telemetry counters, session ID. It is set once at startup and mutated directly – no reactivity. A minimal reactive store (34 lines, Zustand-shaped) drives the UI: messages, input mode, tool approvals, progress indicators. The separation is intentional: infrastructure state changes rarely and does not need to trigger re-renders; UI state changes constantly and must. Chapter 3 covers the two-tier architecture in depth.

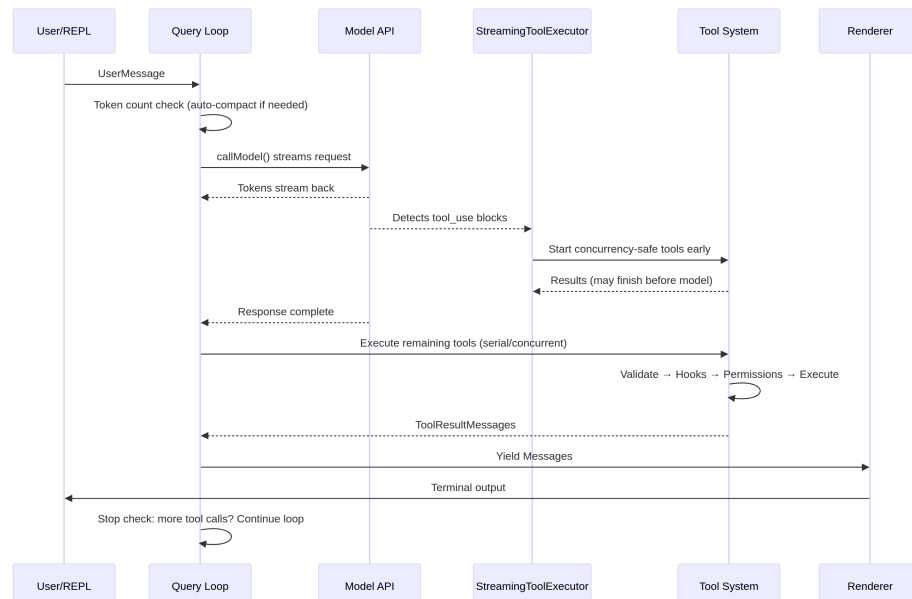
5. Memory (`memdir/`). The agent’s persistent context across sessions. Three tiers: project-level (`CLAUDE.md` files in the repo), user-level (`~/.claude/MEMORY.md`), and team-level (shared via symlinks). At session start, the system scans for all memory files, parses frontmatter, and an LLM selects which memories are relevant to the current conversation. Memory is how Claude Code “remembers” your codebase conventions, architectural

decisions, and debugging history.

6. Hooks (`hooks/`, `utils/hooks/`). User-defined lifecycle interceptors that fire at 27 distinct events across 4 execution types: shell commands, single-shot LLM prompts, multi-turn agent conversations, and HTTP webhooks. Hooks can block tool execution, modify inputs, inject additional context, or short-circuit the entire query loop. The permission system itself is partially implemented through hooks – `PreToolUse` hooks can deny tool calls before the interactive permission prompt ever fires.

1.3 The Golden Path: From Keystroke to Output

Trace a single request through the system. The user types “add error handling to the login function” and presses Enter.



Three things to notice about this flow.

First, the query loop is a generator, not a callback chain. The REPL pulls messages from it via `for await`, which means backpressure is natural – if the UI cannot keep up, the generator pauses. This is a deliberate choice over event emitters or observable streams.

Second, tool execution overlaps with model streaming. The `StreamingToolExecutor` does not wait for the model to finish before starting concurrency-safe tools. A `Read` call can complete and return its results while the model is still generating the rest of its response. This is speculative execution – if the model’s final output invalidates the tool call (rare but possible), the result is discarded.

Third, the entire loop is re-entrant. When the model makes tool calls, the results are appended to the message history, and the loop calls the model again with the updated context. There is no separate “tool result handling” phase – it is all one loop. The model decides when it is done by simply not making any more tool calls.

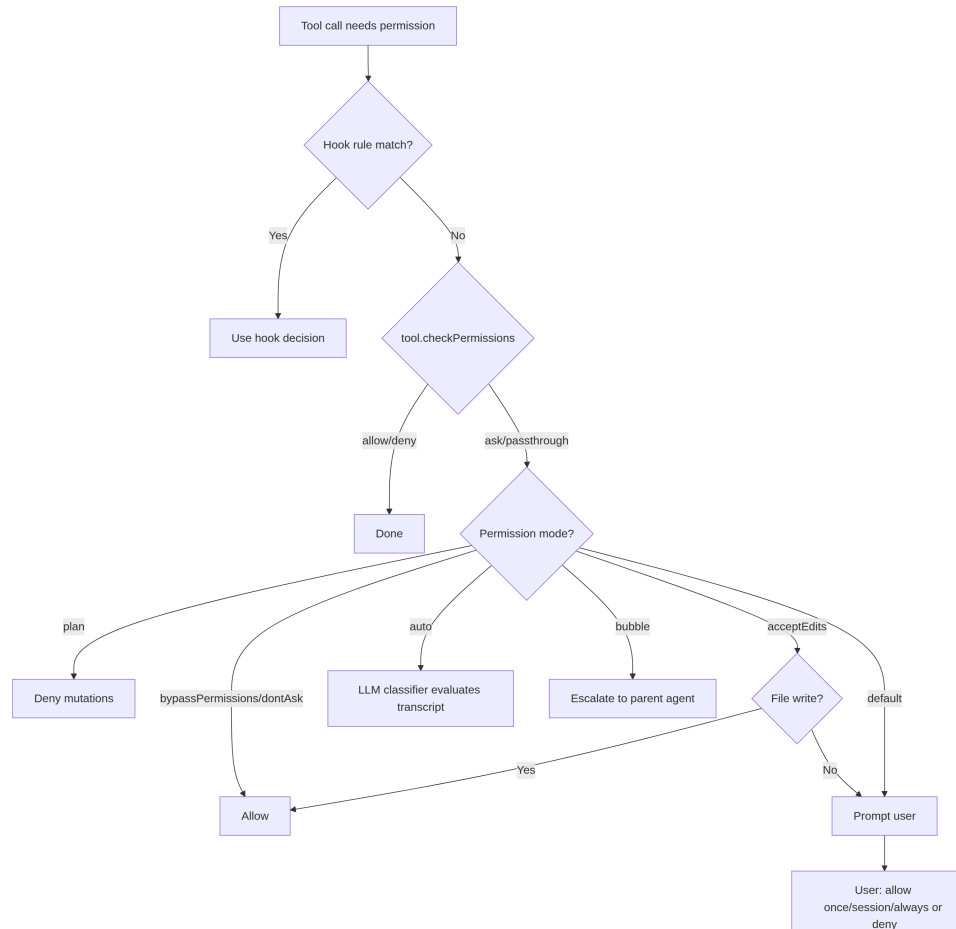
1.4 The Permission System

Claude Code runs arbitrary shell commands on your machine. It edits your files. It can spawn sub-processes, make network requests, and modify your git history. Without a permission system, this is a security catastrophe.

The system defines seven permission modes, ordered from most to least permissive:

Mode	Behavior
<code>bypassPermissions</code>	Everything allowed. No checks. Internal/testing only.
<code>dontAsk</code>	All allowed, but still logged. No user prompts.
<code>auto</code>	Transcript classifier (LLM) decides allow/deny.
<code>acceptEdits</code>	File edits auto-approved; all other mutations prompt.
<code>default</code>	Standard interactive mode. User approves each action.
<code>plan</code>	Read-only. All mutations blocked.
<code>bubble</code>	Escalate decision to parent agent (sub-agent mode).

When a tool call needs permission, the resolution follows a strict chain:

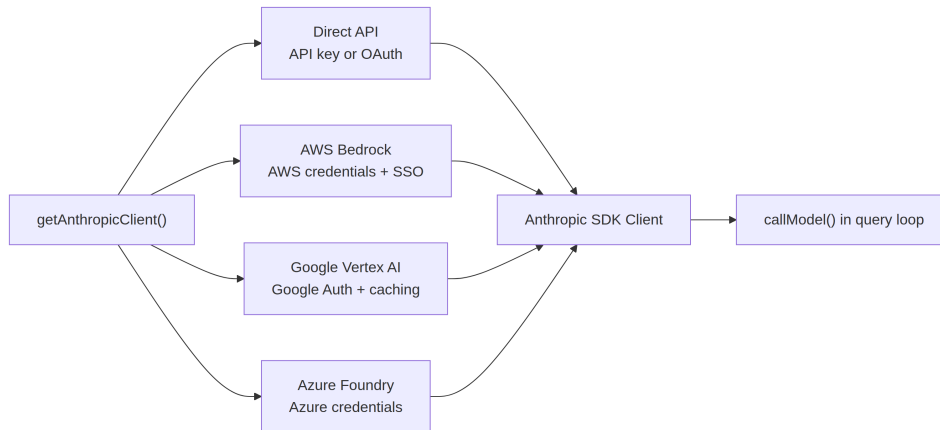


The **auto** mode deserves special attention. It runs a separate, lightweight LLM call that classifies the tool invocation against the conversation transcript. The classifier sees a compact representation of the tool input and decides whether the action is consistent with what the user asked for. This is the mode that lets Claude Code work semi-autonomously – approving routine operations while flagging anything that looks like it deviates from the user’s intent.

Sub-agents default to **bubble** mode, which means they cannot approve their own dangerous actions. Permission requests propagate up to the parent agent or ultimately to the user. This prevents a sub-agent from silently running destructive commands that the user never saw.

1.5 Multi-Provider Architecture

Claude Code talks to Claude through four different infrastructure paths, all transparent to the rest of the system.



The key insight is that the Anthropic SDK provides wrapper classes for each cloud provider that present the same interface as the direct API client. The `getAnthropicClient()` factory reads environment variables and configuration to determine which provider to use, constructs the appropriate client, and returns it. From that point forward, `callModel()` and every other consumer treats it as a generic Anthropic client.

Provider selection is determined at startup and stored in `STATE`. The query loop never checks which provider is active. This means switching from Direct API to Bedrock is a configuration change, not a code change – the agent loop, tool system, and permission model are entirely provider-agnostic.

1.6 The Build System

Claude Code ships as both an internal Anthropic tool and a public npm package. The same codebase serves both, with compile-time feature flags controlling what gets included.

```
// Conditional imports guarded by feature flags
const reactiveCompact = feature('REACTIVE_COMPACT')
  ? require('./services/compact/reactiveCompact.js')
  : null
```

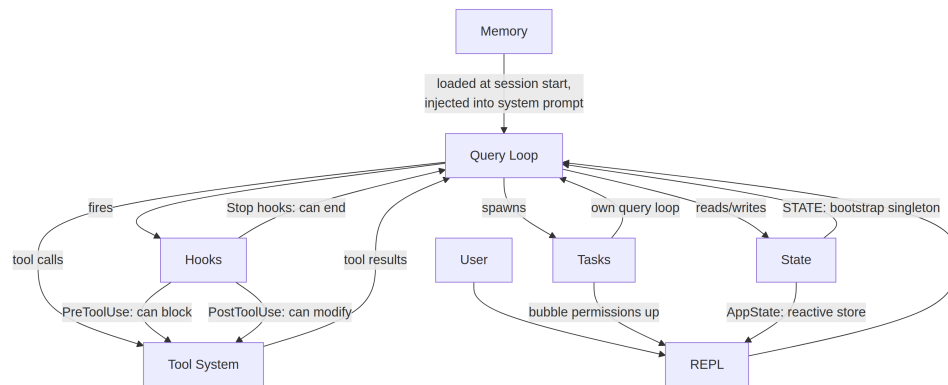
The `feature()` function comes from `bun:bundle`, Bun’s built-in bundler API. At build time, each feature flag resolves to a boolean literal. The bundler’s dead code elimination then strips the `require()` call entirely when the flag is false – the module is never loaded, never included in the bundle, and never shipped.

The pattern is consistent: a top-level `feature()` guard wrapping a `require()` call. The `require()` is used instead of `import` specifically because dynamic `require()` can be fully eliminated by the bundler when the guard is false, while dynamic `import()` cannot (it returns a Promise that the bundler must preserve).

There is an irony worth noting. The source maps published with early npm releases contained `sourcesContent` – the full original TypeScript source, including the internal-only code paths. The feature flags successfully stripped the runtime code but left the source in the maps. This is how the Claude Code source became publicly readable.

1.7 How the Pieces Connect

The six abstractions form a dependency graph:



Memory feeds into the query loop as part of the system prompt. The query loop drives tool execution. Tool results feed back into the query loop as messages. Tasks are recursive query loops with isolated message histories. Hooks intercept the query loop at defined points. State is read and written by everything, with the reactive store bridging to the UI.

The circular dependency between the query loop and the tool system is the system’s defining characteristic. The model generates tool calls. Tools execute and produce results. Results are appended to the message history. The model sees the results and decides what to do next. This cycle continues until the model stops generating tool calls or an external constraint (token budget, max turns, user abort) terminates it.

Here is how they connect to the chapters that follow: the golden path from input to output is the thread that runs through the entire book. Chapter 2 traces how the system boots to the point where this path can execute. Chapter 3 explains the two-tier state architecture that the path reads and writes. Chapter 4 covers the API layer that the query loop calls. Each subsequent chapter zooms into one segment of the path you have just seen end-to-end.

1.8 Apply This

If you are building an agentic system – any system where an LLM decides what actions to take at runtime – here are the patterns from Claude Code’s architecture that transfer.

The generator loop pattern. Use an async generator as your agent loop, not callbacks or event emitters. The generator gives you natural backpressure (consumers pull at their own pace), clean cancellation (`.return()` on the generator), and a typed return value for terminal states. The problem it solves: in callback-based agent loops, it is difficult to know when the loop is “done” and why. Generators make termination a first-class part of the type system.

The self-describing tool interface. Every tool should declare its own concurrency safety, permission requirements, and rendering behavior. Do not put this logic in a central orchestrator that “knows about” each tool. The problem it solves: a central orchestrator becomes a god object that must be updated every time a tool is added. Self-describing tools scale linearly – adding tool $N+1$ requires zero changes to existing code.

Separate infrastructure state from reactive state. Not all state needs to trigger UI updates. Session configuration, cost tracking, and telemetry belong in a plain mutable object. Message history, progress indicators, and approval queues belong in a reactive store. The problem it solves: making

everything reactive adds subscription overhead and complexity to state that changes once at startup and is read a thousand times. Two tiers match two access patterns.

Permission modes, not permission checks. Define a small set of named modes (plan, default, auto, bypass) and resolve every permission decision through the mode. Do not scatter `if (isAllowed)` checks through tool implementations. The problem it solves: inconsistent permission enforcement. When every tool goes through the same mode-based resolution chain, you can reason about the system's security posture by knowing which mode is active.

Recursive agent architecture via tasks. Sub-agents should be new instances of the same agent loop with their own message history, not special-cased code paths. Permission escalation flows upward via `bubble` mode. The problem it solves: sub-agent logic that diverges from the main agent loop, leading to subtle differences in behavior and error handling. If the sub-agent is the same loop, it inherits all the same guarantees.

Chapter 2

Chapter 2: Starting Fast – The Bootstrap Pipeline

If Chapter 1 gave you the map of Claude Code’s architecture, this chapter gives you the route it takes to reach a working state. Every component from the six abstractions – the query loop, the tool system, the state layers, hooks, memory – must be initialized before the user sees a cursor. The budget for all of it: 300 milliseconds.

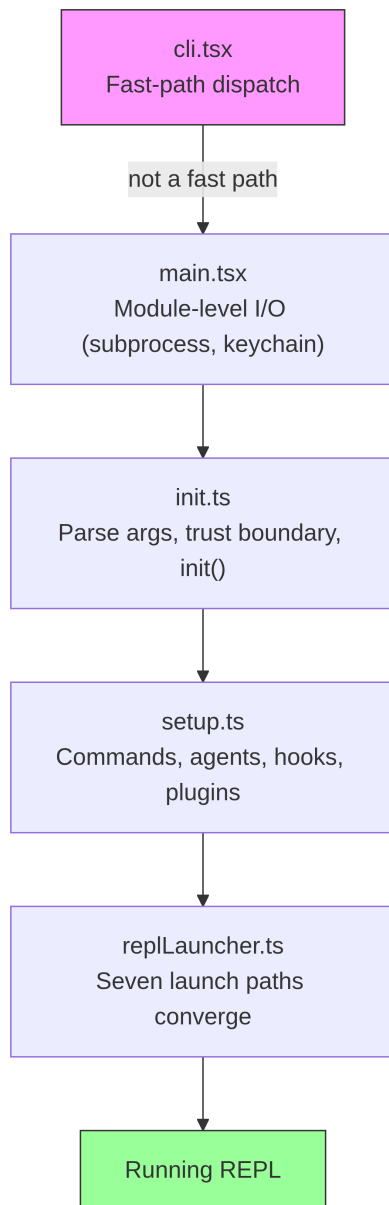
Three hundred milliseconds is the threshold where humans perceive a tool as instant. Cross it, and the CLI feels sluggish. Miss it by a lot, and developers stop using it. Everything in this chapter exists to stay under that line.

Bootstrap must accomplish four things: validate the environment, establish security boundaries, configure the communication layer, and render the UI. It must do all four in under 300ms. The architectural insight is that these four jobs can be partially overlapped, carefully ordered, and aggressively pruned to fit inside a budget that feels impossible for a system this complex.

A note on methodology: the timestamps in this chapter are approximate, derived from the codebase’s own profiling checkpoints. They represent typical warm-start timings on modern hardware. Cold starts are slower. The absolute numbers matter less than the relative structure: which operations overlap, which block, and which are deferred.

2.1 The Shape of the Pipeline

The startup pipeline lives in five files, executed in sequence. Each file narrows the scope of what the system needs to do next:



Each file does the minimum work necessary before passing control to the

next. `cli.tsx` tries to exit before importing anything heavy. `main.tsx` fires slow operations as side effects during import evaluation. `init.ts` resolves configuration and establishes the trust boundary. `setup.ts` registers capabilities. `replLauncher.ts` picks the right entry point and starts the UI.

Three parallelism strategies make this fast:

1. **Module-level subprocess dispatch.** Fire keychain and MDM reads as side effects *during import evaluation*. The subprocesses run while the remaining ~135ms of static imports load.
2. **Promise parallelism in setup.** Socket binding, hook snapshotting, command loading, and agent definition loading all run concurrently.
3. **Post-render deferred prefetches.** Everything the user does not need before typing their first message – git status, model capabilities, AWS credentials – runs after the prompt is visible.

A fourth strategy is less visible but equally important: **dynamic imports to defer module evaluation**. The codebase uses `await import('./module.js')` in at least a dozen places to avoid loading code until it is needed. OpenTelemetry (400KB + 700KB gRPC) loads only when telemetry initializes. React components load only when rendering. Each dynamic import trades cold-path latency (first use triggers module evaluation) for hot-path speed (startup does not pay for modules it might never use).

2.2 Phase 0: Fast-Path Dispatch (cli.tsx)

The first file the process enters, `cli.tsx`, has one job: determine whether the full bootstrap pipeline is needed at all. Many invocations – `claude --version`, `claude --help`, `claude mcp list` – need a specific answer and nothing else. Loading React, initializing telemetry, reading the keychain, and setting up the tool system would be pure waste.

The pattern is: check `argv`, dynamically import only the handler you need, and exit before the rest of the system loads.

```
// Pseudocode for the fast-path pattern
if (args.length === 1 && args[0] === '--version') {
  const { printVersion } = await import('./commands/version.js')
  await printVersion()
```

```
process.exit(0)
}
```

There are roughly a dozen fast paths covering version, help, configuration, MCP server management, and update checks. The specifics do not matter – the pattern does. Each path dynamically imports exactly one module, calls one function, and exits. The rest of the codebase never loads.

This is the first instance of a principle that recurs throughout bootstrap: **do less by knowing more about intent**. The argv array reveals the user’s intent. If the intent is narrow, the execution path should be narrow too.

If no fast path matches, `cli.tsx` falls through to the full `main.tsx` import, and the real startup begins.

2.3 Phase 1: Module-Level I/O (`main.tsx`)

When `main.tsx` is imported, its module-level side effects fire during evaluation – before any function in the file is called. This is the most performance-critical technique in the entire bootstrap:

```
// These run at import time, not at call time
const mdmPromise = startMDMSubprocess()
const keychainPromise = readKeychainCredentials()
```

While the JavaScript engine evaluates the rest of `main.tsx` and its transitive imports (~138ms of module evaluation), these two promises are already in flight. The MDM (Mobile Device Management) subprocess checks organizational security policies. The keychain read fetches stored credentials. Both are I/O-bound operations that would otherwise serialize on the critical path.

The insight: module evaluation is not idle time – it is time you can overlap with I/O. By the time `main.tsx`’s exported functions are first called, these promises are often already resolved.

This technique requires suppressing ESLint’s top-level-await and side-effect-in-module-scope rules in the relevant files. The codebase has a custom ESLint rule specifically for `process.env` access patterns that allows controlled side effects at module scope while preventing uncontrolled ones elsewhere.

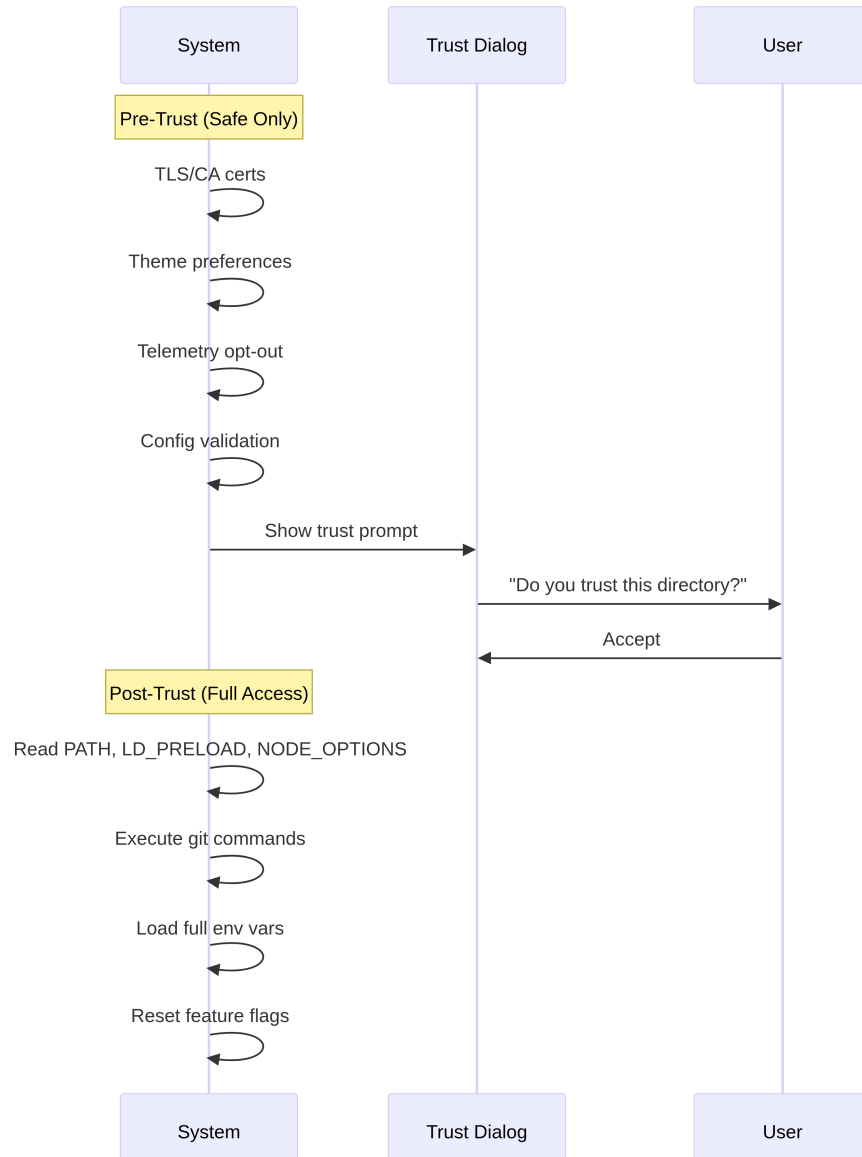
2.4 Phase 2: Parse and Trust (init.ts)

The `init()` function is memoized – calling it multiple times is safe and returns the same result. This is important because multiple entry points (the REPL, print mode, SDK mode) may each call `init()`, and the memoization guarantees it runs exactly once.

The function resolves command-line arguments via Commander, loads configuration from multiple sources (global settings, project settings, environment variables), and then hits the most important boundary in the pipeline.

2.4.1 The Trust Boundary

Before the trust boundary, the system operates in a restricted mode. After it, full capabilities are available. The boundary exists because Claude Code reads environment variables – and environment variables can be poisoned.



The trust boundary is not about the user trusting Claude Code. It is about Claude Code trusting the *environment*. A malicious `.bashrc` could set `LD_PRELOAD` to inject code into every subprocess. The trust dialog ensures the user explicitly consents to operating in a directory that may have been configured by someone else.

The system has ten distinct trust-sensitive operations. Before the user

accepts the trust dialog, only safe operations run: TLS certificate configuration, theme preferences, telemetry opt-out. After trust, the system reads potentially dangerous environment variables (PATH, LD_PRELOAD, NODE_OPTIONS), executes git commands, and applies the full environment configuration.

2.4.2 The preAction Hook

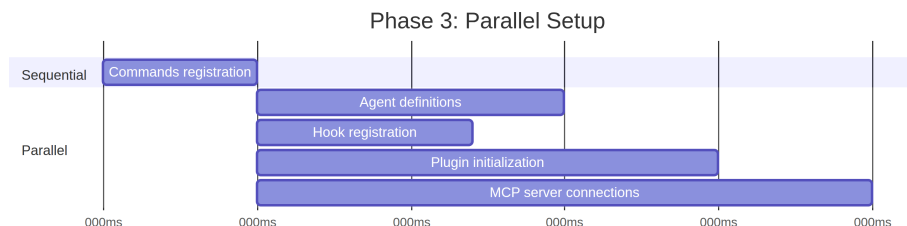
Commander’s `preAction` hook is the architectural linchpin. Commander parses the command structure (flags, subcommands, positional arguments) *without* executing anything. The `preAction` hook fires after parsing but before the matched command handler runs:

```
program.hook('preAction', async (thisCommand) => {
  await init(thisCommand)
})
```

This separation means fast-path commands (handled in `cli.tsx` before Commander loads) never pay the `init()` cost. Only commands that need the full environment trigger initialization.

2.5 Phase 3: Setup (setup.ts)

After `init()` completes, `setup()` registers all the capabilities the system needs:



Commands, agents, hooks, and plugins all register in parallel where possible. The setup phase is where the system transitions from “I know my configuration” to “I have all my capabilities.” After setup, every tool is registered, every hook is wired, and the system is ready to handle user input.

Setup also handles the security hook snapshot. The hook configuration is

read from disk once, frozen into an immutable snapshot, and used for the rest of the session. Later modifications to the hooks configuration file on disk are ignored. This prevents an attacker from modifying hook rules after the session starts – the frozen snapshot is the only source of truth for permission decisions.

2.6 Phase 4: Launch (`replLauncher.ts`)

Seven different code paths converge on `replLauncher.ts`: interactive REPL, print mode (`--print`), SDK mode, resume (`--resume`), continue (`--continue`), pipe mode, and headless. The launcher inspects the configuration produced by `init()` and dispatches to the right entry point.

Two examples illustrate the range:

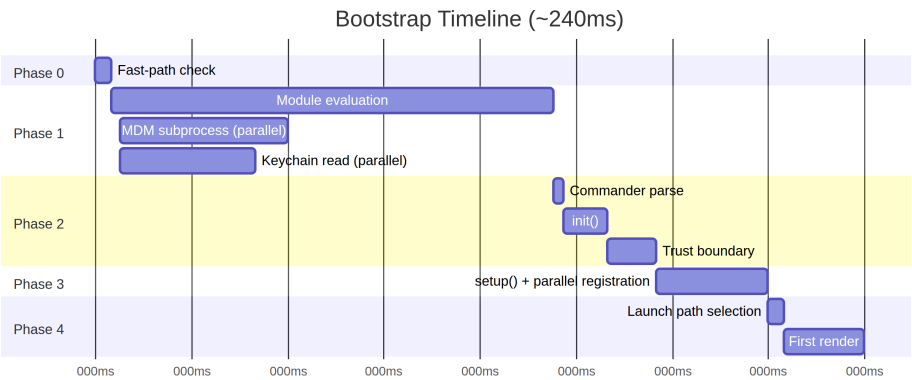
Interactive REPL – the standard case. The launcher mounts the React/Ink component tree, starts the terminal renderer, and enters the event loop. The user sees a prompt and can start typing.

Print mode (`--print`) – a single prompt from `argv`. The launcher creates a headless query loop with no React tree, runs it to completion, streams the output to `stdout`, and exits. Same agent loop, different presentation.

The important detail: all seven paths eventually call `query()` – the same agent loop from Chapter 1. The launch path determines *how* the loop is presented (interactive terminal, single-shot, SDK protocol), not *what* it does. This convergence is what makes the architecture testable and predictable: regardless of how the user invokes Claude Code, the core behavior is identical.

2.7 The Startup Timeline

Here is what the full pipeline looks like in time:



The critical path runs through module evaluation (the single longest phase at ~138ms), then Commander parse, init, and setup. The parallel I/O operations (MDM, keychain) overlap with module evaluation and are typically resolved before they are needed.

2.7.1 The Performance Budget

Phase	Time	What Happens
Fast-path check	~5ms	Check argv, exit early if possible
Module evaluation	~138ms	Import tree, fire parallel I/O
Commander parse	~3ms	Parse flags and subcommands
init()	~14ms	Config resolution, trust boundary
setup()	~35ms	Commands, agents, hooks, plugins
Launch + first render	~25ms	Pick path, mount React, first paint
Total	~240ms	Under 300ms budget

The total is approximately 240ms on a modern machine – 60ms of headroom under the 300ms budget. Cold starts (first run after reboot, OS cache empty) can push module evaluation to 200ms+, bringing the total closer to the limit.

2.8 The Migration System

A brief note on one subsystem that runs during init: schema migrations. Claude Code stores configuration and session data in local files and directories.

When the format changes between versions, migrations run automatically at startup.

Each migration is a function with a version number. The system checks the current schema version against the highest migration version, runs pending migrations in order, and updates the version. Migrations are idempotent and fast (operating on small local files, not databases). The entire migration pass typically completes in under 5ms. If a migration fails, it logs the error and continues – availability beats strict consistency for local configuration.

2.9 What Startup Teaches About System Design

The bootstrap pipeline is a study in narrowing scopes. Each phase reduces the space of possibilities:

- Phase 0 narrows from “any CLI invocation” to “needs full bootstrap”
- Phase 1 narrows from “everything must load” to “load in parallel with I/O”
- Phase 2 narrows from “unknown environment” to “trusted, configured environment”
- Phase 3 narrows from “no capabilities” to “fully registered”
- Phase 4 narrows from “seven possible modes” to “one concrete launch path”

By the time the REPL renders, every decision has been made. The query loop receives a fully configured environment with no ambiguity about what mode it is in, which tools are available, or what permissions apply. The 300ms budget is not just a performance target – it is a forcing function that prevents bootstrap from becoming a lazy initialization system where decisions are deferred and scattered throughout the codebase.

2.10 Apply This

Overlap I/O with initialization. Fire slow operations (subprocess spawns, credential reads, network checks) at module evaluation time, before they are needed. The JavaScript engine is doing synchronous work anyway – use that

time for parallel I/O. The pattern: `const promise = startSlowThing()` at the top of the file, `await promise` at the point of use.

Narrow scope as early as possible. The bootstrap pipeline’s five files form a funnel: each phase eliminates work that subsequent phases do not need to do. Fast-path dispatch is the most dramatic example, but the principle applies everywhere. If you can determine at parse time that a code path is unnecessary, skip it.

Establish trust boundaries explicitly. If your application reads from an environment it does not control (environment variables, configuration files, shell settings), draw a clear line between “safe to read before the user consents” and “only read after consent.” The trust boundary prevents a class of attacks where a malicious environment poisons the application before the user has a chance to evaluate it.

Memoize your init function. Make initialization idempotent – calling it twice produces the same result. This eliminates ordering bugs when multiple entry points may each trigger initialization. The memoization pattern is trivial but eliminates an entire class of double-initialization bugs.

Capture early input before yielding. In an event-driven system, user input that arrives during initialization can be lost. Claude Code captures the initial prompt from `argv` before any async work begins, ensuring that `claude "fix the bug"` does not drop the prompt if initialization takes longer than expected.

Chapter 3

Chapter 3: State – The Two-Tier Architecture

Chapter 2 traced the bootstrap pipeline from process start to first render. By the end, the system had a fully configured environment. But configured with *what*? Where does the session ID live? The current model? The message history? The cost tracker? The permission mode? Where does state live, and why does it live there?

Every long-running application eventually faces this question. For a simple CLI tool the answer is trivial – a few variables in `main()`. But Claude Code is not a simple CLI tool. It is a React application rendered through Ink, with a process lifecycle that spans hours, a plugin system that loads at arbitrary times, an API layer that must construct prompts from cached context, a cost tracker that survives process restarts, and dozens of infrastructure modules that need to read and write shared data without importing each other.

The naive approach – a single global store – fails immediately. If the cost tracker updated the same store that drives React re-renders, every API call would trigger a full component tree reconciliation. Infrastructure modules (bootstrap, context building, cost tracking, telemetry) cannot import React. They run before React mounts. They run after React unmounts. They run in contexts where no component tree exists at all. Putting everything into a React-aware store would create circular dependencies across the entire import graph.

Claude Code solves this with a two-tier architecture: a mutable process

singleton for infrastructure state, and a minimal reactive store for UI state. This chapter explains both tiers, the side-effect system that bridges them, and the supporting subsystems that depend on this foundation. Every subsequent chapter assumes you understand where state lives and why it lives there.

3.1 3.1 Bootstrap State – The Process Singleton

3.1.1 Why a Mutable Singleton

The bootstrap state module (`bootstrap/state.ts`) is a single mutable object created once at process start:

```
const STATE: State = getInitialState()
```

The comment above this line reads: **AND ESPECIALLY HERE**. Two lines above the type definition: **DO NOT ADD MORE STATE HERE – BE JUDICIOUS WITH GLOBAL STATE**. These comments have the tone of engineers who learned the cost of an ungoverned global object the hard way.

A mutable singleton is the right choice here for three reasons. First, bootstrap state must be available before any framework initializes – before React mounts, before the store is created, before plugins load. Module-scope initialization is the only mechanism that guarantees availability at import time. Second, the data is inherently process-scoped: session IDs, telemetry counters, cost accumulators, cached paths. There is no meaningful “previous state” to diff against, no subscribers to notify, no undo history. Third, the module must be a leaf in the import dependency graph. If it imported React, or the store, or any service module, it would create cycles that break the bootstrap sequence described in Chapter 2. By depending on nothing but utility types and `node:crypto`, it remains importable from anywhere.

3.1.2 The ~80 Fields

The `State` type contains approximately 80 fields. A sampling reveals the breadth:

Identity and paths – `originalCwd`, `projectRoot`, `cwd`, `sessionId`, `parentSessionId`. The `originalCwd` is resolved through `realpathSync` and NFC-normalized at process start. It never changes.

Cost and metrics – `totalCostUSD`, `totalAPIDuration`, `totalLinesAdded`, `totalLinesRemoved`. These accumulate monotonically through the session and persist to disk on exit.

Telemetry – `meter`, `sessionCounter`, `costCounter`, `tokenCounter`. Open-Telemetry handles, all nullable (null until telemetry initializes).

Model configuration – `mainLoopModelOverride`, `initialMainLoopModel`. The override is set when the user changes models mid-session.

Session flags – `isInteractive`, `kairosActive`, `sessionTrustAccepted`, `hasExitedPlanMode`. Booleans that gate behavior for the session duration.

Cache optimization – `promptCache1hAllowlist`, `promptCache1hEligible`, `systemPromptSectionCache`, `cachedClaudeMdContent`. These exist to prevent redundant computation and prompt cache busting.

3.1.3 The Getter/Setter Pattern

The STATE object is never exported. All access goes through approximately 100 individual getter and setter functions:

```
// Pseudocode - illustrates the pattern
export function getProjectRoot(): string {
  return STATE.projectRoot
}

export function setProjectRoot(dir: string): void {
  STATE.projectRoot = dir.normalize('NFC') // NFC normalization on every path setter
}
```

This pattern enforces encapsulation, NFC normalization on every path setter (preventing Unicode mismatches on macOS), type narrowing, and bootstrap isolation. The trade-off is verbosity – a hundred functions for eighty fields. But in a codebase where a stray mutation could bust a 50,000-token prompt cache, explicitness wins.

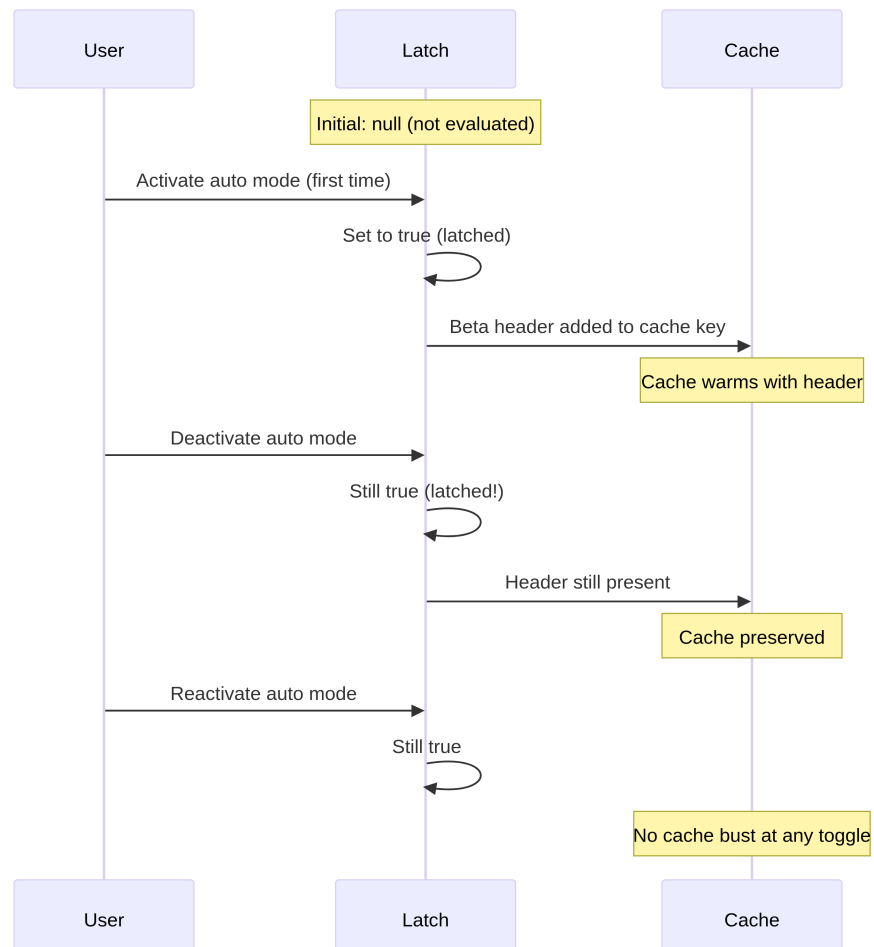
3.1.4 The Signal Pattern

Bootstrap cannot import listeners (it is a DAG leaf), so it uses a minimal pub/sub primitive called `createSignal`. The `sessionSwitched` signal has exactly one consumer: `concurrentSessions.ts`, which keeps PID files in sync. The signal is exposed as `onSessionSwitch =`

`sessionSwitched.subscribe`, letting callers register themselves without bootstrap knowing who they are.

3.1.5 The Five Sticky Latches

The most subtle fields in bootstrap state are five boolean latches that follow the same pattern: once a feature is first activated during a session, a corresponding flag stays `true` for the rest of the session. They all exist for one reason: prompt cache preservation.



Claude’s API supports server-side prompt caching. When consecutive requests share the same system prompt prefix, the server reuses cached computations. But the cache key includes HTTP headers and request body

fields. If a beta header appears in request N but not request N+1, the cache is busted – even if the prompt content is identical. For a system prompt exceeding 50,000 tokens, a cache miss is expensive.

The five latches:

Latch	What It Prevents
<code>afkModeHeaderLatched</code>	Shift+Tab auto mode toggling flips the AFK beta header on/off
<code>fastModeHeaderLatched</code>	Fast mode cooldown enter/exit flips the fast mode header
<code>cacheEditingHeaderLatched</code>	Cache feature flag changes bust every active user's cache
<code>thinkingClearLatched</code>	Triggered on confirmed cache miss (>1h idle). Prevents re-enabling thinking blocks from busting freshly warmed cache
<code>pendingPostCompaction</code>	Consume-once flag for telemetry: distinguishes compaction-induced cache misses from TTL-expiry misses

All five use a three-state type: `boolean | null`. The `null` initial value means “not yet evaluated.” `true` means “latched on.” They never return to `null` or `false` once set to `true`. This is the defining property of a latch.

The implementation pattern:

```
function shouldSendBetaHeader(featureCurrentlyActive: boolean): boolean {
  const latched = getAfkModeHeaderLatched()
  if (latched === true) return true           // Already latched -- always send
  if (featureCurrentlyActive) {
    setAfkModeHeaderLatched(true)             // First activation -- latch it
    return true
  }
  return false                                // Never activated -- don't send
}
```

Why not just always send all beta headers? Because headers are part of the cache key. Sending an unrecognized header creates a different cache namespace. The latch ensures you only enter a cache namespace when you actually need it, then stay there.

3.2 3.2 AppState – The Reactive Store

3.2.1 The 34-Line Implementation

The UI state store lives in `state/store.ts`:

The store implementation is approximately 30 lines: a closure over a `state` variable, an `Object.is` equality check to prevent spurious updates, synchronous listener notification, and an `onChange` callback for side effects. The skeleton looks like:

```
// Pseudocode - illustrates the pattern
function makeStore(initial, onTransition) {
  let current = initial
  const subs = new Set()
  return {
    read:      () => current,
    update:    (fn) => { /* Object.is guard, then notify */ },
    subscribe: (cb) => { subs.add(cb); return () => subs.delete(cb) },
  }
}
```

Thirty-four lines. No middleware, no devtools, no time-travel debugging, no action types. Just a closure over a mutable variable, a Set of listeners, and an `Object.is` equality check. This is Zustand without the library.

The design decisions worth examining:

Updater function pattern. There is no `setState(newValue)` – only `setState((prev) => next)`. Every mutation receives the current state and must produce the next state, eliminating stale-state bugs from concurrent mutations.

Object.is equality check. If the updater returns the same reference, the mutation is a no-op. No listeners fire. No side effects run. Critical for performance – components that spread-and-set without changing values produce no re-renders.

onChange fires before listeners. The optional `onChange` callback receives both old and new state and fires synchronously before any subscriber is

notified. This is used for side effects (Section 3.4) that must complete before the UI re-renders.

No middleware, no devtools. This is not an oversight. When your store needs exactly three operations (get, set, subscribe), an `Object.is` equality check, and a synchronous `onChange` hook, 34 lines of code you own is better than a dependency. You control the exact semantics. You can read the entire implementation in thirty seconds.

3.2.2 The AppState Type

The `AppState` type (~452 lines) is the shape of everything the UI needs to render. It is wrapped in `DeepImmutable<>` for most fields, with explicit exclusions for fields containing function types:

```
export type AppState = DeepImmutable<{
  settings: SettingsJson
  verbose: boolean
  // ... ~150 more fields
}> & {
  tasks: { [taskId: string]: TaskState } // Contains abort controllers
  agentNameRegistry: Map<string, AgentId>
}
```

The intersection type lets most fields be deeply immutable while exempting fields that hold functions, Maps, and mutable refs. Full immutability is the default, with surgical escape hatches where the type system would fight the runtime semantics.

3.2.3 React Integration

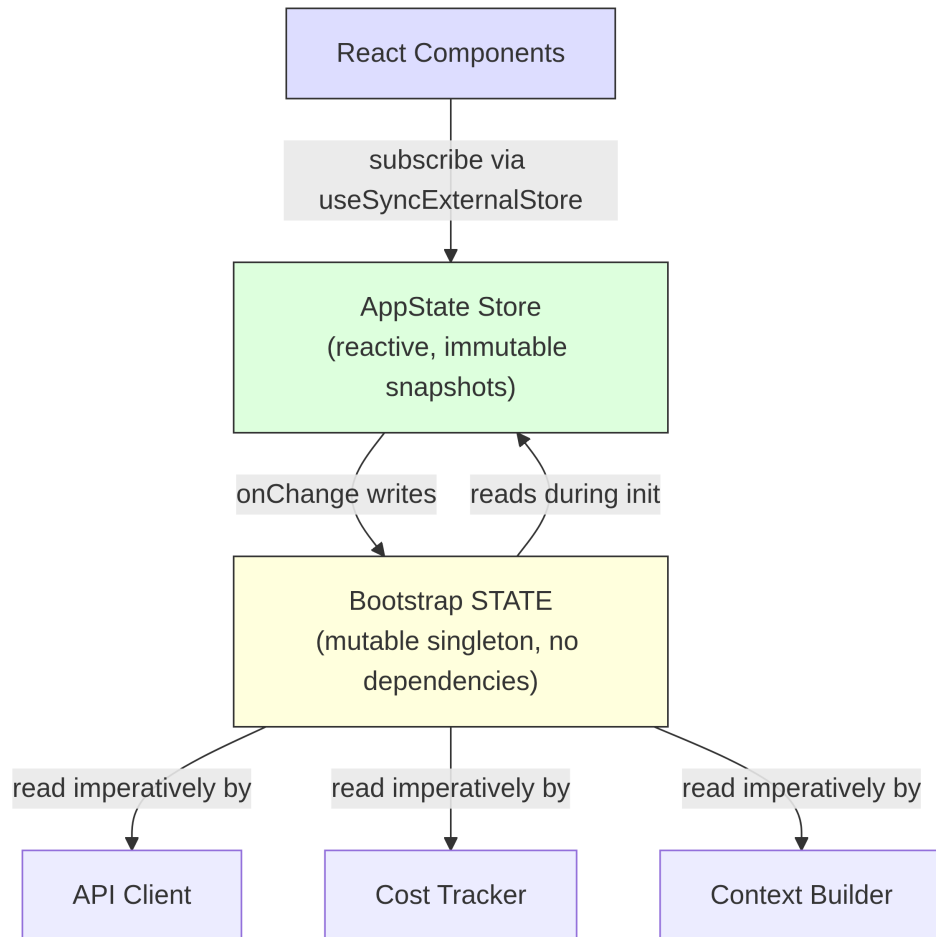
The store integrates with React through `useSyncExternalStore`:

```
// Standard React pattern - useSyncExternalStore with a selector
export function useAppState<T>(selector: (state: AppState) => T): T {
  const store = useContext(AppStoreContext)
  return useSyncExternalStore(
    store.subscribe,
    () => selector(store.getState()),
  )
}
```

The selector must return an existing sub-object reference (not a freshly constructed object) for `Object.is` comparison to prevent unnecessary re-renders. If you write `useAppState(s => ({ a: s.a, b: s.b })),` every render produces a new object reference, and the component re-renders on every state change. This is the same constraint Zustand users face – cheaper comparisons, but the selector author must understand reference identity.

3.3 How the Two Tiers Relate

The two tiers communicate through explicit, narrow interfaces.



Bootstrap state flows into AppState during initialization: `getDefaultAppState()` reads settings from disk (which bootstrap helped locate), checks feature flags (which bootstrap evaluated), and sets the initial model (which bootstrap resolved from CLI args and settings).

AppState flows back to bootstrap state through side effects: when the user changes the model, `onChangeAppState` calls `setMainLoopModelOverride()` in bootstrap. When settings change, credential caches in bootstrap are cleared.

But the two tiers never share a reference. A module that imports bootstrap state does not need to know about React. A component that reads AppState does not need to know about the process singleton.

A concrete example clarifies the data flow. When the user types `/model claude-sonnet-4`:

1. The command handler calls `store.setState(prev => ({ ...prev, mainLoopModel: 'claude-sonnet-4' }))`
2. The store's `Object.is` check detects a change
3. `onChangeAppState` fires, detects the model changed, calls `setMainLoopModelOverride()` (updates bootstrap) and `updateSettingsForSource()` (persists to disk)
4. All store subscribers fire – React components re-render to show the new model name
5. The next API call reads the model from `getMainLoopModelOverride()` in bootstrap state

Steps 1-4 are synchronous. The API client in step 5 may run seconds later. But it reads from bootstrap state (updated in step 3), not from AppState. This is the two-tier handoff: the UI store is the source of truth for what the user chose, but bootstrap state is the source of truth for what the API client uses.

The DAG property – bootstrap depends on nothing, AppState depends on bootstrap for init, React depends on AppState – is enforced by an ESLint rule that prevents `bootstrap/state.ts` from importing modules outside its allowed set.

3.4 3.4 Side Effects: `onChangeAppState`

The `onChange` callback is where the two tiers synchronize. Every `setState` call triggers `onChangeAppState`, which receives both previous and new state and decides what external effects to fire.

Permission mode sync is the primary use case. Prior to this centralized handler, permission mode was synced to the remote session (CCR) by only 2 of 8+ mutation paths. The other six – Shift+Tab cycling, dialog options, slash commands, rewind, bridge callbacks – all mutated `AppState` without telling CCR. The external metadata drifted out of sync.

The fix: stop scattering notifications across mutation sites and instead hook the diff in one place. The comment in the source code lists every mutation path that was broken and notes that “the scattered callsites above need zero changes.” This is the architectural benefit of centralized side effects – coverage is structural, not manual.

Model changes keep bootstrap state in sync with what the UI renders. **Settings changes** clear credential caches and re-apply environment variables. **Verbose toggle** and **expanded view** are persisted to global config.

The pattern – centralized side effects on a diffable state transition – is essentially the Observer pattern applied at the granularity of a state diff rather than individual events. It scales better than scattered event emissions because the number of side effects grows much more slowly than the number of mutation sites.

3.5 3.5 Context Building

Three memoized async functions in `context.ts` build the system prompt context prepended to every conversation. Each is computed once per session, not per turn.

`getGitStatus` runs five git commands in parallel (`Promise.all`), producing a block with the current branch, default branch, recent commits, and working tree status. The `--no-optional-locks` flag prevents git from taking write locks that could interfere with concurrent git operations in another terminal.

`getUserContext` loads `CLAUDE.md` content and caches it in bootstrap state via `setCachedClaudeMdContent`. This cache breaks a circular dependency:

the auto-mode classifier needs CLAUDE.md content, but CLAUDE.md loading goes through the filesystem, which goes through permissions, which calls the classifier. By caching in bootstrap state (a DAG leaf), the cycle is broken.

All three context functions use Lodash's `memoize` (compute once, cache forever) rather than TTL-based caching. The reasoning: if git status were re-computed every 5 minutes, the change would bust the server-side prompt cache. The system prompt even tells the model: "This is the git status at the start of the conversation. Note that this status is a snapshot in time."

3.6 3.6 Cost Tracking

Every API response flows through `addToTotalSessionCost`, which accumulates per-model usage, updates bootstrap state, reports to OpenTelemetry, and recursively processes advisor tool usage (nested model calls within a response).

Cost state survives process restarts through save-and-restore to a project config file. The session ID is used as a guard – costs are only restored if the persisted session ID matches the session being resumed.

Histograms use reservoir sampling (Algorithm R) to maintain bounded memory while accurately representing distributions. The 1,024-entry reservoir produces p50, p95, and p99 percentiles. Why not a simple running average? Because averages hide distribution shape. A session where 95% of API calls take 200ms and 5% take 10 seconds has the same average as one where all calls take 690ms, but the user experience is radically different.

3.7 3.7 What We Learned

The codebase has grown from a simple CLI to a system with ~450 lines of state type definitions, ~80 fields of process state, a side-effect system, multiple persistence boundaries, and cache optimization latches. None of this was designed upfront. The sticky latches were added when cache busting became a measurable cost problem. The `onChange` handler was centralized when 6

of 8 permission sync paths were discovered to be broken. The CLAUDE.md cache was added when a circular dependency emerged.

This is the natural growth pattern of state in a complex application. The two-tier architecture provides enough structure to contain the growth – new bootstrap fields do not affect React rendering, new AppState fields do not create import cycles – while remaining flexible enough to accommodate patterns that were not anticipated in the original design.

3.8 3.8 State Architecture Summary

Property	Bootstrap State	AppState
Location	Module-scope singleton	React context
Mutability	Mutable through setters	Immutable snapshots via updater
Subscribers	Signal (pub/sub) for specific events	<code>useSyncExternalStore</code> for React
Availability	Import time (before React)	After provider mounts
Persistence	Process exit handlers	Via <code>onChange</code> to disk
Equality	N/A (imperative reads)	<code>Object.is</code> reference check
Dependencies	DAG leaf (imports nothing)	Imports types from across codebase
Test reset	<code>resetStateForTests()</code>	Create new store instance
Primary consumers	API client, cost tracker, context builder	React components, side effects

3.9 Apply This

Separate state by access pattern, not by domain. Session ID belongs in the singleton not because it is “infrastructure” in the abstract, but because it must be readable before React mounts and writable without notifying subscribers. Permission mode belongs in the reactive store because changing

it must trigger re-renders and side effects. Let the access pattern drive the tier, and the architecture follows naturally.

The sticky latch pattern. Any system that interacts with a cache (prompt cache, CDN, query cache) faces the same problem: feature toggles that change the cache key mid-session cause invalidation. Once a feature is activated, its cache key contribution stays active for the session. The three-state type (`boolean | null`, meaning “not evaluated / on / never off”) makes the intent self-documenting. Especially valuable when the cache is not under your control.

Centralize side effects on state diffs. When multiple code paths can change the same state, do not scatter notifications across mutation sites. Hook the store’s `onChange` callback and detect which fields changed. Coverage becomes structural (any mutation triggers the effect) rather than manual (each mutation site must remember to notify).

Prefer 34 lines you own over a library you do not. When your requirements are exactly get, set, subscribe, and a change callback, a minimal implementation gives you full control over the semantics. In a system where state management bugs can cost real money, that transparency has value. The key insight is recognizing when you do *not* need a library.

Use process exit as a persistence boundary with intention. Multiple subsystems persist state on process exit. The trade-off is explicit: non-graceful termination (SIGKILL, OOM) loses accumulated data. This is acceptable because the data is diagnostic, not transactional, and writing to disk on every state change would be too expensive for counters that increment hundreds of times per session.

The two-tier architecture established in this chapter – bootstrap singleton for infrastructure, reactive store for UI, side effects bridging them – is the foundation that every subsequent chapter builds on. The conversation loop (Chapter 4) reads context from the memoized builders. The tool system (Chapter 5) checks permissions from AppState. The agent system (Chapter 8) creates task entries in AppState while tracking costs in bootstrap state. Understanding where state lives, and why, is prerequisite to understanding how any of these systems work.

Some fields straddle the boundary. The main loop model exists in both tiers: `mainLoopModel` in AppState (for UI rendering) and

`mainLoopModelOverride` in bootstrap state (for API client consumption). The `onChangeAppState` handler keeps them synchronized. This duplication is the cost of the two-tier split. But the alternative – having the API client import the React store, or having React components read from the process singleton – would violate the dependency direction that keeps the architecture sound. A small amount of controlled duplication, bridged by a centralized synchronization point, is preferable to a tangled dependency graph.

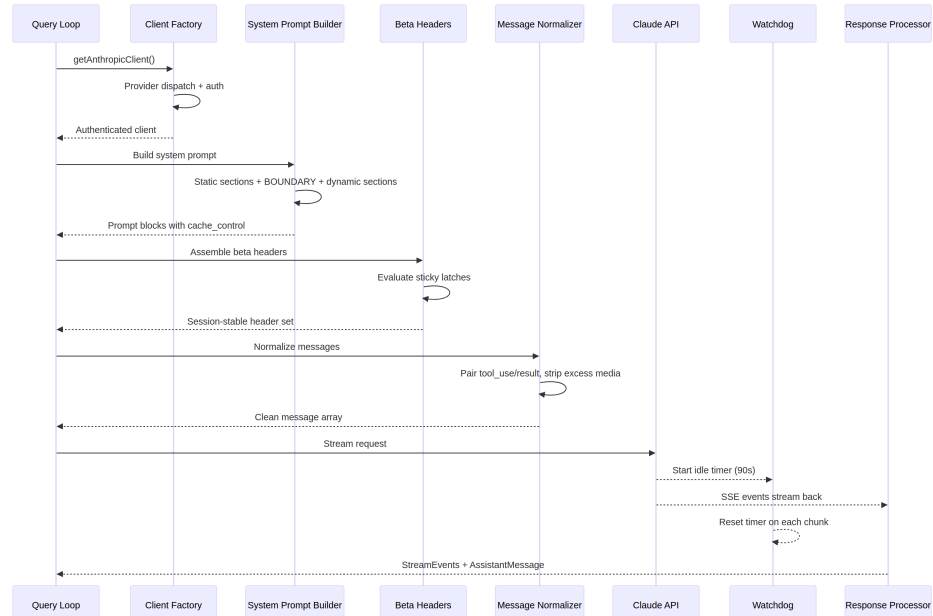
Chapter 4

Chapter 4: Talking to Claude – The API Layer

Chapter 3 established where state lives and how the two tiers communicate. Now we follow what happens when that state is put to use: the system needs to talk to a language model. Everything in Claude Code – the bootstrap sequence, the state system, the permission framework – exists to serve this moment.

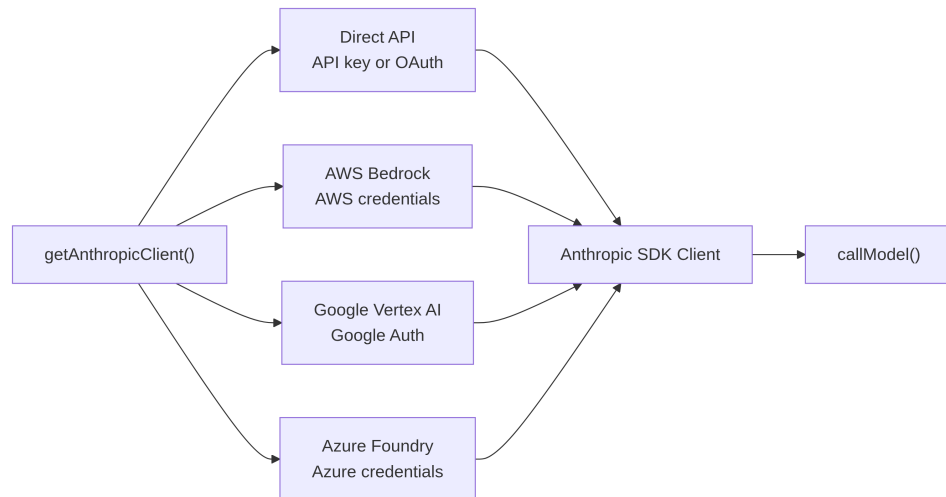
This layer handles more failure modes than any other part of the system. It must route through four cloud providers via a single transparent interface. It must construct system prompts with byte-level awareness of how the server’s prompt cache works, because a single misplaced section can bust a cache worth 50,000+ tokens. It must stream responses with active failure detection, because TCP connections die silently. And it must maintain session-stable invariants so that mid-conversation changes to feature flags do not cause invisible performance cliffs.

Let us trace a single API call from start to finish.



4.1 The Multi-Provider Client Factory

The `getAnthropicClient()` function is the single factory for all model communication. It returns an Anthropic SDK client configured for whichever provider the deployment targets:



The dispatch is entirely environment-variable driven, evaluated in a fixed priority order. All four provider-specific SDK classes are cast to **Anthropic** via **as unknown as Anthropic**. The comment in the source is refreshingly honest: “we have always been lying about the return type.” This deliberate type erasure means every consumer sees a uniform interface. The rest of the codebase never branches on provider.

Each provider SDK is dynamically imported – **AnthropicBedrock**, **AnthropicFoundry**, **AnthropicVertex** are heavy modules with their own dependency trees. The dynamic import ensures unused providers never load.

Provider selection is determined at startup and stored in bootstrap **STATE**. The query loop never checks which provider is active. Switching from Direct API to Bedrock is a configuration change, not a code change.

4.1.1 The buildFetch Wrapper

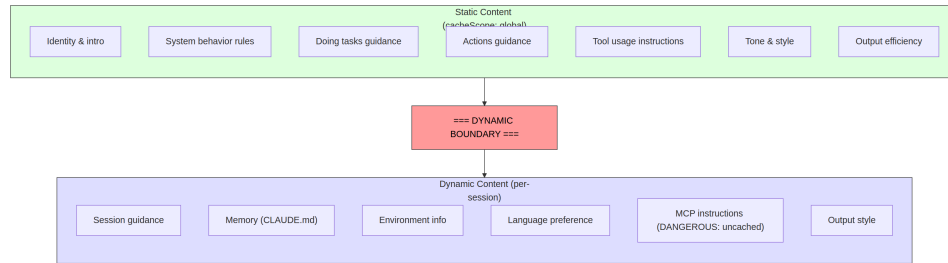
Every outbound fetch gets wrapped to inject an **x-client-request-id** header – a UUID generated per request. When a request times out, the server never assigns a request ID to the response. Without the client-side ID, the API team cannot correlate the timeout with server-side logs. This header bridges that gap. It is only sent to first-party Anthropic endpoints – third-party providers might reject unknown headers.

4.2 System Prompt Construction

The system prompt is the most cache-sensitive artifact in the entire system. Claude’s API provides server-side prompt caching: identical prompt prefixes across requests can be cached, saving both latency and cost. A 200K-token conversation might have 50-70K tokens that are identical to the previous turn. Busting that cache forces the server to re-process all of it.

4.2.1 The Dynamic Boundary Marker

The prompt is built as an array of string sections with a critical dividing line:



Everything before the boundary is identical across sessions, users, and organizations – it gets the highest tier of server-side caching. Everything after contains user-specific content and drops to per-session caching.

The naming convention for sections is deliberately loud. Adding a new section requires choosing between `systemPromptSection` (safe, cached) and `DANGEROUS_uncachedSystemPromptSection` (cache-breaking, requires a reason string). The `_reason` parameter is unused at runtime but serves as mandatory documentation – every cache-breaking section carries its justification in the source code.

4.2.2 The 2^N Problem

A comment in `prompts.ts` explains why conditional sections must go after the boundary:

Each conditional here is a runtime bit that would otherwise multiply the Blake2b prefix hash variants (2^N).

Every boolean condition before the boundary doubles the number of unique global cache entries. Three conditionals create 8 variants; five create 32. The static sections are deliberately unconditional. Compile-time feature flags (resolved by the bundler) are acceptable before the boundary. Runtime checks (is this Haiku? does the user have auto mode?) must go after.

This is the kind of constraint that is invisible until you violate it. A well-intentioned engineer adding a user-setting-gated section before the boundary could silently fragment the global cache and double the fleet's prompt processing costs.

4.3 Streaming

4.3.1 Raw SSE Over SDK Abstractions

The streaming implementation uses the raw `Stream<BetaRawMessageStreamEvent>` rather than the SDK's higher-level `BetaMessageStream`. The reason: `BetaMessageStream` calls `partialParse()` on every `input_json_delta` event. For tool calls with large JSON inputs (file edits with hundreds of lines), this re-parses the growing JSON string from scratch on every chunk – $O(n^2)$ behavior. Claude Code handles tool input accumulation itself, so the partial parsing is pure waste.

4.3.2 The Idle Watchdog

TCP connections can die without notification. The server may crash, a load balancer may silently drop the connection, or a corporate proxy may time out. The SDK's request timeout only covers the initial fetch – once HTTP 200 arrives, the timeout is satisfied. If the streaming body stops, nothing catches it.

The watchdog: a `setTimeout` that resets on every received chunk. If no chunks arrive for 90 seconds, the stream is aborted and the system falls back to a non-streaming retry. A warning fires at the 45-second mark. When the watchdog fires, it logs the event with the client request ID for correlation.

4.3.3 Non-Streaming Fallback

When streaming fails mid-response (network error, stall, truncation), the system falls back to a synchronous `messages.create()` call. This handles proxy failures where the proxy returns HTTP 200 with a non-SSE body, or truncates the SSE stream partway through.

The fallback can be disabled when streaming tool execution is active, since a fallback would re-execute the entire request and potentially run tools twice.

4.4 Prompt Cache System

4.4.1 Three Tiers

Prompt caching operates at three levels:

Ephemeral cache (default): Per-session caching with a server-defined TTL (~5 minutes). All users get this.

1-hour TTL: Eligible users get extended caching. Eligibility is determined by subscription status and latched in bootstrap state – the `promptCache1hEligible` sticky latch from Chapter 3 ensures a mid-session overage flip does not change the TTL.

Global scope: System prompt cache entries get cross-session, cross-organization sharing. The static portions of the prompt are identical for all Claude Code users, so a single cached copy serves everyone. Global scope is disabled when MCP tools are present, because MCP tool definitions are user-specific and would fragment the cache into millions of unique prefixes.

4.4.2 The Sticky Latches in Action

The five sticky latches from Chapter 3 are evaluated here, during request construction. Each latch starts as `null` and, once set to `true`, remains `true` for the session. The comment above the latch block is precise: “Sticky-on latches for dynamic beta headers. Each header, once first sent, keeps being sent for the rest of the session so mid-session toggles don’t change the server-side cache key and bust ~50-70K tokens.”

See Chapter 3, Section 3.1 for the full explanation of the latch pattern, the five specific latches, and why always-send-all-headers is not the right solution.

4.5 The queryModel Generator

The `queryModel()` function is an async generator (~700 lines) that orchestrates the entire API call lifecycle. It yields `StreamEvent`, `AssistantMessage`, and `SystemAPIErrorMessage` objects.

The request assembly follows a carefully ordered sequence:

1. **Kill switch check** – safety valve for the most expensive model tier
2. **Beta header assembly** – model-specific, with sticky latches applied
3. **Tool schema building** – parallel via `Promise.all()`, deferred tools excluded until discovered
4. **Message normalization** – repair orphaned `tool_use/tool_result` mismatches, strip excess media, remove stale blocks

5. **System prompt block construction** – split at the dynamic boundary, assign cache scopes
6. **Retry-wrapped streaming** – handles 529 (overloaded), model fallback, thinking downgrade, OAuth refresh

4.5.1 Output Token Cap

The default output cap is 8,000 tokens, not the typical 32K or 64K. Production data showed that p99 output is 4,911 tokens – standard limits over-reserve by 8-16x. When a response hits the cap (<1% of requests), it gets one clean retry at 64K. This saves significant cost at fleet scale.

4.5.2 Error Handling and Retry

The `withRetry()` function is itself an async generator that yields `SystemAPIErrorMessage` events so the UI can display retry status. Retry strategies:

- **529 (overloaded)**: Wait and retry, optionally downgrading fast mode
- **Model fallback**: Primary model fails, try a fallback (e.g., Opus to Sonnet)
- **Thinking downgrade**: Context window overflow triggers reduced thinking budget
- **OAuth 401**: Refresh token and retry once

The generator pattern means retry progress (“Server overloaded, retrying in 5s...”) appears as a natural part of the event stream, not as a side-channel notification.

4.6 Apply This

Treat prompt caching as an architectural constraint, not a feature toggle. Most LLM applications “turn on” caching. Claude Code treats it as a design constraint that shapes prompt ordering, section memoization, header latching, and configuration management. The difference between a well-structured prompt (cache hit on 50K tokens) and a poorly-structured one (full reprocessing every turn) is the single largest cost lever in the system.

Use the DANGEROUS naming convention for costly escape hatches. When a codebase has an invariant that is easy to violate

accidentally, naming the escape hatch with a loud prefix does three things: makes violations visible in code review, forces documentation (the required reason parameter), and creates psychological friction toward the safe default. This generalizes beyond caching to any operation with invisible cost.

Build streaming with a watchdog, not just a timeout. The SDK’s request timeout satisfies on HTTP 200, but the response body can stop arriving at any point. A `setTimeout` that resets on every chunk catches this. The non-streaming fallback handles proxy failure modes (HTTP 200 with non-SSE body, mid-stream truncation) that are more common than you expect in corporate environments.

Make retry strategies yield-based, not exception-based. By making the retry wrapper an async generator that yields status events, the caller displays retry progress as a natural part of the event stream. The model fallback pattern (Opus fails, try Sonnet) is particularly useful for production resilience.

Separate the fast path from the full pipeline. Not every API call needs tool search, advisor integration, thinking budgets, and streaming infrastructure. Claude Code’s `queryHaiku()` function provides a streamlined path for internal operations (compaction, classification) that skips all agentic concerns. A separate function with a simplified interface prevents accidental complexity leakage.

4.7 Looking Ahead

The API layer sits at the foundation of everything that follows. Chapter 5 will show how the query loop uses the streaming response to drive tool execution – including how tools begin executing before the model finishes its response. Chapter 6 will explain how the compaction system preserves cache efficiency when conversations approach the context limit. Chapter 7 will show how each agent thread gets its own message array and request chain.

All of those systems inherit the constraints established here: cache stability as an architectural invariant, provider transparency through the client factory, and session-stable configuration through the latch system. The API layer does not just send requests – it defines the rules by which every other system operates.

Part II

Part II: The Core Loop

Chapter 5

Chapter 5: The Agent Loop

5.1 The Beating Heart

Chapter 4 showed how the API layer transforms configuration into streaming HTTP requests – how the client is built, how system prompts are assembled, how responses arrive as server-sent events. That layer handles the *mechanics* of talking to the model. But a single API call is not an agent. An agent is a loop: call the model, execute tools, feed results back, call the model again, until the work is done.

Every system has a center of gravity. In a database, it is the storage engine. In a compiler, it is the intermediate representation. In Claude Code, it is `query.ts` – a single 1,730-line file containing the async generator that runs every interaction, from the first keystroke in the REPL to the last tool call of a headless `--print` invocation.

This is not an exaggeration. There is exactly one code path that talks to the model, executes tools, manages context, recovers from errors, and decides when to stop. That code path is the `query()` function. The REPL calls it. The SDK calls it. Sub-agents call it. The headless runner calls it. If you are using Claude Code, you are inside `query()`.

The file is dense, but it is not complex in the way that tangled inheritance hierarchies are complex. It is complex in the way that a submarine is complex: a single hull with many redundant systems, each one added because the ocean found a way in. Every `if` branch has a story. Every withheld error message represents a real bug where an SDK consumer disconnected mid-recovery.

Every circuit breaker threshold was tuned against real sessions that burned thousands of API calls in infinite loops.

This chapter traces the entire loop, start to finish. By the end, you will understand not just what happens, but why each mechanism exists and what breaks without it.

5.2 Why an Async Generator

The first architectural question: why is the agent loop a generator and not a callback-based event emitter?

```
// Simplified - shows the concept, not the exact types
async function* agentLoop(params: LoopParams): AsyncGenerator<Message | Event, T,
```

The actual signature yields several message and event types and returns a discriminated union encoding why the loop stopped.

Three reasons, in order of importance.

Backpressure. An event emitter fires whether the consumer is ready or not. A generator yields only when the consumer calls `.next()`. When the REPL’s React renderer is busy painting the previous frame, the generator naturally pauses. When an SDK consumer is processing a tool result, the generator waits. No buffer overflow, no dropped messages, no “fast producer / slow consumer” problem.

Return value semantics. The generator’s return type is `Terminal` – a discriminated union encoding exactly why the loop stopped. Was it a normal completion? A user abort? A token budget exhaustion? A stop hook intervention? A max-turns limit? An unrecoverable model error? There are 10 distinct terminal states. Callers do not need to subscribe to an “end” event and hope the payload contains the reason. They get it as a typed return value from `for await...of` or `yield*`.

Composability via `yield*`. The outer `query()` function delegates to `queryLoop()` with `yield*`, which transparently forwards every yielded value and the final return. Sub-generators like `handleStopHooks()` use the same pattern. This creates a clean chain of responsibility without callbacks, without promises wrapping promises, without event forwarding boilerplate.

The choice has a cost – async generators in JavaScript cannot be “rewound” or forked. But the agent loop does not need either. It is a strictly forward-moving state machine.

One more subtlety: the `function*` syntax makes the function *lazy*. The body does not execute until the first `.next()` call. This means `query()` returns instantly – all the heavy initialization (config snapshot, memory prefetch, budget tracker) happens only when the consumer starts pulling values. In the REPL, this means the React rendering pipeline is already set up before the first line of the loop runs.

5.3 What Callers Provide

Before tracing the loop, it helps to know what goes in:

```
// Simplified - illustrates the key fields
type LoopParams = {
  messages: Message[]
  prompt: SystemPrompt
  permissionCheck: CanUseToolFn
  context: ToolUseContext
  source: QuerySource           // 'repl', 'sdk', 'agent:xyz', 'compact', etc.
  maxTurns?: number
  budget?: { total: number }    // API-level task budget
  deps?: LoopDeps              // Injected for testing
}
```

The notable fields:

- **querySource**: A string discriminant like `'repl_main_thread'`, `'sdk'`, `'agent:xyz'`, `'compact'`, or `'session_memory'`. Many conditionals branch on this. The compact agent uses `querySource: 'compact'` so the blocking limit guard does not deadlock (the compact agent needs to run to *reduce* the token count).
- **taskBudget**: The API-level task budget (`output_config.task_budget`). Distinct from the +500k auto-continue token budget feature. `total` is the budget for the whole agentic turn; `remaining` is computed per iteration from cumulative API usage and adjusted across compaction boundaries.

- **deps**: Optional dependency injection. Defaults to `productionDeps()`. This is the seam where tests swap in fake model calls, fake compaction, and deterministic UUIDs.
 - **canUseTool**: A function that returns whether a given tool is allowed. This is the permission layer – it checks trust settings, hook decisions, and the current permission mode.
-

5.4 The Two-Layer Entry Point

The public API is a thin wrapper around the real loop:

The outer function wraps the inner loop, tracking which queued commands were consumed during the turn. After the inner loop completes, consumed commands are marked as `'completed'`. If the loop throws or the generator is closed via `.return()`, the completion notifications never fire – a failed turn should not mark commands as successfully processed. Commands queued during a turn (via `/` slash commands or task notifications) are marked `'started'` inside the loop and `'completed'` in the wrapper. If the loop throws or the generator is closed via `.return()`, the completion notifications never fire. This is intentional – a failed turn should not mark commands as successfully processed.

5.5 The State Object

The loop carries its state in a single typed object:

```
// Simplified - illustrates the key fields
type LoopState = {
  messages: Message[]
  context: ToolUseContext
  turnCount: number
  transition: Continue | undefined
  // ... plus recovery counters, compaction tracking, pending summaries, etc.
}
```

Ten fields. Each one earns its place:

Field	Why It Exists
<code>messages</code>	The conversation history, grown each iteration
<code>toolUseContext</code>	Mutable context: tools, abort controller, agent state, options
<code>autoCompactTracking</code>	Tracks compaction state: turn counter, turn ID, consecutive failures, compacted flag
<code>maxOutputTokensRecoveryCount</code>	How many multi-turn recovery attempts for output token limits (max 3)
<code>hasAttemptedReactiveCompact</code>	One-shot guard against infinite reactive compaction loops
<code>maxOutputTokensOverride</code>	Set to 64K during escalation, cleared after
<code>pendingToolUseSummary</code>	A promise from the previous iteration's Haiku summary, resolved during current streaming
<code>stopHookActive</code>	Prevents re-running stop hooks after a blocking retry
<code>turnCount</code>	Monotonic counter, checked against <code>maxTurns</code>
<code>transition</code>	Why the previous iteration continued – <code>undefined</code> on first iteration

5.5.1 Immutable Transitions in a Mutable Loop

Here is the pattern that appears at every `continue` statement in the loop:

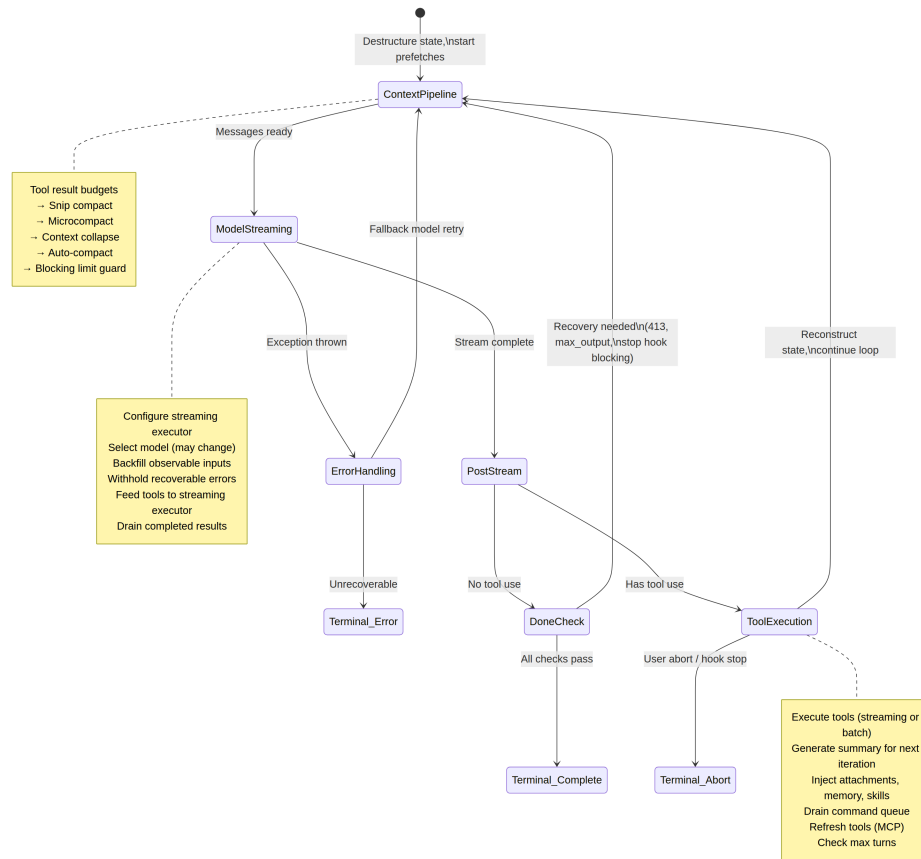
```
const next: State = {
  messages: [...messagesForQuery, ...assistantMessages, ...toolResults],
  toolUseContext: toolUseContextWithQueryTracking,
  autoCompactTracking: tracking,
  turnCount: nextTurnCount,
  maxOutputTokensRecoveryCount: 0,
  hasAttemptedReactiveCompact: false,
  pendingToolUseSummary: nextPendingToolUseSummary,
  maxOutputTokensOverride: undefined,
  stopHookActive,
  transition: { reason: 'next_turn' },
}
state = next
```

Every `continue` site constructs a complete new `State` object. Not `state.messages = newMessages`. Not `state.turnCount++`. A full reconstruction. The benefit is that every transition is self-documenting. You can

read any `continue` site and see exactly which fields change and which are preserved. The `transition` field on the new state records *why* the loop is continuing – tests assert on this to verify that the correct recovery path fired.

5.6 The Loop Body

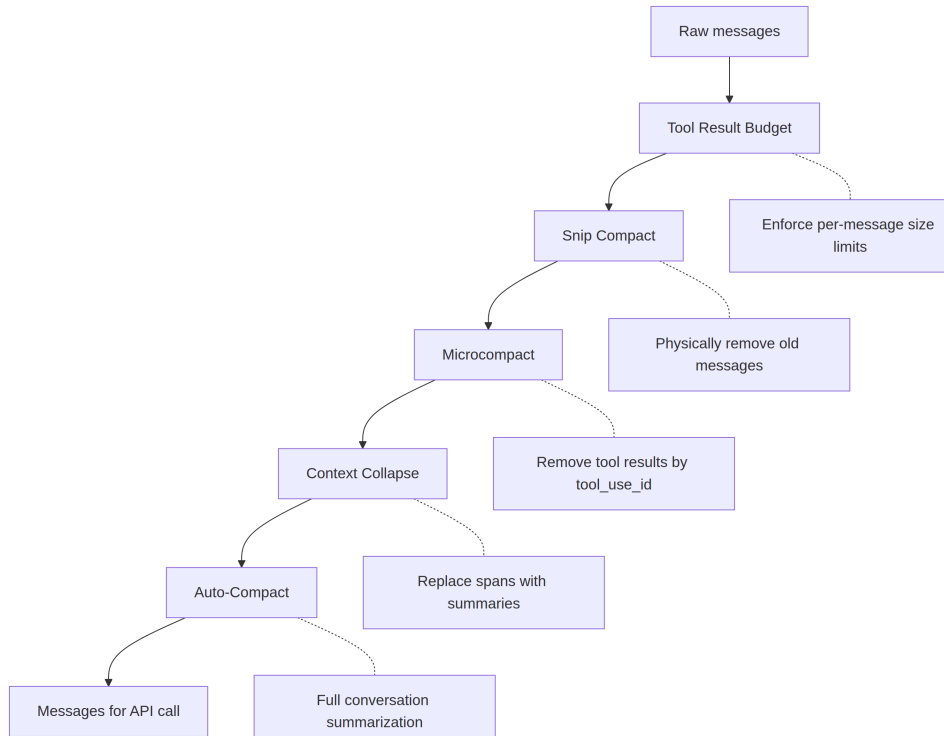
Here is the full execution flow of a single iteration, compressed to its skeleton:



That is the entire loop. Every feature in Claude Code – from memory to sub-agents to error recovery – feeds into or consumes from this single iteration structure.

5.7 Context Management: Four Compression Layers

Before each API call, the message history passes through up to four context management stages. They run in a specific order, and that order matters.



5.7.1 Layer 0: Tool Result Budget

Before any compression, `applyToolResultBudget()` enforces per-message size limits on tool results. Tools without a finite `maxResultSizeChars` are exempted.

5.7.2 Layer 1: Snip Compact

The lightest operation. Snip physically removes old messages from the array, yielding a boundary message to signal the removal to the UI. It reports how many tokens were freed, and that number is plumbed into auto-compact's threshold check.

5.7.3 Layer 2: Microcompact

Microcompact removes tool results that are no longer needed, identified by `tool_use_id`. For cached microcompact (which edits the API cache), the boundary message is deferred until after the API response. The reason: client-side token estimates are unreliable. The actual `cache_deleted_input_tokens` from the API response tells you what was really freed.

5.7.4 Layer 3: Context Collapse

Context collapse replaces spans of conversation with summaries. It runs before auto-compact, and the ordering is deliberate: if collapse reduces the context below the auto-compact threshold, auto-compact becomes a no-op. This preserves granular context instead of replacing everything with a single monolithic summary.

5.7.5 Layer 4: Auto-Compact

The heaviest operation: it forks an entire Claude conversation to summarize the history. The implementation has a circuit breaker – after 3 consecutive failures, it stops trying. This prevents the nightmare scenario observed in production: sessions stuck over the context limit burning 250K API calls per day in an infinite compact-fail-retry loop.

5.7.6 Auto-Compact Thresholds

The thresholds are derived from the model’s context window:

```
effectiveContextWindow = contextWindow - min(modelMaxOutput, 20000)
```

Thresholds (relative to `effectiveContextWindow`):

```
Auto-compact fires:      effectiveWindow - 13,000
Blocking limit (hard):   effectiveWindow - 3,000
```

Constant	Value	Purpose
<code>AUTOCOMPACT_BUFFER_TOKENS</code>	13,000	Headroom below effective window for auto-compact trigger
<code>MANUAL_COMPACT_BUFFER_TOKENS</code>	3,000	Reserves space so /compact still works

Constant	Value	Purpose
<code>MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES</code>		Circuit breaker threshold

The 13,000-token buffer means auto-compact fires well before the hard limit. The gap between the auto-compact threshold and the blocking limit is where reactive compact operates – if the proactive auto-compact fails or is disabled, reactive compact catches the 413 error and compacts on demand.

5.7.7 Token Counting

The canonical function `tokenCountWithEstimation` combines authoritative API-reported token counts (from the most recent response) with a rough estimate for messages added after that response. The approximation is conservative – it errs toward higher counts, which means auto-compact fires slightly early rather than slightly late.

5.8 Model Streaming

5.8.1 The `callModel()` Loop

The API call happens inside a `while(attemptWithFallback)` loop that enables model fallback:

```
let attemptWithFallback = true
while (attemptWithFallback) {
  attemptWithFallback = false
  try {
    for await (const message of deps.callModel({ messages, systemPrompt, tools, signal })) {
      // Process each streamed message
    }
  } catch (innerError) {
    if (innerError instanceof FallbackTriggeredError && fallbackModel) {
      currentModel = fallbackModel
      attemptWithFallback = true
      continue
    }
    throw innerError
  }
}
```

```
}  
}
```

When enabled, a `StreamingToolExecutor` starts executing tools as soon as their `tool_use` blocks arrive during streaming – not after the full response completes. How tools are orchestrated into concurrent batches is the subject of Chapter 7.

5.8.2 The Withholding Pattern

This is one of the most important patterns in the file. Recoverable errors are suppressed from the yield stream:

```
let withheld = false  
if (contextCollapse?.isWithheldPromptTooLong(message)) withheld = true  
if (reactiveCompact?.isWithheldPromptTooLong(message)) withheld = true  
if (isWithheldMaxOutputTokens(message)) withheld = true  
if (!withheld) yield yieldMessage
```

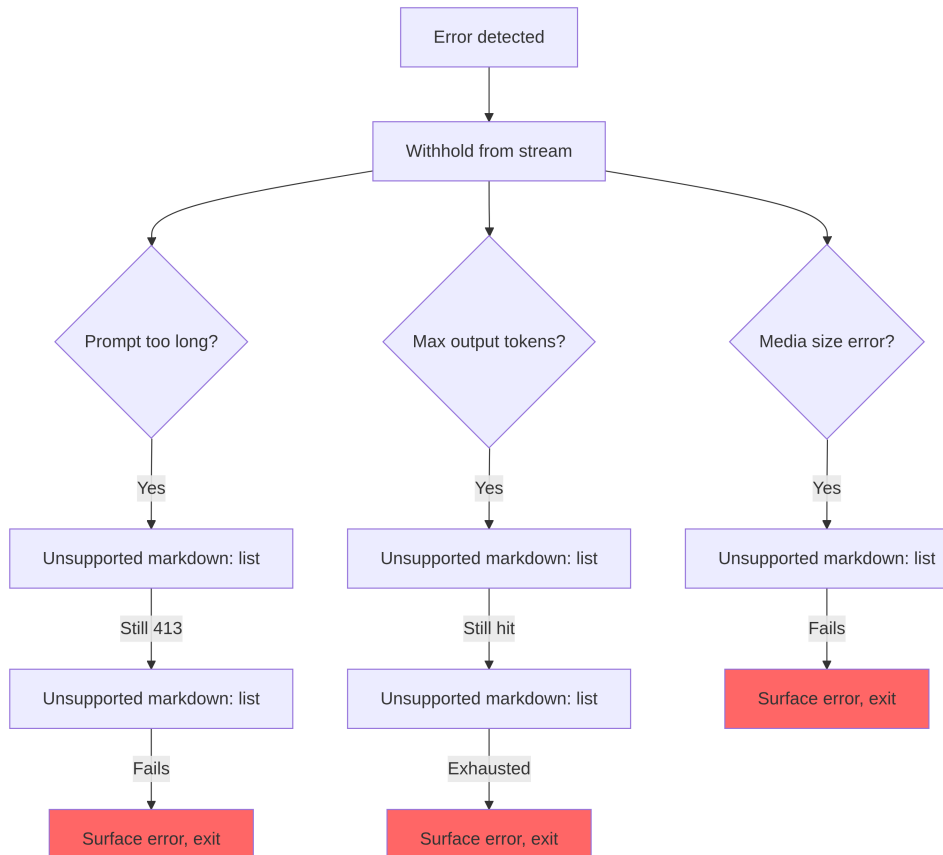
Why withhold? Because SDK consumers – Cowork, the desktop app – terminate the session on any message with an `error` field. If you yield a prompt-too-long error and then successfully recover via reactive compaction, the consumer has already disconnected. The recovery loop keeps running, but nobody is listening. So the error is withheld, pushed to `assistantMessages` so downstream recovery checks can find it. If all recovery paths fail, the withheld message is finally surfaced.

5.8.3 Model Fallback

When a `FallbackTriggeredError` is caught (high demand on the primary model), the loop switches models and retries. But thinking signatures are model-bound – replaying a protected-thinking block from one model to a different fallback model causes a 400 error. The code strips signature blocks before retry. All orphaned assistant messages from the failed attempt are tombstoned so the UI removes them.

5.9 Error Recovery: The Escalation Ladder

Error recovery in `query.ts` is not a single strategy. It is a ladder of increasingly aggressive interventions, each triggered when the previous one fails.



5.9.1 The Death Spiral Guard

The most dangerous failure mode is an infinite loop. The code has multiple guards:

1. **hasAttemptedReactiveCompact**: One-shot flag. Reactive compact fires once per error type.
2. **MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3**: Hard cap on multi-turn recovery attempts.
3. **Circuit breaker on auto-compact**: After 3 consecutive failures, auto-compact stops trying entirely.

4. **No stop hooks on error responses:** The code explicitly returns before reaching stop hooks when the last message is an API error. The comment explains: “error -> hook blocking -> retry -> error -> ... (the hook injects more tokens each cycle).”
5. **Preserved `hasAttemptedReactiveCompact` across stop hook retries:** When a stop hook returns blocking errors and forces a retry, the reactive compact guard is preserved. The comment documents the bug: “Resetting to false here caused an infinite loop burning thousands of API calls.”

Each of these guards was added because someone hit the failure mode in production.

5.10 Worked Example: “Fix the Bug in `auth.ts`”

To make the loop concrete, let us trace a real interaction through three iterations.

The user types: Fix the null pointer bug in `src/auth/validate.ts`

Iteration 1: The model reads the file.

The loop enters. Context management runs (no compression needed – the conversation is short). The model streams a response: “Let me look at the file.” It emits a single `tool_use` block: `Read({ file_path: "src/auth/validate.ts" })`. The streaming executor sees a concurrency-safe tool and starts it immediately. By the time the model finishes its response text, the file contents are already in memory.

Post-stream processing: the model used a tool, so we enter the tool-use path. The `Read` result (file contents with line numbers) is pushed to `toolResults`. A Haiku summary promise is kicked off in the background. State is reconstructed with the new messages, `transition: { reason: 'next_turn' }`, and the loop continues.

Iteration 2: The model edits the file.

Context management runs again (still under the threshold). The model streams: “I see the bug on line 42 – `userId` can be null.” It emits `Edit({ file_path: "src/auth/validate.ts", old_string: "const user =`


```
getUser(userId)", new_string: "if (!userId) return { error:
'unauthorized' }\nconst user = getUser(userId)" }).
```

Edit is not concurrency-safe, so the streaming executor queues it until the response completes. Then the 14-step execution pipeline fires: Zod validation passes, input backfill expands the path, the PreToolUse hook checks permissions (the user approves), and the edit is applied. The pending Haiku summary from iteration 1 resolves during streaming – its result is yielded as a `ToolUseSummaryMessage`. State is reconstructed, loop continues.

Iteration 3: The model declares completion.

The model streams: “I’ve fixed the null pointer bug by adding a guard clause.” No `tool_use` blocks. We enter the “done” path. Prompt-too-long recovery? Not needed. Max output tokens? No. Stop hooks run – no blocking errors. Token budget check passes. The loop returns `{ reason: 'completed' }`.

Total: three API calls, two tool executions, one user permission prompt. The loop handled streaming tool execution, Haiku summarization overlapping with the API call, and the full permission pipeline – all through the same `while(true)` structure.

5.11 Token Budgets

Users can request a token budget for a turn (e.g., +500k). The budget system decides whether to continue or stop after the model completes a response.

`checkTokenBudget` makes a binary continue/stop decision with three rules:

1. **Subagents always stop.** Budget is a top-level concept only.
2. **Completion threshold at 90%.** If `turnTokens < budget * 0.9`, continue.
3. **Diminishing returns detection.** After 3+ continuations, if both the current and previous delta are below 500 tokens, stop early. The model is producing less and less output per continuation.

When the decision is “continue,” a nudge message is injected telling the model how much budget remains.

5.12 Stop Hooks: Forcing the Model to Keep Working

Stop hooks run when the model finishes without requesting any tool use – it thinks it is done. The hooks evaluate whether it actually *is* done.

The pipeline runs template job classification, fires background tasks (prompt suggestion, memory extraction), and then executes stop hooks proper. When a stop hook returns blocking errors – “you said you were done, but the linter found 3 errors” – the errors are appended to the message history and the loop continues with `stopHookActive: true`. This flag prevents re-running the same hooks on the retry.

When a stop hook signals `preventContinuation`, the loop exits immediately with `{ reason: 'stop_hook_prevented' }`.

5.13 State Transitions: The Complete Catalog

Every exit from the loop is one of two types: a **Terminal** (the loop returns) or a **Continue** (the loop iterates).

5.13.1 Terminal States (10 reasons)

Reason	Trigger
<code>blocking_limit</code>	Token count at hard limit, auto-compact OFF
<code>image_error</code>	ImageSizeError, ImageResizeError, or unrecoverable media error
<code>model_error</code>	Unrecoverable API/model exception
<code>aborted_streaming</code>	User abort during model streaming
<code>prompt_too_long</code>	Withheld 413 after all recovery exhausted
<code>completed</code>	Normal completion (no tool use, or budget exhausted, or API error)
<code>stop_hook_prevented</code>	Stop hook explicitly blocked continuation
<code>aborted_tools</code>	User abort during tool execution
<code>hook_stopped</code>	PreToolUse hook stopped continuation

5.14. ORPHANED TOOL RESULTS: THE PROTOCOL SAFETY NET65

Reason	Trigger
<code>max_turns</code>	Hit the <code>maxTurns</code> limit

5.13.2 Continue States (7 reasons)

Reason	Trigger
<code>collapse_drain_retry</code>	Context collapse drained staged collapses on 413
<code>reactive_compact_retry</code>	Reactive compact succeeded after 413 or media error
<code>max_output_tokens_escalate</code>	8K cap hit, escalating to 64K
<code>max_output_tokens_recovery</code>	64K still hit, multi-turn recovery (up to 3 attempts)
<code>stop_hook_blocking</code>	Stop hook returned blocking errors, must retry
<code>token_budget_continuation</code>	Token budget not exhausted, nudge message injected
<code>next_turn</code>	Normal tool-use continuation

5.14 Orphaned Tool Results: The Protocol Safety Net

The API protocol requires that every `tool_use` block is followed by a `tool_result`. The function `yieldMissingToolResultBlocks` creates error `tool_result` messages for every `tool_use` block that the model emitted but that never got a corresponding result. Without this safety net, a crash during streaming would leave orphaned `tool_use` blocks that would cause a protocol error on the next API call.

It fires in three places: the outer error handler (model crash), the fallback handler (model switch mid-stream), and the abort handler (user interruption). Each path has a different error message, but the mechanism is identical.

5.15 Abort Handling: Two Paths

Aborts can happen at two points: during streaming and during tool execution. Each has distinct behavior.

Abort during streaming: The streaming executor (if active) drains remaining results, generating synthetic `tool_results` for queued tools. Without the executor, `yieldMissingToolResultBlocks` fills the gap. The `signal.reason` check distinguishes between a hard abort (Ctrl+C) and a submit-interrupt (user typed a new message) – submit-interrupts skip the interruption message because the queued user message already provides context.

Abort during tool execution: Similar logic, with a `toolUse: true` parameter on the interruption message signaling to the UI that tools were in progress.

5.16 The Thinking Rules

Claude’s thinking/redacted_thinking blocks have three inviolable rules:

1. A message containing a thinking block must be part of a query whose `max_thinking_length > 0`
2. A thinking block may not be the last block in a message
3. Thinking blocks must be preserved for the duration of an assistant trajectory

Violating any of these produces opaque API errors. The code handles them in several places: the fallback handler strips signature blocks (which are model-bound), the compaction pipeline preserves the protected tail, and the microcompact layer never touches thinking blocks.

5.17 Dependency Injection

The `QueryDeps` type is intentionally narrow – four dependencies, not forty:

Four injected dependencies: the model caller, the compactor, the microcompactor, and a UUID generator. Tests pass `deps` into the loop params to inject fakes directly. Using `typeof fn` for the type definitions keeps the signatures

in sync automatically. Alongside the mutable `State` and the injectable `QueryDeps`, an immutable `QueryConfig` is snapshotted once at `query()` entry – feature flags, session state, and environment variables captured once and never re-read. The three-way separation (mutable state, immutable config, injectable deps) makes the loop testable and makes the eventual refactor to a pure `step(state, event, config)` reducer straightforward.

5.18 Apply This: Building Your Own Agent Loop

Use a generator, not callbacks. The backpressure is free. The return value semantics are free. The composability via `yield*` is free. Agent loops are strictly forward-moving – you never need to rewind or fork.

Make state transitions explicit. Reconstruct the full state object at every `continue` site. The verbosity is the feature – it prevents partial-update bugs and makes each transition self-documenting.

Withhold recoverable errors. If your consumers disconnect on errors, do not yield errors until you know recovery has failed. Push them to an internal buffer, attempt recovery, and surface only on exhaustion.

Layer your context management. Light operations first (removal), heavy operations last (summarization). This preserves granular context when possible and falls back to monolithic summaries only when necessary.

Add circuit breakers for every retry. Every recovery mechanism in `query.ts` has an explicit limit: 3 auto-compact failures, 3 max-output recovery attempts, 1 reactive compact attempt. Without these limits, the first production session that triggers a retry-on-failure loop will burn your API budget overnight.

The minimal agent loop skeleton, if you are starting from scratch:

```
async function* agentLoop(params) {
  let state = initState(params)
  while (true) {
    const context = compressIfNeeded(state.messages)
    const response = await callModel(context)
    if (response.error) {
      if (canRecover(response.error, state)) { state = recoverState(state); continue }
      return { reason: 'error' }
    }
  }
}
```

```

    }
    if (!response.toolCalls.length) return { reason: 'completed' }
    const results = await executeTools(response.toolCalls)
    state = { ...state, messages: [...context, response.message, ...results] }
  }
}

```

Every feature in Claude Code’s loop is an elaboration of one of these steps. The four compression layers elaborate step 3 (compress). The withholding pattern elaborates the model call. The escalation ladder elaborates error recovery. Stop hooks elaborate the “no tool use” exit. Start with this skeleton. Add each elaboration only when you hit the problem it solves.

5.19 Summary

The agent loop is 1,730 lines of a single `while(true)` that does everything. It streams model responses, executes tools concurrently, compresses context through four layers, recovers from five categories of errors, tracks token budgets with diminishing returns detection, runs stop hooks that can force the model back to work, manages prefetch pipelines for memory and skills, and produces a typed discriminated union of exactly why it stopped.

It is the most important file in the system because it is the only file that touches every other subsystem. The context pipeline feeds into it. The tool system feeds out of it. The error recovery wraps around it. The hooks intercept it. The state layer persists through it. The UI renders from it.

If you understand `query()`, you understand Claude Code. Everything else is a peripheral.

Chapter 6

Chapter 6: Tools – From Definition to Execution

6.1 The Nervous System

Chapter 5 showed you the agent loop – the `while(true)` that streams model responses, collects tool calls, and feeds results back. The loop is the heartbeat. But the heartbeat is meaningless without the nervous system that translates “the model wants to run `git status`” into an actual shell command, with permission checks, result budgeting, and error handling.

The tool system is that nervous system. It spans 40+ tool implementations, a centralized registry with feature-flag gating, a 14-step execution pipeline, a permission resolver with seven modes, and a streaming executor that starts tools before the model finishes its response.

Every tool call in Claude Code – every file read, every shell command, every `grep`, every sub-agent dispatch – flows through the same pipeline. The uniformity is the point: whether the tool is a built-in Bash executor or a third-party MCP server, it gets the same validation, the same permission checks, the same result budgeting, the same error classification.

The `Tool` interface has approximately 45 members. That sounds overwhelming, but only five matter for understanding how the system works:

1. `call()` – execute the tool
2. `inputSchema` – validate and parse the input
3. `isConcurrencySafe()` – can this run in parallel?

4. `checkPermissions()` – is this allowed?
5. `validateInput()` – does this input make semantic sense?

Everything else – the 12 rendering methods, the analytics hooks, the search hints – exists to support the UI and telemetry layers. Start with the five, and the rest falls into place.

6.2 The Tool Interface

6.2.1 Three Type Parameters

Every tool is parameterized over three types:

```
Tool<Input extends AnyObject, Output, P extends ToolProgressData>
```

`Input` is a Zod object schema that serves double duty: it generates the JSON Schema sent to the API (so the model knows what parameters to provide), and it validates the model’s response at runtime via `safeParse`. `Output` is the TypeScript type of the tool’s result. `P` is the progress event type the tool emits while running – `BashTool` emits stdout chunks, `GrepTool` emits match counts, `AgentTool` emits sub-agent transcripts.

6.2.2 buildTool() and Fail-Closed Defaults

No tool definition directly constructs a `Tool` object. Every tool passes through `buildTool()`, a factory that spreads a defaults object under the tool-specific definition:

```
// Pseudocode - illustrates the fail-closed defaults pattern
const SAFE_DEFAULTS = {
  isEnabled:      () => true,
  isParallelSafe:  () => false,    // Fail-closed: new tools run serially
  isReadOnly:     () => false,    // Fail-closed: treated as writes
  isDestructive:  () => false,
  checkPermissions: (input) => ({ behavior: 'allow', updatedInput: input }),
}

function buildTool(definition) {
  return { ...SAFE_DEFAULTS, ...definition } // Definition overrides defaults
}
```


The defaults are deliberately fail-closed where it matters for safety. A new tool that forgets to implement `isConcurrencySafe` defaults to `false` – it runs serially, never in parallel. A tool that forgets `isReadOnly` defaults to `false` – the system treats it as a write operation. A tool that forgets `toAutoClassifierInput` returns an empty string – the auto-mode security classifier skips it, which means the general permission system handles it instead of an automated bypass.

The one default that is *not* fail-closed is `checkPermissions`, which returns `allow`. This seems backwards until you understand the layered permission model: `checkPermissions` is tool-specific logic that runs *after* the general permission system has already evaluated rules, hooks, and mode-based policies. A tool returning `allow` from `checkPermissions` is saying “I have no tool-specific objection” – it is not granting blanket access. The grouping into sub-objects (`options`, named fields like `readFileState`) provides the structure that focused interfaces would provide, without the ceremony of declaring, implementing, and threading five separate interface types through 40+ call sites.

6.2.3 Concurrency Is Input-Dependent

The signature `isConcurrencySafe(input: z.infer<Input>): boolean` takes the parsed input because the same tool can be safe for some inputs and unsafe for others. `BashTool` is the canonical example: `ls -la` is read-only and concurrency-safe, but `rm -rf /tmp/build` is not. The tool parses the command, classifies each subcommand against known-safe sets, and returns `true` only when every non-neutral part is a search or read operation.

6.2.4 The ToolResult Return Type

Every `call()` returns a `ToolResult<T>`:

```
type ToolResult<T> = {
  data: T
  newMessages?: (UserMessage | AssistantMessage | AttachmentMessage | SystemMessage)[]
  contextModifier?: (context: ToolUseContext) => ToolUseContext
}
```

`data` is the typed output that gets serialized into the API’s `tool_result` content block. `newMessages` lets a tool inject additional messages into the conversation – `AgentTool` uses this to append sub-agent transcripts.

`contextModifier` is a function that mutates the `ToolUseContext` for subsequent tools – this is how `EnterPlanMode` switches the permission mode. Context modifiers are only honored for non-concurrency-safe tools; if your tool runs in parallel, its modifier is queued until the batch completes.

6.3 ToolUseContext: The God Object

`ToolUseContext` is the massive context bag threaded through every tool call. It has approximately 40 fields. It is, by any reasonable definition, a god object. It exists because the alternative is worse.

A tool like `BashTool` needs the abort controller, the file state cache, the app state, the message history, the tool set, MCP connections, and half a dozen UI callbacks. Threading these as individual parameters would produce function signatures with 15+ arguments. The pragmatic solution is a single context object, grouped by concern:

Configuration (`options` sub-object): The tool set, model name, MCP connections, debug flags. Set once at query start, mostly immutable.

Execution state: The `abortController` for cancellation, `readFileState` for the LRU file cache, `messages` for the full conversation history. These change during execution.

UI callbacks: `setToolJSX`, `addNotification`, `requestPrompt`. Only wired in interactive (REPL) contexts. SDK and headless modes leave them undefined.

Agent context: `agentId`, `renderedSystemPrompt` (frozen parent prompt for fork sub-agents – re-rendering could diverge due to feature flag warm-up and bust the cache).

The sub-agent variant of `ToolUseContext` is particularly revealing. When `createSubagentContext()` builds a context for a child agent, it makes deliberate choices about which fields to share and which to isolate: `setAppState` becomes a no-op for async agents, `localDenialTracking` gets a fresh object, `contentReplacementState` is cloned from the parent. Each choice encodes a lesson learned from a production bug.

6.4 The Registry

6.4.1 getAllBaseTools(): The Single Source of Truth

The function `getAllBaseTools()` returns the exhaustive list of every tool that could exist in the current process. Always-present tools come first, then conditionally-included tools gated by feature flags:

```
const SleepTool = feature('PROACTIVE') || feature('KAIROS')
  ? require('./tools/SleepTool/SleepTool.js').SleepTool
  : null
```

The `feature()` import from `bun:bundle` is resolved at bundle time. When `feature('AGENT_TRIGGERS')` is statically false, the bundler eliminates the entire `require()` call – dead code elimination that keeps the binary small.

6.4.2 assembleToolPool(): Merging Built-in and MCP Tools

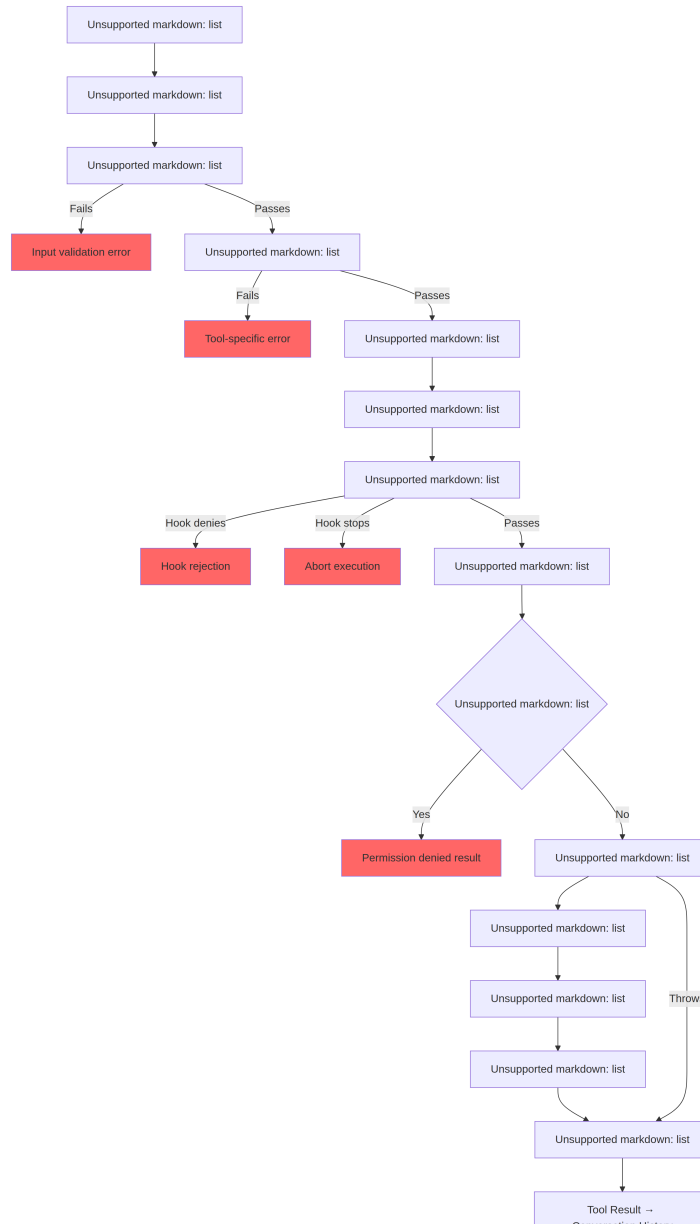
The final tool set that reaches the model comes from `assembleToolPool()`:

1. Get built-in tools (with deny-rule filtering, REPL mode hiding, and `isEnabled()` checks)
2. Filter MCP tools by deny rules
3. Sort each partition alphabetically by name
4. Concatenate built-ins (prefix) + MCP tools (suffix)

The sort-then-concatenate approach is not aesthetic preference. The API server places a prompt-cache breakpoint after the last built-in tool. A flat sort across all tools would interleave MCP tools into the built-in list, and adding or removing an MCP tool would shift built-in tool positions, invalidating the cache.

6.5 The 14-Step Execution Pipeline

The function `checkPermissionsAndCallTool()` is where intent becomes action. Every tool call passes through these 14 steps.



6.5.1 Steps 1-4: Validation

Tool Lookup falls back to `getAllBaseTools()` for alias matches, handling transcripts from older sessions where a tool was renamed. **Abort Check** prevents wasted computation on tool calls queued before Ctrl+C propagated.

Zod Validation catches type mismatches; for deferred tools, the error appends a hint to call ToolSearch first. **Semantic Validation** goes beyond schema conformance – FileEditTool rejects no-op edits, BashTool blocks standalone `sleep` when MonitorTool is available.

6.5.2 Steps 5-6: Preparation

Speculative Classifier Start kicks off the auto-mode security classifier in parallel for Bash commands, shaving hundreds of milliseconds off the common path. **Input Backfill** clones the parsed input and adds derived fields (expanding `~/foo.txt` to absolute paths) for hooks and permissions, preserving the original for transcript stability.

6.5.3 Steps 7-9: Permission

PreToolUse Hooks are the extension mechanism – they can make permission decisions, modify inputs, inject context, or stop execution entirely. **Permission Resolution** bridges hooks and the general permission system: if a hook already decided, that is final; otherwise `canUseTool()` triggers rule matching, tool-specific checks, mode-based defaults, and interactive prompts. **Permission Denied Handling** builds an error message and executes `PermissionDenied` hooks.

6.5.4 Steps 10-14: Execution and Cleanup

Tool Execution runs the actual `call()` with the original input. **Result Budgeting** persists oversized output to `~/.claude/tool-results/{hash}.txt` and replaces it with a preview. **PostToolUse Hooks** can modify MCP output or block continuation. **New Messages** are appended (sub-agent transcripts, system reminders). **Error Handling** classifies errors for telemetry, extracts safe strings from potentially mangled names, and emits OTel events.

6.6 The Permission System

6.6.1 Seven Modes

Mode	Behavior
<code>default</code>	Tool-specific checks; prompt user for unrecognized operations
<code>acceptEdits</code>	Auto-allow file edits; prompt for other operations
<code>plan</code>	Read-only – deny all write operations
<code>dontAsk</code>	Auto-deny anything that would normally prompt (background agents)
<code>bypassPermissions</code>	Allow everything without prompting
<code>auto</code>	Use the transcript classifier to decide (feature-flagged)
<code>bubble</code>	Internal mode for sub-agents that escalate to the parent

6.6.2 The Resolution Chain

When a tool call reaches permission resolution:

1. **Hook decision:** If a `PreToolUse` hook already returned `allow` or `deny`, that is final.
2. **Rule matching:** Three rule sets – `alwaysAllowRules`, `alwaysDenyRules`, `alwaysAskRules` – match on tool name and optional content patterns. `Bash(git *)` matches any Bash command starting with `git`.
3. **Tool-specific check:** The tool’s `checkPermissions()` method. Most return `passthrough`.
4. **Mode-based default:** `bypassPermissions` allows everything. `plan` denies writes. `dontAsk` denies prompts.
5. **Interactive prompt:** In `default` and `acceptEdits` modes, unresolved decisions show a prompt.
6. **Auto-mode classifier:** A two-stage classifier (fast model, then extended thinking for ambiguous cases).

The `safetyCheck` variant has a `classifierApprovable` boolean: `.claude/` and `.git/` edits are `classifierApprovable: true` (unusual but sometimes legitimate), while Windows path bypass attempts are `classifierApprovable: false` (almost always adversarial).

6.6.3 Permission Rules and Matching

Permission rules are stored as `PermissionRule` objects with three parts: a `source` tracing provenance (userSettings, projectSettings, localSettings, cliArg, policySettings, session, etc.), a `ruleBehavior` (allow, deny, ask), and a `ruleValue` with the tool name and optional content pattern.

The `ruleContent` field enables fine-grained matching. `Bash(git *)` allows any Bash command starting with `git`. `Edit(/src/**)` allows edits only within `/src`. `Fetch(domain:example.com)` allows fetching from a specific domain. Rules without `ruleContent` match all invocations of that tool.

BashTool's permission matcher parses the command via `parseForSecurity()` (a bash AST parser) and splits compound commands into subcommands. If AST parsing fails (complex syntax with heredocs or nested subshells), the matcher returns `() => true` – fail-safe, meaning the hook always runs. The assumption is that if the command is too complex to parse, it is too complex to confidently exclude from safety checks.

6.6.4 Bubble Mode for Sub-Agents

Sub-agents in coordinator-worker patterns cannot show permission prompts – they have no terminal. The `bubble` mode causes permission requests to propagate up to the parent context. The coordinator agent, running in the main thread with terminal access, handles the prompt and sends the decision back down.

6.7 Tool Deferred Loading

Tools with `shouldDefer: true` are sent to the API with `defer_loading: true` – names and descriptions but not full parameter schemas. This reduces initial prompt size. To use a deferred tool, the model must first call `ToolSearchTool` to load its schema. The failure mode is instructive: calling a deferred tool without loading it causes Zod validation to fail (all typed parameters arrive as strings), and the system appends a targeted recovery hint.

Deferred loading also improves cache hit rates: tools sent with `defer_loading: true` contribute only their name to the prompt, so

adding or removing a deferred MCP tool changes the prompt by a few tokens rather than hundreds.

6.8 Result Budgeting

6.8.1 Per-Tool Size Limits

Each tool declares `maxResultSizeChars`:

Tool	<code>maxResultSizeChars</code>	Rationale
<code>BashTool</code>	30,000	Enough for most useful output
<code>FileEditTool</code>	100,000	Diffs can be large but the model needs them
<code>GrepTool</code>	100,000	Search results with context lines add up fast
<code>FileReadTool</code>	Infinity	Self-bounds via its own token limits; persisting would create circular Read loops

When a result exceeds the threshold, the full content is saved to disk and replaced with a `<persisted-output>` wrapper containing a preview and file path. The model can then use `Read` to access the full output if needed.

6.8.2 Per-Conversation Aggregate Budget

Beyond per-tool limits, `ContentReplacementState` tracks an aggregate budget across the entire conversation, preventing death by a thousand cuts – many tools each returning 90% of their individual limit can still overwhelm the context window.

6.9 Individual Tool Highlights

6.9.1 BashTool: The Most Complex Tool

BashTool is the system’s most complex tool by far. It parses compound commands, classifies subcommands as read-only or write, manages background tasks, detects image output by magic bytes, and implements a sed simulation for safe edit previews.

The compound command parsing is particularly interesting. `splitCommandWithOperators()` breaks a command like `cd /tmp && mkdir build && ls build` into individual subcommands. Each is classified against known-safe command sets (`BASH_SEARCH_COMMANDS`, `BASH_READ_COMMANDS`, `BASH_LIST_COMMANDS`). A compound command is read-only only if ALL non-neutral parts are safe. The neutral set (`echo`, `printf`) is ignored – they do not make a command read-only, but they also do not make it write-only.

The sed simulation (`_simulatedSedEdit`) deserves special attention. When a user approves a sed command in the permission dialog, the system pre-computes the result by running the sed command in a sandbox and capturing the output. The pre-computed result is injected into the input as `_simulatedSedEdit`. When `call()` executes, it applies the edit directly, bypassing shell execution. This guarantees that what the user previewed is exactly what gets written – not a re-execution that might produce different results if the file changed between preview and execution.

6.9.2 FileEditTool: Staleness Detection

FileEditTool integrates with `readFileState`, the LRU cache of file contents and timestamps maintained across the conversation. Before applying an edit, it checks whether the file has been modified since the model last read it. If the file is stale – modified by a background process, another tool, or the user – the edit is rejected with a message telling the model to re-read the file first.

The fuzzy matching in `findActualString()` handles the common case where the model gets whitespace slightly wrong. It normalizes whitespace and quote styles before matching, so an edit targeting `old_string` with trailing spaces still matches the file’s actual content. The `replace_all` flag enables bulk replacements; without it, non-unique matches are rejected, requiring the model to provide enough context to identify a single location.

6.9.3 FileReadTool: The Versatile Reader

FileReadTool is the only built-in tool with `maxResultSizeChars: Infinity`. If Read output were persisted to disk, the model would need to Read the persisted file, which could itself exceed the limit, creating an infinite loop. The tool instead self-bounds via token estimation and truncates at the source.

The tool is remarkably versatile: it reads text files with line numbers, images (returning base64 multimodal content blocks), PDFs (via `extractPDFPages()`), Jupyter notebooks (via `readNotebook()`), and directories (falling back to `ls`). It blocks dangerous device paths (`/dev/zero`, `/dev/random`, `/dev/stdin`) and handles macOS screenshot filename quirks (U+202F narrow no-break space vs regular space in “Screen Shot” filenames).

6.9.4 GrepTool: Pagination via `head_limit`

GrepTool wraps `ripGrep()` and adds a pagination mechanism via `head_limit`. The default is 250 entries – enough for useful results but small enough to avoid context bloat. When truncation occurs, the response includes `appliedLimit: 250`, signaling the model to use `offset` on the next call to paginate. An explicit `head_limit: 0` disables the limit entirely.

GrepTool automatically excludes six VCS directories (`.git`, `.svn`, `.hg`, `.bzz`, `.jj`, `.sl`). Searching inside `.git/objects` is almost never what the model wants, and accidental inclusion of binary pack files would blow through token budgets.

6.9.5 AgentTool and Context Modifiers

AgentTool spawns sub-agents that run their own query loops. Its `call()` returns `newMessages` containing the sub-agent’s transcript, and optionally a `contextModifier` that propagates state changes back to the parent. Because AgentTool is not concurrency-safe by default, multiple Agent tool calls in a single response run serially – each sub-agent’s context modifier is applied before the next sub-agent starts. In coordinator mode, the pattern inverts: the coordinator dispatches sub-agents for independent tasks, and the `isAgentSwarmsEnabled()` check unlocks parallel agent execution.

6.10 How Tools Interact with the Message History

Tool results do not simply return data to the model. They participate in the conversation as structured messages.

The API expects tool results as `ToolResultBlockParam` objects that reference the original `tool_use` block by ID. Most tools serialize to text. `FileReadTool` can serialize to image content blocks (base64-encoded) for multimodal responses. `BashTool` detects image output by inspecting magic bytes in stdout and switches to image blocks accordingly.

`ToolResult.newMessages` is how tools extend the conversation beyond the simple call-and-response pattern. **Agent transcripts:** `AgentTool` injects the sub-agent’s message history as attachment messages. **System reminders:** Memory tools inject system messages that appear after the tool result – visible to the model on the next turn but stripped at the `normalizeMessagesForAPI` boundary. **Attachment messages:** Hook results, additional context, and error details carry structured metadata that the model can reference in subsequent turns.

The `contextModifier` function is the mechanism for tools that change the execution environment. When `EnterPlanMode` executes, it returns a modifier that sets the permission mode to `'plan'`. When `ExitWorktree` executes, it modifies the working directory. These modifiers are the only way for a tool to affect subsequent tools – direct mutation of `ToolUseContext` is not possible because the context is spread-copied before each tool call. The serial-only restriction is enforced by the orchestration layer: if two concurrent tools both modify the working directory, which wins?

6.11 Apply This: Designing a Tool System

Fail-closed defaults. New tools should be conservative until explicitly marked otherwise. A developer who forgets to set a flag gets the safe behavior, not the dangerous one.

Input-dependent safety. `isConcurrencySafe(input)` and `isReadOnly(input)` take the parsed input because the same tool at different inputs has different safety profiles. A tool registry that marks `BashTool` as “always serial” is correct but wasteful.

Layer your permissions. Tool-specific checks, rule-based matching, mode-based defaults, interactive prompts, and automated classifiers each handle different cases. No single mechanism is sufficient.

Budget results, not just inputs. Token limits on input are standard. But tool results can be arbitrarily large and they accumulate across turns. Per-tool limits prevent individual explosions. Aggregate conversation limits prevent cumulative overflow.

Make error classification telemetry-safe. In minified builds, `error.constructor.name` is mangled. The `classifyToolError()` function extracts the most informative safe string available – telemetry-safe messages, errno codes, stable error names – without ever logging the raw error message to analytics.

6.12 What Comes Next

This chapter traced how a single tool call flows from definition through validation, permission, execution, and result budgeting. But the model rarely requests just one tool at a time. How tools are orchestrated into concurrent batches is the subject of Chapter 7.

Chapter 7

Chapter 7: Concurrent Tool Execution

7.1 The Cost of Waiting

Chapter 6 traced the lifecycle of a single tool call – from the raw `tool_use` block in the API response through input validation, permission checks, execution, and result formatting. That pipeline handles one tool. But the model rarely requests just one.

A typical Claude Code interaction involves three to five tool calls per turn. “Read these two files, grep for this pattern, then edit this function.” The model emits all of those in a single response. If each tool takes 200 milliseconds, running them sequentially costs a full second. If the Read and Grep calls are independent – and they are – running them in parallel cuts that to 200 milliseconds. Five-to-one improvement, free.

But not all tools are independent. An Edit that modifies `config.ts` cannot run concurrently with another Edit that modifies `config.ts`. A Bash command that creates a directory must complete before a Bash command that writes a file into that directory. Concurrency is not a global property of a tool. It is a property of a specific tool invocation with specific inputs.

This is the insight that drives the entire concurrency system: **safety is per-call, not per-tool-type**. `Bash("ls -la")` is safe to parallelize. `Bash("rm -rf build/")` is not. The same tool, different inputs, different concurrency classification. The system must inspect the input before deciding.

Claude Code implements two layers of concurrency optimization. The first is **batch orchestration**: after the model’s response is fully received, partition the tool calls into concurrent and serial groups, then execute each group appropriately. The second is **speculative execution**: start running tools *while the model is still streaming its response*, harvesting results before the response is even complete. Together, these two mechanisms eliminate most of the wall-clock time that would otherwise be spent waiting.

7.2 The Partition Algorithm

The entry point is `partitionToolCalls()` in `toolOrchestration.ts`. It takes an ordered array of `ToolUseBlock` messages and produces an array of batches, where each batch is either “all concurrent-safe” or “a single serial tool.”

```
// Pseudocode - illustrates the partition algorithm
type Group = { parallel: boolean; calls: ToolCall[] }

function groupBySafety(calls: ToolCall[], registry: ToolRegistry): Group[] {
  return calls.reduce((groups, call) => {
    const def = registry.lookup(call.name)
    const input = def?.schema.safeParse(call.input)
    // Fail-closed: parse failure or exception → serial
    const safe = input?.success
      ? tryCatch(() => def.isParallelSafe(input.data), false)
      : false
    // Merge consecutive safe calls into one group
    if (safe && groups.at(-1)?.parallel) {
      groups.at(-1)!.calls.push(call)
    } else {
      groups.push({ parallel: safe, calls: [call] })
    }
    return groups
  }, [] as Group[])
}
```

The algorithm walks the array left to right. For each tool call:

1. **Look up the tool definition** by name.

2. **Parse the input** with the tool's Zod schema via `safeParse()`. If parsing fails, the tool is conservatively classified as not concurrency-safe.
3. **Call `isConcurrencySafe(parsedInput)`** on the tool definition. This is where per-input classification happens. The Bash tool parses the command string, checks if every subcommand is read-only (`ls`, `grep`, `cat`, `git status`), and returns `true` only if the entire compound command is a pure read. The Read tool always returns `true`. The Edit tool always returns `false`. The call is wrapped in try-catch – if `isConcurrencySafe` throws (say, the Bash command string can't be parsed by the shell-quote library), the tool defaults to serial.
4. **Merge or create a batch.** If the current tool is concurrency-safe AND the most recent batch is also concurrency-safe, append to that batch. Otherwise, start a new batch.

The result is a sequence of batches that alternates between concurrent groups and individual serial entries. Walk through a concrete example:

Model requests: [Read, Read, Grep, Edit, Read]

```

Step 1: Read  → concurrent-safe → new batch {safe, [Read]}
Step 2: Read  → concurrent-safe → append  {safe, [Read, Read]}
Step 3: Grep  → concurrent-safe → append  {safe, [Read, Read, Grep]}
Step 4: Edit  → NOT safe        → new batch {serial, [Edit]}
Step 5: Read  → concurrent-safe → new batch {safe, [Read]}

```

Result: 3 batches

```

Batch 1: [Read, Read, Grep] - run concurrently
Batch 2: [Edit]             - run alone
Batch 3: [Read]             - run concurrently (just one tool)

```

The partitioning is greedy and order-preserving. Consecutive safe tools accumulate into a single batch. Any unsafe tool breaks the run and starts a new batch. This means the order in which the model emits tool calls matters – if it interleaves a Write between two Reads, you get three batches instead of two. In practice, models tend to cluster their reads together, which is the common case the algorithm is optimized for.

7.3 Batch Execution

The `runTools()` generator iterates through the partitioned batches and dispatches each one to the appropriate executor.

7.3.1 Concurrent Batches

For a concurrent batch, `runToolsConcurrently()` fires all tools in parallel using an `all()` utility that caps active generators at the concurrency limit:

```
// Pseudocode - illustrates the concurrent dispatch pattern
async function* dispatchParallel(calls, context) {
  yield* boundedAll(
    calls.map(async function* (call) {
      context.markInProgress(call.id)
      yield* executeSingle(call, context)
      context.markComplete(call.id)
    }),
    MAX_CONCURRENCY, // Default: 10
  )
}
```

The concurrency limit defaults to 10, configurable via `CLAUDE_CODE_MAX_TOOL_USE_CONCURRENCY`. Ten is generous – you rarely see more than five or six tool calls in a single model response. The limit exists as a safety valve for pathological cases, not as a typical constraint.

The `all()` utility is a generator-aware variant of `Promise.all` with bounded concurrency. It starts up to N generators simultaneously, yields results from whichever completes first, and starts the next queued generator as each one finishes. The mechanics are similar to a semaphore-guarded task pool, but adapted for async generators that yield intermediate results.

Context modifier queuing is the subtle part. Some tools produce *context modifiers* – functions that transform the `ToolUseContext` for subsequent tools. When tools run concurrently, you cannot apply these modifiers immediately because other tools in the same batch are reading the same context. Instead, modifiers are collected in a map keyed by tool use ID:

```
const queuedContextModifiers: Record<
  string,
  ((context: ToolUseContext) => ToolUseContext) []
```



```
> = {}
```

After the entire concurrent batch finishes, the modifiers are applied in tool-order (not completion-order), preserving deterministic context evolution:

```
for (const block of blocks) {  
  const modifiers = queuedContextModifiers[block.id]  
  if (!modifiers) continue  
  for (const modifier of modifiers) {  
    currentContext = modifier(currentContext)  
  }  
}
```

In practice, none of the current concurrency-safe tools produce context modifiers – the comment in the codebase acknowledges this explicitly. But the infrastructure exists because tools can be added by MCP servers, and a custom read-only MCP tool might legitimately want to modify context (updating a “files seen” set, for instance).

7.3.2 Serial Batches

Serial execution is straightforward. Each tool runs, its context modifiers are applied immediately, and the next tool sees the updated context:

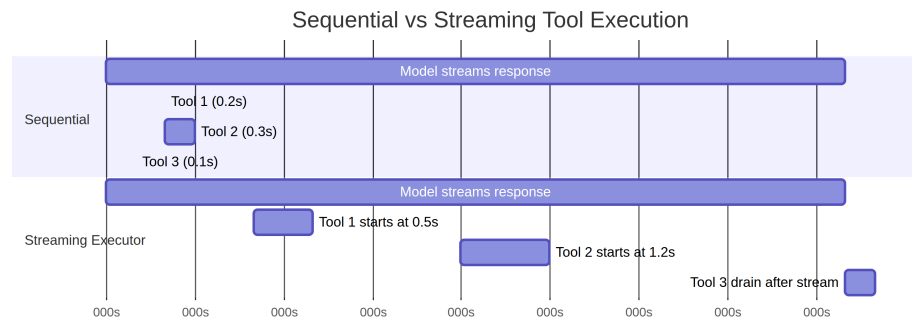
```
for (const toolUse of toolUseMessages) {  
  for await (const update of runToolUse(toolUse, /* ... */)) {  
    if (update.contextModifier) {  
      currentContext = update.contextModifier.modifyContext(currentContext)  
    }  
    yield { message: update.message, newContext: currentContext }  
  }  
}
```

This is the critical difference. Serial tools can change the world for subsequent tools. An Edit modifies a file; the next Read sees the modified version. A Bash command creates a directory; the next Bash command writes into it. Context modifiers are the formalization of this dependency: they let a tool say “the execution environment has changed, here’s how.”

7.4 The Streaming Tool Executor

Batch orchestration eliminates unnecessary serialization *after* the model’s response arrives. But there is a bigger opportunity: the model’s response takes time to stream. A typical multi-tool response might take 2-3 seconds to fully arrive. The first tool call is parseable after 500 milliseconds. Why wait for the remaining 2 seconds?

The `StreamingToolExecutor` class implements speculative execution. As the model streams its response, each `tool_use` block is handed to the executor the moment it is fully parsed. The executor starts running it immediately – while the model is still generating the next tool call. By the time the response finishes streaming, several tools may have already completed.

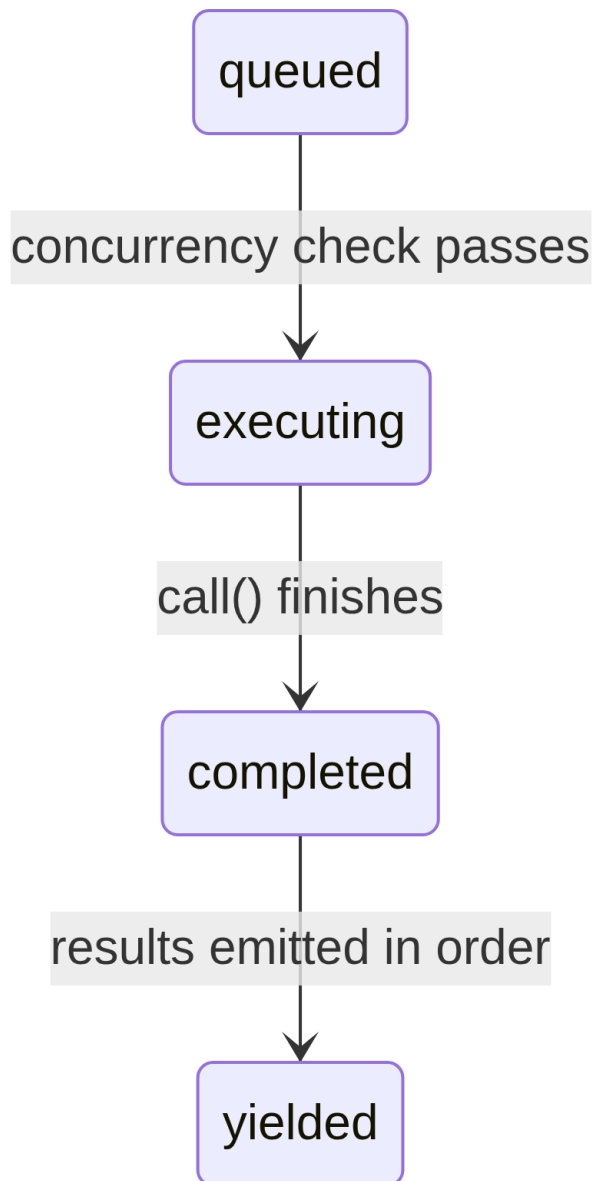


Sequential total: 3.1s. Streaming total: 2.6s – tools 1 and 2 completed during streaming, saving 16% of wall-clock time.

The savings compound. When the model requests five read-only tools and the response takes 3 seconds to stream, all five tools can start and finish during that 3 seconds. The post-stream drain phase has nothing left to do. The user sees results almost immediately after the last character of the model’s response appears.

7.4.1 Tool Lifecycle

Each tool tracked by the executor progresses through four states:



- **queued**: The `tool_use` block has been parsed and registered. Waiting for concurrency conditions to allow execution.
- **executing**: The tool's `call()` function is running. Results accumulate in a buffer.
- **completed**: Execution finished. Results are ready to be yielded to

the conversation.

- **yielded**: Results have been emitted. Terminal state.

7.4.2 addTool(): Queuing During the Stream

```
addTool(block: ToolUseBlock, assistantMessage: AssistantMessage): void
```

Called by the streaming response parser each time a complete `tool_use` block arrives. The method:

1. Looks up the tool definition. If not found, immediately creates a **completed** entry with an error message – no point in queuing a tool that does not exist.
2. Parses the input and determines `isConcurrencySafe` using the same logic as `partitionToolCalls()`.
3. Pushes a `TrackedTool` with status 'queued'.
4. Calls `processQueue()` – which may start the tool immediately.

The call to `processQueue()` is fire-and-forget (`void this.processQueue()`). The executor does not await it. This is intentional: `addTool()` is called from the streaming parser's event handler, and blocking there would stall response parsing. The tool starts executing in the background while the parser continues consuming the stream.

7.4.3 processQueue(): The Admission Check

The admission check is a single predicate:

```
// Pseudocode - illustrates the mutual exclusion rule
canRun = noToolsRunning || (newToolIsSafe && allRunningAreSafe)
```

A tool can start executing if and only if: - **No tools are currently executing** (the queue is empty), OR - **Both the new tool and all currently executing tools are concurrency-safe**.

This is a mutual exclusion contract. A non-concurrent tool requires exclusive access – nothing else can be running. Concurrent tools can share the runway with other concurrent tools, but a single non-concurrent tool in the executing set blocks everyone.

The `processQueue()` method iterates through all tools in order. For each queued tool, it checks `canExecuteTool()`. If the tool can run, it starts. If a

non-concurrent tool cannot run yet, the loop *breaks* – it stops checking subsequent tools entirely, because non-concurrent tools must maintain ordering. If a concurrent tool cannot run (blocked by an executing non-concurrent tool), the loop *continues* – but in practice this rarely helps, because concurrent tools after a non-concurrent blocker are typically dependent on its results anyway.

7.4.4 `executeTool()`: The Core Execution Loop

This method is where the real complexity lives. It manages abort controllers, error cascades, progress reporting, and context modifiers.

Child abort controllers. Each tool gets its own `AbortController` that is a child of a shared sibling-level controller.

The hierarchy is three levels deep: the query-level controller (owned by the REPL, fires on user Ctrl+C) parents the sibling controller (owned by the streaming executor, fires on Bash errors) which parents each tool’s individual controller. Aborting the sibling controller kills all running tools. Aborting a tool’s individual controller kills only that tool – but it also bubbles up to the query controller if the abort reason is not a sibling error. This bubble-up prevents the system from silently discarding the executor when, for example, a permission denial should end the entire turn.

This bubble-up is essential for permission denial. When a user rejects a tool in the permission dialog, the tool’s abort controller fires. That signal must reach the query loop so it can end the turn. Without it, the query loop would continue as if nothing happened, sending a stale rejection message to the model.

The sibling error cascade. When a tool produces an error result, the executor checks whether to cancel sibling tools. The rule: **only Bash errors cascade**. When a shell command errors, the executor records the failure, captures a description of the errored tool, and aborts the sibling controller – which cancels all other running tools in the batch.

The rationale is pragmatic. Bash commands often form implicit dependency chains: `mkdir build && cp src/* build/ && tar -czf dist.tar.gz build/`. If `mkdir` fails, running `cp` and `tar` is pointless. Canceling siblings immediately saves time and avoids confusing error messages.

Read and Grep errors, by contrast, are independent. If one file read fails because the file was deleted, that has no bearing on a concurrent grep

searching a different directory. Canceling the grep would waste work for no reason.

The error cascade produces synthetic error messages for sibling tools:

```
Cancelled: parallel tool call Bash(mkdir build) errored
```

The description includes the first 40 characters of the errored tool’s command or file path, giving the model enough context to understand what went wrong.

Progress messages are handled separately from results. While results are buffered and yielded in order, progress messages (status updates like “Reading file...” or “Searching...”) go to a **pendingProgress** array and are yielded immediately via `getCompletedResults()`. A resolve callback wakes up the `getRemainingResults()` loop when new progress arrives, preventing the UI from appearing frozen during long-running tools.

Queue re-processing. After each tool completes, `processQueue()` is called again:

```
void promise.finally(() => {
    void this.processQueue()
})
```

This is how serial tools that were blocked by a concurrent batch get started. When the last concurrent tool finishes, the subsequent non-concurrent tool’s `canExecuteTool()` check passes, and it begins executing.

7.4.5 Result Harvesting

The streaming executor exposes two harvesting methods, designed for two different phases of the response lifecycle.

getCompletedResults() – mid-stream harvesting. This is a synchronous generator called between chunks of the streaming API response. It walks the tools array in order and yields results for any tools that have completed:

`getCompletedResults()` is a synchronous generator that walks the tools array in submission order. For each tool, it first drains any pending progress messages. If the tool is completed, it yields the results and marks it as yielded. The critical rule: if a non-concurrent tool is still executing, the walk **breaks** – nothing after it can be yielded, even if subsequent tools have already completed. Results after a serial tool might depend on its context

modifications, so they must wait. For concurrent tools, this restriction does not apply; the loop skips executing concurrent tools and continues checking subsequent entries.

This break is the order-preservation mechanism. If a non-concurrent tool is still executing, nothing after it can be yielded – even if subsequent tools have already completed. Results after a serial tool might depend on its context modifications, so they must wait. For concurrent tools, this restriction does not apply; the loop skips executing concurrent tools and continues checking subsequent entries.

getRemainingResults() – post-stream drain. Called after the model’s response is fully received. This async generator loops until every tool is yielded:

getRemainingResults() is the post-stream drain. It loops until every tool is yielded. On each iteration, it processes the queue (starting any newly-unblocked tools), yields any completed results via **getCompletedResults()**, and then – if tools are still executing but nothing new has completed – uses **Promise.race** to idle-wait on whichever finishes first: any executing tool’s promise, or a progress-available signal. This avoids busy-polling while still waking up the moment something happens. When no tools have completed and nothing new can start, the executor waits for any executing tool to finish (or for progress to arrive). This avoids busy-polling while still waking up the moment something happens.

7.4.6 Order Preservation

Results are yielded in the order tools were *received*, not the order they *completed*. This is a deliberate design choice.

Consider a model response that requests `[Read("a.ts"), Read("b.ts"), Read("c.ts")]`. All three start concurrently. `c.ts` finishes first (it is smaller), then `a.ts`, then `b.ts`. If results were yielded in completion order, the conversation would show:

```
Tool result: c.ts contents
Tool result: a.ts contents
Tool result: b.ts contents
```

But the model emitted them in a-b-c order. The conversation history must match the model’s expectation, or the next turn will be confused about

which result corresponds to which request. By yielding in arrival order, the conversation stays coherent:

```
Tool result: a.ts contents (completed second, yielded first)
Tool result: b.ts contents (completed third, yielded second)
Tool result: c.ts contents (completed first, yielded third)
```

The cost is minor: if tool 1 is slow and tools 2-5 are fast, the fast results sit in buffers until tool 1 finishes. But the alternative – conversation incoherence – is far worse.

7.4.7 `discard()`: The Streaming Fallback Escape Hatch

When the API response stream fails mid-way (network error, server disconnect), the system retries with a new API call. But the streaming executor may have already started tools from the failed attempt. Those results are now orphaned – they correspond to a response that was never fully received.

```
discard(): void {
    this.discarded = true
}
```

Setting `discarded = true` causes:

- `getCompletedResults()` returns immediately with no results.
- `getRemainingResults()` returns immediately with no results.
- Any tool that starts executing checks `getAbortReason()`, sees `streaming_fallback`, and gets a synthetic error instead of actually running.

The discarded executor is abandoned. A fresh executor is created for the retry attempt.

7.5 Tool Concurrency Properties

Each built-in tool declares its concurrency characteristics through the `isConcurrencySafe()` method. The classification is not arbitrary – it reflects the tool’s actual effect on shared state.

Tool	Concurrency Safe	Condition	Rationale
Read	Always	–	Pure read. No side effects.

Tool	Concurrency Safe	Condition	Rationale
Grep	Always	–	Pure read.
Glob	Always	–	Wraps ripgrep. Pure read. File listing.
Fetch	Always	–	HTTP GET. No local side effects.
WebSearch	Always	–	API call to search provider.
Bash	Sometimes	Read-only commands only	<code>isReadOnly()</code> parses the command and classifies subcommands. <code>ls</code> , <code>git status</code> , <code>cat</code> , <code>grep</code> are safe. <code>rm</code> , <code>mkdir</code> , <code>mv</code> are not.
Edit	Never	–	Modifies files. Two concurrent edits to the same file corrupt it.
Write	Never	–	Creates or overwrites files. Same corruption risk.
Notebook	Never	–	Modifies <code>.ipynb</code> files.

The Bash tool’s classification deserves elaboration. It uses `splitCommandWithOperators()` to decompose compound commands (`&&`, `||`, `;`, `|`), then classifies each subcommand against known-safe sets:

- **Search commands:** `grep`, `rg`, `find`, `fd`, `ag`, `ack`
- **Read commands:** `cat`, `head`, `tail`, `wc`, `jq`, `less`, `file`, `stat`
- **List commands:** `ls`, `tree`, `du`, `df`
- **Neutral commands:** `echo`, `printf` (no side effects but not “reads”)

A compound command is read-only only if every non-neutral subcommand is in the search, read, or list set. `ls -la && cat README.md` is safe. `ls -la && rm -rf build/` is not – the `rm` contaminates the entire command.

7.6 The Interrupt Behavior Contract

While tools are executing, the user can type a new message. What should happen? The answer depends on the tool.

Each tool declares an `interruptBehavior()` method that returns either `'cancel'` or `'block'`:

- **'cancel'**: Stop the tool immediately, discard partial results, and process the new user message. Used by tools where partial execution is harmless (reads, searches).
- **'block'**: Keep the tool running to completion. The user's new message waits. Used by tools where interruption would leave the system in an inconsistent state (writes mid-flight, long-running bash commands). This is the default.

The streaming executor tracks the interruptible state of the current tool set:

The interruptible state is updated by checking all currently executing tools: the set is interruptible only when every executing tool supports cancellation. If even one tool's interrupt behavior is `'block'`, the entire set is treated as non-interruptible.

The UI only shows an “interruptible” indicator when ALL executing tools support cancellation. If even one tool is `'block'`, the entire set is treated as non-interruptible. This is conservative but correct: you cannot meaningfully interrupt a batch where one tool would keep running anyway.

When the user does interrupt and all tools are cancellable, the abort controller fires with reason `'interrupt'`. The executor's `getAbortReason()` method checks each tool's interrupt behavior individually – a `'cancel'` tool gets a synthetic `user_interrupted` error, while a `'block'` tool (which would not be present in a fully interruptible set, but the code handles the edge case) continues running.

7.7 Context Modifiers: The Serial-Only Contract

Context modifiers are functions of type `(context: ToolUseContext) => ToolUseContext`. They let a tool say “I’ve changed something about the execution environment that subsequent tools need to know about.”

The contract is simple: **context modifiers are only applied for serial (non-concurrent-safe) tools**. This is stated explicitly in the source:

```
// NOTE: we currently don't support context modifiers for concurrent
//       tools. None are actively being used, but if we want to use
//       them in concurrent tools, we need to support that here.
if (!tool.isConcurrencySafe && contextModifiers.length > 0) {
  for (const modifier of contextModifiers) {
    this.toolUseContext = modifier(this.toolUseContext)
  }
}
```

In the batch orchestration path (`toolOrchestration.ts`), concurrent batch modifiers are collected and applied after the batch completes, in tool-submission order. This means concurrent tools within a batch cannot see each other’s context changes, but the batch after them can.

The asymmetry is intentional. If Tool A modifies context and Tool B reads that context, they have a data dependency. Data dependencies mean they cannot run concurrently. By definition, if two tools are concurrency-safe, neither should depend on the other’s context modifications. The system enforces this by deferring application.

7.8 Apply This

The concurrency patterns in Claude Code generalize to any system that orchestrates multiple independent operations. Three principles are worth extracting.

Partition by safety, not by type. The `isConcurrencySafe(input)` method receives the parsed input, not just the tool name. This per-invocation classification is more precise than a static “this tool type is always safe” declaration. In your own systems, inspect the operation’s arguments before deciding whether to parallelize. A database read is safe to parallelize; a

database write to the same row is not. The operation type alone does not tell you enough.

Speculative execution during I/O waits. The streaming executor starts tools while the API response is still arriving. The same pattern applies anywhere you have a slow producer and fast consumers: start processing early items while later items are still being generated. HTTP/2 server push, compiler pipeline parallelism, and speculative CPU execution all share this structure. The key requirement is that you can identify independent work before the full instruction set is available.

Preserve submission order in results. Yielding results in completion order is tempting – it minimizes latency to first result. But if the consumer (in this case, the language model) expects results in a specific order, reordering them creates confusion that costs more time to resolve than the latency savings. Buffer completed results and release them in the order they were requested. The implementation cost is a simple array walk; the correctness benefit is absolute.

The streaming executor pattern is particularly powerful for agent systems. Any time your agent loop involves a “think, then act” cycle where the thinking phase produces multiple independent actions, you can overlap the tail of thinking with the beginning of acting. The savings are proportional to the ratio of think-time to act-time. For language model agents, where think-time (API response generation) dominates, the savings are substantial.

7.9 Summary

Claude Code’s concurrency system operates at two levels. The partition algorithm (`partitionToolCalls`) groups consecutive concurrency-safe tools into batches that run in parallel, while isolating unsafe tools into serial batches where each tool sees the effects of the one before it. The streaming tool executor (`StreamingToolExecutor`) goes further, starting tools speculatively as they arrive during model response streaming, overlapping tool execution with response generation.

The safety model is conservative by design. Concurrency safety is determined per-invocation by inspecting parsed inputs. Unknown tools default to serial. Parsing failures default to serial. Exceptions in safety checks default to serial.

The system never guesses that something is safe to parallelize – the tool must affirmatively declare it.

Error handling follows the dependency structure of the tools. Bash errors cascade to siblings because shell commands often form implicit pipelines. Read and search errors are isolated because they are independent operations. The abort controller hierarchy – query controller, sibling controller, per-tool controller – gives each level the ability to cancel its scope without disrupting the level above.

The result is a system that extracts maximum parallelism from the model's tool requests while maintaining the invariant that the conversation history reflects a coherent, ordered sequence of actions. The model sees results in the order it requested them. The user sees tools complete as fast as the underlying operations allow. The gap between those two – execution speed vs. presentation order – is bridged by buffering, and that buffer is the simplest part of the entire system.

Part III

Part III: Multi-Agent Orchestration

Chapter 8

Chapter 8: Spawning Sub-Agents

8.1 The Multiplication of Intelligence

A single agent is powerful. It can read files, edit code, run tests, search the web, and reason about the results. But there is a hard ceiling on what one agent can do in a single conversation: the context window fills up, the task branches in directions that demand different capabilities, and the serial nature of tool execution becomes a bottleneck. The solution is not a bigger model. It is more agents.

Claude Code’s sub-agent system lets the model request help. When the parent agent encounters a task that would benefit from delegation – a codebase search that should not pollute the main conversation, a verification pass that demands adversarial thinking, a set of independent edits that could run in parallel – it calls the **Agent** tool. That call spawns a child: a fully independent agent with its own conversation loop, its own tool set, its own permission boundary, and its own abort controller. The child does its work and returns a result. The parent never sees the child’s internal reasoning, only the final output.

This is not a convenience feature. It is the architectural foundation for everything from parallel file exploration to coordinator-worker hierarchies to multi-agent swarm teams. And it all flows through two files: **AgentTool.tsx**, which defines the model-facing interface, and **runAgent.ts**, which implements the lifecycle.

The design challenge is significant. A sub-agent needs enough context to do its job but not so much that it wastes tokens on irrelevant information. It needs permission boundaries that are strict enough for safety but flexible enough for utility. It needs lifecycle management that cleans up every resource it touches without requiring the caller to remember what to clean up. And all of this must work for a spectrum of agent types – from a cheap, fast, read-only Haiku searcher to an expensive, thorough, Opus-powered verification agent running adversarial tests in the background.

This chapter traces the path from the model’s “I need help” to a fully operational child agent. We will examine the tool definition that the model sees, the fifteen-step lifecycle that creates the execution environment, the six built-in agent types and what each optimizes for, the frontmatter system that lets users define custom agents, and the design principles that emerge from all of it.

A note on terminology: throughout this chapter, “parent” refers to the agent that calls the **Agent** tool, and “child” refers to the agent that is spawned. The parent is usually (but not always) the top-level REPL agent. In coordinator mode, the coordinator spawns workers, which are children. In nested scenarios, a child can itself spawn grandchildren – the same lifecycle applies recursively.

The orchestration layer spans approximately 40 files across `tools/AgentTool/`, `tasks/`, `coordinator/`, `tools/SendMessageTool/`, and `utils/swarm/`. This chapter focuses on the spawning mechanics – the `AgentTool` definition and the `runAgent` lifecycle. The next chapter covers the runtime: progress tracking, result retrieval, and multi-agent coordination patterns.

8.2 The AgentTool Definition

The `AgentTool` is registered under the name “**Agent**” with a legacy alias “**Task**” for backward compatibility with older transcripts, permission rules, and hook configurations. It is built with the standard `buildTool()` factory, but its schema is more dynamic than any other tool in the system.

8.2.1 The Input Schema

The input schema is constructed lazily via `lazySchema()` – a pattern we saw in Chapter 6 that defers zod compilation until first use. There are two

layers: a base schema and a full schema that adds multi-agent and isolation parameters.

The base fields are always present:

Field	Type	Required	Purpose
<code>description</code>	<code>string</code>	Yes	Short 3-5 word summary of the task
<code>prompt</code>	<code>string</code>	Yes	The full task description for the agent
<code>subagent_type</code>	<code>string</code>	No	Which specialized agent to use
<code>model</code>	<code>enum('sonnet', 'noopus', 'haiku')</code>	No	Model override for this agent
<code>run_in_background</code>	<code>boolean</code>	No	Launch asynchronously

The full schema adds multi-agent parameters (when swarm features are active) and isolation controls:

Field	Type	Purpose
<code>name</code>	<code>string</code>	Makes the agent addressable via <code>SendMessage({to: name})</code>
<code>team_name</code>	<code>string</code>	Team context for spawning
<code>mode</code>	<code>PermissionMode</code>	Permission mode for spawned teammate
<code>isolation</code>	<code>enum('worktree', 'filesystem')</code>	Filesystem isolation strategy
<code>cwd</code>	<code>string</code>	Absolute path override for working directory

The multi-agent fields enable the swarm pattern covered in Chapter 9: named agents that can send messages to each other via `SendMessage({to: name})` while running concurrently. The isolation fields enable filesystem safety: `worktree` isolation creates a temporary git worktree so the agent operates on

a copy of the repository, preventing conflicting edits when multiple agents work on the same codebase simultaneously.

What makes this schema unusual is that it is **dynamically shaped by feature flags**:

```
// Pseudocode - illustrates the feature-gated schema pattern
inputSchema = lazySchema(() => {
  let schema = baseSchema()
  if (!featureEnabled('ASSISTANT_MODE')) schema = schema.omit({ cwd: true })
  if (backgroundDisabled || forkMode) schema = schema.omit({ run_in_background: true })
  return schema
})
```

When the fork experiment is active, `run_in_background` disappears from the schema entirely because all spawns are forced async under that path. When background tasks are disabled (via `CLAUDE_CODE_DISABLE_BACKGROUND_TASKS`), the field is also stripped. When the KAIROS feature flag is off, `cwd` is omitted. The model never sees fields it cannot use.

This is a subtle but important design choice. The schema is not just validation – it is the model’s instruction manual. Every field in the schema is described in the tool definition that the model reads. Removing fields the model should not use is more effective than adding “do not use this field” to the prompt. The model cannot misuse what it cannot see.

8.2.2 The Output Schema

The output is a discriminated union with two public variants:

- { `status: 'completed', prompt, ...AgentToolResult` } – synchronous completion with the agent’s final output
- { `status: 'async_launched', agentId, description, prompt, outputFile` } – background launch acknowledgment

Two additional internal variants (`TeammateSpawnedOutput` and `RemoteLaunchedOutput`) exist but are excluded from the exported schema to enable dead code elimination in external builds. The bundler strips these variants and their associated code paths when the corresponding feature flags are disabled, keeping the distributed binary smaller.

The `async_launched` variant is notable for what it includes: the `outputFile` path where the agent’s results will be written when it completes. This lets

the parent (or any other consumer) poll or watch the file for results, providing a filesystem-based communication channel that survives process restarts.

8.2.3 The Dynamic Prompt

The `AgentTool` prompt is generated by `getPrompt()` and is context-sensitive. It adapts based on available agents (listed inline or as an attachment to avoid busting prompt cache), whether fork is active (adds “When to fork” guidance), whether the session is in coordinator mode (slim prompt since the coordinator system prompt already covers usage), and subscription tier. Non-pro users get a note about launching multiple agents concurrently.

The attachment-based agent list is worth highlighting. The codebase comments reference “approximately 10.2% of fleet cache_creation tokens” being caused by dynamic tool descriptions. Moving the agent list from the tool description to an attachment message keeps the tool description static, so connecting an MCP server or loading a plugin does not bust the prompt cache for every subsequent API call.

This is a pattern worth internalizing for any system that uses tool definitions with dynamic content. The Anthropic API caches the prompt prefix – system prompt, tool definitions, and conversation history – and reuses the cached computation for subsequent requests that share the same prefix. If the tool definition changes between API calls (because an agent was added or an MCP server connected), the entire cache is invalidated. Moving volatile content from the tool definition (which is part of the cached prefix) to an attachment message (which is appended after the cached portion) preserves the cache while still delivering the information to the model.

With the tool definition understood, we can now trace what happens when the model actually calls it.

8.2.4 Feature Gating

The sub-agent system has the most complex feature gating in the codebase. At least twelve feature flags and GrowthBook experiments control which agents are available, which parameters appear in the schema, and which code paths are taken:

Feature Gate	Controls
<code>FORK_SUBAGENT</code>	Fork agent path

Feature Gate	Controls
BUILTIN_EXPLORE_PLAN_AGENTS	Explore and Plan agents
VERIFICATION_AGENT	Verification agent
KAIROS	cwd override, assistant force-async
TRANSCRIPT_CLASSIFIER	Handoff classification, auto mode override
PROACTIVE	Proactive module integration

Each gate uses `feature()` from Bun's dead code elimination system (compile-time) or `getFeatureValue_CACHED_MAY_BE_STALE()` from GrowthBook (runtime A/B testing). The compile-time gates are string-replaced during the build – when `FORK_SUBAGENT` is 'ant', the entire fork code path is included; when it is 'external', it may be excluded entirely. The GrowthBook gates allow live experimentation: the `tengu_amber_stoat` experiment can A/B test whether removing Explore and Plan agents changes user behavior, without shipping a new binary.

8.2.5 The `call()` Decision Tree

Before `runAgent()` is ever invoked, the `call()` method in `AgentTool.tsx` routes the request through a decision tree that determines *what kind* of agent to spawn and *how* to spawn it:

1. Is this a teammate spawn? (`team_name + name` both set)
 - YES -> `spawnTeammate()` -> return `teammate_spawned`
 - NO -> continue
2. Resolve effective agent type
 - `subagent_type` provided -> use it
 - `subagent_type` omitted, fork enabled -> undefined (fork path)
 - `subagent_type` omitted, fork disabled -> "general-purpose" (default)
3. Is this the fork path? (`effectiveType === undefined`)
 - YES -> Recursive fork guard check -> Use `FORK_AGENT` definition
4. Resolve agent definition from `activeAgents` list
 - Filter by permission deny rules
 - Filter by `allowedAgentTypes`

- Throw if not found or denied
- 5. Check required MCP servers (wait up to 30s for pending)
- 6. Resolve isolation mode (param overrides agent def)
 - "remote" -> teleportToRemote() -> return remote_launched
 - "worktree" -> createAgentWorktree()
 - null -> normal execution
- 7. Determine sync vs async
 - shouldRunAsync = run_in_background || selectedAgent.background ||
 - isCoordinator || forceAsync || isProactiveActive
- 8. Assemble worker tool pool
- 9. Build system prompt and prompt messages
- 10. Execute (async -> registerAsyncAgent + void lifecycle; sync -> iterate runAgent)

Steps 1 through 6 are pure routing – no agent has been created yet. The actual lifecycle begins at `runAgent()`, which the sync path iterates directly and the async path wraps in `runAsyncAgentLifecycle()`.

The routing is done in `call()` rather than `runAgent()` for a reason: `runAgent()` is a pure lifecycle function that does not know about teammates, remote agents, or the fork experiment. It receives a resolved agent definition and executes it. The decision of *which* definition to resolve, *how* to isolate the agent, and *whether* to run synchronously or asynchronously belongs to the layer above. This separation keeps `runAgent()` testable and reusable – it is called from both the normal AgentTool path and from the async lifecycle wrapper when resuming a backgrounded agent.

The fork guard in step 3 deserves attention. Fork children keep the **Agent** tool in their pool (for cache-identical tool definitions with the parent), but recursive forking would be pathological. Two guards prevent it: `querySource === 'agent:builtin:fork'` (set on the child's context options, survives autocompact) and `isInForkChild(messages)` (scans conversation history for the `<fork-boilerplate>` tag as a fallback). Belt and suspenders – the primary guard is fast and reliable; the fallback catches edge cases where `querySource` was not threaded.

8.3 The runAgent Lifecycle

`runAgent()` in `runAgent.ts` is an async generator that drives a sub-agent's entire lifecycle. It yields `Message` objects as the agent works. Every sub-agent – fork, built-in, custom, coordinator worker – flows through this single function. The function is approximately 400 lines, and every line exists for a reason.

The function signature reveals the complexity of the problem:

```
export async function* runAgent({
  agentDefinition,           // What kind of agent
  promptMessages,           // What to tell it
  toolUseContext,           // Parent's execution context
  canUseTool,               // Permission callback
  isAsync,                  // Background or blocking?
  canShowPermissionPrompts,
  forkContextMessages,      // Parent's history (fork only)
  querySource,              // Origin tracking
  override,                 // System prompt, abort controller, agent ID overrides
  model,                    // Model override from caller
  maxTurns,                 // Turn limit
  availableTools,           // Pre-assembled tool pool
  allowedTools,             // Permission scoping
  onCacheSafeParams,        // Callback for background summarization
  useExactTools,            // Fork path: use parent's exact tools
  worktreePath,             // Isolation directory
  description,              // Human-readable task description
  // ...
}): { ... }: AsyncGenerator<Message, void>
```

Seventeen parameters. Each one represents a dimension of variation that the lifecycle must handle. This is not over-engineering – it is the natural consequence of a single function serving fork agents, built-in agents, custom agents, sync agents, async agents, worktree-isolated agents, and coordinator workers. The alternative would be seven different lifecycle functions with duplicated logic, which is worse.

The `override` object is particularly important – it is the escape hatch for fork agents and resumed agents that need to inject pre-computed values (system prompt, abort controller, agent ID) into the lifecycle without re-deriving

them.

Here are the fifteen steps.

8.3.1 Step 1: Model Resolution

```
const resolvedAgentModel = getAgentModel(
  agentDefinition.model,           // Agent's declared preference
  toolUseContext.options.mainLoopModel, // Parent's model
  model,                           // Caller's override (from input)
  permissionMode,                  // Current permission mode
)
```

The resolution chain is: **caller override > agent definition > parent model > default**. The `getAgentModel()` function handles special values like `'inherit'` (use whatever the parent uses) and GrowthBook-gated overrides for specific agent types. The Explore agent, for example, defaults to Haiku for external users – the cheapest and fastest model, appropriate for a read-only search specialist that runs 34 million times per week.

Why this order matters: the caller (the parent model) can override the agent definition's preference by passing a `model` parameter in the tool call. This lets the parent promote a normally-cheap agent to a more capable model for a particularly complex search, or demote an expensive agent when the task is simple. But the agent definition's model is the default, not the parent's – a Haiku Explore agent should not accidentally inherit the parent's Opus model just because no one specified otherwise.

Understanding the model resolution chain is important because it establishes a design principle that recurs throughout the lifecycle: **explicit overrides beat declarations, declarations beat inheritance, inheritance beats defaults**. This same principle governs permission modes, abort controllers, and system prompts. The consistency makes the system predictable – once you understand one resolution chain, you understand them all.

8.3.2 Step 2: Agent ID Creation

```
const agentId = override?.agentId ? override.agentId : createAgentId()
```

Agent IDs follow the pattern `agent-<hex>` where the hex part is derived from `crypto.randomUUID()`. The branded type `AgentId` prevents accidental

string confusion at the type level. The override path exists for resumed agents that need to keep their original ID for transcript continuity.

8.3.3 Step 3: Context Preparation

Fork agents and fresh agents diverge here:

```
const contextMessages: Message[] = forkContextMessages
  ? filterIncompleteToolCalls(forkContextMessages)
  : []
const initialMessages: Message[] = [...contextMessages, ...promptMessages]

const agentReadFileState = forkContextMessages !== undefined
  ? cloneFileStateCache(toolUseContext.readFileState)
  : createFileStateCacheWithSizeLimit(READ_FILE_STATE_CACHE_SIZE)
```

For fork agents, the parent’s entire conversation history is cloned into `contextMessages`. But there is a critical filter: `filterIncompleteToolCalls()` strips any `tool_use` blocks that lack matching `tool_result` blocks. Without this filter, the API would reject the malformed conversation. This happens when the parent is mid-tool-execution at the moment of forking – the `tool_use` has been emitted but the result has not arrived yet.

The file state cache follows the same fork-or-fresh pattern. Fork children get a clone of the parent’s cache (they already “know” which files have been read). Fresh agents start empty. The clone is a shallow copy – file content strings are shared via reference, not duplicated. This matters for memory: a fork child with a 50-file cache does not duplicate 50 file contents, it duplicates 50 pointers. The LRU eviction behavior is independent – each cache evicts based on its own access pattern.

8.3.4 Step 4: CLAUDE.md Stripping

Read-only agents like Explore and Plan have `omitClaudeMd: true` in their definitions:

```
const shouldOmitClaudeMd =
  agentDefinition.omitClaudeMd &&
  !override?.userContext &&
  getFeatureValue_CACHED_MAY_BE_STALE('tengu_slim_subagent_claudemd', true)
const { claudeMd: _omittedClaudeMd, ...userContextNoClaudeMd } = baseUserContext
const resolvedUserContext = shouldOmitClaudeMd
```

```
? userContextNoClaudeMd
: baseUserContext
```

CLAUDE.md files contain project-specific instructions about commit messages, PR conventions, lint rules, and coding standards. A read-only search agent does not need any of this – it cannot commit, cannot create PRs, cannot edit files. The parent agent has full context and will interpret the search results. Dropping CLAUDE.md here saves billions of tokens per week across the fleet – an aggregate cost reduction that justifies the added complexity of conditional context injection.

Similarly, Explore and Plan agents have `gitStatus` stripped from system context. The git status snapshot taken at session start can be up to 40KB and is explicitly labeled as stale. If these agents need git information, they can run `git status` themselves and get fresh data.

These are not premature optimizations. At 34 million Explore spawns per week, every unnecessary token compounds into measurable cost. The kill-switch (`tengu_slim_subagent_claudemd`) defaults to true but can be flipped via GrowthBook if the stripping causes regressions.

8.3.5 Step 5: Permission Isolation

This is the most intricate step. Each agent gets a custom `getAppState()` wrapper that overlays its permission configuration onto the parent's state:

```
const agentGetAppState = () => {
  const state = toolUseContext.getAppState()
  let toolPermissionContext = state.toolPermissionContext

  // Override mode unless parent is in bypassPermissions, acceptEdits, or auto
  if (agentPermissionMode && canOverride) {
    toolPermissionContext = {
      ...toolPermissionContext,
      mode: agentPermissionMode,
    }
  }

  // Auto-deny prompts for agents that can't show UI
  const shouldAvoidPrompts =
    canShowPermissionPrompts !== undefined
```

```

    ? !canShowPermissionPrompts
    : agentPermissionMode === 'bubble'
    ? false
    : isAsync
  if (shouldAvoidPrompts) {
    toolPermissionContext = {
      ...toolPermissionContext,
      shouldAvoidPermissionPrompts: true,
    }
  }

  // Scope tool allow rules
  if (allowedTools !== undefined) {
    toolPermissionContext = {
      ...toolPermissionContext,
      alwaysAllowRules: {
        cliArg: state.toolPermissionContext.alwaysAllowRules.cliArg,
        session: [...allowedTools],
      },
    }
  }

  return { ...state, toolPermissionContext, effortValue }
}

```

There are four distinct concerns layered together:

Permission mode cascade. If the parent is in `bypassPermissions`, `acceptEdits`, or `auto` mode, the parent’s mode always wins – the agent definition cannot weaken it. Otherwise, the agent definition’s `permissionMode` is applied. This prevents a custom agent from downgrading security when the user has explicitly set a permissive mode for the session.

Prompt avoidance. Background agents cannot show permission dialogs – there is no terminal attached. So `shouldAvoidPermissionPrompts` is set to `true`, which causes the permission system to auto-deny rather than block. The exception is `bubble` mode: these agents surface prompts to the parent’s terminal, so they can always show prompts regardless of sync/async status.

Automated check ordering. Background agents that *can* show prompts (bubble mode) set `awaitAutomatedChecksBeforeDialog`. This means the

classifier and permission hooks run first; the user is only interrupted if automated resolution fails. For background work, waiting an extra second for the classifier is fine – the user should not be interrupted unnecessarily.

Tool permission scoping. When `allowedTools` is provided, it replaces the session-level allow rules entirely. This prevents parent approvals from leaking through to scoped agents. But SDK-level permissions (from `--allowedTools` CLI flag) are preserved – those represent the embedding application’s explicit security policy and should apply everywhere.

8.3.6 Step 6: Tool Resolution

```
const resolvedTools = useExactTools
  ? availableTools
  : resolveAgentTools(agentDefinition, availableTools, isAsync).resolvedTools
```

Fork agents use `useExactTools: true`, which passes the parent’s tool array through unchanged. This is not just convenience – it is a cache optimization. Different tool definitions serialize differently (different permission modes produce different tool metadata), and any divergence in the tool block busts the prompt cache. Fork children need byte-identical prefixes.

For normal agents, `resolveAgentTools()` applies a layered filter: - `tools: ['*']` means all tools; `tools: ['Read', 'Bash']` means only those - `disallowedTools: ['Agent', 'FileEdit']` removes those from the pool - Built-in agents and custom agents have different base disallowed tool sets - Async agents get filtered through `ASYNC_AGENT_ALLOWED_TOOLS`

The result is that each agent type sees exactly the tools it should have. The Explore agent cannot call `FileEdit`. The Verification agent cannot call `Agent` (no recursive spawning from a verifier). Custom agents have a more restrictive default deny list than built-ins.

8.3.7 Step 7: System Prompt

```
const agentSystemPrompt = override?.systemPrompt
  ? override.systemPrompt
  : asSystemPrompt(
    await getAgentSystemPrompt(
      agentDefinition, toolUseContext,
      resolvedAgentModel, additionalWorkingDirectories, resolvedTools
```

```
)
)
```

Fork agents receive the parent's pre-rendered system prompt via `override.systemPrompt`. This is threaded from `toolUseContext.renderedSystemPrompt` – the exact bytes the parent used in its last API call. Recomputing the system prompt via `getSystemPrompt()` could diverge. GrowthBook features might have transitioned from cold to warm between the parent's call and the child's. A single byte difference in the system prompt busts the entire prompt cache prefix.

For normal agents, `getAgentSystemPrompt()` calls the agent definition's `getSystemPrompt()` function, then enhances with environment details – absolute paths, emoji guidance (Claude tends to over-use emojis in certain contexts), and model-specific instructions.

8.3.8 Step 8: Abort Controller Isolation

```
const agentAbortController = override?.abortController
  ? override.abortController
  : isAsync
    ? new AbortController()
    : toolUseContext.abortController
```

Three lines, three behaviors:

- **Override:** Used when resuming a backgrounded agent or for special lifecycle management. Takes precedence.
- **Async agents get a new, unlinked controller.** When the user presses Escape, the parent's abort controller fires. Async agents should survive this – they are background work that the user chose to delegate. Their independent controller means they keep running.
- **Sync agents share the parent's controller.** Escape kills both. The child is blocking the parent; if the user wants to stop, they want to stop everything.

This is one of those decisions that seems obvious in retrospect but would be catastrophic if wrong. An async agent that aborts when the parent aborts would lose all its work every time the user pressed Escape to ask a follow-up question. A sync agent that ignored the parent's abort would leave the user staring at a frozen terminal.

8.3.9 Step 9: Hook Registration

```
if (agentDefinition.hooks && hooksAllowedForThisAgent) {
  registerFrontmatterHooks(
    rootSetAppState, agentId, agentDefinition.hooks,
    `agent '${agentDefinition.agentType}'`, true
  )
}
```

Agent definitions can declare their own hooks (PreToolUse, PostToolUse, etc.) in frontmatter. These hooks are scoped to the agent’s lifecycle via the `agentId` – they only fire for this agent’s tool calls, and they are automatically cleaned up in the `finally` block when the agent terminates.

The `isAgent: true` flag (the final `true` parameter) converts `Stop` hooks to `SubagentStop` hooks. Sub-agents trigger `SubagentStop`, not `Stop`, so the conversion ensures the hooks fire at the right event.

Security matters here. When `strictPluginOnlyCustomization` is active for hooks, only plugin, built-in, and policy-settings agent hooks are registered. User-controlled agents (from `.claude/agents/`) have their hooks silently skipped. This prevents a malicious or misconfigured agent definition from injecting hooks that bypass security controls.

8.3.10 Step 10: Skill Preloading

```
const skillsToPreload = agentDefinition.skills ?? []
if (skillsToPreload.length > 0) {
  const allSkills = await getSkillToolCommands(getProjectRoot())
  // resolve names, load content, prepend to initialMessages
}
```

Agent definitions can specify `skills: ["my-skill"]` in their frontmatter. The resolution tries three strategies: exact match, prefix with the agent’s plugin name (e.g., "my-skill" becomes "plugin:my-skill"), and suffix match on `:"skillName"` for plugin-namespaced skills. The three-strategy resolution ensures that skill references work regardless of whether the agent author used the fully-qualified name, the short name, or the plugin-relative name.

Loaded skills become user messages prepended to the agent’s conversation. This means the agent “reads” its skill instructions before seeing the task

prompt – the same mechanism used for slash commands in the main REPL, repurposed for automated skill injection. The skill content is loaded concurrently via `Promise.all()` to minimize startup latency when multiple skills are specified.

8.3.11 Step 11: MCP Initialization

```
const { clients: mergedMcpClients, tools: agentMcpTools, cleanup: mcpCleanup } =
  await initializeAgentMcpServers(agentDefinition, toolUseContext.options.mcpCli
```

Agents can define their own MCP servers in frontmatter, additive to the parent's clients. Two forms are supported:

- **Reference by name:** "slack" looks up an existing MCP config and gets a shared, memoized client
- **Inline definition:** { "my-server": { command: "...", args: [...] } } creates a new client that is cleaned up when the agent finishes

Only newly created (inline) clients are cleaned up. Shared clients are memoized at the parent level and persist beyond the agent's lifetime. This distinction prevents an agent from accidentally tearing down an MCP connection that other agents or the parent are still using.

The MCP initialization happens *after* hook registration and skill preloading but *before* context creation. This ordering matters: the MCP tools must be merged into the tool pool before `createSubagentContext()` snapshots the tools into the agent's options. Reordering these steps would mean the agent either has no MCP tools or has them but they are not in its tool pool.

8.3.12 Step 12: Context Creation

```
const agentToolUseContext = createSubagentContext(toolUseContext, {
  options: agentOptions,
  agentId,
  agentType: agentDefinition.agentType,
  messages: initialMessages,
  readFileState: agentReadFileState,
  abortController: agentAbortController,
  getAppState: agentGetAppState,
  shareSetAppState: !isAsync,
```



```

shareSetResponseLength: true,
criticalSystemReminder_EXPERIMENTAL:
  agentDefinition.criticalSystemReminder_EXPERIMENTAL,
contentReplacementState,
})

```

`createSubagentContext()` in `utils/forkedAgent.ts` assembles the new `ToolUseContext`. The key isolation decisions:

- **Sync agents share `setAppState`** with the parent. State changes (like permission approvals) are immediately visible to both. The user sees one coherent state.
- **Async agents get isolated `setAppState`**. The parent's copy is a no-op for the child's writes. But `setAppStateForTasks` reaches the root store – the child can still update task state (progress, completion) that the UI observes.
- **Both share `setResponseLength`** for response metrics tracking.
- **Fork agents inherit `thinkingConfig`** for cache-identical API requests. Normal agents get `{ type: 'disabled' }` – thinking (extended reasoning tokens) is disabled to control output costs. The parent pays for thinking; the children execute.

The `createSubagentContext()` function is worth examining for what it *isolates* versus what it *shares*. The isolation boundary is not all-or-nothing – it is a carefully chosen set of shared and isolated channels:

Concern	Sync Agent	Async Agent
<code>setAppState</code>	Shared (parent sees changes)	Isolated (parent's copy is no-op)
<code>setAppStateForTasks</code>	Shared	Shared (task state must reach root)
<code>setResponseLength</code>	Shared	Shared (metrics need global view)
<code>readFileState</code>	Own cache	Own cache
<code>abortController</code>	Parent's	Independent
<code>thinkingConfig</code>	Fork: inherited / Normal: disabled	Fork: inherited / Normal: disabled
<code>messages</code>	Own array	Own array

The asymmetry between `setAppState` (isolated for async) and

`setAppStateForTasks` (always shared) is a key design decision. An async agent cannot push state changes to the parent’s reactive store – that would cause the parent’s UI to jump unexpectedly. But the agent must still be able to update the global task registry, because that is how the parent knows the background agent has completed. The split channel solves both requirements.

8.3.13 Step 13: Cache-Safe Params Callback

```
if (onCacheSafeParams) {
  onCacheSafeParams({
    systemPrompt: agentSystemPrompt,
    userContext: resolvedUserContext,
    systemContext: resolvedSystemContext,
    toolUseContext: agentToolUseContext,
    forkContextMessages: initialMessages,
  })
}
```

This callback is consumed by background summarization. When an async agent is running, the summarization service can fork the agent’s conversation – using these exact params to construct a cache-identical prefix – and generate periodic progress summaries without disturbing the main conversation. The params are “cache-safe” because they produce the same API request prefix the agent is using, maximizing cache hits.

8.3.14 Step 14: The Query Loop

```
try {
  for await (const message of query({
    messages: initialMessages,
    systemPrompt: agentSystemPrompt,
    userContext: resolvedUserContext,
    systemContext: resolvedSystemContext,
    canUseTool,
    toolUseContext: agentToolUseContext,
    querySource,
    maxTurns: maxTurns ?? agentDefinition.maxTurns,
  })) {
    // Forward API request starts for metrics
  }
}
```

```

    // Yield attachment messages
    // Record to sidechain transcript
    // Yield recordable messages to caller
  }
}

```

The same `query()` function from Chapter 3 drives the sub-agent’s conversation. The sub-agent’s messages are yielded back to the caller – either `AgentTool.call()` for sync agents (which iterates the generator inline) or `runAsyncAgentLifecycle()` for async agents (which consumes the generator in a detached async context).

Each yielded message is recorded to a sidechain transcript via `recordSidechainTranscript()` – an append-only JSONL file per agent. This enables resume: if the session is interrupted, the agent can be reconstructed from its transcript. The recording is $O(1)$ per message, appending only the new message with a reference to the previous UUID for chain continuity.

8.3.15 Step 15: Cleanup

The `finally` block runs on normal completion, abort, or error. It is the most comprehensive cleanup sequence in the codebase:

```

finally {
  await mcpCleanup() // Tear down agent-specific MCP servers
  clearSessionHooks(rootSetAppState, agentId) // Remove agent-scoped hooks
  cleanupAgentTracking(agentId) // Prompt cache tracking state
  agentToolUseContext.readFileState.clear() // Release file state cache memory
  initialMessages.length = 0 // Release fork context (GC hint)
  unregisterPerfettoAgent(agentId) // Perfetto trace hierarchy
  clearAgentTranscriptSubdir(agentId) // Transcript subdir mapping
  rootSetAppState(prev => { // Remove agent's todo entries
    const { [agentId]: _removed, ...todos } = prev.todos
    return { ...prev, todos }
  })
  killShellTasksForAgent(agentId, ...) // Kill orphaned bash processes
}

```

Every subsystem the agent touched during its lifetime gets cleaned up. MCP connections, hooks, cache tracking, file state, perfetto tracing, todo entries,

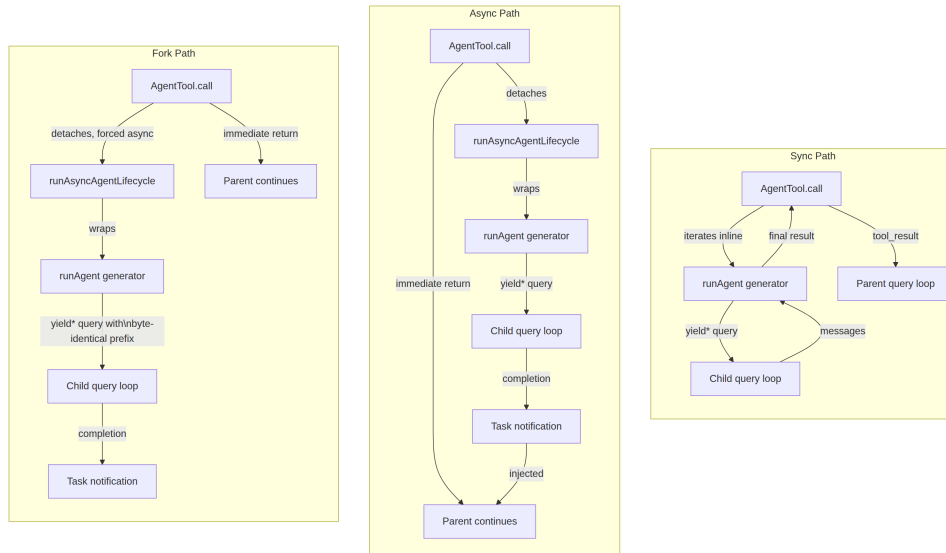
and orphaned shell processes. The comment about “whale sessions” spawning hundreds of agents is telling – without this cleanup, each agent would leave small leaks that accumulate into measurable memory pressure over long sessions.

The `initialMessages.length = 0` line is a manual GC hint. For fork agents, `initialMessages` contains the parent’s entire conversation history. Setting the length to zero releases those references so the garbage collector can reclaim the memory. In a session with a 200K-token context that spawns five fork children, that is a megabyte of duplicated message objects per child.

There is a lesson here about resource management in long-running agent systems. Each of the cleanup steps addresses a different kind of leak: MCP connections (file descriptors), hooks (memory in the app state store), file state caches (in-memory file content), Perfetto registrations (tracing metadata), todo entries (reactive state keys), and shell processes (OS-level processes). An agent interacts with many subsystems during its lifetime, and each subsystem must be notified when the agent is done. The `finally` block is the single place where all these notifications happen, and the generator protocol guarantees it runs. This is why the generator-based architecture is not just a convenience – it is a correctness requirement.

8.3.16 The Generator Chain

Before examining the built-in agent types, it is worth stepping back to see the structural pattern that makes all of this work. The entire sub-agent system is built on async generators. The chain flows:



This generator-based architecture enables four critical capabilities:

Streaming. Messages flow through the system incrementally. The parent (or the async lifecycle wrapper) can observe each message as it is produced – updating progress indicators, forwarding metrics, recording transcripts – without buffering the entire conversation.

Cancellation. Returning the async iterator triggers the `finally` block in `runAgent()`. The fifteen-step cleanup runs regardless of whether the agent completed normally, was aborted by the user, or threw an error. JavaScript’s async generator protocol guarantees this.

Backgrounding. A sync agent that is taking too long can be backgrounded mid-execution. The iterator is handed off from the foreground (where `AgentTool.call()` is iterating it) to an async context (where `runAsyncAgentLifecycle()` takes over). The agent does not restart – it continues from where it was.

Progress tracking. Each yielded message is an observation point. The async lifecycle wrapper uses these observation points to update the task state machine, compute progress percentages, and generate notifications when the agent completes.

8.4 Built-In Agent Types

Built-in agents are registered via `getBuiltInAgents()` in `builtInAgents.ts`. The registry is dynamic – which agents are available depends on feature flags, GrowthBook experiments, and the session’s entrypoint type. Six built-in agents ship with the system, each optimized for a specific class of work.

8.4.1 General-Purpose

The default agent when `subagent_type` is omitted and `fork` is not active. Full tool access, no CLAUDE.md omission, model determined by `getDefaultSubagentModel()`. Its system prompt positions it as a completion-oriented worker: “Complete the task fully – don’t gold-plate, but don’t leave it half-done.” It includes guidelines for search strategy (broad first, then narrow) and file creation discipline (never create files unless the task requires it).

This is the workhorse. When the model does not know what kind of agent it needs, it gets a general-purpose agent that can do everything the parent can do, minus spawning its own sub-agents. The “minus spawning” restriction is important: without it, a general-purpose child could spawn its own children, which could spawn theirs, creating an exponential fan-out that burns through API budget in seconds. The `Agent` tool is in the default disallowed list for good reason.

8.4.2 Explore

A read-only search specialist. Uses Haiku (the cheapest, fastest model). Omits CLAUDE.md and git status. Has `FileEdit`, `FileWrite`, `NotebookEdit`, and `Agent` removed from its tool pool, enforced at both the tooling level and via a `=== CRITICAL: READ-ONLY MODE ===` section in its system prompt.

The Explore agent is the most aggressively optimized built-in because it is the most frequently spawned – 34 million times per week across the fleet. It is marked as a one-shot agent (`ONE_SHOT_BUILTIN_AGENT_TYPES`), which means the `agentId`, `SendMessage` instructions, and usage trailer are skipped from its prompt, saving approximately 135 characters per invocation. At 34 million invocations, those 135 characters add up to roughly 4.6 billion characters per week of saved prompt tokens.

Availability is gated by the `BUILTIN_EXPLORE_PLAN_AGENTS` feature flag

AND the `tengu_amber_stoat` GrowthBook experiment, which A/B tests the impact of removing these specialized agents.

8.4.3 Plan

A software architect agent. Same read-only tool set as Explore but uses `'inherit'` for its model (same capability as the parent). Its system prompt guides it through a structured four-step process: Understand Requirements, Explore Thoroughly, Design Solution, Detail the Plan. It must end with a “Critical Files for Implementation” list.

The Plan agent inherits the parent’s model because architecture requires the same reasoning capability as implementation. You do not want a Haiku-class model making design decisions that an Opus-class model will have to execute. The model mismatch would produce plans that the executing agent cannot follow – or worse, plans that sound plausible but are subtly wrong in ways that only a more capable model would catch.

Same availability gate as Explore (`BUILTIN_EXPLORE_PLAN_AGENTS + tengu_amber_stoat`).

8.4.4 Verification

The adversarial tester. Read-only tools, `'inherit'` model, always runs in background (`background: true`), displayed in red in the terminal. Its system prompt is the most elaborate of any built-in agent at approximately 130 lines.

What makes the Verification agent interesting is its anti-avoidance programming. The prompt explicitly lists excuses the model might reach for and instructs it to “recognize them and do the opposite.” Every check must include a “Command run” block with actual terminal output – no hand-waving, no “this should work.” The agent must include at least one adversarial probe (concurrency, boundary, idempotency, orphan cleanup). And before reporting a failure, it must check whether the behavior is intentional or handled elsewhere.

The `criticalSystemReminder_EXPERIMENTAL` field injects a reminder after every tool result, reinforcing that this is verification-only. This is a guardrail against the model drifting from “verify” to “fix” – a tendency that would undermine the entire purpose of an independent verification pass. Language models have a strong inclination to be helpful, and “helpful” in most contexts

means “fix the problem.” The Verification agent’s entire value proposition depends on resisting that inclination.

The **background: true** flag means the Verification agent always runs asynchronously. The parent does not wait for verification results – it continues working while the verifier probes in the background. When the verifier finishes, a notification appears with the results. This mirrors how human code review works: the developer does not stop coding while the reviewer reads their PR.

Availability is gated by the **VERIFICATION_AGENT** feature flag AND the **tengu_hive_evidence** GrowthBook experiment.

8.4.5 Claude Code Guide

A documentation-fetching agent for questions about Claude Code itself, the Claude Agent SDK, and the Claude API. Uses Haiku, runs with **dontAsk** permission mode (no user prompts needed – it only reads documentation), and has two hardcoded documentation URLs.

Its **getSystemPrompt()** is unique because it receives the **toolUseContext** and dynamically includes context about the project’s custom skills, custom agents, configured MCP servers, plugin commands, and user settings. This lets it answer “how do I configure X?” by knowing what is already configured.

Excluded when the entrypoint is SDK (TypeScript, Python, or CLI), since SDK users are not asking Claude Code how to use Claude Code. They are building their own tools on top of it.

The Guide agent is an interesting case study in agent design because it is the only built-in agent whose system prompt is dynamic in a way that depends on the user’s project. It needs to know what is configured to answer “how do I configure X?” effectively. This makes its **getSystemPrompt()** function more complex than the others, but the trade-off is worth it – a documentation agent that does not know what the user has already set up gives worse answers than one that does.

8.4.6 Statusline Setup

A specialized agent for configuring the terminal status line. Uses Sonnet, displayed in orange, limited to **Read** and **Edit** tools only. Knows how to convert shell PS1 escape sequences to shell commands, write to

`~/claude/settings.json`, and handle the `statusLine` command's JSON input format.

This is the most narrowly-scoped built-in agent – it exists because status line configuration is a self-contained domain with specific formatting rules that would clutter a general-purpose agent's context. Always available, no feature gate.

The Statusline Setup agent illustrates an important principle: **sometimes a specialized agent is better than a general-purpose agent with more context**. A general-purpose agent given the status line documentation as context would probably configure it correctly. But it would also be more expensive (bigger model), slower (more context to process), and more likely to get confused by the interaction between status line syntax and the task at hand. A dedicated Sonnet agent with Read and Edit tools and a focused system prompt does the job faster, cheaper, and more reliably.

8.4.7 The Worker Agent (Coordinator Mode)

Not in the `built-in/` directory but loaded dynamically when coordinator mode is active:

```
if (isEnvTruthy(process.env.CLAUDE_CODE_COORDINATOR_MODE)) {
  const { getCoordinatorAgents } = require('../..//coordinator/workerAgent.js')
  return getCoordinatorAgents()
}
```

The worker agent replaces all standard built-in agents in coordinator mode. It has a single type `"worker"` and full tool access. This simplification is deliberate – when a coordinator is orchestrating workers, the coordinator decides what each worker does. The worker does not need the specialization of Explore or Plan; it needs the flexibility to do whatever the coordinator assigns.

8.5 Fork Agents

Fork agents – where the child inherits the parent's full conversation history, system prompt, and tool array for prompt cache exploitation – are the subject of Chapter 9. The fork path triggers when the model omits `subagent_type` from the Agent tool call and the fork experiment is active. Every design

decision in the fork system traces back to a single goal: byte-identical API request prefixes across parallel children, enabling 90% cache discounts on shared context.

8.6 Agent Definitions from Frontmatter

Users and plugins can define custom agents by placing markdown files in `.claude/agents/`. The frontmatter schema supports the full range of agent configuration:

```
---
description: "When to use this agent"
tools:
  - Read
  - Bash
  - Grep
disallowedTools:
  - FileWrite
model: haiku
permissionMode: dontAsk
maxTurns: 50
skills:
  - my-custom-skill
mcpServers:
  - slack
  - my-inline-server:
      command: node
      args: ["/server.js"]
hooks:
  PreToolUse:
    - command: "echo validating"
      event: PreToolUse
color: blue
background: false
isolation: worktree
effort: high
---
```

```
# My Custom Agent

You are a specialized agent for...
```

The markdown body becomes the agent’s system prompt. The frontmatter fields map directly to the `AgentDefinition` interface that `runAgent()` consumes. The loading pipeline in `loadAgentsDir.ts` validates the frontmatter against `AgentJsonSchema`, resolves the source (user, plugin, or policy), and registers the agent in the available agents list.

Four sources of agent definitions exist, in priority order:

1. **Built-in agents** – hardcoded in TypeScript, always available (subject to feature gates)
2. **User agents** – markdown files in `.claude/agents/`
3. **Plugin agents** – loaded via `loadPluginAgents()`
4. **Policy agents** – loaded via organizational policy settings

When the model calls `Agent` with a `subagent_type`, the system resolves the name against this combined list, filtering by permission rules (deny rules for `Agent(AgentName)`) and by `allowedAgentTypes` from the tool spec. If the requested agent type is not found or is denied, the tool call fails with an error.

This design means that organizations can ship custom agents via plugins (a code review agent, a security audit agent, a deployment agent) and have them appear seamlessly alongside the built-in agents. The model sees them in the same list, with the same interface, and delegates to them the same way.

The power of frontmatter-defined agents is that they require zero TypeScript. A team lead who wants a “PR review” agent writes a markdown file with the right frontmatter, drops it in `.claude/agents/`, and it appears in every team member’s agent list on their next session. The system prompt is the markdown body. The tool restrictions, model preference, and permission mode are declared in YAML. The `runAgent()` lifecycle handles everything else – the same fifteen steps, the same cleanup, the same isolation guarantees.

This also means that agent definitions are version-controlled alongside the codebase. A repository can ship agents tailored to its architecture, conventions, and tooling. The agents evolve with the code. When the team adopts a new testing framework, the verification agent’s prompt is updated in the

same commit that adds the framework dependency.

There is one important security consideration: the trust boundary. User agents (from `.claude/agents/`) are user-controlled – their hooks, MCP servers, and tool configurations are subject to `strictPluginOnlyCustomization` restrictions when those policies are active. Plugin agents and policy agents are admin-trusted and bypass these restrictions. Built-in agents are part of the Claude Code binary itself. The system tracks the `source` of each agent definition precisely so that security policies can distinguish between “the user wrote this” and “the organization approved this.”

The `source` field is not just metadata – it gates real behavior. When a plugin-only policy is active for MCP, user agent frontmatter that declares MCP servers is silently skipped (the MCP connections are not established). When a plugin-only policy is active for hooks, user agent frontmatter hooks are not registered. The agent still runs – it just runs without the untrusted extensions. This is a principle of graceful degradation: the agent is useful even when its full capabilities are restricted by policy.

8.7 Apply This: Designing Agent Types

The built-in agents demonstrate a pattern language for agent design. If you are building a system that spawns sub-agents – whether using Claude Code’s AgentTool directly or designing your own multi-agent architecture – the design space breaks down into five dimensions.

8.7.1 Dimension 1: What Can It See?

The combination of `omitClaudeMd`, git status stripping, and skill preloading controls the agent’s awareness. Read-only agents see less (they do not need project conventions). Specialized agents see more (preloaded skills inject domain knowledge).

The key insight is that context is not free. Every token in the system prompt, user context, or conversation history costs money and displaces working memory. Claude Code strips `CLAUDE.md` from Explore agents not because those instructions are harmful, but because they are irrelevant – and irrelevance at 34 million spawns per week becomes a line item on the

infrastructure bill. When designing your own agent types, ask: “What does this agent need to know to do its job?” and strip everything else.

8.7.2 Dimension 2: What Can It Do?

The `tools` and `disallowedTools` fields set hard boundaries. The Verification agent cannot edit files. The Explore agent cannot write anything. The General-Purpose agent can do everything except spawn sub-agents of its own.

Tool restrictions serve two purposes: **safety** (the Verification agent cannot accidentally “fix” what it finds, preserving its independence) and **focus** (an agent with fewer tools spends less time deciding which tool to use). The pattern of combining tool-level restrictions with system prompt guidance (Explore’s `=== CRITICAL: READ-ONLY MODE ===`) is defense in depth – the tools enforce the boundary mechanically, and the prompt explains *why* the boundary exists so the model does not waste turns trying to work around it.

8.7.3 Dimension 3: How Does It Interact with the User?

The `permissionMode` and `canShowPermissionPrompts` settings determine whether the agent asks for permission, auto-denies, or bubbles prompts to the parent’s terminal. Background agents that cannot interrupt the user must either work within pre-approved boundaries or bubble.

The `awaitAutomatedChecksBeforeDialog` setting is a nuance worth understanding. Background agents that *can* show prompts (bubble mode) wait for the classifier and permission hooks to run before interrupting the user. This means the user is only interrupted for genuinely ambiguous permissions – not for things the automated system could have resolved. In a multi-agent system where five background agents are running simultaneously, this is the difference between a usable interface and a permission-prompt barrage.

8.7.4 Dimension 4: How Does It Relate to the Parent?

Sync agents block the parent and share its state. Async agents run independently with their own abort controller. Fork agents inherit the full conversation context. The choice shapes both the user experience (does the parent wait?) and the system behavior (does Escape kill the child?).

The abort controller decision in Step 8 crystallizes this: sync agents share the parent’s controller (Escape kills both), async agents get their own (Escape

leaves them running). Fork agents go further – they inherit the parent’s system prompt, tool array, and message history to maximize prompt cache sharing. Each relationship type has a clear use case: sync for sequential delegation (“do this then I’ll continue”), async for parallel work (“do this while I do something else”), and fork for context-heavy delegation (“you know everything I know, now go handle this part”).

8.7.5 Dimension 5: How Expensive Is It?

The model choice, thinking config, and context size all contribute to cost. Haiku for cheap read-only work. Sonnet for moderate tasks. Inherit-from-parent for tasks requiring the parent’s reasoning capability. Thinking is disabled for non-fork agents to control output token costs – the parent pays for reasoning; the children execute.

The economic dimension is often an afterthought in multi-agent system design, but it is central to Claude Code’s architecture. An Explore agent that used Opus instead of Haiku would work fine for any individual invocation. But at 34 million invocations per week, the model choice is a multiplicative cost factor. The one-shot optimization that saves 135 characters per Explore invocation translates to 4.6 billion characters per week of saved prompt tokens. These are not micro-optimizations – they are the difference between a viable product and an unaffordable one.

8.7.6 The Unified Lifecycle

The `runAgent()` lifecycle implements all five dimensions through its fifteen steps, assembling a unique execution environment for each agent type from the same set of building blocks. The result is a system where spawning a sub-agent is not “run another copy of the parent.” It is the creation of a precisely-scoped, resource-controlled, isolated execution context – tailored to the work at hand, and cleaned up completely when the work is done.

The architectural elegance is in the uniformity. Whether the agent is a Haiku-powered read-only searcher or an Opus-powered fork child with full tool access and bubble permissions, it flows through the same fifteen steps. The steps do not branch based on agent type – they parameterize. Model resolution picks the right model. Context preparation picks the right file state. Permission isolation picks the right mode. The agent type is not encoded in control flow; it is encoded in configuration. And that is what makes the system extensible: adding a new agent type means writing a

definition, not modifying the lifecycle.

8.7.7 The Design Space Summarized

The six built-in agents cover a spectrum:

Agent	Model	Tools	Context	Sync/Async	Purpose
General-Purpose	Default	All	Full	Either	Workhorse delegation
Explore	Haiku	Read-only	Stripped	Sync	Fast, cheap search
Plan	Inherit	Read-only	Stripped	Sync	Architecture design
Verification	Inherit	Read-only	Full	Always async	Adversarial testing
Guide	Haiku	Read + Web	Dynamic	Sync	Documentation lookup
Statusline	Sonnet	Read + Edit	Minimal	Sync	Config task

No two agents make the same choices across all five dimensions. Each is optimized for its specific use case. And the `runAgent()` lifecycle handles all of them through the same fifteen steps, parameterized by the agent definition. This is the power of the architecture: the lifecycle is a universal machine, and the agent definitions are the programs that run on it.

The next chapter examines fork agents in depth – the prompt cache exploitation mechanism that makes parallel delegation economically viable. Chapter 10 then follows with the orchestration layer: how async agents report progress through the task state machine, how the parent retrieves results, and how the coordinator pattern orchestrates dozens of agents working toward a single goal. If this chapter was about *creating* agents, Chapter 9 is about making them cheap, and Chapter 10 is about *managing* them.

Chapter 9

Chapter 9: Fork Agents and the Prompt Cache

9.1 The Ninety-Five Percent Insight

When a parent agent spawns five child agents in parallel, the overwhelming majority of each child’s API request is identical. The system prompt is the same. The tool definitions are the same. The conversation history is the same. The assistant message that triggered the spawns is the same. The only thing that differs is the final directive: “you handle the database migration,” “you write the tests,” “you update the docs.”

On a typical fork with a warm conversation, the shared prefix might be 80,000 tokens. The per-child directive might be 200 tokens. That is 99.75% overlap. Anthropic’s prompt cache gives a 90% discount on cached input tokens. If you can make those 80,000 tokens hit the cache for children 2 through 5, you just cut the input cost of those four requests by 90%. For the parent, this is the difference between spending \$4 and spending \$0.50 on the same parallel dispatch.

The catch is that prompt caching is byte-exact. Not “similar enough.” Not “semantically equivalent.” The bytes must match, character for character, from the first byte of the system prompt through to the last byte before the per-child content diverges. One extra space, one reordered tool definition, one stale feature flag changing a system prompt fragment – and the cache misses. The entire prefix is reprocessed at full price.

Fork agents are Claude Code’s answer to this constraint. They are not just a convenience for “spawn a child with context” – they are a prompt cache exploitation mechanism disguised as an orchestration feature. Every design decision in the fork system traces back to one question: how do we guarantee byte-identical prefixes across parallel children?

9.2 What a Fork Child Inherits

A fork agent inherits four things from its parent, and it inherits them by reference or byte-exact copy, not by recomputation.

- 1. The system prompt.** Not regenerated – threaded. The parent’s already-rendered system prompt bytes are passed via `override.systemPrompt`, pulled from `toolUseContext.renderedSystemPrompt`. This is the exact string that was sent in the parent’s most recent API call.
- 2. The tool definitions.** The fork agent definition declares `tools: ['*']`, but with the `useExactTools` flag set to true, the child receives the parent’s assembled tool array directly. No filtering, no reordering, no re-serialization.
- 3. The conversation history.** Every message the parent has exchanged with the API – user turns, assistant turns, tool calls, tool results – is cloned into the child’s context via `forkContextMessages`.
- 4. The thinking configuration and model.** The fork definition specifies `model: 'inherit'`, which resolves to the parent’s exact model. Same model means same tokenizer, same context window, same cache namespace.

The fork agent definition itself is minimal – almost a no-op:

The fork agent definition is deliberately minimal – it inherits everything from the parent. It specifies all tools (`'*'`), inherits the parent’s model, uses bubble mode for permissions (so prompts surface in the parent’s terminal), and provides a no-op system prompt function that is never actually called – the real prompt arrives via the override channel, already rendered and byte-stable.

9.3 The Byte-Identical Prefix Trick

The API request to Claude has a specific structure: system prompt, then tools, then messages. For the prompt cache to hit, every byte from the start of the request through to some prefix boundary must be identical across requests.

Fork agents achieve this by ensuring three layers are frozen:

Layer 1: System prompt via threading, not recomputation.

When the parent agent's system prompt was rendered for its last API call, the result was captured in `toolUseContext.renderedSystemPrompt`. This is the string after all dynamic interpolation – GrowthBook feature flags, environment details, MCP server descriptions, skill content, CLAUDE.md files. The fork child receives this exact string.

Why not just call `getSystemPrompt()` again? Because system prompt generation is not pure. GrowthBook flags transition from cold to warm state as the SDK fetches remote config. A flag that returned `false` during the parent's first turn might return `true` by the time the fork child spins up. If the system prompt includes a conditional block gated by that flag, the re-rendered prompt diverges by even a single character. Cache busted. Full-price reprocessing of 80,000 tokens, times five children.

Threading the rendered bytes eliminates this entire class of divergence.

Layer 2: Tool definitions via exact passthrough.

Normal sub-agents go through `resolveAgentTools()`, which filters the tool pool based on the agent definition's `tools` and `disallowedTools` arrays, applies permission mode differences, and potentially reorders tools. The resulting serialized tool array would differ from the parent's – different subset, different order, different permission annotations.

Fork agents skip this entirely:

```
const resolvedTools = useExactTools
  ? availableTools // parent's exact array
  : resolveAgentTools(agentDefinition, availableTools, isAsync).resolvedTools
```

The `useExactTools` flag is set to true only on the fork path. The child gets the parent's tool pool as-is. Same tools, same order, same serialization. This includes keeping the Agent tool itself in the child's pool, even though the

child is forbidden from using it – removing it would change the tool array and bust the cache.

Layer 3: Message array construction.

This is where `buildForkedMessages()` does its careful work. The function constructs the final two messages that sit between the shared history and the per-child directive:

The `buildForkedMessages()` function constructs the final two messages that sit between the shared history and the per-child directive. The algorithm:

1. Clone the parent’s assistant message (preserving all `tool_use` blocks with their original IDs).
2. For each `tool_use` block, create a `tool_result` with a constant placeholder string (identical across all children).
3. Build a single user message containing all the placeholder results followed by the per-child directive wrapped in the boilerplate tag.
4. Return `[clonedAssistantMessage, userMessageWithPlaceholdersAndDirective]`.

```
// Pseudocode - illustrates the message construction
function buildChildMessages(directive, parentAssistant) {
  const cloned = cloneMessage(parentAssistant)
  const placeholders = parentAssistant.toolUseBlocks.map(b =>
    toolResult(b.id, CONSTANT_PLACEHOLDER) // Byte-identical across children
  )
  const userMsg = createUserMessage([...placeholders, wrapDirective(directive)])
  return [cloned, userMsg]
}
```

The resulting message array for each child looks like:

```
[...shared_history, assistant(all_tool_uses), user(placeholder_results..., directive)]
```

Every element before the directive is identical across children. The `FORK_PLACEHOLDER_RESULT` – a constant string `'Fork started -- processing in background'` – ensures even the tool result blocks are byte-identical. The `tool_use_id` values are identical because they reference the same assistant message. Only the final text block, containing the per-child directive, varies.

The cache boundary falls right before that final text block. Everything above it – potentially tens of thousands of tokens of system prompt, tool definitions, conversation history, and placeholder results – hits the cache at

a 90% discount for every child after the first.

9.4 The Fork Boilerplate Tag

Each child’s directive is wrapped in a boilerplate XML tag that serves two purposes: it instructs the child on how to behave, and it acts as a marker for recursive fork detection.

The boilerplate contains approximately 10 rules. The key ones:

- **Override the parent’s forking instruction.** The parent’s system prompt says “default to forking” – the boilerplate explicitly tells the child: “that instruction is for the parent. You ARE the fork. Do NOT spawn sub-agents.”
- **Execute silently, report once.** No conversational text between tool calls. Use tools directly, then produce a structured summary.
- **Stay within scope.** The child must not expand beyond its directive.
- **Structured output format.** The response must follow a Scope/Result/Key files/Files changed/Issues template that makes results easy for the parent to parse when multiple children report back simultaneously.

Rule 1 is particularly interesting. The parent’s system prompt – which the fork child inherits verbatim for cache reasons – contains instructions like “default to forking when you have parallel work.” If the child followed that instruction, it would try to fork its own children, creating an infinite recursion of agents. The boilerplate explicitly overrides: “that instruction is for the parent. You ARE the fork.”

The structured output format (Scope/Result/Key files/Files changed/Issues) is not decorative. It constrains the child’s output to factual reporting, which makes the results easier for the parent to parse and aggregate when five children report back simultaneously.

9.5 Recursive Fork Prevention

The fork child keeps the Agent tool in its tool pool. It has to – removing it would change the serialized tool array and bust the prompt cache. But if

the child actually invokes the Agent tool without `subagent_type`, the fork path would trigger again, creating a grandchild fork. This grandchild would inherit an even larger context (parent + child conversation), spawn its own forks, and so on.

Two guards prevent this:

Primary guard: `querySource` check. When a fork child is spawned, its `context.options.querySource` is set to `'agent:builtin:fork'`. The `call()` method checks this before allowing the fork path:

```
// In AgentTool.call():
if (effectiveType === undefined) {
  // Fork path -- but are we already in a fork?
  if (querySource === 'agent:builtin:fork') {
    // Reject: already a fork child
  }
}
```

This is the fast path. It checks a single string in the options object.

Fallback guard: message scanning. Fork prevention uses two guards: the `querySource` tag set at spawn time (the fast path – a single string comparison), and a fallback that scans message history for the boilerplate XML tag. The fallback exists because the `querySource` survives autocompact, but in edge cases where it was not properly threaded, the message-scanning fallback catches the recursion. It is a belt-and-suspenders approach where the cost of the check (scanning messages) is trivial compared to the cost of accidental recursive forking (runaway API spend).

Why the fallback? Because Claude Code has an autocompact feature that rewrites the message array when context gets too long. Autocompact can rewrite message content but preserves the `querySource` in options. In theory, `querySource` alone is sufficient. In practice, the message-scanning fallback catches edge cases where `querySource` was not properly threaded – a belt-and-suspenders approach where the cost of the check (scanning messages) is trivial compared to the cost of accidental recursive forking (runaway API spend).

9.6 The Sync-to-Async Transition

A fork child starts running in the foreground: its messages stream to the parent's terminal, and the parent blocks waiting for completion. But what if the child is taking too long? Claude Code allows mid-execution backgrounding – the user (or an auto-timeout) can push a running foreground agent into the background without losing any work.

The mechanism is surprisingly clean:

1. When a foreground agent is registered via `registerAgentForeground()`, a background signal promise is created.
2. The parent's sync loop races between the agent's message stream and the background signal:

```
while (true) {
  const result = await Promise.race([
    iterator.next(),          // next message from agent
    backgroundSignal,         // "move to background" trigger
  ])
  if (result === BACKGROUND_SIGNAL) break
  // ... process message
}
```

3. When the background signal fires, the foreground iterator is gracefully terminated via `iterator.return()`. This triggers the generator's `finally` block, which handles cleanup.
4. A new `runAgent()` instance is spawned with `isAsync: true`, using the same agent ID and the message history accumulated so far. The agent continues from where it left off, now running in the background.
5. The original synchronous `call()` returns `{ status: 'async_launched' }`, and the parent continues its conversation.

No work is lost because the message history is the agent's state. The sidechain transcript on disk has every message the agent has produced. The new async instance replays from this transcript and picks up where the sync instance stopped.

9.7 Auto-Backgrounding

When the `CLAUDE_AUTO_BACKGROUND_TASKS` environment variable or the `tengu_auto_background_agents` GrowthBook flag is enabled, foreground agents are automatically backgrounded after 120 seconds:

When enabled via environment variable or feature flag, foreground agents are automatically backgrounded after 120 seconds. When disabled, the function returns 0 (no auto-backgrounding).

This is a UX decision with cost implications. A foreground agent blocks the parent terminal – the user cannot type, cannot issue new instructions, cannot spawn other agents. Two minutes is long enough for the agent to complete most quick tasks synchronously (where the streaming output is useful feedback), but short enough that long-running tasks do not hold the terminal hostage.

Under the fork experiment, the auto-backgrounding question is moot: all fork spawns are forced async from the start. The `run_in_background` parameter is hidden from the schema entirely. Every fork child runs in the background, reports back via a `<task-notification>` when done, and the parent never blocks.

9.8 When Fork Is NOT Used

Fork is one of several orchestration modes, and it is deliberately excluded in three cases:

Coordinator mode. Coordinator mode and fork mode are mutually exclusive. A coordinator has a structured delegation model: it maintains a plan, assigns tasks to workers with explicit prompts, and tracks progress. Fork’s “inherit everything” approach would undermine this. A forked coordinator would inherit the parent coordinator’s system prompt (which says “you are the coordinator, delegate work”), and the child would try to orchestrate instead of execute. The `isForkSubagentEnabled()` function checks `isCoordinatorMode()` first and returns false if active.

Non-interactive sessions. SDK and API consumers (`--print` mode, Claude Agent SDK) operate without a terminal. Fork’s `permissionMode: 'bubble'` surfaces permission prompts to the parent terminal – which does

not exist in non-interactive mode. Rather than building a separate permission flow, the fork path is simply disabled. SDK consumers use explicit `subagent_type` selection instead.

Explicit `subagent_type`. When the model specifies a `subagent_type` (e.g., "Explore", "Plan", "general-purpose"), the fork path is not triggered. Fork only fires when `subagent_type` is omitted. This lets the model choose between “I want a specialized agent with its own system prompt and tool set” (explicit type) and “I want a context-inheriting clone of myself to handle this in parallel” (omitted type).

9.9 The Economics

Consider a concrete scenario. A developer asks Claude Code to refactor a module. The parent agent analyzes the codebase, forms a plan, and dispatches five fork children in parallel: one to update the database schema, one to rewrite the service layer, one to update the router, one to fix the tests, and one to update the types.

At this point in the conversation, the shared context is substantial: - System prompt: ~4,000 tokens - Tool definitions (40+ tools): ~12,000 tokens - Conversation history (analysis + planning): ~30,000 tokens - Assistant message with five `tool_use` blocks: ~2,000 tokens - Placeholder tool results: ~500 tokens

Total shared prefix: ~48,500 tokens. Per-child directive: ~200 tokens.

Without fork (five independent agents, each with fresh context and their own system prompt): - Each child processes its own system prompt + tools + task prompt - No cache sharing (different system prompts, different tool sets) - Cost: 5 x full input processing

With fork (byte-identical prefixes): - Child 1: 48,700 tokens at full price (cache miss on first request) - Children 2-5: 48,500 tokens at 10% price (cache hit) + 200 tokens at full price each - Effective cost for children 2-5: $\sim 4,850 + 200 = \sim 5,050$ tokens equivalent each

The savings scale with context size and child count. For a warm session with 100K tokens of history spawning 8 parallel forks, the cache savings can exceed 90% of what the input tokens would have cost without sharing.

This is why every design decision in the fork system – the threading instead of recomputation, the exact tool passthrough, the placeholder results, even keeping the Agent tool in the child’s pool despite it being forbidden – optimizes for one thing: byte-identical prefixes. Each decision trades a small amount of elegance or safety for a measurable reduction in API cost.

9.10 Design Tensions

The fork system makes explicit trade-offs that are worth understanding:

Isolation vs. cache efficiency. Fork children inherit everything, including conversation history that may be irrelevant to their task. A child rewriting tests does not need the 15 messages where the parent discussed database schema design. But including those messages is what makes the prefix identical. Stripping irrelevant history would save context window space at the cost of busting the cache. The design bet is that cache savings outweigh the context overhead.

Safety vs. cache efficiency. The Agent tool stays in the fork child’s tool pool even though the child must not use it. Removing it would be safer (the child cannot even attempt to fork), but would change the tool array serialization. The boilerplate tag and recursive fork guards are the compensating controls – runtime prevention instead of static removal.

Simplicity vs. cache efficiency. The placeholder tool results are a lie. The child sees `'Fork started -- processing in background'` for every `tool_use` block in the parent’s assistant message, regardless of what those tool calls actually did. This is fine because the child’s directive tells it what to do – it does not need accurate tool results from the parent’s dispatching turn. But it means the child’s conversation history is technically incoherent. The placeholder is chosen for brevity and uniformity, not accuracy.

Each of these trade-offs reflects the same priority: when you are paying per-token for API calls at scale, byte-identical prefixes are worth contorting the architecture around.

9.11 Apply This: Designing for Prompt Cache Efficiency

The fork agent pattern generalizes beyond Claude Code. Any system that dispatches multiple parallel LLM calls from the same context can benefit from cache-aware request construction. The principles:

- 1. Thread rendered prompts, do not recompute.** If your system prompt includes any dynamic content – feature flags, timestamps, user preferences, A/B test variants – capture the rendered result and pass it to children by value. Recomputing risks divergence.
- 2. Freeze the tool array.** If your children need different tool sets, you are giving up cache sharing on the tools block. Consider keeping the full tool set and using runtime guards (like the fork boilerplate’s “do not use Agent”) instead of compile-time removal.
- 3. Maximize the shared prefix, minimize the per-child suffix.** Structure your message array so that everything shared comes first and per-child content is appended at the end. Interleaving shared and per-child content fragments the cache boundary.
- 4. Use constant placeholders for variable content.** When the message structure requires responses to previous tool calls, use identical placeholder strings across all children rather than actual (divergent) results.
- 5. Measure the break-even.** Cache sharing has overhead: larger context windows per child (they carry irrelevant history), runtime guards instead of static safety, architectural complexity. Calculate whether your parallelism pattern (how many children, how large the shared prefix) actually saves money after accounting for the extra context tokens.

The fork agent system is, at its core, a prompt cache exploitation engine. It answers a question that every multi-agent system builder eventually faces: when the cache gives you a 90% discount on repeated prefixes, how far will you restructure your architecture to claim that discount? Claude Code’s answer is: very far.

Chapter 10

Chapter 10: Tasks, Coordination, and Swarms

10.1 The Limits of a Single Thread

Chapter 8 showed how to create a sub-agent – the fifteen-step lifecycle that builds an isolated execution context from an agent definition. Chapter 9 showed how to make parallel spawns economical through prompt cache exploitation. But creating agents and managing agents are different problems. This chapter addresses the second.

A single agent loop – one model, one conversation, one tool at a time – can accomplish a remarkable amount of work. It can read files, edit code, run tests, search the web, and reason about complex problems. But it hits a ceiling.

The ceiling is not intelligence. It is parallelism and scope. A developer working on a large refactoring needs to update 40 files, run tests after each batch, and verify nothing broke. A codebase migration touches frontend, backend, and database layers simultaneously. A thorough code review reads dozens of files while running the test suite in the background. These are not harder problems – they are wider ones. They require the ability to do multiple things at once, to delegate work to specialists, and to coordinate the results.

Claude Code’s answer to this problem is not one mechanism but a layered stack of orchestration patterns, each suited to a different shape of work.

Background tasks for fire-and-forget commands. Coordinator mode for manager-worker hierarchies. Swarm teams for peer-to-peer collaboration. And a unified communication protocol that ties them all together.

The orchestration layer spans approximately 40 files across `tools/AgentTool/`, `tasks/`, `coordinator/`, `tools/SendMessageTool/`, and `utils/swarm/`. Despite this breadth, the design is anchored by a single state machine that all patterns share. Understanding that state machine – the **Task** abstraction in `Task.ts` – is the prerequisite for understanding everything else.

This chapter traces the full stack, from the foundational task state machine up through the most sophisticated multi-agent topologies.

10.2 The Task State Machine

Every background operation in Claude Code – a shell command, a sub-agent, a remote session, a workflow script – is tracked as a *task*. The task abstraction lives in `Task.ts` and provides the unified state model that the rest of the orchestration layer builds on.

10.2.1 Seven Types

The system defines seven task types, each representing a different execution model:

The seven task types are: `local_bash` (background shell commands), `local_agent` (background sub-agents), `remote_agent` (remote sessions), `in_process_teammate` (swarm teammates), `local_workflow` (workflow script executions), `monitor_mcp` (MCP server monitors), and `dream` (speculative background thinking).

`local_bash` and `local_agent` are the workhorses – background shell commands and background sub-agents, respectively. `in_process_teammate` is the swarm primitive. `remote_agent` bridges to remote Claude Code Runtime environments. `local_workflow` runs multi-step scripts. `monitor_mcp` watches MCP server health. `dream` is the most unusual – a background task that lets the agent think speculatively while waiting for user input.

Each type gets a single-character ID prefix for instant visual identification:

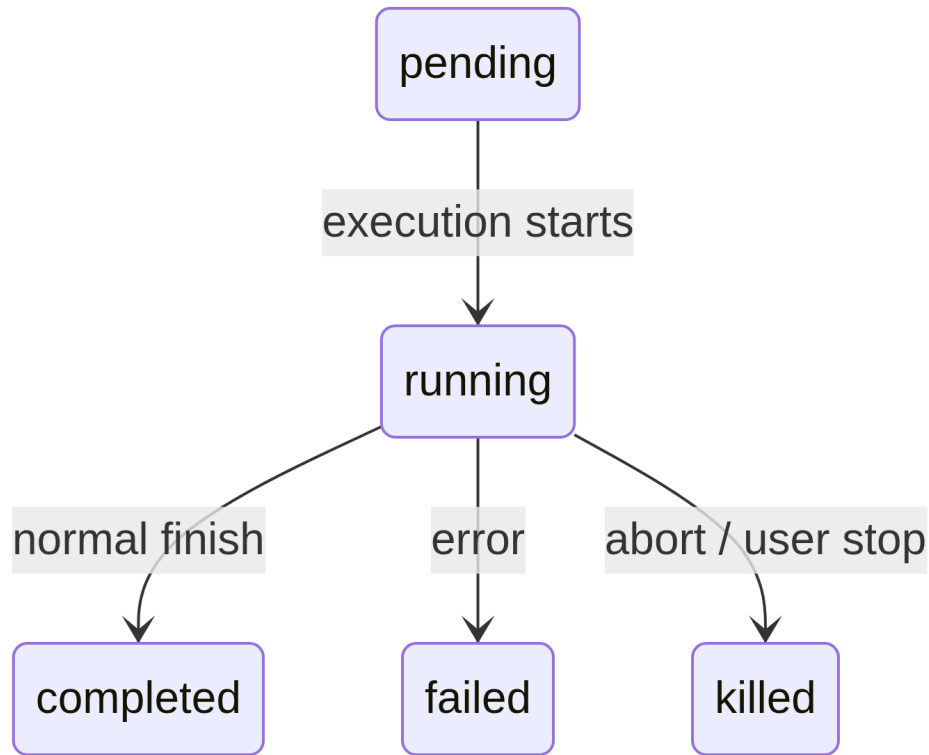
Type	Prefix	Example ID
local_bash	b	b4k2m8x1
local_agent	a	a7j3n9p2
remote_agent	r	r1h5q6w4
in_process_teammate	t	t3f8s2v5
local_workflow	w	w6c9d4y7
monitor_mcp	m	m2g7k1z8
dream	d	d5b4n3r6

Task IDs use a single-character prefix (a for agents, b for bash, t for teammates, etc.) followed by 8 random alphanumeric characters drawn from a case-insensitive-safe alphabet (digits plus lowercase letters). This yields approximately 2.8 trillion combinations – enough to resist brute-force symlink attacks against the task output files on disk.

When you see `a7j3n9p2` in a log line, you know immediately it is a background agent. When you see `b4k2m8x1`, a shell command. The prefix is a micro-optimization for human readers, but in a system that can have dozens of concurrent tasks, it matters.

10.2.2 Five Statuses

The lifecycle is a simple directed graph with no cycles:



`pending` is the brief state between registration and first execution. `running` means the task is actively doing work. The three terminal states are `completed` (success), `failed` (error), and `killed` (explicitly stopped by the user, the coordinator, or an abort signal). A helper function guards against interacting with dead tasks:

```
export function isTerminalTaskStatus(status: TaskStatus): boolean {
  return status === 'completed' || status === 'failed' || status === 'killed'
}
```

This function appears everywhere – in message injection guards, eviction logic, orphan cleanup, and the `SendMessage` routing that decides whether to queue a message or resume a dead agent.

10.2.3 The Base State

Every task state extends `TaskStateBase`, which carries the fields that all seven types share:


```
export type TaskStateBase = {
  id: string           // Prefixed random ID
  type: TaskType        // Discriminator
  status: TaskStatus    // Current lifecycle position
  description: string   // Human-readable summary
  toolUseId?: string   // The tool_use block that spawned this task
  startTime: number     // Creation timestamp
  endTime?: number     // Terminal-state timestamp
  totalPausedMs?: number // Accumulated pause time
  outputFile: string    // Disk path for streaming output
  outputOffset: number  // Read cursor for incremental output
  notified: boolean     // Whether completion was reported to parent
}
```

Two fields deserve attention. `outputFile` is the bridge between async execution and the parent’s conversation – every task writes its output to a file on disk, and the parent can read it incrementally via `outputOffset`. `notified` prevents duplicate completion messages; once the parent has been told a task finished, the flag flips to `true` and the notification is never sent again. Without this guard, a task that completes between two consecutive polls of the notification queue would generate duplicate notifications, confusing the model into thinking two tasks finished when only one did.

10.2.4 The Agent Task State

`LocalAgentTaskState` is the most complex variant, carrying everything needed to manage a background sub-agent’s full lifecycle:

```
export type LocalAgentTaskState = TaskStateBase & {
  type: 'local_agent'
  agentId: string
  prompt: string
  selectedAgent?: AgentDefinition
  agentType: string
  model?: string
  abortController?: AbortController
  pendingMessages: string[] // Queued via SendMessage
  isBackgrounded: boolean   // Was this originally a foreground agent?
  retain: boolean          // UI is holding this task
  diskLoaded: boolean      // Sidechain transcript loaded
}
```

```

    evictAfter?: number // GC deadline
    progress?: AgentProgress
    lastReportedToolCount: number
    lastReportedTokenCount: number
    // ... additional lifecycle fields
}

```

Three fields reveal important design decisions. `pendingMessages` is the inbox – when `SendMessage` targets a running agent, the message is queued here rather than injected immediately. Messages are drained at tool-round boundaries, which preserves the agent’s turn structure. `isBackgrounded` distinguishes agents that were born `async` from those that started as foreground `sync` agents and were later backgrounded by the user pressing a key. `evictAfter` is a garbage collection mechanism: non-retained completed tasks get a grace period before their state is purged from memory.

All task states are stored in `AppState.tasks` as a `Record<string, TaskState>`, keyed by the prefixed ID. This is a flat map, not a tree – the system does not model parent-child relationships in the state store. The parent-child relationship is implicit in the conversation flow: the parent holds the `toolUseId` that spawned the child.

10.2.5 The Task Registry

Each task type is backed by a `Task` object with a minimal interface:

```

export type Task = {
  name: string
  type: TaskType
  kill(taskId: string, setAppState: SetAppState): Promise<void>
}

```

The registry collects all task implementations:

```

export function getAllTasks(): Task[] {
  return [
    LocalShellTask,
    LocalAgentTask,
    RemoteAgentTask,
    DreamTask,
    ...(LocalWorkflowTask ? [LocalWorkflowTask] : []),
    ...(MonitorMcpTask ? [MonitorMcpTask] : []),
  ]
}

```

```
]
}
```

Notice the conditional inclusion – `LocalWorkflowTask` and `MonitorMcpTask` are feature-gated and may not exist at runtime. The `Task` interface is deliberately minimal. Earlier iterations included `spawn()` and `render()` methods, but these were removed when it became clear that spawning and rendering were never called polymorphically. Each task type has its own spawn logic, its own state management, and its own rendering. The only operation that genuinely needs to dispatch by type is `kill()`, and so that is all the interface requires.

This is an example of interface evolution through subtraction. The initial design imagined that all task types would share a common lifecycle interface. In practice, the types diverged enough that the shared interface became a fiction – `spawn()` for a shell command and `spawn()` for an in-process teammate have almost nothing in common. Rather than maintain a leaky abstraction, the team removed everything except the one method that actually benefits from polymorphism.

10.3 Communication Patterns

A task that runs in the background is only useful if the parent can observe its progress and receive its results. Claude Code supports three communication channels, each optimized for a different access pattern.

10.3.1 Foreground: The Generator Chain

When an agent runs synchronously, the parent iterates its `runAgent()` async generator directly, yielding each message back up the call stack. The interesting mechanism here is the background escape hatch – the sync loop races between “next message from agent” and “background signal”:

```
const agentIterator = runAgent({ ...params })[Symbol.asyncIterator]()

while (true) {
  const nextMessagePromise = agentIterator.next()
  const raceResult = backgroundPromise
    ? await Promise.race([nextMessagePromise.then(...), backgroundPromise])
```

```

    : { type: 'message', result: await nextMessagePromise }

    if (raceResult.type === 'background') {
      // User triggered backgrounding -- transition to async
      await agentIterator.return(undefined)
      void runAgent({ ...params, isAsync: true })
      return { data: { status: 'async_launched' } }
    }

    agentMessages.push(message)
  }

```

If the user decides mid-execution that a sync agent should become a background task, the foreground iterator is cleanly returned (triggering its `finally` block for resource cleanup), and the agent is re-spawned as an async task with the same ID. The transition is seamless – no work is lost, and the agent continues from where it left off with an async abort controller that is unlinked from the parent’s ESC key.

This is a genuinely difficult state transition to get right. The foreground agent shares the parent’s abort controller (ESC kills both). The background agent needs its own controller (ESC should not kill it). The agent’s messages need to transfer from the foreground generator stream to the background notification system. The task state needs to flip `isBackgrounded` so the UI knows to show it in the background panel. And all of this must happen atomically – no messages lost in the transition, no zombie iterators left running. The `Promise.race` between the next message and the background signal is the mechanism that makes this possible.

10.3.2 Background: Three Channels

Background agents communicate through disk, notifications, and queued messages.

Disk output files. Every task writes to an `outputFile` path – a symlink to the agent’s transcript in JSONL format. The parent (or any observer) can read this file incrementally using `outputOffset`, which tracks how far into the file has been consumed. The `TaskOutputTool` exposes this to the model:

```

inputSchema = z.strictObject({
  task_id: z.string(),

```

```

    block: z.boolean().default(true),
    timeout: z.number().default(30000),
  })

```

When `block: true`, the tool polls until the task reaches a terminal state or the timeout expires. This is the primary mechanism for a coordinator that spawns a worker and waits for its result.

Task notifications. When a background agent completes, the system generates an XML notification and enqueues it for delivery into the parent’s conversation:

```

<task-notification>
  <task-id>a7j3n9p2</task-id>
  <tool-use-id>toolu_abc123</tool-use-id>
  <output-file>/path/to/output</output-file>
  <status>completed</status>
  <summary>Agent "Investigate auth bug" completed</summary>
  <result>Found null pointer in src/auth/validate.ts:42...</result>
  <usage>
    <total_tokens>15000</total_tokens>
    <tool_uses>8</tool_uses>
    <duration_ms>12000</duration_ms>
  </usage>
</task-notification>

```

The notification is injected as a user-role message in the parent’s conversation, which means the model sees it in its normal message flow. It does not need a special tool to check for completions – they arrive as context. The `notified` flag on the task state prevents duplicate delivery.

Command queue. The `pendingMessages` array on `LocalAgentTaskState` is the third channel. When `SendMessage` targets a running agent, the message is queued:

```

if (isLocalAgentTask(task) && task.status === 'running') {
  queuePendingMessage(agentId, input.message, setAppState)
  return { data: { success: true, message: 'Message queued...' } }
}

```

These messages are drained at tool-round boundaries by `drainPendingMessages()` and injected as user messages into the agent’s conversation. This is a crucial

design choice – messages arrive between tool rounds, not mid-execution. The agent finishes its current thought, then receives the new information. No race conditions, no corrupted state.

10.3.3 Progress Tracking

The `ProgressTracker` provides real-time visibility into agent activity:

```
export type ProgressTracker = {
  toolUseCount: number
  latestInputTokens: number // Cumulative (latest value, not sum)
  cumulativeOutputTokens: number // Summed across turns
  recentActivities: ToolActivity[] // Last 5 tool uses
}
```

The distinction between input and output token tracking is deliberate and reflects a subtlety of the API’s billing model. Input tokens are cumulative per API call because the full conversation is re-sent each time – the 15th turn includes all 14 previous turns, so the input token count reported by the API already reflects the total. Keeping the latest value is the correct aggregation. Output tokens are per-turn – the model generates new tokens each time – so summing is the correct aggregation. Getting this wrong would either dramatically overcount (summing cumulative input tokens) or dramatically undercount (keeping only the latest output tokens).

The `recentActivities` array (capped at 5 entries) provides a human-readable stream of what the agent is doing: “Read src/auth/validate.ts”, “Bash: npm test”, “Edit src/auth/validate.ts”. This appears in the VS Code subagent panel and the terminal’s background task indicator, giving users visibility into agent work without requiring them to read full transcripts.

For background agents, progress is written to `AppState` via `updateAsyncAgentProgress()` and emitted as SDK events via `emitTaskProgress()`. The VS Code subagent panel consumes these events to render live progress bars, tool counts, and activity streams. The progress tracking is not just cosmetic – it is the primary feedback mechanism that tells users whether a background agent is making progress or stuck in a loop.

10.4 Coordinator Mode

Coordinator mode transforms Claude Code from a single agent with background helpers into a true manager-worker architecture. It is the most opinionated orchestration pattern in the system, and its design reveals deep thinking about how LLMs should and should not delegate work.

10.4.1 The Problem Coordinator Mode Solves

The standard agent loop has a single conversation and a single context window. When it spawns a background agent, the background agent runs independently and reports results via task notifications. This works well for simple delegation – “run the tests while I keep editing” – but breaks down for complex multi-step workflows.

Consider a codebase migration. The agent needs to: (1) understand the current patterns across 200 files, (2) design the migration strategy, (3) apply changes to each file, and (4) verify nothing broke. Steps 1 and 3 benefit from parallelism. Step 2 requires synthesizing the results of step 1. Step 4 depends on step 3. A single agent doing this sequentially would spend most of its token budget re-reading files. Multiple background agents doing this without coordination would produce inconsistent changes.

Coordinator mode solves this by splitting the “thinking” agent from the “doing” agents. The coordinator handles steps 1 and 2 (dispatching research workers, then synthesizing). Workers handle steps 3 and 4 (applying changes, running tests). The coordinator sees the full picture; workers see their specific task.

10.4.2 Activation

A single environment variable flips the switch:

```
export function isCoordinatorMode(): boolean {
  if (feature('COORDINATOR_MODE')) {
    return isEnvTruthy(process.env.CLAUDE_CODE_COORDINATOR_MODE)
  }
  return false
}
```

On session resume, `matchSessionMode()` checks whether the resumed session’s stored mode matches the current environment. If they diverge, the

environment variable is flipped to match. This prevents the confusing scenario where a coordinator session resumes as a regular agent (losing awareness of its workers) or a regular session resumes as a coordinator (losing access to its tools). The session’s mode is the source of truth; the environment variable is the runtime signal.

10.4.3 Tool Restrictions

The coordinator’s power comes not from having more tools, but from having fewer. In coordinator mode, the coordinator agent gets exactly three tools:

- **Agent** – spawn workers
- **SendMessage** – communicate with existing workers
- **TaskStop** – terminate running workers

That is it. No file reading. No code editing. No shell commands. The coordinator cannot directly touch the codebase. This restriction is not a limitation – it is the core design principle. The coordinator’s job is to think, plan, decompose, and synthesize. Workers do the work.

Workers, conversely, get the full tool set minus internal coordination tools:

```
const INTERNAL_WORKER_TOOLS = new Set([
  TEAM_CREATE_TOOL_NAME,
  TEAM_DELETE_TOOL_NAME,
  SEND_MESSAGE_TOOL_NAME,
  SYNTHETIC_OUTPUT_TOOL_NAME,
])
```

Workers cannot spawn their own sub-teams or send messages to peers. They report results through the normal task completion mechanism, and the coordinator synthesizes across them.

10.4.4 The 370-Line System Prompt

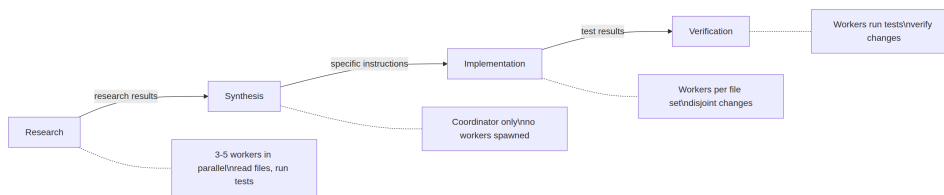
The coordinator system prompt is, line for line, the most instructive document in the codebase about how to use LLMs for orchestration. It runs approximately 370 lines and encodes hard-won lessons about delegation patterns. The key teachings:

“Never delegate understanding.” This is the central thesis. The coordinator must synthesize research findings into specific prompts with file paths, line numbers, and exact changes. The prompt explicitly calls out

anti-patterns like “based on your findings, fix the bug” – a prompt that delegates *comprehension* to the worker, forcing it to re-derive context the coordinator already has. The correct pattern is: “In `src/auth/validate.ts` at line 42, the `userId` parameter can be null when called from the OAuth flow. Add a null check that returns a 401 response.”

“Parallelism is your superpower.” The prompt establishes a clear concurrency model. Read-only tasks run freely in parallel – research, exploration, file reading. Write-heavy tasks serialize per file set. The coordinator is expected to reason about which tasks can overlap and which must sequence. A good coordinator spawns five research workers simultaneously, waits for all of them, synthesizes, then spawns three implementation workers that touch disjoint file sets. A bad coordinator spawns one worker, waits, spawns the next, waits again – serializing work that could have been parallel.

Task workflow phases. The prompt defines four phases:



1. **Research** – workers explore the codebase in parallel, reading files, running tests, gathering information
2. **Synthesis** – the coordinator (not a worker) reads all research results and builds a unified understanding
3. **Implementation** – workers receive precise instructions derived from the synthesis
4. **Verification** – workers run tests and verify the changes

The coordinator should not skip phases. The most common failure mode is jumping from research directly to implementation without synthesis. When this happens, the coordinator delegates understanding to the implementation workers – each one must re-derive context from scratch, leading to inconsistent changes and wasted tokens.

The continue-vs-spawn decision. When a worker finishes and the coordinator has follow-up work, should it send a message to the existing worker (via `SendMessage`) or spawn a fresh one (via `Agent`)? The decision is a function of context overlap:

- **High overlap, same files:** Continue. The worker already has the file contents in its context, understands the patterns, and can build on its previous work. Spawning fresh would force re-reading the same files and re-deriving the same understanding.
- **Low overlap, different domain:** Spawn fresh. A worker that just investigated the authentication system carries 20,000 tokens of auth-specific context that is dead weight for a CSS refactoring task. Starting clean is cheaper.
- **High overlap but the worker failed:** Spawn fresh with explicit guidance about what went wrong. Continuing a failed worker often means fighting against confused context. A fresh start with “the previous attempt failed because X, avoid Y” is more reliable.
- **Follow-up requires the worker’s output:** Continue, with the output included in the `SendMessage`. The worker does not need to re-derive its own results.

Worker prompt writing and anti-patterns. The prompt teaches the coordinator how to write effective worker prompts and explicitly flags bad patterns:

Anti-pattern: *“Based on your research findings, implement the fix.”* This delegates comprehension. The worker was not the one who did the research – the coordinator read the research results.

Anti-pattern: *“Fix the bug in the auth module.”* No file paths, no line numbers, no description of the bug. The worker must search the entire codebase from scratch.

Anti-pattern: *“Make the same change to all the other files.”* Which files? What change? The coordinator knows; it should enumerate them.

Good pattern: *“In `src/auth/validate.ts` at line 42, the `userId` parameter can be null when called from `src/oauth/callback.ts:89`. Add a null check: if `userId` is null, return { `error: 'unauthorized', status: 401` }. Then update the test in `src/auth/__tests__/validate.test.ts` to cover the null case.”*

The cost of writing a specific prompt is borne once, by the coordinator. The benefit – a worker that executes correctly on the first try – is enormous. Vague prompts create a false economy: the coordinator saves 30 seconds of prompt writing and the worker wastes 5 minutes of exploration.

10.4.5 Worker Context

The coordinator injects information about available tools into its own context, so the model knows what workers can do:

```
export function getCoordinatorUserContext(mcpClients, scratchpadDir?) {
  return {
    workerToolsContext: `Workers spawned via Agent have access to: ${workerTools}`
      + (mcpClients.length > 0
        ? `\nWorkers also have MCP tools from: ${serverNames}` : '')
      + (scratchpadDir ? `\nScratchpad: ${scratchpadDir}` : '')
  }
}
```

The scratchpad directory (gated by the `tengu_scratch` feature flag) is a shared filesystem location where workers can read and write without permission prompts. It enables durable cross-worker knowledge sharing – one worker’s research notes become another worker’s input, mediated through the filesystem rather than through the coordinator’s token window.

This is significant because it solves a fundamental limitation of the coordinator pattern. Without a scratchpad, all information flows through the coordinator: Worker A produces findings, the coordinator reads them via `TaskOutput`, synthesizes them into Worker B’s prompt. The coordinator’s context window becomes the bottleneck – it must hold all intermediate results long enough to synthesize them. With a scratchpad, Worker A writes findings to `/tmp/scratchpad/auth-analysis.md`, and the coordinator tells Worker B: “Read the auth analysis at `/tmp/scratchpad/auth-analysis.md` and apply the pattern to the OAuth module.” The coordinator moves information by reference, not by value.

10.4.6 Mutual Exclusion with Fork

Coordinator mode and fork-based subagents are mutually exclusive:

```
export function isForkSubagentEnabled(): boolean {
  if (feature('FORK_SUBAGENT')) {
    if (isCoordinatorMode()) return false
    // ...
  }
}
```

The conflict is fundamental. Fork agents inherit the parent’s entire conversa-

tion context – they are cheap clones that share prompt cache. Coordinator workers are independent agents with fresh context and specific instructions. These are opposing philosophies of delegation, and the system enforces the choice at the feature flag level.

10.5 The Swarm System

Coordinator mode is hierarchical: one manager, many workers, top-down control. The swarm system is the peer-to-peer alternative – multiple Claude Code instances working as a team, with a leader coordinating multiple teammates through message passing.

10.5.1 Team Context

Teams are identified by a `teamName` and tracked in `AppState.teamContext`:

```
teamContext?: {
  teamName: string
  teammates: {
    [id: string]: { name: string; color?: string; ... }
  }
}
```

Each teammate gets a name (for addressing) and a color (for visual distinction in the UI). The team file is persisted on disk so that team membership survives process restarts.

10.5.2 Agent Name Registry

Background agents can be given names at spawn time, which makes them addressable by human-readable identifiers instead of random task IDs:

```
if (name) {
  rootSetAppState(prev => {
    const next = new Map(prev.agentNameRegistry)
    next.set(name, asAgentId(asyncAgentId))
    return { ...prev, agentNameRegistry: next }
  })
}
```

The `agentNameRegistry` is a `Map<string, AgentId>`. When `SendMessage` resolves a `to` field, the registry is checked first:

```
const registered = appState.agentNameRegistry.get(input.to)
const agentId = registered ?? toAgentId(input.to)
```

This means you can send a message to "researcher" instead of `a7j3n9p2`. The indirection is simple but it enables the coordinator to think in terms of roles rather than IDs – a significant improvement for the model's ability to reason about multi-agent workflows.

10.5.3 In-Process Teammates

In-process teammates run in the same Node.js process as the leader, isolated via `AsyncLocalStorage`. Their state extends the base with team-specific fields:

```
export type InProcessTeammateTaskState = TaskStateBase & {
  type: 'in_process_teammate'
  identity: TeammateIdentity
  prompt: string
  messages?: Message[] // Capped at 50
  pendingUserMessages: string[]
  isIdle: boolean
  shutdownRequested: boolean
  awaitingPlanApproval: boolean
  permissionMode: PermissionMode
  onIdleCallbacks?: Array<() => void>
  currentWorkAbortController?: AbortController
}
```

The `messages` cap at 50 entries deserves explanation. During development, analysis revealed that each in-process agent accumulates approximately 20MB of RSS at 500+ turns. Whale sessions – power users running extended workflows – were observed launching 292 agents in 2 minutes, driving RSS to 36.8GB. The 50-message cap for the UI representation is a memory safety valve. The agent's actual conversation continues with full history; only the UI-facing snapshot is truncated.

The `isIdle` flag enables a work-stealing pattern. An idle teammate is not consuming tokens or API calls – it is simply waiting for the next message. The `onIdleCallbacks` array lets the system hook into the transition from

active to idle, enabling orchestration patterns like “wait for all teammates to finish, then proceed.”

The `currentWorkAbortController` is distinct from the teammate’s main abort controller. Aborting the current work controller cancels the teammate’s ongoing turn but does not kill the teammate. This enables a “redirect” pattern: the leader sends a higher-priority message, the teammate’s current work is aborted, and the teammate picks up the new message. The main abort controller, when aborted, kills the teammate entirely. Two levels of interruption for two levels of intent.

The `shutdownRequested` flag implements cooperative termination. When the leader sends a shutdown request, this flag is set. The teammate can check it at natural stopping points and wind down gracefully – finishing its current file write, committing its changes, or sending a final status update. This is gentler than a hard kill, which might leave files in an inconsistent state.

10.5.4 The Mailbox

Teammates communicate via a file-based mailbox system. When `SendMessage` targets a teammate, the message is written to the recipient’s mailbox file on disk:

```
await writeToMailbox(recipientName, {
  from: senderName,
  text: content,
  summary,
  timestamp: new Date().toISOString(),
  color: senderColor,
}, teamName)
```

Messages can be plain text, structured protocol messages (shutdown requests, plan approvals), or broadcasts (`to: "*"` sends to all team members excluding the sender). A poller hook processes incoming messages and routes them into the teammate’s conversation.

The file-based approach is deliberately simple. There is no message broker, no event bus, no shared memory channel. Files are durable (surviving process crashes), inspectable (you can `cat` a mailbox), and cheap (no infrastructure dependencies). For a system where message volumes are measured in tens per session, not thousands per second, this is the right trade-off. A Redis-backed

message queue would add operational complexity, a dependency, and failure modes – all for a throughput requirement that a filesystem call handles trivially.

The broadcast mechanism deserves a note. When a message is sent to "*", the sender iterates all team members from the team file, skips itself (case-insensitive comparison), and writes to each member's mailbox individually:

```
for (const member of teamFile.members) {
  if (member.name.toLowerCase() === senderName.toLowerCase()) continue
  recipients.push(member.name)
}
for (const recipientName of recipients) {
  await writeToMailbox(recipientName, { from: senderName, text: content, ... }, teamName)
}
```

There is no fan-out optimization – each recipient gets a separate file write. Again, at the scale of agent teams (typically 3-8 members), this is perfectly adequate. If a team had 100 members, this would need rethinking. But the 50-message memory cap that prevents 36GB RSS scenarios also implicitly caps the effective team size.

10.5.5 Permission Forwarding

Swarm workers operate with restricted permissions but can escalate to the leader when they need approval for sensitive operations:

```
const request = createPermissionRequest({
  toolName, toolUseId, input, description, permissionSuggestions
})
registerPermissionCallback({ requestId, toolUseId, onAllow, onReject })
void sendPermissionRequestViaMailbox(request)
```

The flow is: worker hits a tool that requires permission, the bash classifier attempts auto-approval, and if that fails, the request is forwarded to the leader via the mailbox system. The leader sees the request in their UI and can approve or reject. The callback fires and the worker proceeds. This lets workers operate autonomously for safe operations while maintaining human oversight for dangerous ones.

10.6 Inter-Agent Communication: SendMessage

`SendMessageTool` is the universal communication primitive. It handles four distinct routing modes through a single tool interface, selected by the shape of the `to` field.

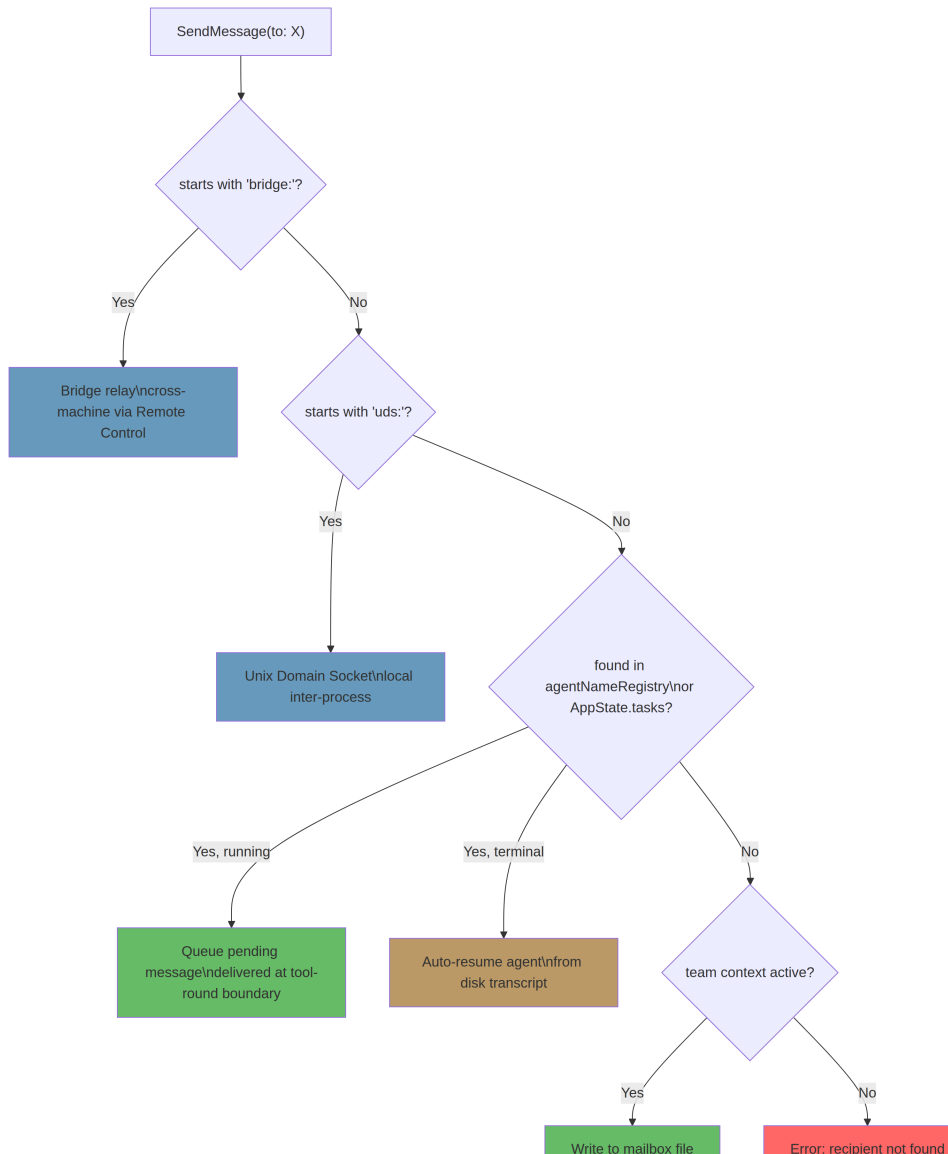
1001 Transit Clubhouse

```
inputSchema = z.object({
  to: z.string(),
  // "teammate-name", "*", "uds:<socket>", "bridge:<session-id>"
  summary: z.string().optional(),
  message: z.union([
    z.string(),
    z.discriminatedUnion('type', [
      z.object({ type: z.literal('shutdown_request'), reason: z.string().optional(),
        },
      z.object({ type: z.literal('shutdown_response'), request_id, approve, reason: z.string().optional(),
        },
      z.object({ type: z.literal('plan_approval_response'), request_id, approve,
        },
    ]),
  ]),
})
```

The `message` field is a union of plain text and structured protocol messages. This means `SendMessage` serves double duty – it is both the informal chat channel (“here are my findings”) and the formal protocol layer (“I approve your plan” / “please shut down”).

10.6.2 Routing Dispatch

The `call()` method follows a priority-ordered dispatch chain:



1. Bridge messages (bridge:<session-id>). Cross-machine communication via Anthropic’s Remote Control servers. This is the widest reach – two Claude Code instances on different machines, potentially different continents, communicating through a relay. The system requires explicit user consent before sending bridge messages – a safety check that prevents one agent from unilaterally establishing communication with a remote instance. Without this gate, a compromised or confused agent could exfiltrate information to a

remote session. The consent check uses `postInterClaudeMessage()`, which handles serialization and transport over the Remote Control relay.

2. UDS messages (`uds:<socket-path>`). Local inter-process communication via Unix Domain Sockets. This is for Claude Code instances running on the same machine but in different processes – for example, a VS Code extension hosting one instance and a terminal hosting another. UDS communication is fast (no network round-trip), secure (filesystem permissions control access), and reliable (the kernel handles delivery). The `sendToUdsSocket()` function serializes the message and writes it to the socket path specified in the `to` field. Peers discover each other via a `ListPeers` tool that scans for active UDS endpoints.

3. In-process subagent routing (plain name or agent ID). This is the most common path. The routing logic:

- Look up `input.to` in the `agentNameRegistry`
- If found and running: `queuePendingMessage()` – the message waits for the next tool-round boundary
- If found but in a terminal state: `resumeAgentBackground()` – the agent is transparently restarted
- If not in `AppState`: attempt to resume from the disk transcript

4. Team mailbox (fallback when team context is active). Named recipients get messages written to their mailbox files. The `"*"` wildcard triggers a broadcast to all team members.

10.6.3 Structured Protocols

Beyond plain text, `SendMessage` carries two formal protocols.

The shutdown protocol. The leader sends `{ type: 'shutdown_request', reason: '...' }` to a teammate. The teammate responds with `{ type: 'shutdown_response', request_id, approve: true/false, reason }`. If approved, in-process teammates abort their controller; tmux-based teammates receive a `gracefulShutdown()` call. The protocol is cooperative – a teammate can reject a shutdown request if it is in the middle of critical work, and the leader must handle that case.

The plan approval protocol. Teammates operating in plan mode must get approval before executing. They submit a plan, and the leader responds with `{ type: 'plan_approval_response', request_id, approve, feedback }`. Only the team lead can issue approvals. This creates a review

gate – the leader can examine a worker’s intended approach before any files are touched, catching misunderstandings early.

10.6.4 The Auto-Resume Pattern

The most elegant feature of the routing system is transparent agent resumption. When `SendMessage` targets a completed or killed agent, instead of returning an error, it resurrects the agent:

```
if (task.status !== 'running') {
  const result = await resumeAgentBackground({
    agentId,
    prompt: input.message,
    toolUseContext: context,
    canUseTool,
  })
  return {
    data: {
      success: true,
      message: `Agent "${input.to}" was stopped; resumed with your message`
    }
  }
}
```

The `resumeAgentBackground()` function reconstructs the agent from its disk transcript:

1. Reads the sidechain JSONL transcript
2. Reconstructs the message history, filtering orphaned thinking blocks and unresolved tool uses
3. Rebuilds the content replacement state for prompt cache stability
4. Resolves the original agent definition from stored metadata
5. Re-registers as a background task with a fresh abort controller
6. Calls `runAgent()` with the restored history plus the new message as prompt

From the coordinator’s perspective, sending a message to a dead agent and sending a message to a live agent are the same operation. The routing layer handles the complexity. This means coordinators do not need to track which agents are alive – they simply send messages and the system figures it out.

The implications are significant. Without auto-resume, the coordinator

would need to maintain a mental model of agent liveness: “Is **researcher** still running? Let me check. It completed. I need to spawn a new agent. But wait, should I use the same name? Will it have the same context?” With auto-resume, all of that collapses to: “Send **researcher** a message.” If it is alive, the message is queued. If it is dead, it is resurrected with its full history. The coordinator’s prompt complexity drops dramatically.

There is a cost, of course. Resuming from a disk transcript means re-reading potentially thousands of messages, reconstructing internal state, and making a new API call with a full context window. For a long-lived agent, this can be expensive in both latency and tokens. But the alternative – requiring the coordinator to manually manage agent lifecycles – is worse. The coordinator is an LLM. It is good at reasoning about problems and writing instructions. It is bad at bookkeeping. Auto-resume plays to the LLM’s strengths by eliminating a category of bookkeeping entirely.

10.7 TaskStop: The Kill Switch

`TaskStopTool` is the complement to `Agent` and `SendMessage` – it terminates running tasks:

```
inputSchema = z.strictObject({
  task_id: z.string().optional(),
  shell_id: z.string().optional(), // Deprecated backward compat
})
```

The implementation delegates to `stopTask()`, which dispatches based on task type:

1. Look up the task in `AppState.tasks`
2. Call `getTaskByType(task.type).kill(taskId, setAppState)`
3. For agents: abort the controller, set status to 'killed', start the eviction timer
4. For shells: kill the process group

The tool has a legacy alias `"KillShell"` – a reminder that the task system evolved from simpler origins where the only background operation was a shell command.

The kill mechanism varies by task type, but the pattern is consistent. For agents, killing means aborting the abort controller (which causes the `query()`

loop to exit at the next yield point), setting the status to 'killed', and starting an eviction timer so the task state is cleaned up after a grace period. For shells, killing means sending a signal to the process group – **SIGTERM** first, then **SIGKILL** if the process does not exit within a timeout. For in-process teammates, killing also triggers a shutdown notification to the team so other members know the teammate is gone.

The eviction timer is worth noting. When an agent is killed, its state is not immediately purged. It lingers in `AppState.tasks` for a grace period (controlled by `evictAfter`) so that the UI can show the killed status, any final output can be read, and auto-resume via `SendMessage` remains possible. After the grace period, the state is garbage collected. This is the same pattern used for completed tasks – the system distinguishes between “finished” (result available) and “forgotten” (state purged).

10.8 Choosing Between Patterns

(A note on naming: the codebase also contains `TaskCreate/TaskGet/TaskList/TaskUpdate` tools that manage a structured todo list – a completely separate system from the background task state machine described here. `TaskStop` operates on `AppState.tasks`; `TaskUpdate` operates on a project tracking data store. The naming overlap is historical and a recurring source of model confusion.)

With three orchestration patterns available – background delegation, coordinator mode, and swarm teams – the natural question is when to use each.

Simple delegation (Agent tool with `run_in_background: true`) is appropriate when the parent has one or two independent tasks to offload. Run the tests in the background while continuing to edit. Search the codebase while waiting for a build. The parent stays in control, checks results when ready, and never needs a complex communication protocol. The overhead is minimal – one task state entry, one disk output file, one notification on completion.

Coordinator mode is appropriate when the problem decomposes into a research phase, a synthesis phase, and an implementation phase – and when the coordinator needs to reason across the results of multiple workers before directing the next step. The coordinator cannot touch files, which forces clean separation of concerns: thinking happens in one context, doing happens

in another. The 370-line system prompt is not ceremony – it encodes patterns that prevent the most common failure mode of LLM delegation, which is delegating comprehension instead of delegating action.

Swarm teams are appropriate for long-running collaborative sessions where agents need peer-to-peer communication, where the work is ongoing rather than batch-oriented, and where agents may need to idle and resume based on incoming messages. The mailbox system supports asynchronous patterns that coordinator mode (which is synchronous spawn-wait-synthesize) does not. Plan approval gates add a review layer. Permission forwarding maintains security without requiring every agent to have full privileges.

A practical decision table:

Scenario	Pattern	Why
Run tests while editing	Simple delegation	One background task, no coordination needed
Search codebase for all usages	Simple delegation	Fire-and-forget, read output when done
Refactor 40 files across 3 modules	Coordinator	Research phase finds patterns, synthesis plans changes, workers execute in parallel per module
Multi-day feature development with review gates	Swarm	Long-lived agents, plan approval protocol, peer communication

Scenario	Pattern	Why
Fix a bug with known location	Neither – single agent	Orchestration overhead exceeds the benefit for focused, sequential work
Migrate database schema + update API + update frontend	Coordinator	Three independent workstreams after a shared re-search/planning phase
Pair programming with user oversight	Swarm with plan mode	Worker proposes, leader approves, worker executes

The patterns are not mutually exclusive in principle, but they are in practice. Coordinator mode disables fork subagents. Swarm teams have their own communication protocol that does not mix with coordinator task notifications. The choice is made at session startup via environment variables and feature flags, and it shapes the entire interaction model.

One final observation: the simplest pattern is almost always the right starting point. Most tasks do not need coordinator mode or swarm teams. A single agent with occasional background delegation handles the vast majority of development work. The sophisticated patterns exist for the 5% of cases where the problem is genuinely wide, genuinely parallel, or genuinely long-running. Reaching for coordinator mode on a single-file bug fix is like deploying Kubernetes for a static website – technically possible, architecturally inappropriate.

10.9 The Cost of Orchestration

Before examining what the orchestration layer reveals philosophically, it is worth acknowledging what it costs practically.

Every background agent is a separate API conversation. It has its own context window, its own token budget, and its own prompt cache slot. A coordinator that spawns 5 research workers is making 6 concurrent API calls, each with its own system prompt, tool definitions, and CLAUDE.md injection. The token overhead is not trivial – the system prompt alone can be thousands of tokens, and each worker re-reads files that other workers may have already read.

The communication channels add latency. Disk output files require filesystem I/O. Task notifications are delivered at tool-round boundaries, not instantly. The command queue introduces a full round-trip delay – the coordinator sends a message, the message waits for the worker to finish its current tool use, the worker processes the message, and the result is written to disk for the coordinator to read.

The state management adds complexity. Seven task types, five statuses, and dozens of fields per task state. The eviction logic, the garbage collection timers, the memory caps – all of this exists because unbounded state growth caused real production incidents (36.8GB RSS).

None of this means orchestration is wrong. It means orchestration is a tool with a cost, and the cost should be weighed against the benefit. Running 5 parallel workers to search a codebase is worthwhile when the search would take 5 sequential minutes. Running a coordinator to fix a typo in one file is pure overhead.

10.10 What the Orchestration Layer Reveals

The most interesting aspect of this system is not any individual mechanism – task states, mailboxes, and notification XML are all straightforward engineering. What is interesting is the *design philosophy* that emerges from how they fit together.

The coordinator prompt’s “never delegate understanding” is not just good advice for LLM orchestration. It is a statement about the fundamental

limitation of context-window-based reasoning. A worker with a fresh context window cannot understand what the coordinator understood after reading 50 files and synthesizing three research reports. The only way to bridge that gap is for the coordinator to distill its understanding into a specific, actionable prompt. Vague delegation is not just inefficient – it is information-theoretically lossy.

The auto-resume pattern in `SendMessage` reveals a preference for *apparent simplicity over actual simplicity*. The implementation is complex – reading disk transcripts, reconstructing content replacement state, re-resolving agent definitions. But the interface is trivial: send a message, and it works regardless of whether the recipient is alive or dead. The complexity is absorbed by the infrastructure so that the model (and the user) can reason in simpler terms.

And the 50-message memory cap on in-process teammates is a reminder that orchestration systems operate under real physical constraints. 292 agents in 2 minutes reaching 36.8GB of RSS is not a theoretical concern – it happened in production. The abstractions are elegant, but they run on hardware with finite memory, and the system must degrade gracefully when users push it to extremes.

There is also a lesson in the layered architecture itself. The task state machine is agnostic – it does not know about coordinators or swarms. The communication channels are agnostic – `SendMessage` does not know whether it is being called by a coordinator, a swarm leader, or a standalone agent. The coordinator prompt is layered on top, adding methodology without changing the underlying machinery. Each layer can be understood independently, tested independently, and evolved independently. When the team added the swarm system, they did not need to modify the task state machine. When they added the coordinator prompt, they did not need to modify `SendMessage`.

This is the hallmark of well-factored orchestration: the primitives are general, and the patterns are composed from them. A coordinator is just an agent with restricted tools and a detailed system prompt. A swarm leader is just an agent with a team context and mailbox access. A background worker is just an agent with an independent abort controller and a disk output file. The seven task types, five statuses, and four routing modes combine to produce orchestration patterns that are greater than the sum of their parts.

The orchestration layer is where Claude Code stops being a single-threaded

tool executor and becomes something closer to a development team. The task state machine provides the bookkeeping. The communication channels provide the information flow. The coordinator prompt provides the methodology. And the swarm system provides the peer-to-peer topology for problems that do not fit a strict hierarchy. Together, they make it possible for a language model to do what no single model invocation can: work on wide problems, in parallel, with coordination.

The next chapter examines the permission system – the safety layer that determines which of these agents can do what, and how dangerous operations are escalated from workers to humans. Orchestration without permission controls would be a force multiplier for mistakes. The permission system ensures that more agents means more capability, not more risk.

Part IV

Part IV: Persistence and Intelligence

Chapter 11

Chapter 11: Memory – Learning Across Conversations

11.1 The Stateless Problem

Every chapter so far has described machinery that exists within a single session. The agent loop runs, tools execute, sub-agents coordinate, and when the process exits, all of it vanishes. The next conversation starts with the same system prompt, the same tool definitions, the same model – and zero knowledge of what happened before.

This is the fundamental limitation of a stateless architecture. A developer corrects the model’s testing approach on Monday, and on Tuesday the model makes the same mistake. A user explains their role, their project’s constraints, their preferences for code style, and every new session requires them to explain it again. The model is not forgetful – it never knew. Each conversation is an independent universe.

The problem is not theoretical. It manifests in concrete ways that erode trust. A user says “remember, we use real database instances in tests, not mocks” – and next week the model generates mocked tests. A user explains they are a senior engineer who does not need beginner explanations – and the next session opens with a tutorial-level walkthrough. Without memory, every session starts at zero. The agent is perpetually a new hire on their

first day.

The standard solution in the industry is Retrieval-Augmented Generation (RAG): embed documents into vectors, store them in a vector database, and retrieve relevant chunks at query time. This works well for knowledge bases – documentation, FAQs, reference material. But it is architecturally mismatched for what an agent actually needs to remember across sessions. An agent’s memory is not a knowledge base. It is a collection of observations: who the user is, what they have corrected, what the project’s current constraints are, where to find things. These observations are small, change frequently, and must be human-editable. A vector database solves the wrong problem.

Claude Code’s memory system is a different bet entirely: files on disk, Markdown format, LLM-powered recall, no infrastructure. The bet is that simplicity in storage, combined with intelligence in retrieval, produces a better system than sophistication in both.

The design philosophy has consequences that shape the entire system:

- **Human-readable.** A user who wants to see what Claude Code remembers can open `~/.claude/projects/<slug>/memory/MEMORY.md` in any text editor. No special tools, no decryption, no export command.
- **Human-editable.** A stale memory can be corrected with `vim`. A wrong memory can be deleted with `rm`. The user has full agency over the agent’s knowledge.
- **Version-controllable.** Team memories can be committed to `git`. Memory changes diff cleanly because they are Markdown.
- **Zero infrastructure.** The memory system works offline, works without a server, works on any OS that has a filesystem. There is no migration path because there is no schema.
- **Debuggable.** When memory behaves unexpectedly, the diagnosis path is `ls` and `cat`, not query logs and database inspection.

The model both reads and writes memories using `FileWriteTool` and `FileEditTool` – the same tools it uses to edit source code (introduced in Chapter 6). No special memory API exists. The system prompt teaches the model a two-step write protocol (create file, update index), and the model executes it with its existing capabilities under new instructions. This is tool reuse as architectural principle – the memory system is not a subsystem bolted onto the agent, it is an emergent behavior of the agent using its existing capabilities.

There is a deeper reason the file-based choice works here. Memory, for an AI

agent, is fundamentally different from memory in a traditional application. A traditional application’s database holds authoritative state – the source of truth for the system’s data. An agent’s memory holds *observations* – things that were true at a point in time and may or may not still be true. Files communicate this epistemological status naturally. They have modification times that reveal when the observation was recorded. They can be read, edited, and deleted by humans who know the observation is wrong. A database suggests permanence and authority; a Markdown file suggests a note that someone wrote down and might need to update. The storage medium communicates the nature of the data – these are working notes, not gospel.

11.1.1 Per-Project Scoping

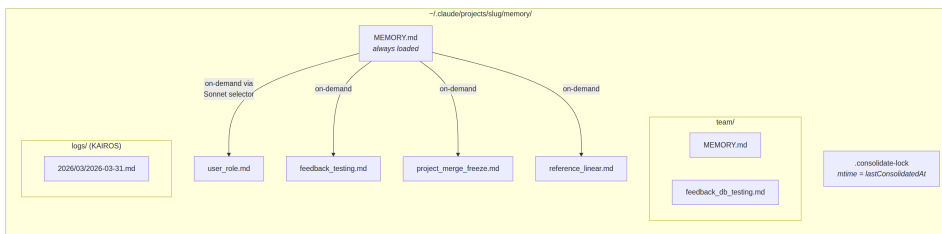
Memory is scoped to the git repository root, not the working directory. If a user opens a terminal in `src/components/` and another in `tests/`, both sessions share the same memory directory. The resolution logic finds the canonical git root first, falling back to the project root:

The base path resolution finds the canonical git root first, falling back to the project root. This ensures all git worktrees of the same repository share a single memory directory.

The `findCanonicalGitRoot` call ensures that all git worktrees of the same repository share a single memory directory. The git root is sanitized (slashes become dashes, via `sanitizePath()`) to produce a flat directory name:

```
~/ .claude/projects/-Users-alex-code-myapp/memory/
```

A fully populated memory directory reveals the system’s structure:



The naming convention is semantic: `<type>_<topic>.md`. The type prefix is not enforced by code but is part of the prompt’s instructions, making it easy to visually scan the directory and understand the memory landscape.

11.2 The Four-Type Taxonomy

Not everything is worth remembering. The memory system constrains all memories to exactly four types:

The four types are: **user**, **feedback**, **project**, and **reference**.

The taxonomy is designed around a single criterion: **is this knowledge derivable from the current project state?** Code patterns, architecture, file structure, git history – all of these can be re-derived by reading the codebase. They are excluded. The four types capture what cannot be re-derived.

User memories record information about the person: their role, goals, responsibilities, expertise level. A senior Go engineer who is new to React gets different explanations than a first-time programmer.

Feedback memories capture guidance about how to approach work – both corrections and confirmations. The system explicitly instructs the model to record both: “if you only save corrections, you will drift away from approaches the user has already validated.” Each feedback memory has a specific structure: the rule itself, then a ****Why:**** line with the reason (often a past incident), then a ****How to apply:**** line with the trigger conditions.

Project memories record ongoing work context – who is doing what, why, by when. The prompt emphasizes converting relative dates to absolute: “Thursday” becomes “2026-03-05” so the memory remains interpretable weeks later.

Reference memories are bookmarks – pointers to where information lives in external systems. A Linear project URL, a Grafana dashboard, a Slack channel. These tell the model where to look, not what to find.

11.2.1 The Taxonomy as Filter

The four types are not just categories – they are a filter. By defining exactly what counts as a memory, the system implicitly defines what does not. Without the taxonomy, an eager model would save everything: code patterns, architecture diagrams, error messages. All derivable from the codebase. Saving it creates a parallel, potentially stale copy of information that is better sourced from its origin.

The taxonomy also prevents a subtler failure: memory as crutch. If the model saves architectural decisions as memories, it stops reading the codebase to

understand architecture. By excluding derivable information, the system forces the model to stay grounded in the current state of the code.

The exclusion list is explicit: code patterns, git history, debugging solutions, anything in CLAUDE.md, ephemeral task details. These exclusions apply even when the user explicitly asks to save. If a user says “remember this PR list,” the model is instructed to push back – “what was *surprising* or *non-obvious* about it?” That surprising part is worth keeping. The raw list is not. This instruction was validated through evals, going from 0/2 to 3/3 when the exclusion-override instruction was added.

11.2.2 Frontmatter as Contract

Every memory file uses YAML frontmatter with three required fields:

```
---
name: {{memory name}}
description: {{one-line description -- used to decide relevance}}
type: {{user, feedback, project, reference}}
---
```

The **description** is the most load-bearing field. It is what the relevance selector (a Sonnet side-query, discussed below) uses to decide whether to surface this memory. A vague description like “testing stuff” will either match too broadly or fail to match at all. A specific description like “Integration tests must hit real DB, not mocks – burned by mock divergence Q4” matches exactly the conversations where it matters. The description is the memory’s search index – consumed not by a search engine but by a language model that can understand nuance, context, and intent.

The frontmatter is also the only part of the file that the scanning system reads during recall. `scanMemoryFiles()` reads each file only to its first 30 lines to extract the header. The body is private until the file is explicitly selected and loaded.

11.3 The Write Path

Writing a memory is a two-step process executed with standard file tools.

Step 1: Write the memory file. The model creates a `.md` file in the memory directory with YAML frontmatter:

```

---
name: Testing Policy
description: Integration tests must hit real DB, not mocks
type: feedback
---

Don't mock the database in integration tests.

**Why:** We got burned last quarter when mocked tests passed but production
queries hit edge cases the mocks didn't cover.

**How to apply:** Any test file under `__tests__` that touches database
operations should use the real PGlite instance from test-utils.

```

Step 2: Update the index. The model adds a one-line pointer to `MEMORY.md`:

```
- [Testing Policy](feedback_testing.md) -- integration tests must hit real DB
```

Each entry must stay under approximately 150 characters. The index is a table of contents, not a knowledge base.

When the model learns new information that modifies an existing memory, it uses `FileEditTool` to update the existing file rather than creating a duplicate. The system does not version memories internally – the file is on the local filesystem, and the user has `git` if they want versioning. Before the prompt is built, `ensureMemoryDirExists()` creates the memory directory, and the prompt tells the model the directory already exists, avoiding wasted turns on `ls` and `mkdir -p`.

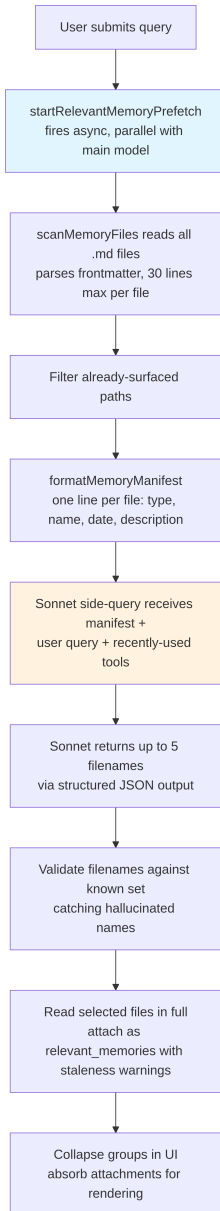
11.4 The Recall Path

Writing memories is necessary but not sufficient. The harder problem is retrieval: given a user's query, which of the potentially hundreds of memory files should be loaded into the model's context? Loading all of them would exhaust the token budget. Loading none would defeat the purpose. Loading the wrong ones would waste tokens on irrelevant information while missing the knowledge that would have changed the model's behavior.

The recall system operates in two tiers. The `MEMORY.md` index is always

loaded into context at session start, providing orientation. Individual memory files are surfaced on-demand through an LLM-powered relevance query that selects up to five memories per turn.

11.4.1 The Full Recall Pipeline



The async prefetch in step 2 is the key performance decision. By the time the main model reaches a point where recalled context would be useful, the side-query has usually already completed. The user experiences no additional latency.

11.4.2 The Sonnet Side-Query

The manifest is sent to a Sonnet model as a side-query. The system prompt for this selector is precise:

The system prompt for the selector instructs it to be conservative: include only memories that will be useful for the current query, skip memories if uncertain, and avoid selecting API/usage documentation for tools already in active use (since the model already has those tools loaded) – but still surface warnings, gotchas, or known issues about those tools.

The response uses structured output – `{ selected_memories: string[] }` – and filenames are validated against the known set.

This approach trades latency for precision, and the tradeoff analysis is instructive. **Keyword matching** would be fast but has no understanding of context – it cannot express “do not select memories for tools already in active use.” **Embedding similarity** handles semantic matching but introduces infrastructure (embedding model, vector store, update pipeline) and struggles with negation – the embedding of “do NOT use database mocks” is very close to “use database mocks.” **The Sonnet side-query** understands semantic relevance, reasons about context, handles negation, and requires zero infrastructure. The latency cost is bounded (hundreds of milliseconds) and hidden behind the main model’s initial processing.

The telemetry system tracks selection rates even when no memories are selected. A selection rate of 0/150 means something different from 0/3 – the first indicates a precision problem, the second a coverage problem.

11.5 Staleness

The staleness system addresses a failure mode that emerged from real usage. Users reported that old memories – containing file:line citations to code that had since changed – were being asserted as fact by the model. The citation made the stale claim sound *more* authoritative, not less.

The solution is not expiration. Old memories are not deleted – they may contain institutional knowledge valid for years. Instead, the system attaches age warnings:

The staleness function computes the memory’s age in days. Memories from today or yesterday get no warning (the function returns an empty string). Everything older gets a caveat injected alongside the memory content: a message stating the age in days and warning that code behavior claims or file:line citations may be outdated, advising verification against current code.

Memories from today or yesterday get no warning. Everything older gets a staleness caveat injected alongside the memory content. The human-readable format – “today,” “yesterday,” “47 days ago” – exists because models are poor at date arithmetic. A raw ISO timestamp does not trigger staleness reasoning the way “47 days ago” does. This is an empirical observation about model behavior, validated through evals: the action-cue framing “Before recommending from memory” scored 3/3 versus 0/3 for the more abstract “Trusting what you recall,” with identical body text.

There is a philosophical tension worth naming. The staleness system treats memories as hypotheses, not facts. But the model’s natural tendency is to present information confidently. The staleness warning is fighting the model’s own voice – using its instruction-following capability to override its confidence-generation tendency.

11.6 MEMORY.md as the Always-Loaded Index

Every conversation begins with `MEMORY.md` in context. It is not a memory – it is an index, a table of contents for the actual memory files.

The index has two hard caps:

The index has two hard caps: 200 lines and 25,000 bytes.

The 200-line cap catches normal growth. The 25KB byte cap catches an observed failure mode: users packing long lines that stay under 200 lines but consume enormous token budgets. At the 97th percentile, a `MEMORY.md` with only 197 lines weighed 197KB. When either cap fires, actionable guidance tells the user what to fix: “Keep index entries to one line under ~200 chars; move detail into topic files.”

This two-tier architecture – lightweight always-on index plus heavy on-demand content – is the design that allows memory to scale. A project with 150 memories has a 150-line index consuming perhaps 3,000 tokens, not 150 full files consuming 100,000.

The transition from individual memory to shared knowledge is natural. A testing policy, a deployment convention, a known gotcha in the build system – these need to be shared across a team.

11.7 Team Memory

Team memory is a subdirectory of the auto-memory directory at `<autoMemPath>/team/`, gated behind a feature flag and requiring auto-memory to be enabled. The architectural nesting is deliberate: disabling auto-memory transitively disables team memory.

11.7.1 Defense in Depth

Team memory introduces an attack surface that individual memory does not have. Team-synced files come from other users, and a malicious teammate could attempt path traversal. The security model uses three layers of defense.

Layer 1: Input sanitization. The `sanitizePathKey()` function validates against null bytes, URL-encoded traversals (`%2e%2e%2f`), Unicode normalization attacks (fullwidth characters that normalize to `../`), backslashes, and absolute paths.

Layer 2: String-level path validation. After sanitization, `path.resolve()` normalizes remaining `..` segments, and the resolved path is checked against the team directory prefix (including a trailing separator to prevent `team-evil/` from matching `team/`).

Layer 3: Symlink resolution. `realpathDeepestExisting()` resolves symlinks on the deepest existing ancestor, catching attacks that string-level validation cannot detect. If `team/evil` is a symlink pointing to `/etc/`, string validation sees a valid prefix, but `realpath` reveals the true target.

All validation failures produce a `PathTraversalError`. No partial successes, no fallbacks. Fail closed.

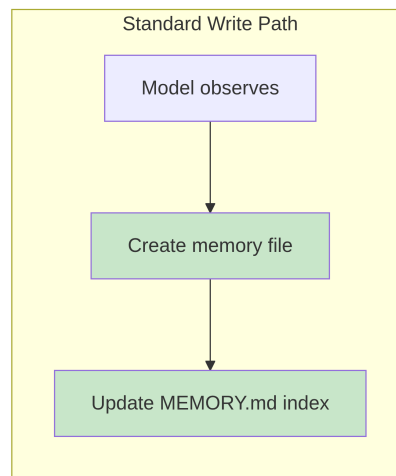
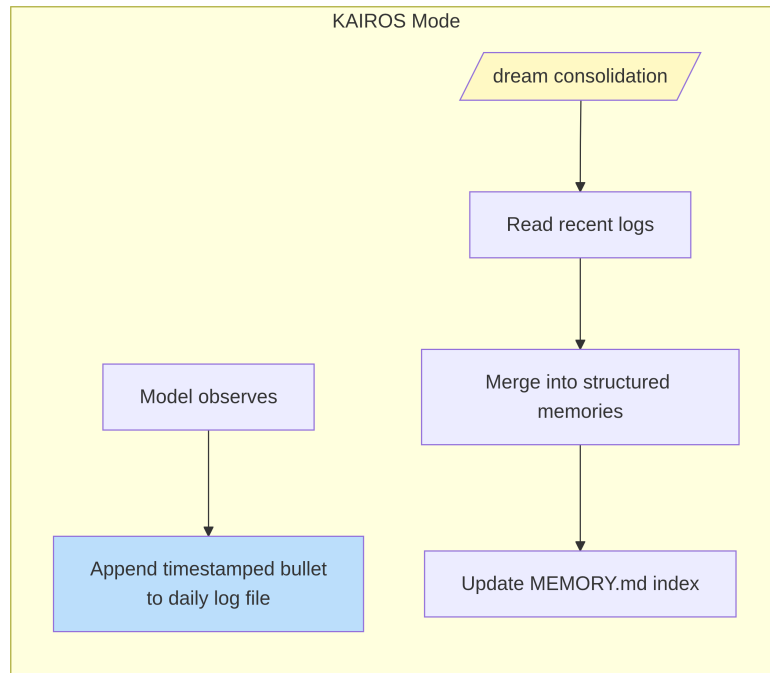
11.7.2 Scope Guidance

The prompt teaches the model about private vs. shared memory. User memories are always private. Reference memories are usually team. Feedback memories default to private unless they represent project-wide conventions. The cross-checking instruction – “Before saving a private feedback memory, check that it does not contradict a team feedback memory” – prevents conflicting guidance from surfacing unpredictably depending on which memory is recalled first.

11.8 KAIROS Mode: Append-Only Daily Logs

Standard memory assumes discrete sessions. KAIROS mode (Claude Code’s assistant mode) breaks this assumption – sessions are long-lived, potentially running for days. The two-step write pattern does not scale to continuous operation.

The solution is architectural separation between capture and consolidation:



In KAIROS mode, the model appends to date-named log files (`<autoMemPath>/logs/YYYY/MM/YYYY-MM-DD.md`). Each entry is a short timestamped bullet. The model is instructed: “Do not rewrite or reorganize the log” – restructuring during capture loses the chronological signal that consolidation needs.

The path in the prompt is described as a *pattern* rather than today’s literal date. This is a caching optimization: the memory prompt is cached and not invalidated when the date changes at midnight. The model derives the current date from a separate `date_change` attachment.

11.8.1 The /dream Consolidation

Consolidation runs in four phases: **Orient** (list directory, read index, skim existing files), **Gather** (search logs, check for drifted memories), **Consolidate** (write or update files, merge rather than duplicate), **Prune** (update index under 200 lines, remove stale pointers). The emphasis on merging into existing files rather than creating new ones is important – without it, the memory directory would grow linearly with usage.

11.8.2 The Consolidation Lock

The lock file `.consolidate-lock` serves dual purpose: its content is the holder’s PID (mutual exclusion), its mtime *is* `lastConsolidatedAt` (scheduling state). The auto-dream fires when three gates pass, evaluated cheapest-first: hours since last consolidation exceeds 24, sessions modified since then exceeds 5, and no other process holds the lock. Crash recovery detects dead PIDs via `process.kill(pid, 0)`, with a one-hour staleness timeout as defense against PID reuse.

11.9 Background Extraction

The main agent has full instructions for writing memories proactively. But agents are imperfect – and the imperfection is predictable. When a user says “remember to always use integration tests” and then immediately asks “now fix the login bug,” the model’s attention shifts entirely to the bug. The memory-saving instruction was processed but may not execute.

At the end of each complete query loop, a forked agent – sharing the parent’s prompt cache – analyzes recent messages and writes any memories the main agent missed. When the main agent has already written memories in the current turn range, the extraction agent skips that range. The extraction agent has a constrained tool budget: read-only tools plus write access only to memory directory paths. Its prompt instructs a two-turn strategy: turn 1 reads in parallel, turn 2 writes in parallel.

The interaction is cooperative, not competitive. The main agent’s prompt always contains the full save instructions. When the main agent saves, the background agent defers. When it does not, the background agent catches the gap. This pattern – a primary path with a background safety net – makes memory capture more reliable without burdening the primary interaction. Neither alone would be sufficient.

11.10 Path Resolution and Security

The auto-memory path is resolved through a priority chain:

1. **CLAUDE_COWORK_MEMORY_PATH_OVERRIDE** – Full-path override for Cowork.
2. **autoMemoryDirectory** in **settings.json** – Only trusted settings sources. Project settings are intentionally excluded.
3. **Default computed path** – `~/.claude/projects/<sanitized-git-root>/memory/`.

The exclusion of project settings is a security decision. A malicious repository could commit `.claude/settings.json` with `autoMemoryDirectory: "~/ssh"`, and the permission carve-out for memory files would grant the model automatic write access to SSH keys. By limiting the override to policy, flag, local, and user settings – none committable to a repository – this attack vector is closed.

The `isAutoMemPath()` function normalizes paths before prefix-checking to prevent traversal, and the trailing separator convention ensures prefix matching requires a directory boundary.

11.10.1 The Enable/Disable Chain

Whether auto-memory is active is determined by `isAutoMemoryEnabled()`, implementing its own priority chain: environment variable, bare mode, CCR without persistent storage, settings, default enabled. When disabled, both the prompt section is dropped (so the model receives no memory instructions) and the background processes stop (extract-memories, auto-dream, team sync). Both gates must align – removing the prompt alone would not stop the extraction agent, which has its own prompt.

11.11 Apply This: Designing Agent Memory

The memory system’s complexity is in the behavioral layer – prompt instructions, LLM-powered recall, staleness management, background extraction – not in storage infrastructure. This distribution of complexity is itself a design principle.

Files beat databases for agent memory. Files are inspectable, editable, and version-controllable. Transparency builds trust. When the alternative is a database users cannot easily read, files win on trust alone.

Constrain what gets saved, not just how. The derivability test – can this knowledge be re-derived from the current project state? – eliminates the majority of potential memories while preserving the ones that actually matter.

Use an LLM for recall, not keywords or embeddings. An LLM side-query understands context, reasons about what is already available in conversation, handles negation, and requires no index maintenance. The latency cost is real but bounded and hidden behind the main model’s processing.

Warn about staleness, do not expire. Institutional knowledge may remain valid for years. Attaching age warnings lets the model treat old memories as hypotheses rather than facts. The human-readable age format triggers the right reasoning in a way that raw timestamps do not.

Build a safety net for capture. The main agent will miss memories. A background extraction agent that reviews recent conversation makes the system more reliable without burdening the primary interaction. When the main agent saves, the background agent defers.

The agent can now learn across sessions – accumulating knowledge about its user, their preferences, their project’s state, and the corrections they have made. The memory system makes a philosophical commitment: that an agent’s relationship with its user should deepen over time, not reset on every interaction. The file-based implementation makes that commitment tangible – visible on disk, editable by humans, version-controlled alongside code. The agent’s memory is not a black box. It is a collection of notes in a folder, written in a language that both the model and the human can read.

The next chapter examines how Claude Code extends its capabilities beyond

the core: the skills system that teaches the model new behaviors, and the hooks system that lets external code constrain and modify those behaviors at over two dozen lifecycle points.

Chapter 12

Chapter 12: Extensibility – Skills and Hooks

12.1 Two Dimensions of Extension

Every extensibility system answers two questions: what can the system do, and when does it do it. Most frameworks conflate the two – a plugin registers both capabilities and lifecycle callbacks in the same object, and the boundary between “adding a feature” and “intercepting a feature” blurs into a single registration API.

Claude Code separates them cleanly. Skills extend what the model can do. They are markdown files that become slash commands, injecting new instructions into the conversation when invoked. Hooks extend when and how things happen. They are lifecycle interceptors that fire at over two dozen distinct points during a session, running arbitrary code that can block actions, modify inputs, force continuation, or silently observe.

The separation is not accidental. Skills are content – they expand the model’s knowledge and capabilities by adding prompt text. Hooks are control flow – they modify the execution path without changing what the model knows. A skill might teach the model how to run your team’s deployment process. A hook might ensure no deployment command executes without a passing test suite. The skill adds capability; the hook adds constraint.

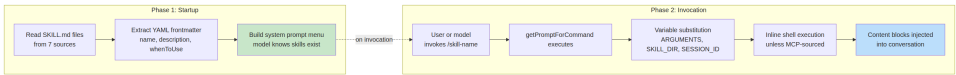
This chapter covers both systems in depth, then examines where they intersect: skill-declared hooks that register as session-scoped lifecycle interceptors

when the skill is invoked.

12.2 Skills: Teaching the Model New Tricks

12.2.1 Two-Phase Loading

The core optimization of the skills system is that frontmatter loads at startup, but full content loads only on invocation.



Phase 1 reads each `SKILL.md` file, splits YAML frontmatter from the markdown body, and extracts metadata. The frontmatter fields become part of the system prompt so the model knows the skill exists. The markdown body is captured in a closure but not processed. A project with 50 skills pays the token cost of 50 short descriptions, not 50 full documents.

Phase 2 fires when the model or user invokes a skill. `getPromptForCommand` prepends the base directory, substitutes variables (`$ARGUMENTS`, `${CLAUDE_SKILL_DIR}`, `${CLAUDE_SESSION_ID}`), and executes inline shell commands (backtick-prefixed with `!`). The result is returned as content blocks injected into the conversation.

12.2.2 Seven Sources with Priority

Skills arrive from seven distinct sources, loaded in parallel and merged by precedence:

Priority	Source	Location	Notes
1	Managed (Policy)	<MANAGED_PATH>/ <code>.claude/policies/</code>	Enterprise-controlled
2	User	~/ <code>.claude/skills/</code>	Personal, available everywhere
3	Project	<code>.claude/skills/</code> (walked up to home)	Checked into version control

Priority	Source	Location	Notes
4	Additional Dirs	<code><add-dir>/.claude/skills/</code>	<code>--add-dir</code> flag
5	Legacy Commands	<code>.claude/commands/</code>	Backwards-compatible
6	Bundled	Compiled into the binary	Feature-gated
7	MCP	MCP server prompts	Remote, untrusted

Deduplication uses `realpath` to resolve symlinks and overlapping parent directories. The first-seen source wins. The `getFileIdentity` function resolves to canonical paths via `realpath` rather than relying on inode values, which are unreliable on container/NFS mounts and ExFAT.

12.2.3 The Frontmatter Contract

Key frontmatter fields that control skill behavior:

YAML Field	Purpose
<code>name</code>	User-facing display name
<code>description</code>	Shown in autocomplete and system prompt
<code>when_to_use</code>	Detailed usage scenarios for model discovery
<code>allowed-tools</code>	Which tools the skill can use
<code>disable-model-invocation</code>	Block autonomous model use
<code>context</code>	'fork' to run as sub-agent
<code>hooks</code>	Lifecycle hooks registered on invocation
<code>paths</code>	Glob patterns for conditional activation

The `context: 'fork'` option runs the skill as a sub-agent with its own context window, essential for skills that need significant work without polluting the main conversation's token budget. The `disable-model-invocation` and `user-invocable` fields control two distinct access paths – setting both to true makes the skill invisible, useful for hooks-only skills.

12.2.4 The MCP Security Boundary

After variable substitution, inline shell commands execute. The security boundary is absolute: **MCP skills never execute inline shell commands.** MCP servers are external systems. An MCP prompt containing `!`rm -rf /`` would execute with the user's full permissions if allowed. The system treats MCP skills as content-only. This trust boundary connects to the broader MCP security model discussed in Chapter 15.

12.2.5 Dynamic Discovery

Skills are not only loaded at startup. When the model touches files, `discoverSkillDirsForPaths` walks up from each path looking for `.claude/skills/` directories. Skills with `paths` frontmatter are stored in a `conditionalSkills` map and activate only when touched paths match their patterns. A skill declaring `paths: "packages/database/**"` remains invisible until the model reads or edits a database file – context-sensitive capability expansion.

12.3 Hooks: Controlling When Things Happen

Hooks are Claude Code's mechanism for intercepting and modifying behavior at lifecycle points. The main execution engine exceeds 4,900 lines. The system serves three audiences: individual developers (custom linting, validation), teams (shared quality gates checked into the project), and enterprises (policy-managed compliance rules).

12.3.1 A Real-World Hook: Preventing Commits to Main

Before diving into the machinery, here is what a hook looks like in practice. Suppose your team wants to prevent the model from committing directly to the `main` branch.

Step 1: The `settings.json` configuration:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
```



```

    "hooks": [
      {
        "type": "command",
        "command": "/path/to/check-not-main.sh",
        "if": "Bash(git commit*)"
      }
    ]
  }
}
}
}

```

Step 2: The shell script:

```

#!/bin/bash
BRANCH=$(git rev-parse --abbrev-ref HEAD 2>/dev/null)
if [ "$BRANCH" = "main" ]; then
  echo "Cannot commit directly to main. Create a feature branch first." >&2
  exit 2 # Exit 2 = blocking error
fi
exit 0

```

Step 3: What the model experiences. When the model tries `git commit` on the main branch, the hook fires before the command executes. The script checks the branch, writes to `stderr`, and exits with code 2. The model sees a system message: “Cannot commit directly to main. Create a feature branch first.” The commit never runs. The model creates a branch and commits there instead.

The `if: "Bash(git commit*)"` condition means the script only runs for `git commit` commands – not for every `Bash` invocation. Exit code 2 blocks; exit code 0 passes; any other exit code produces a non-blocking warning. This is the complete protocol.

12.3.2 Four User-Configurable Types

Claude Code defines six hook types – four user-configurable, two internal.

Command hooks spawn a shell process. Hook input JSON is piped to `stdin`; the hook communicates back via exit code and `stdout/stderr`. This is the workhorse type.

Prompt hooks make a single LLM call, returning `{"ok": true}` or `{"ok": false, "reason": "..."}.` Lightweight AI-powered validation without a full agent loop.

Agent hooks run a multi-turn agentic loop (max 50 turns, `dontAsk` permissions, thinking disabled). Each gets its own session scope. This is the heavy machinery for “verify that the test suite passes and covers the new feature.”

HTTP hooks POST the hook input to a URL. Enables remote policy servers and audit logging without local process spawning.

The two internal types are **callback hooks** (registered programmatically, -70% overhead on the hot path via a fast path that skips span tracking) and **function hooks** (session-scoped TypeScript callbacks for structured output enforcement in agent hooks).

12.3.3 The Five Most Important Lifecycle Events

The hook system fires at over two dozen lifecycle points. Five dominate real-world usage:

PreToolUse – fires before every tool execution. Can block, modify input, auto-approve, or inject context. Permission behavior follows strict precedence: deny > ask > allow. The most common hook point for quality gates.

PostToolUse – fires after successful execution. Can inject context or replace MCP tool output entirely. Useful for automated feedback on tool results.

Stop – fires before Claude concludes its response. A blocking hook forces continuation. This is the mechanism for automated verification loops: “are you really done?”

SessionStart – fires at session beginning. Can set environment variables, override the first user message, or register file watch paths. Cannot block (a hook cannot prevent a session from starting).

UserPromptSubmit – fires when the user submits a prompt. Can block processing, enabling input validation or content filtering before the model sees it.

Reference table – remaining events:

Category	Events
Tool lifecycle	PostToolUseFailure, PermissionDenied, PermissionRequest
Session	SessionEnd (1.5s timeout), Setup
Subagent	SubagentStart, SubagentStop
Compaction	PreCompact, PostCompact
Notification	Notification, Elicitation, ElicitationResult
Configuration	ConfigChange, InstructionsLoaded, CwdChanged, FileChanged, TaskCreated, TaskCompleted, TeammateIdle

The blocking asymmetry is intentional. Events representing recoverable decisions (tool calls, stop conditions) support blocking. Events representing irrevocable facts (session started, API failed) do not.

12.3.4 Exit Code Semantics

For command hooks, exit codes carry specific meaning:

Exit Code	Meaning	Blocks
0	Success, stdout parsed if JSON	No
2	Blocking error, stderr shown as system message	Yes
Other	Non-blocking warning, shown to user only	No

Exit code 2 was chosen deliberately. Exit code 1 is too common – any unhandled exception, assertion failure, or syntax error produces exit 1. Using exit 2 prevents accidental enforcement.

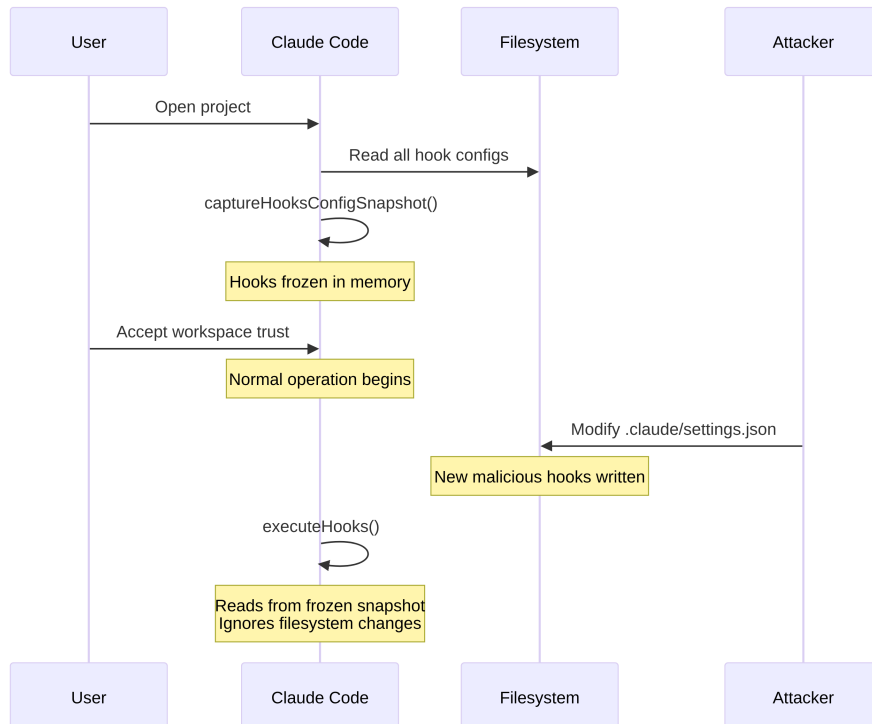
12.3.5 Six Hook Sources

Source	Trust Level	Notes
<code>userSettings</code>	User	<code>~/.claude/settings.json</code> , highest priority
<code>projectSettings</code>	Project	<code>.claude/settings.json</code> , version-controlled
<code>localSettings</code>	Local	<code>.claude/settings.local.json</code> , gitignored
<code>policySettings</code>	Enterprise	Cannot be overridden
<code>pluginHook</code>	Plugin	Priority 999 (lowest)
<code>sessionHook</code>	Session	In-memory only, registered by skills

12.4 The Snapshot Security Model

Hooks execute arbitrary code. A project's `.claude/settings.json` can define hooks that fire before every tool call. What happens if a malicious repository modifies its hooks after the user accepts the workspace trust dialog?

Nothing. The hooks configuration is frozen at startup.

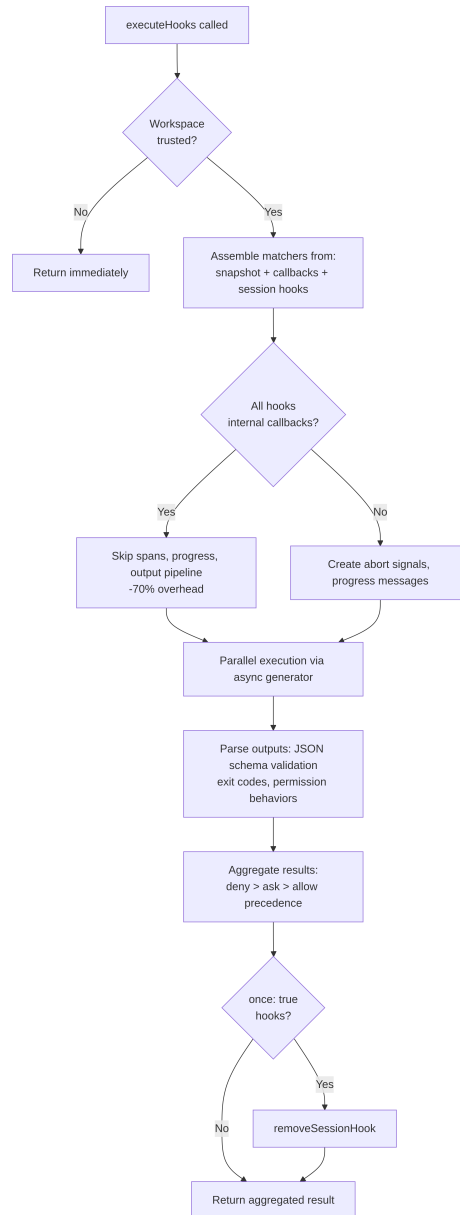


`captureHooksConfigSnapshot()` is called once during startup. From that point, `executeHooks()` reads from the snapshot, never re-reading settings files implicitly. The snapshot is only updated through explicit channels: the `/hooks` command or a file watcher detection, both of which rebuild through `updateHooksConfigSnapshot()`.

The policy enforcement cascade: `disableAllHooks` in policy settings clears everything. `allowManagedHooksOnly` excludes user and project hooks. A user can disable their own hooks by setting `disableAllHooks`, but they cannot disable enterprise-managed hooks. The policy layer always wins.

The trust check itself (`shouldSkipHookDueToTrust()`) was introduced after two vulnerabilities: `SessionEnd` hooks executing when a user *declined* the trust dialog, and `SubagentStop` hooks firing before trust was presented. Both shared the same root cause – hooks firing in lifecycle states where the user had not consented to workspace code execution. The fix is a centralized gate at the top of `executeHooks()`.

12.5 Execution Flow



The fast path for internal callbacks is a significant optimization. When all matched hooks are internal (file access analytics, commit attribution), the system skips span tracking, abort signal creation, progress messages, and

the full output processing pipeline. Most `PostToolUse` invocations hit only internal callbacks.

Hook input JSON is serialized once via a lazy `getJsonInput()` closure and reused across all parallel hooks. Environment injection sets `CLAUDE_PROJECT_DIR`, `CLAUDE_PLUGIN_ROOT`, and for certain events, `CLAUDE_ENV_FILE` where hooks can write environment exports.

12.6 Integration: Where Skills Meet Hooks

When a skill is invoked, its frontmatter-declared hooks register as session-scoped hooks. The `skillRoot` becomes `CLAUDE_PLUGIN_ROOT` for the hook's shell commands:

```
my-skill/
  SKILL.md          # The skill content
  validate.sh       # Called by a PreToolUse hook declared in frontmatter
```

The skill's frontmatter declares:

```
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "${CLAUDE_PLUGIN_ROOT}/validate.sh"
          once: true
```

When the user invokes `/my-skill`, the skill content loads into the conversation AND the `PreToolUse` hook registers. The next `Bash` tool call triggers `validate.sh`. Because `once: true` is set, the hook removes itself after the first successful execution.

For agents, `Stop` hooks declared in frontmatter are automatically converted to `SubagentStop` hooks, because subagents trigger `SubagentStop`, not `Stop`. Without the conversion, an agent's stop-verification hook would never fire.

12.6.1 Permission Behavior Precedence

`executePreToolHooks()` can block (via `blockingError`), auto-approve (via `permissionBehavior: 'allow'`), force ask (via `'ask'`), deny (via `'deny'`),

modify input (via `updatedInput`), or add context (via `additionalContext`). When multiple hooks return different behaviors, `deny` always wins. This is the correct default for security-relevant decisions.

12.6.2 Stop Hooks: Forcing Continuation

When a Stop hook returns exit code 2, the stderr is shown to the model as feedback and the conversation continues. This turns a single-shot prompt-response into a goal-directed loop. The Stop hook is arguably the most powerful integration point in the entire system.

12.7 Apply This: Designing an Extensibility System

Separate content from control flow. Skills add capabilities; hooks constrain behavior. Conflating the two makes it impossible to reason about what a plugin does versus what it prevents.

Freeze configuration at trust boundaries. The snapshot mechanism captures hooks at the moment of consent and never re-reads implicitly. If your system executes user-provided code, this eliminates TOCTOU attacks.

Use uncommon exit codes for semantic signals. Exit code 1 is noise – every unhandled error produces it. Exit code 2 as the blocking signal prevents accidental enforcement. Choose signals that require deliberate intent.

Validate at the socket level, not the application level. The SSRF guard runs at DNS lookup time, not as a pre-flight check. This eliminates the DNS rebinding window. When validating network destinations, the check must be atomic with the connection.

Optimize for the common case. The internal callback fast path (-70% overhead) recognizes that most hook invocations hit only internal callbacks. The two-phase skill loading recognizes that most skills are never invoked in a given session. Each optimization targets the actual distribution of usage.

The extensibility system reflects a mature understanding of the tension between power and safety. Skills give the model new capabilities bounded by the MCP security line (Chapter 15). Hooks give external code influence over the model's actions bounded by the snapshot mechanism, exit code

semantics, and policy cascade. Neither system trusts the other – and that mutual distrust is what makes the combination safe to deploy at scale.

The next chapter turns to the visual layer: how Claude Code renders a reactive terminal UI at 60fps and processes input across five terminal protocols.

Part V

Part V: The Interface

Chapter 13

Chapter 13: The Terminal UI

13.1 Why Build a Custom Renderer?

The terminal is not a browser. There is no DOM, no CSS engine, no compositor, no retained-mode graphics pipeline. There is a stream of bytes going to stdout and a stream of bytes coming from stdin. Everything between those two streams – layout, styling, diffing, hit-testing, scrolling, selection – has to be invented from scratch.

Claude Code needs a reactive UI. It has a prompt input, streaming markdown output, permission dialogs, progress spinners, scrollable message lists, search highlighting, and a vim-mode editor. React is the obvious choice for declaring this kind of component tree. But React needs a host environment to render into, and terminals do not provide one.

Ink is the standard answer: a React renderer for terminals, built on Yoga for flexbox layout. Claude Code started with Ink, then forked it beyond recognition. The stock version allocates one JavaScript object per cell per frame – on a 200x120 terminal, that is 24,000 objects created and garbage-collected every 16ms. It diffs at the string level, comparing entire rows of ANSI-encoded text. It has no concept of blit optimization, no double buffering, no cell-level dirty tracking. For a simple CLI dashboard refreshing once per second, this is fine. For an LLM agent streaming tokens at 60fps while the user scrolls through a conversation with hundreds of messages, it is a non-starter.

What remains in Claude Code is a custom rendering engine that shares

Ink’s conceptual DNA – React reconciler, Yoga layout, ANSI output – but reimplements the critical path: packed typed arrays instead of object-per-cell, pool-based string interning instead of string-per-frame, double-buffered rendering with cell-level diffing, and an optimizer that merges adjacent terminal writes into minimal escape sequences.

The result runs at 60fps on a 200-column terminal while streaming tokens from Claude. To understand how, we need to examine four layers: the custom DOM that React reconciles against, the rendering pipeline that converts that DOM into terminal output, the pool-based memory management that keeps the system alive for hours-long sessions without drowning in garbage collection, and the component architecture that ties it all together.

13.2 The Custom DOM

React’s reconciler needs something to reconcile against. In the browser, that’s the DOM. In Claude Code’s terminal, it is a custom in-memory tree with seven element types and one text node type.

The element types map directly to terminal rendering concepts:

- **ink-root** – the document root, one per Ink instance
- **ink-box** – a flexbox container, the terminal equivalent of a `<div>`
- **ink-text** – a text node with a Yoga measure function for word wrapping
- **ink-virtual-text** – nested styled text inside another text node (automatically promoted from **ink-text** when inside a text context)
- **ink-link** – a hyperlink, rendered via OSC 8 escape sequences
- **ink-progress** – a progress indicator
- **ink-raw-ansi** – pre-rendered ANSI content with known dimensions, used for syntax-highlighted code blocks

Each `DOMElement` carries the state that the rendering pipeline needs:

```
// Illustrative - actual interface extends this significantly
interface DOMElement {
  yogaNode: YogaNode;           // Flexbox layout node
  style: Styles;                 // CSS-like properties mapped to Yoga
  attributes: Map<string, DOMNodeAttribute>;
  childNodes: (DOMElement | TextNode)[];
  dirty: boolean;               // Needs re-rendering
}
```

```

_eventHandlers: EventHandlerMap; // Separated from attributes
scrollTop: number;               // Imperative scroll state
pendingScrollDelta: number;
stickyScroll: boolean;
debugOwnerChain?: string;       // React component stack for debug
}

```

The separation of `_eventHandlers` from `attributes` is deliberate. In React, handler identity changes on every render (unless manually memoized). If handlers were stored as attributes, every render would mark the node dirty and trigger a full repaint. By storing them separately, the reconciler's `commitUpdate` can update handlers without dirtying the node.

The `markDirty()` function is the bridge between DOM mutations and the rendering pipeline. When any node's content changes, `markDirty()` walks up through every ancestor, setting `dirty = true` on each element and calling `yogaNode.markDirty()` on leaf text nodes. This is how a single character change in a deeply nested text node schedules a re-render of the entire path to the root – but only that path. Sibling subtrees remain clean and can be blitted from the previous frame.

The `ink-raw-ansi` element type deserves special mention. When a code block has already been syntax-highlighted (producing ANSI escape sequences), re-parsing those sequences to extract characters and styles would be wasteful. Instead, the pre-highlighted content is wrapped in an `ink-raw-ansi` node with `rawWidth` and `rawHeight` attributes that tell Yoga the exact dimensions. The rendering pipeline writes the raw ANSI content directly to the output buffer without decomposing it into individual styled characters. This makes syntax-highlighted code blocks essentially zero-cost after the initial highlighting pass – the most expensive visual element in the UI is also the cheapest to render.

The `ink-text` node's measure function is worth understanding because it runs inside Yoga's layout pass, which is synchronous and blocking. The function receives the available width and must return the text's dimensions. It performs word wrapping (respecting the `wrap` style prop: `wrap`, `truncate`, `truncate-start`, `truncate-middle`), accounts for grapheme cluster boundaries (so it does not split a multi-codepoint emoji across lines), measures CJK double-width characters correctly (each counts as 2 columns), and strips ANSI escape codes from the width calculation (escape sequences have zero visual width). All of this must complete in microseconds per node, because

a conversation with 50 visible text nodes means 50 measure function calls per layout pass.

13.3 The React Fiber Container

The reconciler bridge uses `react-reconciler` to create a custom host config. This is the same API that React DOM and React Native use. The key difference: Claude Code runs in `ConcurrentRoot` mode.

```
createContainer(rootNode, ConcurrentRoot, ...)
```

`ConcurrentRoot` enables React's concurrent features – Suspense for lazy-loaded syntax highlighting, transitions for non-blocking state updates during streaming. The alternative, `LegacyRoot`, would force synchronous rendering and block the event loop during heavy markdown re-parses.

The host config methods map React operations to the custom DOM:

- **`createInstance(type, props)`** creates a `DOMElement` via `createNode()`, applies initial styles and attributes, attaches event handlers, and captures the React component owner chain for debug attribution. The owner chain is stored as `debugOwnerChain` and used by the `CLAUDE_CODE_DEBUG_REPAINTS` mode to attribute full-screen resets to specific components
- **`createTextInstance(text)`** creates a `TextNode` – but only if we are inside a text context. The reconciler enforces that raw strings must be wrapped in `<Text>`. Attempting to create a text node outside a text context throws, catching a class of bugs at reconciliation time rather than at render time
- **`commitUpdate(node, type, oldProps, newProps)`** diffs old and new props via a shallow comparison, then applies only what changed. Styles, attributes, and event handlers each have their own update path. The diff function returns `undefined` if nothing changed, avoiding unnecessary DOM mutations entirely
- **`removeChild(parent, child)`** removes the node from the tree, recursively frees Yoga nodes (calling `unsetMeasureFunc()` before `free()` to avoid accessing freed WASM memory), and notifies the focus manager
- **`hideInstance(node)` / `unhideInstance(node)`** toggles `isHidden` and switches the Yoga node between `Display.None` and `Display.Flex`. This is React's mechanism for Suspense fallback transitions

- `resetAfterCommit(container)` is the critical hook: it calls `rootNode.onComputeLayout()` to run Yoga, then `rootNode.onRender()` to schedule the terminal paint

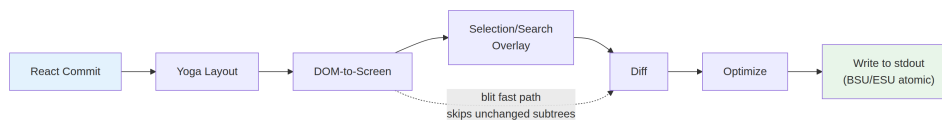
The reconciler tracks two performance counters per commit cycle: Yoga layout time (`lastYogaMs`) and total commit time (`lastCommitMs`). These flow into the `FrameEvent` that the Ink class reports, enabling performance monitoring in production.

The event system mirrors the browser’s capture/bubble model. A `Dispatcher` class implements full event propagation with three phases: capture (root to target), at-target, and bubble (target to root). Event types map to React scheduling priorities – discrete for keyboard and click (highest priority, processed immediately), continuous for scroll and resize (can be deferred). The dispatcher wraps all event processing in `reconciler.discreteUpdates()` for proper React batching.

When you press a key in the terminal, the resulting `KeyboardEvent` is dispatched through the custom DOM tree, bubbling from the focused element up to the root exactly as a keyboard event would bubble through browser DOM elements. Any handler along the path can call `stopPropagation()` or `preventDefault()`, and the semantics are identical to the browser specification.

13.4 The Rendering Pipeline

Every frame traverses seven stages, each timed individually:



Each stage is timed individually and reported in `FrameEvent.phases`. This per-stage instrumentation is essential for diagnosing performance issues: when a frame takes 30ms, you need to know whether the bottleneck is Yoga re-measuring text (stage 2), the renderer walking a large dirty subtree (stage 3), or stdout backpressure from a slow terminal (stage 7). The answer determines the fix.

Stage 1: React commit and Yoga layout. The reconciler processes

state updates and calls `resetAfterCommit`. This sets the root node's width to `terminalColumns` and runs `yogaNode.calculateLayout()`. Yoga computes the entire flexbox tree in one pass, following the CSS flexbox specification: it resolves flex-grow, flex-shrink, padding, margin, gap, alignment, and wrapping across all nodes. The results – `getComputedWidth()`, `getComputedHeight()`, `getComputedLeft()`, `getComputedTop()` – are cached per node. For `ink-text` nodes, Yoga calls the custom measure function (`measureTextNode`) during layout, which computes text dimensions via word wrapping and grapheme measurement. This is the most expensive per-node operation: it must handle Unicode grapheme clusters, CJK double-width characters, emoji sequences, and ANSI escape codes embedded in text content.

Stage 2: DOM-to-screen. The renderer walks the DOM tree depth-first, writing characters and styles into a `Screen` buffer. Each character becomes a packed cell. The output is a complete frame: every cell on the terminal has a defined character, style, and width.

Stage 3: Overlay. Text selection and search highlighting modify the screen buffer in-place, flipping style IDs on matching cells. Selection applies inverse video to create the familiar “highlighted text” appearance. Search highlighting applies a more aggressive visual treatment: inverse + yellow foreground + bold + underline for the current match, inverse only for other matches. This contaminates the buffer – tracked by a `prevFrameContaminated` flag so the next frame knows to skip the blit fast-path. The contamination is a deliberate tradeoff: modifying the buffer in-place avoids allocating a separate overlay buffer (saving 48KB on a 200x120 terminal), at the cost of one full-damage frame after the overlay is cleared.

Stage 4: Diff. The new screen is compared cell-by-cell against the front frame's screen. Only changed cells produce output. The comparison is two integer comparisons per cell (the two packed `Int32` words), and the diff walks the damage rectangle rather than the full screen. On a steady-state frame (only a spinner ticking), this might produce patches for 3 cells out of 24,000. Each patch is a `{ type: 'stdout', content: string }` object containing the cursor-move sequence and the ANSI-encoded cell content.

Stage 5: Optimize. Adjacent patches on the same row are merged into a single write. Redundant cursor moves are eliminated – if patch N ends at column 10 and patch N+1 starts at column 11, the cursor is already in the right position and no move sequence is needed. Style transitions are pre-serialized via the `StylePool.transition()` cache, so changing from

“bold red” to “dim green” is a single cached string lookup rather than a diff-and-serialize operation. The optimizer typically reduces the byte count by 30-50% compared to naive per-cell output.

Stage 6: Write. The optimized patches are serialized to ANSI escape sequences and written to stdout in a single `write()` call, wrapped in synchronous update markers (BSU/ESU) on terminals that support them. BSU (Begin Synchronized Update, `ESC [ ? 2026 h`) tells the terminal to buffer all following output, and ESU (`ESC [ ? 2026 l`) tells it to flush. This eliminates visible tearing on terminals that support the protocol – the entire frame appears atomically.

Every frame reports its timing breakdown via a `FrameEvent` object:

```
interface FrameEvent {
  durationMs: number;
  phases: {
    renderer: number; // DOM-to-screen
    diff: number; // Screen comparison
    optimize: number; // Patch merging
    write: number; // stdout write
    yoga: number; // Layout computation
  };
  yogaVisited: number; // Nodes traversed
  yogaMeasured: number; // Nodes that ran measure()
  yogaCacheHits: number; // Nodes with cached layout
  flickers: FlickerEvent[]; // Full-reset attributions
}
```

When `CLAUDE_CODE_DEBUG_REPAINTS` is enabled, full-screen resets are attributed to their source React component via `findOwnerChainAtRow()`. This is the terminal equivalent of React DevTools’ “Highlight Updates” – it shows you which component caused the entire screen to repaint, which is the most expensive thing that can happen in the rendering pipeline.

The blit optimization deserves special attention. When a node is not dirty and its position has not changed since the previous frame (checked via a node cache), the renderer copies cells directly from `prevScreen` to the current screen instead of re-rendering the subtree. This makes steady-state frames extremely cheap – on a typical frame where only a spinner is ticking, the blit covers 99% of the screen and only the spinner’s 3-4 cells are re-rendered from scratch.

The blit is disabled under three conditions:

1. **prevFrameContaminated is true** – the selection overlay or a search highlight modified the front frame’s screen buffer in-place, so those cells cannot be trusted as the “correct” previous state
2. **An absolute-positioned node was removed** – absolute positioning means the node could have painted over non-sibling cells, and those cells need to be re-rendered from the elements that actually own them
3. **Layout shifted** – any node’s cached position differs from its current computed position, meaning the blit would copy cells to the wrong coordinates

The damage rectangle (`screen.damage`) tracks the bounding box of all written cells during rendering. The diff only examines rows within this rectangle, skipping entirely unchanged regions. On a 120-row terminal where a streaming message occupies rows 80-100, the diff checks 20 rows instead of 120 – a 6x reduction in comparison work.

13.5 Double-Buffer Rendering and Frame Scheduling

The Ink class maintains two frame buffers:

```
private frontFrame: Frame; // Currently displayed on terminal
private backFrame: Frame;  // Being rendered into
```

Each Frame contains:

- `screen: Screen` – the cell buffer (packed `Int32Array`)
- `viewport: Size` – terminal dimensions at render time
- `cursor: { x, y, visible }` – where to park the terminal cursor
- `scrollHint` – DECSTBM (scroll region) optimization hint for alt-screen mode
- `scrollDrainPending` – whether a `ScrollBar` has remaining scroll delta to process

After each render, the frames swap: `backFrame = frontFrame;` `frontFrame = newFrame.` The old front frame becomes the next back frame, providing the `prevScreen` for blit optimization and the baseline for cell-level diffing.

13.5. DOUBLE-BUFFER RENDERING AND FRAME SCHEDULING 219

This double-buffer design eliminates allocation. Instead of creating a new **Screen** every frame, the renderer reuses the back frame's buffer. The swap is a pointer assignment. The pattern is borrowed from graphics programming, where double buffering prevents tearing by ensuring the display reads from a complete frame while the renderer writes to the other. In the terminal context, tearing is not the concern (the BSU/ESU protocol handles that); the concern is GC pressure from allocating and discarding **Screen** objects containing 48KB+ of typed arrays every 16ms.

Render scheduling uses lodash **throttle** at 16ms (approximately 60fps), with leading and trailing edges enabled:

```
const deferredRender = () => queueMicrotask(this.onRender);
this.scheduleRender = throttle(deferredRender, FRAME_INTERVAL_MS, {
  leading: true,
  trailing: true,
});
```

The microtask deferral is not accidental. **resetAfterCommit** runs before React's layout effects phase. If the renderer ran synchronously here, it would miss cursor declarations set in **useLayoutEffect**. The microtask runs after layout effects but within the same event-loop tick – the terminal sees a single, consistent frame.

For scroll operations, a separate **setTimeout** at 4ms (**FRAME_INTERVAL_MS** » 2) provides faster scroll frames without interfering with the throttle. Scroll mutations bypass React entirely: **ScrollBox.scrollBy()** mutates DOM node properties directly, calls **markDirty()**, and schedules a render via microtask. No React state update, no reconciliation overhead, no re-rendering of the entire message list for a single wheel event.

Resize handling is synchronous, not debounced. When the terminal resizes, **handleResize** updates dimensions immediately to keep layout consistent. For alt-screen mode, it resets frame buffers and defers **ERASE_SCREEN** into the next atomic BSU/ESU paint block rather than writing it immediately. Writing the erase synchronously would leave the screen blank for the ~80ms the render takes; deferring it into the atomic block means old content stays visible until the new frame is fully ready.

Alt-screen management adds another layer. The **AlternateScreen** component enters DEC 1049 alternate screen buffer on mount, constraining height to terminal rows. It uses **useInsertionEffect** – not **useLayoutEffect** – to

ensure the `ENTER_ALT_SCREEN` escape sequence reaches the terminal before the first render frame. Using `useLayoutEffect` would be too late: the first frame would render to the main screen buffer, producing a visible flash before the switch. `useInsertionEffect` runs before layout effects and before the browser (or terminal) would paint, making the transition seamless.

13.6 Pool-Based Memory: Why Interning Matters

A 200-column by 120-row terminal has 24,000 cells. If each cell were a JavaScript object with a `char` string, a `style` string, and a `hyperlink` string, that is 72,000 string allocations per frame – plus 24,000 object allocations for the cells themselves. At 60fps, that is 5.76 million allocations per second. V8’s garbage collector can handle this, but not without pauses that show up as dropped frames. The GC pauses are typically 1-5ms, but they are unpredictable: they might hit during a streaming token update, causing a visible stutter exactly when the user is watching the output.

Claude Code eliminates this entirely with packed typed arrays and three interning pools. The result: zero per-frame object allocations for the cell buffer. The only allocations are in the pools themselves (amortized, since most characters and styles are interned on the first frame and reused thereafter) and in the patch strings produced by the diff (unavoidable, since `stdout.write` requires string or Buffer arguments).

The cell layout uses two `Int32` words per cell, stored in a contiguous `Int32Array`:

```
word0: charId          (32 bits, index into CharPool)
word1: styleId[31:17] | hyperlinkId[16:2] | width[1:0]
```

A parallel `BigInt64Array` view over the same buffer enables bulk operations – clearing a row is a single `fill()` call on 64-bit words instead of zeroing individual fields.

CharPool interns character strings to integer IDs. It has a fast path for ASCII: a 128-entry `Int32Array` maps character codes directly to pool indices, avoiding the `Map` lookup entirely. Multi-byte characters (emoji, CJK ideographs) fall through to a `Map<string, number>`. Index 0 is always space, index 1 is always empty string.

```

export class CharPool {
  private strings: string[] = [' ', '']
  private ascii: Int32Array = initCharAscii()

  intern(char: string): number {
    if (char.length === 1) {
      const code = char.charCodeAt(0)
      if (code < 128) {
        const cached = this.ascii[code]!
        if (cached !== -1) return cached
        const index = this.strings.length
        this.strings.push(char)
        this.ascii[code] = index
        return index
      }
    }
    // Map fallback for multi-byte characters
    ...
  }
}

```

StylePool interns arrays of ANSI style codes to integer IDs. The clever part: bit 0 of each ID encodes whether the style has a visible effect on space characters (background color, inverse, underline). Foreground-only styles get even IDs; styles visible on spaces get odd IDs. This lets the renderer skip invisible spaces with a single bitmask check – `if (!(styleId & 1) && charId === 0) continue` – without looking up the style definition. The pool also caches pre-serialized ANSI transition strings between any two style IDs, so transitioning from “bold red” to “dim green” is a cached string concatenation, not a diff-and-serialize operation.

HyperlinkPool interns OSC 8 hyperlink URIs. Index 0 means no hyperlink.

All three pools are shared across the front and back frames. This is a critical design decision. Because the pools are shared, interned IDs are valid across frames: the blit optimization can copy packed cell words directly from `prevScreen` to the current screen without re-interning. The diff can compare IDs as integers without string lookups. If each frame had its own pools, the blit would need to re-intern every copied cell (looking up the string by old ID, then interning it in the new pool), which would negate most of the blit’s

performance benefit.

Pools are periodically reset (every 5 minutes) to prevent unbounded growth during long sessions. A migration pass re-interns the front frame's live cells into the fresh pools.

CellWidth handles double-wide characters with a 2-bit classification:

Value	Meaning
0 (Narrow)	Standard single-column character
1 (Wide)	CJK/emoji head cell, occupies two columns
2 (SpacerTail)	Second column of a wide character
3 (SpacerHead)	Soft-wrap continuation marker

This is stored in the low 2 bits of **word1**, making width checks on packed cells free – no field extraction needed for the common case.

Additional per-cell metadata lives in parallel arrays rather than the packed cells:

- **noSelect: Uint8Array** – per-cell flag excluding content from text selection. Used for UI chrome (borders, indicators) that should not appear in copied text
- **softWrap: Int32Array** – per-row marker indicating word-wrap continuation. When the user selects text across a soft-wrapped line, the selection logic knows not to insert a newline at the wrap point
- **damage: Rectangle** – bounding box of all written cells in the current frame. The diff only examines rows within this rectangle, skipping entirely unchanged regions

These parallel arrays avoid widening the packed cell format (which would increase cache pressure in the diff inner loop) while providing the metadata that selection, copy, and optimization need.

The **Screen** also exposes a **createScreen()** factory that takes dimensions and pool references. Creating a screen zeroes the **Int32Array** via **fill(0n)** on the **BigInt64Array** view – a single native call that clears the entire buffer in microseconds. This is used during resize (when new frame buffers are needed) and during pool migration (when the old screen's cells are re-interned into fresh pools).

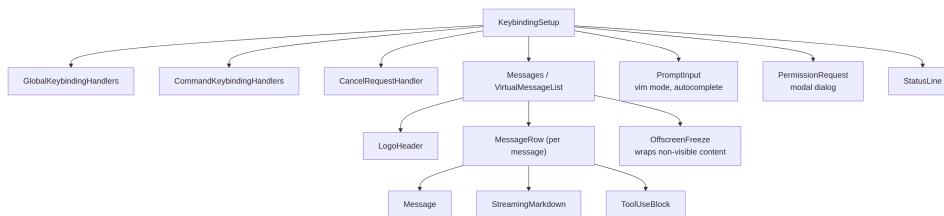
13.7 The REPL Component

The REPL (`REPL.tsx`) is approximately 5,000 lines. It is the largest single component in the codebase, and for good reason: it is the orchestrator of the entire interactive experience. Everything flows through it.

The component is organized into roughly nine sections:

1. **Imports** (~100 lines) – pulls in bootstrap state, commands, history, hooks, components, keybindings, cost tracking, notifications, swarm/team support, voice integration
2. **Feature-flagged imports** – conditional loading of voice integration, proactive mode, brief tool, and coordinator agent via `feature()` guards with `require()`
3. **State management** – extensive `useState` calls covering messages, input mode, pending permissions, dialogs, cost thresholds, session state, tool state, and agent state
4. **QueryGuard** – manages active API call lifecycle, preventing concurrent requests from stepping on each other
5. **Message handling** – processes incoming messages from the query loop, normalizes ordering, manages streaming state
6. **Tool permission flow** – coordinates permission requests between tool use blocks and the `PermissionRequest` dialog
7. **Session management** – resume, switch, export conversations
8. **Keybinding setup** – wires the keybinding providers: `KeybindingSetup`, `GlobalKeybindingHandlers`, `CommandKeybindingHandlers`
9. **Render tree** – composes the final UI from all the above

Its render tree composes the full interface in fullscreen mode:



`OffscreenFreeze` is a performance optimization specific to terminal rendering. When a message scrolls above the viewport, its React element is cached and its subtree is frozen. This prevents timer-based updates (spinners, elapsed time counters) in off-screen messages from triggering terminal resets. Without this, a spinning indicator in message 3 would cause a full repaint

even though the user is looking at message 47.

The component is compiled by the React Compiler throughout. Instead of manual `useMemo` and `useCallback`, the compiler inserts per-expression memoization using slot arrays:

```
const $ = _c(14); // 14 memoization slots
let t0;
if ($[0] !== dep1 || $[1] !== dep2) {
  t0 = expensiveComputation(dep1, dep2);
  $[0] = dep1; $[1] = dep2; $[2] = t0;
} else {
  t0 = $[2];
}
```

This pattern appears in every component in the codebase. It provides finer granularity than `useMemo` (which memoizes at the hook level) – individual expressions within a render function get their own dependency tracking and caching. For a 5,000-line component like the REPL, this eliminates hundreds of potential unnecessary recomputations per render.

13.8 Selection and Search Highlighting

Text selection and search highlighting operate as screen-buffer overlays, applied after the main render but before the diff.

Text selection is alt-screen only. The Ink instance holds a **SelectionState** tracking anchor and focus points, drag mode (character/word/line), and captured rows that have scrolled off-screen. When the user clicks and drags, the selection handler updates these coordinates. During **onRender**, **applySelectionOverlay** walks the affected rows and modifies cell style IDs in-place using **StylePool.withSelectionBg()**, which returns a new style ID with inverse video added. This direct mutation of the screen buffer is why the **prevFrameContaminated** flag exists – the front frame’s buffer has been modified by the overlay, so the next frame cannot trust it for blit optimization and must do a full-damage diff.

Mouse tracking uses SGR 1003 mode, which reports clicks, drags, and motion with column/row coordinates. The **App** component implements multi-click detection: double-click selects a word, triple-click selects a line. The detection

uses a 500ms timeout and 1-cell position tolerance (the mouse can move one cell between clicks without resetting the multi-click counter). Hyperlink clicks are intentionally deferred by this timeout – double-clicking a link selects the word instead of opening the browser, matching the behavior users expect from text editors.

A lost-release recovery mechanism handles the case where the user starts a drag inside the terminal, moves the mouse outside the window, and releases. The terminal reports the press and the drag, but not the release (which happened outside its window). Without recovery, the selection would be stuck in drag mode permanently. The recovery works by detecting mouse motion events with no buttons pressed – if we are in a drag state and receive a no-button motion event, we infer that the button was released outside the window and finalize the selection.

Search highlighting has two mechanisms running in parallel. The scan-based path (`applySearchHighlight`) walks visible cells looking for the query string and applies SGR inverse styling. The position-based path uses pre-computed `MatchPosition[]` from `scanElementSubtree()`, stored message-relative, and applies them at known offsets with a “current match” yellow highlight using stacked ANSI codes (inverse + yellow foreground + bold + underline). The yellow foreground combined with inverse becomes a yellow background – the terminal swaps fg/bg when inverse is active. The underline is the fallback visibility marker for themes where the yellow clashes with existing background colors.

Cursor declaration solves a subtle problem. Terminal emulators render IME (Input Method Editor) preedit text at the physical cursor position. CJK users composing characters need the cursor to be at the text input’s caret, not at the bottom of the screen where the terminal would naturally park it. The `useDeclaredCursor` hook lets a component declare where the cursor should be after each frame. The `Ink` class reads the declared node’s position from `nodeCache`, translates it to screen coordinates, and emits cursor-move sequences after the diff. Screen readers and magnifiers also track the physical cursor, so this mechanism benefits accessibility as well as CJK input.

In main-screen mode, the declared cursor position is tracked separately from `frame.cursor` (which must stay at the content bottom for the log-update’s relative-move invariants). In alt-screen mode, the problem is simpler: every frame begins with `CSI H` (cursor home), so the declared cursor is just an absolute position emitted at the end of the frame.

13.9 Streaming Markdown

Rendering LLM output is the most demanding task the terminal UI faces. Tokens arrive one at a time, 10-50 per second, and each one changes the content of a message that might contain code blocks, lists, bold text, and inline code. The naive approach – re-parse the entire message on every token – would be catastrophic at scale.

Claude Code uses three optimizations:

Token caching. A module-level LRU cache (500 entries) stores `marked.lexer()` results keyed by content hash. The cache survives React unmount/remount cycles during virtual scrolling. When a user scrolls back to a previously visible message, the markdown tokens are served from cache instead of re-parsed.

Fast-path detection. `hasMarkdownSyntax()` checks the first 500 characters for markdown markers via a single regex. If no syntax is found, it constructs a single-paragraph token directly, bypassing the full GFM parser. This saves approximately 3ms per render on plain-text messages – which matters when you are rendering 60 frames per second.

Lazy syntax highlighting. Code block highlighting is loaded via `React Suspense`. The `MarkdownBody` component renders immediately with `highlight={null}` as a fallback, then resolves asynchronously with the `cli-highlight` instance. The user sees the code immediately (unstyled), then it pops into color a frame or two later.

The streaming case adds a wrinkle. When tokens arrive from the model, the markdown content grows incrementally. Re-parsing the entire content on every token would be $O(n^2)$ over the course of a message. The fast-path detection helps – most streaming content is plain text paragraphs, which bypass the parser entirely – but for messages with code blocks and lists, the LRU cache provides the real optimization. The cache key is the content hash, so when 10 tokens arrive and only the last paragraph changes, the cached parse result for the unchanged prefix is reused. The markdown renderer only re-parses the tail that changed.

The `StreamingMarkdown` component is distinct from the static `Markdown` component. It handles the case where the content is still being generated:

incomplete code fences (a ````` without a closing fence), partial bold markers, and truncated list items. The streaming variant is more forgiving in its parsing – it does not error on unclosed syntax because the closing syntax has not arrived yet. When the message finishes streaming, the component transitions to the static **Markdown** renderer, which applies full GFM parsing with strict syntax checking.

Syntax highlighting for code blocks is the most expensive per-element operation in the rendering pipeline. A 100-line code block can take 50-100ms to highlight with `cli-highlight`. Loading the highlighting library itself takes 200-300ms (it bundles grammar definitions for dozens of languages). Both costs are hidden behind **React Suspense**: the code block renders immediately as plain text, the highlighting library loads asynchronously, and when it resolves, the code block re-renders with colors. The user sees code instantly and colors a moment later – a much better experience than a 300ms blank frame while the library loads.

13.10 Apply This: Rendering Streaming Output Efficiently

The terminal rendering pipeline is a case study in eliminating work. Three principles drive the design:

Intern everything. If you have a value that appears in thousands of cells – a style, a character, a URL – store it once and reference it by integer ID. Integer comparison is one CPU instruction. String comparison is a loop. When your inner loop runs 24,000 times per frame at 60fps, the difference between `===` on integers and `===` on strings is the difference between smooth scrolling and visible lag.

Diff at the right level. Cell-level diffing sounds expensive – 24,000 comparisons per frame. But it is two integer comparisons per cell (the packed words), and on a steady-state frame, the diff bails out of most rows after checking the first cell. The alternative – re-rendering the entire screen and writing it to `stdout` – would produce 100KB+ of ANSI escape sequences per frame. The diff typically produces under 1KB.

Separate the hot path from React. Scroll events arrive at mouse-input frequency (potentially hundreds per second). Routing each one through

React’s reconciler – state update, reconciliation, commit, layout, render – adds 5-10ms of latency per event. By mutating DOM nodes directly and scheduling renders via microtask, the scroll path stays under 1ms. React is involved only in the final paint, where it would run anyway.

These principles apply to any streaming output system, not just terminals. If you are building a web application that renders real-time data – a log viewer, a chat client, a monitoring dashboard – the same tradeoffs apply. Intern repeated values. Diff against the previous frame. Keep the hot path out of your reactive framework.

A fourth principle, specific to long-running sessions: **clean up periodically**. Claude Code’s pools grow monotonically as new characters and styles are interned. Over a multi-hour session, the pools could accumulate thousands of entries that are no longer referenced by any live cell. The 5-minute reset cycle bounds this growth: every 5 minutes, fresh pools are created, the front frame’s cells are migrated (re-interned into the new pools), and the old pools become garbage. This is a generational collection strategy, applied at the application level because the JavaScript GC has no visibility into the semantic liveness of pool entries.

The decision to use `Int32Array` over plain objects has a subtler benefit beyond GC pressure: memory locality. When the diff compares 24,000 cells, it walks a contiguous typed array. Modern CPUs prefetch sequential memory accesses, so the entire screen comparison runs within the L1/L2 cache. An object-per-cell layout would scatter cells across the heap, turning every comparison into a cache miss. The performance difference is measurable: on a 200x120 screen, the typed-array diff completes in under 0.5ms, while an equivalent object-based diff takes 3-5ms – enough to blow the 16ms frame budget when combined with the other pipeline stages.

A fifth principle applies to any system that renders into a fixed-size grid: **track damage bounds**. The `damage` rectangle on each screen records the bounding box of cells that were written during rendering. The diff consults this rectangle and skips rows outside it entirely. When a streaming message occupies the bottom 20 rows of a 120-row terminal, the diff examines 20 rows, not 120. Combined with the blit optimization (which populates the damage rectangle only for re-rendered regions, not blitted ones), this means the common case – one message streaming while the rest of the conversation is static – touches a fraction of the screen buffer.

The broader lesson: performance in a rendering system is not about making

any single operation fast. It is about eliminating operations entirely. The blit eliminates re-rendering. The damage rectangle eliminates diffing. The pool sharing eliminates re-interning. The packed cells eliminate allocation. Each optimization removes an entire category of work, and they stack multiplicatively.

To put numbers on it: a worst-case frame (everything dirty, no blit, full-screen damage) on a 200x120 terminal takes approximately 12ms. A best-case frame (one dirty node, blit everything else, 3-row damage rectangle) takes under 1ms. The system spends most of its time in the best case. The streaming token arrival triggers one dirty text node, which dirties its ancestors up to the message container, which is typically 10-30 rows of the screen. The blit handles the other 90-110 rows. The damage rectangle constrains the diff to the dirty region. The pool lookups are integer operations. The steady-state cost of streaming one token is dominated by Yoga layout (which re-measures the dirty text node and its ancestors) and the markdown re-parse – not by the rendering pipeline itself.

Chapter 14

Chapter 14: Input and Interaction

14.1 Raw Bytes, Meaningful Actions

When you press Ctrl+X followed by Ctrl+K in Claude Code, the terminal sends two byte sequences separated by perhaps 200 milliseconds. The first is `0x18` (ASCII CAN). The second is `0x0B` (ASCII VT). Neither of these bytes carries any inherent meaning beyond “control character.” The input system must recognize that these two bytes, arriving in sequence within a timeout window, constitute the chord `ctrl+x ctrl+k`, which maps to the action `chat:killAgents`, which terminates all running sub-agents.

Between the raw bytes and the killed agents, six systems activate: a tokenizer splits escape sequences, a parser classifies them across five terminal protocols, a keybinding resolver matches the sequence against context-specific bindings, a chord state machine manages the multi-key sequence, a handler executes the action, and React batches the resulting state updates into a single render.

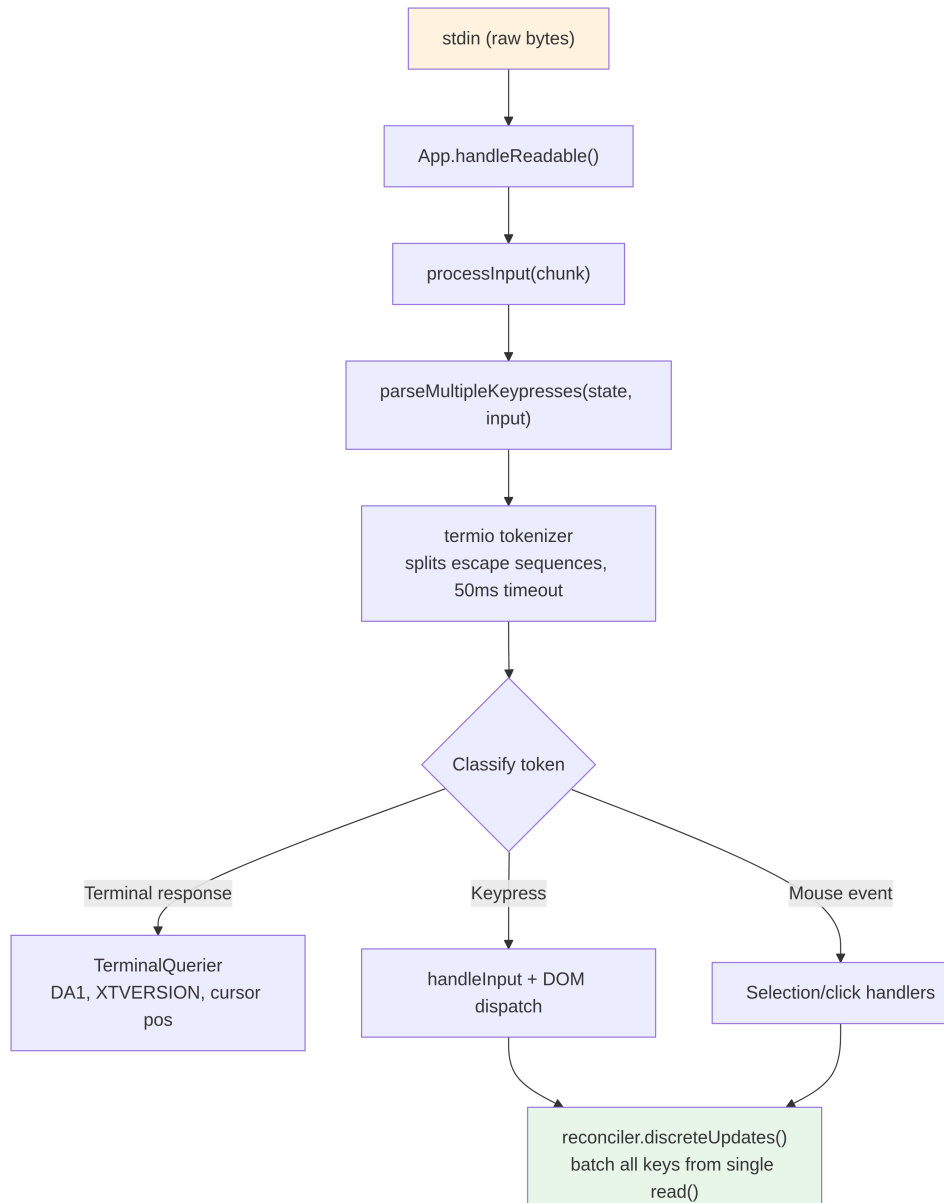
The difficulty is not in any one of these systems. It is in the combinatorial explosion of terminal diversity. iTerm2 sends Kitty keyboard protocol sequences. macOS Terminal sends legacy VT220 sequences. Ghostty over SSH sends xterm `modifyOtherKeys`. tmux may eat, transform, or passthrough any of these depending on its configuration. Windows Terminal has its own quirks with VT mode. The input system must produce correct `ParsedKey` objects from all of them, because a user should not have to know which keyboard protocol their terminal uses.

This chapter traces the path from raw bytes to meaningful actions across that landscape.

The design philosophy is progressive enhancement with graceful degradation. On a modern terminal with Kitty keyboard protocol support, Claude Code gets full modifier detection (Ctrl+Shift+A is distinct from Ctrl+A), super key reporting (Cmd shortcuts), and unambiguous key identification. On a legacy terminal over SSH, it falls back to the best available protocol, losing some modifier distinctions but keeping core functionality intact. The user never sees an error message about their terminal being unsupported. They might not be able to use `ctrl+shift+f` for global search, but `ctrl+r` for history search works everywhere.

14.2 The Key Parsing Pipeline

Input arrives as chunks of bytes on stdin. The pipeline processes them in stages:



The tokenizer is the foundation. Terminal input is a stream of bytes that mixes printable characters, control codes, and multi-byte escape sequences with no explicit framing. A single `read()` from `stdin` might return `\x1b[1;5A` (Ctrl+Up arrow), or it might return `\x1b` in one read and `[1;5A` in the next, depending on how fast bytes arrive from the PTY. The tokenizer maintains

a state machine that buffers partial escape sequences and emits complete tokens.

The incomplete-sequence problem is fundamental. When the tokenizer sees a lone `\x1b`, it cannot know whether this is the Escape key or the start of a CSI sequence. It buffers the byte and starts a 50ms timer. If no continuation arrives, the buffer is flushed and the `\x1b` becomes an Escape keypress. But before flushing, the tokenizer checks `stdin.readableLength` – if bytes are waiting in the kernel buffer, the timer re-arms rather than flushing. This handles the case where the event loop was blocked past 50ms and the continuation bytes are already buffered but not yet read.

For paste operations, the timeout extends to 500ms. Pasted text can be large and arrive in multiple chunks.

All parsed keys from a single `read()` are processed in one `reconciler.discreteUpdates()` call. This batches React state updates so that pasting 100 characters produces one re-render, not 100. The batching is essential: without it, each character in a paste would trigger a full reconciliation cycle – state update, reconciliation, commit, Yoga layout, render, diff, write. At 5ms per cycle, a 100-character paste would take 500ms to process. With batching, the same paste takes one 5ms cycle.

14.2.1 `stdin` Management

The `App` component manages raw mode via reference counting. When any component needs raw input (the prompt, a dialog, vim mode), it calls `setRawMode(true)`, incrementing a counter. When it no longer needs raw input, it calls `setRawMode(false)`, decrementing. Raw mode is only disabled when the counter reaches zero. This prevents a common bug in terminal applications: component A enables raw mode, component B enables raw mode, component A disables raw mode, and suddenly component B's input breaks because raw mode was globally disabled.

When raw mode is first enabled, the `App`:

1. Stops early input capture (the bootstrap-phase mechanism that collects keystrokes before React mounts)
2. Puts `stdin` into raw mode (no line buffering, no echo, no signal processing)
3. Attaches a `readable` listener for async input processing
4. Enables bracketed paste (so pasted text is identifiable)

5. Enables focus reporting (so the app knows when the terminal window gains/loses focus)
6. Enables extended key reporting (Kitty keyboard protocol + `xterm modifyOtherKeys`)

On disable, all of these are reversed in the opposite order. The careful sequencing prevents escape sequence leaks – disabling extended key reporting before disabling raw mode ensures that the terminal does not continue sending Kitty-encoded sequences after the app has stopped parsing them.

The `onExit` signal handler (via the `signal-exit` package) ensures cleanup happens even on unexpected termination. If the process receives `SIGTERM` or `SIGINT`, the handler disables raw mode, restores the terminal state, exits alternate screen if active, and re-shows the cursor before the process exits. Without this cleanup, a crashed Claude Code session would leave the terminal in raw mode with no cursor and no echo – the user would need to blindly type `reset` to recover their terminal.

14.3 Multi-Protocol Support

Terminals do not agree on how to encode keyboard input. A modern terminal emulator like Kitty sends structured sequences with full modifier information. A legacy terminal over SSH sends ambiguous byte sequences that require context to interpret. Claude Code’s parser handles five distinct protocols simultaneously, because the user’s terminal might be any of them.

CSI u (Kitty keyboard protocol) is the modern standard. Format: `ESC [ codepoint [; modifier] u`. Example: `ESC[13;2u` is Shift+Enter, `ESC[27u` is Escape with no modifiers. The codepoint identifies the key unambiguously – there is no ambiguity between Escape-the-key and Escape-as-sequence-prefix. The modifier word encodes shift, alt, ctrl, and super (Cmd) as individual bits. Claude Code enables this protocol on terminals that support it via the `ENABLE_KITTY_KEYBOARD` escape sequence at startup, and disables it on exit via `DISABLE_KITTY_KEYBOARD`. The protocol is detected through a query/response handshake: the application sends `CSI ? u` and the terminal responds with `CSI ? flags u`, where `flags` indicates the supported protocol level.

xterm modifyOtherKeys is the fallback for terminals like Ghostty over SSH, where the Kitty protocol is not negotiated. Format: `ESC [ 27 ;`

modifier ; keycode ~. Note that the parameter order is reversed from CSI `u` – modifier comes before keycode, then keycode. This is a common source of parser bugs. The protocol is enabled via CSI `> 4 ; 2 m` and emitted by Ghostty, tmux, and xterm when the terminal’s TERM identification is not detected (common over SSH where TERM_PROGRAM is not forwarded).

Legacy terminal sequences cover everything else: function keys via ESC `O` and ESC `[` sequences, arrow keys, numpad, Home/End/Insert/Delete, and the full zoo of VT100/VT220/xterm variations accumulated over 40 years of terminal evolution. The parser uses two regular expressions to match these: `FN_KEY_RE` for the ESC `O/N/[/[` prefix pattern (matching function keys, arrow keys, and their modified variants), and `META_KEY_CODE_RE` for meta-key codes (ESC followed by a single alphanumeric, the traditional Alt+key encoding).

The challenge with legacy sequences is ambiguity. ESC `[ 1 ; 2 R` could be Shift+F3 or a cursor position report, depending on context. The parser resolves this with a private-marker check: cursor position reports use CSI `? row ; col R` (with the `?` private marker), while modified function keys use CSI `params R` (without it). This disambiguation is why Claude Code requests DECXCPR (extended cursor position reports) rather than standard CPR – the extended form is unambiguous.

Terminal identification adds another layer of complexity. On startup, Claude Code sends an XTVERSION query (CSI `> 0 q`) to discover the terminal’s name and version. The response (DCS `> | name ST`) survives SSH connections – unlike TERM_PROGRAM, which is an environment variable that does not propagate through SSH. Knowing the terminal identity allows the parser to handle terminal-specific quirks. For example, xterm.js (used by VS Code’s integrated terminal) has different escape sequence behavior from native xterm, and the identification string (`xterm.js(X.Y.Z)`) allows the parser to account for these differences.

SGR mouse events use the format ESC `[ < button ; col ; row M/m`, where `M` is press and `m` is release. Button codes encode the action: 0/1/2 for left/middle/right click, 64/65 for wheel up/down (0x40 OR’d with a wheel bit), 32+ for drag (0x20 OR’d with a motion bit). Wheel events are converted to `ParsedKey` objects so they flow through the keybinding system; click and drag events become `ParsedMouse` objects routed to the selection handler.

Bracketed paste wraps pasted content between ESC `[200~` and ESC `[201~`

markers. Everything between the markers becomes a single `ParsedKey` with `isPasted: true`, regardless of what escape sequences the pasted text might contain. This prevents pasted code from being interpreted as commands – a critical safety feature when a user pastes a code snippet containing `\x03` (which is Ctrl+C as a raw byte).

The output types from the parser form a clean discriminated union:

```
type ParsedKey = {
  kind: 'key';
  name: string;          // 'return', 'escape', 'a', 'f1', etc.
  ctrl: boolean; meta: boolean; shift: boolean;
  option: boolean; super: boolean;
  sequence: string;      // Raw escape sequence for debugging
  isPasted: boolean;     // Inside bracketed paste
}

type ParsedMouse = {
  kind: 'mouse';
  button: number;        // SGR button code
  action: 'press' | 'release';
  col: number; row: number; // 1-indexed terminal coordinates
}

type ParsedResponse = {
  kind: 'response';
  response: TerminalResponse; // Routed to TerminalQuerier
}
```

The `kind` discriminant ensures that downstream code handles each input type explicitly. A key cannot be accidentally processed as a mouse event; a terminal response cannot be accidentally interpreted as a keypress. The `ParsedKey` type also carries the raw `sequence` string for debugging – when a user reports “pressing Ctrl+Shift+A does nothing,” the debug log can show exactly what byte sequence the terminal sent, making it possible to diagnose whether the issue is in the terminal’s encoding, the parser’s recognition, or the keybinding’s configuration.

The `isPasted` flag on `ParsedKey` is critical for security. When bracketed paste is enabled, the terminal wraps pasted content in marker sequences. The parser sets `isPasted: true` on the resulting key event, and the keybinding

resolver skips keybinding matching for pasted keys. Without this, pasting text containing `\x03` (Ctrl+C as a raw byte) or escape sequences would trigger application commands. With it, pasted content is treated as literal text input regardless of its byte content.

The parser also recognizes terminal responses – sequences sent by the terminal itself in answer to queries. These include device attributes (DA1, DA2), cursor position reports, Kitty keyboard flag responses, XTVERSION (terminal identification), and DECRPM (mode status). These are routed to a `TerminalQuerier` rather than the input handler:

```
type TerminalResponse =
  | { type: 'decrpm'; mode: number; status: number }
  | { type: 'da1'; params: number[] }
  | { type: 'da2'; params: number[] }
  | { type: 'kittyKeyboard'; flags: number }
  | { type: 'cursorPosition'; row: number; col: number }
  | { type: 'osc'; code: number; data: string }
  | { type: 'xtversion'; version: string }
```

Modifier decoding follows the XTerm convention: the modifier word is $1 + (\text{shift} ? 1 : 0) + (\text{alt} ? 2 : 0) + (\text{ctrl} ? 4 : 0) + (\text{super} ? 8 : 0)$. The `meta` field in `ParsedKey` maps to Alt/Option (bit 2). The `super` field is distinct (bit 8, Cmd on macOS). This distinction matters because Cmd shortcuts are reserved by the OS and cannot be captured by terminal applications – unless the terminal uses the Kitty protocol, which reports super-modified keys that other protocols silently swallow.

A stdin-gap detector triggers terminal mode re-assertion when no input arrives for 5 seconds after a gap. This handles tmux reattach and laptop wake scenarios, where the terminal’s keyboard mode may have been reset by the multiplexer or the OS. When re-assertion fires, it re-sends `ENABLE_KITTY_KEYBOARD`, `ENABLE_MODIFY_OTHER_KEYS`, bracketed paste, and focus reporting sequences. Without this, detaching from a tmux session and reattaching would silently downgrade the keyboard protocol to legacy mode, breaking modifier detection for the rest of the session.

14.3.1 The Terminal I/O Layer

Beneath the parser sits a structured terminal I/O subsystem in `ink/termio/`:

- **csi.ts** – CSI (Control Sequence Introducer) sequences: cursor move-

ment, erase, scroll regions, bracketed paste enable/disable, focus event enable/disable, Kitty keyboard protocol enable/disable

- **dec.ts** – DEC private mode sequences: alternate screen buffer (1049), mouse tracking modes (1000/1002/1003), cursor visibility, bracketed paste (2004), focus events (1004)
- **osc.ts** – Operating System Commands: clipboard access (OSC 52), tab status, iTerm2 progress indicators, tmux/screen multiplexer wrapping (DCS passthrough for sequences that need to traverse a multiplexer boundary)
- **sgr.ts** – Select Graphic Rendition: the ANSI style code system (colors, bold, italic, underline, inverse)
- **tokenize.ts** – The stateful tokenizer for escape sequence boundary detection

The multiplexer wrapping deserves a note. When Claude Code runs inside tmux, certain escape sequences (like Kitty keyboard protocol negotiation) must pass through to the outer terminal. tmux uses DCS passthrough (ESC P ... ST) to forward sequences it does not understand. The `wrapForMultiplexer` function in `osc.ts` detects the multiplexer environment and wraps sequences appropriately. Without this, Kitty keyboard mode would silently fail inside tmux, and the user would never know why their Ctrl+Shift bindings stopped working.

14.3.2 The Event System

The `ink/events/` directory implements a browser-compatible event system with seven event types: `KeyboardEvent`, `ClickEvent`, `FocusEvent`, `InputEvent`, `TerminalFocusEvent`, and base `TerminalEvent`. Each carries `target`, `currentTarget`, `eventPhase`, and supports `stopPropagation()`, `stopImmediatePropagation()`, and `preventDefault()`.

The `InputEvent` wrapping `ParsedKey` exists for backward compatibility with the legacy `EventEmitter` path, which older components may still use. New components use the DOM-style keyboard event dispatch with capture/bubble phases. Both paths fire from the same parsed key, so they are always consistent – a key that arrives on stdin produces exactly one `ParsedKey`, which spawns both an `InputEvent` (for legacy listeners) and a `KeyboardEvent` (for DOM-style dispatch). This dual-path design allows incremental migration from the `EventEmitter` pattern to the DOM event pattern without breaking existing components.

14.4 The Keybinding System

The keybinding system separates three concerns that are often tangled together: what key triggers what action (bindings), what happens when an action fires (handlers), and which bindings are active right now (contexts).

14.4.1 Bindings: Declarative Configuration

Default bindings are defined in `defaultBindings.ts` as an array of `KeybindingBlock` objects, each scoped to a context:

```
export const DEFAULT_BINDINGS: KeybindingBlock[] = [
  {
    context: 'Global',
    bindings: {
      'ctrl+c': 'app:interrupt',
      'ctrl+d': 'app:exit',
      'ctrl+l': 'app:redraw',
      'ctrl+r': 'history:search',
    },
  },
  {
    context: 'Chat',
    bindings: {
      'escape': 'chat:cancel',
      'ctrl+x ctrl+k': 'chat:killAgents',
      'enter': 'chat:submit',
      'up': 'history:previous',
      'ctrl+x ctrl+e': 'chat:externalEditor',
    },
  },
  // ... 14 more contexts
]
```

Platform-specific bindings are handled at definition time. Image paste is `ctrl+v` on macOS/Linux but `alt+v` on Windows (where `ctrl+v` is system paste). Mode cycling is `shift+tab` on terminals with VT mode support but `meta+m` on Windows Terminal without it. Feature-flagged bindings (quick search, voice mode, terminal panel) are conditionally included.

Users can override any binding via `~/.claude/keybindings.json`. The parser accepts modifier aliases (`ctrl/control`, `alt/opt/option`, `cmd/command/super/win`), key aliases (`esc -> escape`, `return -> enter`), chord notation (space-separated steps like `ctrl+k ctrl+s`), and null actions to unbind default keys. A null action is not the same as not defining a binding – it explicitly blocks the default binding from firing, which is important for users who want to reclaim a key for their terminal’s use.

14.4.2 Contexts: 16 Scopes of Activity

Each context represents a mode of interaction where a specific set of bindings applies:

Context	When Active
Global	Always
Chat	Prompt input is focused
Autocomplete	Completion menu is visible
Confirmation	Permission dialog is showing
Scroll	Alt-screen with scrollable content
Transcript	Read-only transcript viewer
HistorySearch	Reverse history search (<code>ctrl+r</code>)
Task	A background task is running
Help	Help overlay is displayed
MessageSelector	Rewind dialog
MessageActions	Message cursor navigation
DiffDialog	Diff viewer
Select	Generic selection list
Settings	Config panel
Tabs	Tab navigation
Footer	Footer indicators

When a key arrives, the resolver builds a context list from the currently active contexts (determined by React component state), deduplicates it preserving priority order, and searches for a matching binding. The last matching binding wins – this is how user overrides take precedence over defaults. The context list is rebuilt on every keystroke (it is cheap: array concatenation and deduplication of at most 16 strings), so context changes take effect immediately without any subscription or listener mechanism.

The context design handles a tricky interaction pattern: nested modals. When a permission dialog appears during a running task, both **Confirmation** and **Task** contexts might be active. The **Confirmation** context takes priority (it is registered later in the component tree), so **y** triggers “approve” rather than any task-level binding. When the dialog closes, the **Confirmation** context deactivates and **Task** bindings resume. This stacking behavior emerges naturally from the context list’s priority ordering – no special modal-handling code is needed.

14.4.3 Reserved Shortcuts

Not everything can be rebound. The system enforces three tiers of reservation:

Non-rebindable (hardcoded behavior): **ctrl+c** (interrupt/exit), **ctrl+d** (exit), **ctrl+m** (identical to Enter in all terminals – rebinding it would break Enter).

Terminal-reserved (warnings): **ctrl+z** (SIGTSTP), **ctrl+** (SIGQUIT). These can technically be bound, but the terminal will intercept them before the application sees them in most configurations.

macOS-reserved (errors): **cmd+c**, **cmd+v**, **cmd+x**, **cmd+q**, **cmd+w**, **cmd+tab**, **cmd+space**. The OS intercepts these before they reach the terminal. Binding them would create a shortcut that never fires.

14.4.4 The Resolution Flow

When a key arrives, the resolution path is:

1. Build the context list: the component’s registered active contexts plus Global, deduplicated with priority preserved
2. Call `resolveKeyWithChordState(input, key, contexts)` against the merged binding table
3. On **match**: clear any pending chord, call the handler, `stopImmediatePropagation()` on the event
4. On **chord_started**: save the pending keystrokes, stop propagation, start the chord timeout
5. On **chord_cancelled**: clear the pending chord, let the event fall through
6. On **unbound**: clear the chord – this is an explicit unbinding (user set the action to `null`), so propagation is stopped but no handler runs
7. On **none**: fall through to other handlers

The “last wins” resolution strategy means that if both the default bindings and user bindings define `ctrl+k` in the `Chat` context, the user’s binding takes precedence. This is evaluated at match time by iterating bindings in definition order and keeping the last match, rather than building an override map at load time. The advantage: context-specific overrides compose naturally. A user can override `enter` in `Chat` without affecting `enter` in `Confirmation`.

14.5 Chord Support

The `ctrl+x ctrl+k` binding is a chord: two keystrokes that together form a single action. The resolver manages this with a state machine.

When a key arrives:

1. The resolver appends it to any pending chord prefix
2. It checks whether any binding’s chord starts with this prefix. If so, it returns `chord_started` and saves the pending keystrokes
3. If the full chord matches a binding exactly, it returns `match` and clears the pending state
4. If the chord prefix matches nothing, it returns `chord_cancelled`

A `ChordInterceptor` component intercepts all input during the chord wait state. It has a 1000ms timeout – if the second keystroke does not arrive within a second, the chord is cancelled and the first keystroke is discarded. The `KeybindingContext` provides a `pendingChordRef` for synchronous access to the pending state, avoiding React state update delays that could cause the second keystroke to be processed before the first one’s state update completes.

The chord design avoids shadowing readline editing keys. Without chords, the keybinding for “kill agents” might be `ctrl+k` – but that is readline’s “kill to end of line,” which users expect in a terminal text input. By using `ctrl+x` as a prefix (matching readline’s own chord prefix convention), the system gets a namespace of bindings that do not conflict with single-key editing shortcuts.

The implementation handles an edge case that most chord systems miss: what happens when the user presses `ctrl+x` but then types a character that is not part of any chord? Without careful handling, that character would be swallowed – the chord interceptor consumed the input, the chord

was cancelled, and the character is gone. Claude Code’s `ChordInterceptor` returns `chord_cancelled` in this case, which causes the pending input to be discarded but allows the non-matching character to fall through to normal input processing. The character is not lost; only the chord prefix is discarded. This matches the behavior users expect from Emacs-style chord prefixes.

14.6 Vim Mode

14.6.1 The State Machine

The vim implementation is a pure state machine with exhaustive type checking. The types are the documentation:

```
export type VimState =
  | { mode: 'INSERT'; insertedText: string }
  | { mode: 'NORMAL'; command: CommandState }

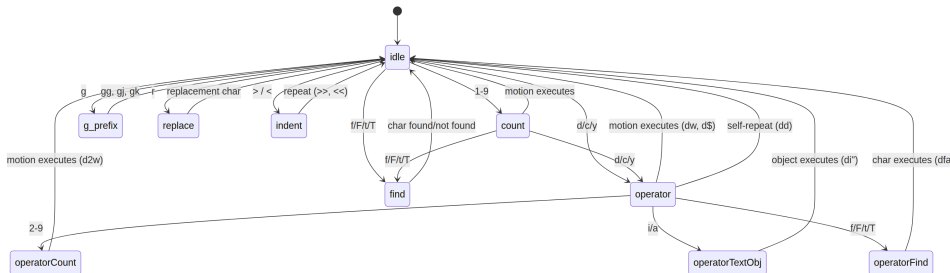
export type CommandState =
  | { type: 'idle' }
  | { type: 'count'; digits: string }
  | { type: 'operator'; op: Operator; count: number }
  | { type: 'operatorCount'; op: Operator; count: number; digits: string }
  | { type: 'operatorFind'; op: Operator; count: number; find: FindType }
  | { type: 'operatorTextObj'; op: Operator; count: number; scope: TextObjScope }
  | { type: 'find'; find: FindType; count: number }
  | { type: 'g'; count: number }
  | { type: 'operatorG'; op: Operator; count: number }
  | { type: 'replace'; count: number }
  | { type: 'indent'; dir: '>' | '<'; count: number }
```

This is a discriminated union with 12 variants. TypeScript’s exhaustive checking ensures that every `switch` statement on `CommandState.type` handles all 12 cases. Adding a new state to the union causes every incomplete switch to produce a compile error. The state machine cannot have dead states or missing transitions – the type system forbids it.

Notice how each state carries exactly the data needed for the next transition. The `operator` state knows which operator (`op`) and the preceding count. The `operatorCount` state adds the digit accumulator (`digits`). The `operatorTextObj` state adds the scope (`inner` or `around`). No state carries

data it does not need. This is not just good taste – it prevents an entire class of bugs where a handler reads stale data from a previous command. If you are in the **find** state, you have a **FindType** and a **count**. You do not have an operator, because no operator is pending. The type makes the impossible state unrepresentable.

The state diagram tells the story:



From **idle**, pressing **d** enters the **operator** state. From **operator**, pressing **w** executes **delete** with the **w** motion. Pressing **d** again (**dd**) triggers a line deletion. Pressing **2** enters **operatorCount**, so **d2w** becomes “delete the next 2 words.” Pressing **i** enters **operatorTextObj**, so **di** becomes “delete inside quotes.” Every intermediate state carries exactly the context needed for the next transition – no more, no less.

14.6.2 Transitions as Pure Functions

The `transition()` function dispatches on the current state type to one of 10 handler functions. Each returns a **TransitionResult**:

```
type TransitionResult = {
  next?: CommandState;    // New state (omitted = stay in current)
  execute?: () => void;    // Side effect (omitted = no action yet)
}
```

Side effects are returned, not executed. The transition function is pure – given a state and a key, it returns the next state and optionally a closure that performs the action. The caller decides when to run the effect. This makes the state machine trivially testable: feed it states and keys, assert on the returned states, ignore the closures. It also means the transition function has no dependencies on the editor state, the cursor position, or the buffer content. Those details are captured by the closure at creation time, not consumed by the state machine at transition time.

The `fromIdle` handler is the entry point and covers the full vim vocabulary:

- **Count prefix:** 1-9 enters the `count` state, accumulating digits. 0 is special – it is the “start of line” motion, not a count digit, unless digits have already been accumulated
- **Operators:** `d`, `c`, `y` enter the `operator` state, waiting for a motion or text object to define the range
- **Find:** `f`, `F`, `t`, `T` enter the `find` state, waiting for a character to search for
- **G-prefix:** `g` enters the `g` state for composite commands (`gg`, `gj`, `gk`)
- **Replace:** `r` enters the `replace` state, waiting for the replacement character
- **Indent:** `>`, `<` enter the `indent` state (for `>>` and `<<`)
- **Simple motions:** `h/j/k/l/w/b/e/W/B/E/O/^/$` execute immediately, moving the cursor
- **Immediate commands:** `x` (delete char), `~` (toggle case), `J` (join lines), `p/P` (paste), `D/C/Y` (operator shortcuts), `G` (go to end), `.` (dot-repeat), `;/`, (find repeat), `u` (undo), `i/I/a/A/o/O` (enter insert mode)

14.6.3 Motions, Operators, and Text Objects

Motions are pure functions mapping a key to a cursor position. `resolveMotion(key, cursor, count)` applies the motion `count` times, short-circuiting if the cursor stops moving (you cannot move left past column 0). This short-circuit is important for `3w` at the end of a line – it stops at the last word rather than wrapping or erroring.

Motions are classified by how they interact with operators:

- **Exclusive** (default) – the character at the destination is NOT included in the range. `dw` deletes up to but not including the first character of the next word
- **Inclusive** (`e`, `E`, `$`) – the character at the destination IS included. `de` deletes through the last character of the current word
- **Linewise** (`j`, `k`, `G`, `gg`, `gj`, `gk`) – when used with operators, the range extends to cover full lines. `dj` deletes the current line and the one below, not just the characters between the two cursor positions

Operators apply to a range. `delete` removes text and saves it to the register. `change` removes text and enters insert mode. `yank` copies to the register without modification. The `cw/cW` special case follows vim convention: change-word goes to the end of the current word, not the start of the next

word (unlike `dw`).

One interesting edge case: `[Image #N]` chip snapping. When a word motion lands inside an image reference chip (rendered as a single visual unit in the terminal), the range extends to cover the entire chip. This prevents partial deletions of what the user perceives as an atomic element – you cannot delete half of `[Image #3]` because the motion system treats the entire chip as a single word.

Additional commands cover the full expected vim vocabulary: `x` (delete character), `r` (replace character), `~` (toggle case), `J` (join lines), `p/P` (paste with linewise/characterwise awareness), `>> / <<` (indent/outdent with 2-space stops), `o/O` (open line below/above and enter insert mode).

Text objects find boundaries around the cursor. They answer the question: “what is the ‘thing’ the cursor is inside?”

Word objects (`iw`, `aw`, `iW`, `aW`) segment text into graphemes, classify each as word-character, whitespace, or punctuation, and expand the selection to the word boundary. The `i` (inner) variant selects just the word. The `a` (around) variant includes surrounding whitespace – trailing whitespace preferred, falling back to leading if at line end. The uppercase variants (`W`, `aW`) treat any non-whitespace sequence as a word, ignoring punctuation boundaries.

Quote objects (`i"`, `a"`, `i'`, `a'`, `i``, `a``) find paired quotes on the current line. Pairs are matched in order (first and second quote form a pair, third and fourth form the next pair, etc.). If the cursor is between the first and second quote, that is the match. The `a` variant includes the quote characters; the `i` variant excludes them.

Bracket objects (`ib/i(`, `ab/a(`, `i[/a[`, `iB/i{/aB/a{`, `i</a<`) do depth-tracking search for matching delimiters. They search outward from the cursor, maintaining a nesting count, until they find the matching pair at depth zero. This correctly handles nested brackets – `d i ( inside foo((bar))` deletes `bar`, not `(bar)`.

14.6.4 Persistent State and Dot-Repeat

The vim mode maintains a `PersistentState` that survives across commands – the “memory” that makes vim feel like vim:

```
interface PersistentState {
  lastChange: RecordedChange; // For dot-repeat
  lastFind: { type: FindType; char: string }; // For ; and ,
  register: string; // Yank buffer
  registerIsLinewise: boolean; // Paste behavior flag
}
```

Every mutating command records itself as a **RecordedChange** – a discriminated union covering insert, operator+motion, operator+textObj, operator+find, replace, delete-char, toggle-case, indent, open-line, and join. The `.` command replays **lastChange** from persistent state, using the recorded count, operator, and motion to reproduce the exact same edit at the current cursor position.

Find-repeat (`;` and `,`) uses **lastFind**. The `;` command repeats the last find in the same direction. The `,` command flips the direction: `f` becomes `F`, `t` becomes `T`, and vice versa. This means after `fa` (find next ‘a’), `;` finds the next ‘a’ forward and `,` finds the next ‘a’ backward – without the user having to remember which direction they were searching.

The register tracks yanked and deleted text. When register content ends with `\n`, it is flagged as linewise, which changes paste behavior: `p` inserts below the current line (not after the cursor), and `P` inserts above. This distinction is invisible to the user but critical for the “delete a line, paste it somewhere else” workflow that vim users rely on constantly.

14.7 Virtual Scrolling

Long Claude Code sessions produce long conversations. A heavy debugging session might generate 200+ messages, each containing markdown, code blocks, tool use results, and permission records. Without virtualization, React would maintain 200+ component subtrees in memory, each with its own state, effects, and memoization caches. The DOM tree would contain thousands of nodes. Yoga layout would visit all of them on every frame. The terminal would be unusable.

The **VirtualMessageList** component solves this by rendering only the messages visible in the viewport plus a small buffer above and below. In a conversation with hundreds of messages, this is the difference between mount-

ing 500 React subtrees (each with markdown parsing, syntax highlighting, and tool use blocks) and mounting 15.

The component maintains:

- **Height cache** per message, invalidated when terminal column count changes
- **Jump handle** for transcript search navigation (jump to index, next/previous match)
- **Search text extraction** with warm-cache support (pre-lowercase all messages when the user enters /)
- **Sticky prompt tracking** – when the user scrolls away from the input, their last prompt text appears at the top as context
- **Message actions navigation** – cursor-based message selection for the rewind feature

The `useVirtualScroll` hook computes which messages to mount based on `scrollTop`, `viewportHeight`, and cumulative message heights. It maintains scroll clamp bounds on the `ScrollBox` to prevent blank screens when burst `scrollTo` calls race past React’s async re-render – a classic problem with virtualized lists where the scroll position can outrun the DOM update.

The interaction between virtual scrolling and the markdown token cache is worth noting. When a message scrolls out of the viewport, its React subtree unmounts. When the user scrolls back, the subtree remounts. Without caching, this would mean re-parsing the markdown for every message the user scrolls past. The module-level LRU cache (500 entries, keyed by content hash) ensures that the expensive `marked.lexer()` call happens at most once per unique message content, regardless of how many times the component mounts and unmounts.

The `ScrollBox` component itself provides an imperative API via `useImperativeHandle`:

- `scrollTo(y)` – absolute scroll, breaks sticky-scroll mode
- `scrollBy(dy)` – accumulates into `pendingScrollDelta`, drained by the renderer at a capped rate
- `scrollToElement(el, offset)` – defers position read to render time via `scrollAnchor`
- `scrollToBottom()` – re-enables sticky-scroll mode
- `setClampBounds(min, max)` – constrains the virtual scroll window

All scroll mutations go directly to DOM node properties and schedule renders

via microtask, bypassing React’s reconciler. The `markScrollActivity()` call signals background intervals (spinners, timers) to skip their next tick, reducing event-loop contention during active scrolling. This is a cooperative scheduling pattern: the scroll path tells background work “I am in a latency-sensitive operation, please yield.” Background intervals check this flag before scheduling their next tick and delay by one frame if scrolling is active. The result is consistently smooth scrolling even when multiple spinners and timers are running in the background.

14.8 Apply This: Building a Context-Aware Key-binding System

Claude Code’s keybinding architecture offers a template for any application with modal input – editors, IDEs, drawing tools, terminal multiplexers. The key insights:

Separate bindings from handlers. Bindings are data (which key maps to which action name). Handlers are code (what happens when the action fires). Keeping them separate means bindings can be serialized to JSON for user customization, while handlers remain in the components that own the relevant state. A user can rebind `ctrl+k` to `chat:submit` without touching any component code.

Context as a first-class concept. Instead of one flat keymap, define contexts that activate and deactivate based on application state. When a dialog opens, the `Confirmation` context activates and its bindings take precedence over `Chat` bindings. When the dialog closes, `Chat` bindings resume. This eliminates the conditional soup of `if (dialogOpen && key === 'y')` scattered through event handlers.

Chord state as an explicit machine. Multi-key sequences (chords) are not a special case of single-key bindings – they are a different kind of binding that requires a state machine with timeout and cancellation semantics. Making this explicit (with a dedicated `ChordInterceptor` component and a `pendingChordRef`) prevents subtle bugs where the second keystroke of a chord is consumed by a different handler because React’s state update had not yet propagated.

Reserve early, warn clearly. Identify keys that cannot be rebound (system

shortcuts, terminal control characters) at definition time, not at resolution time. When a user tries to bind `ctrl+c`, show an error during configuration loading rather than silently accepting a binding that will never fire. This is the difference between a keybinding system that works and one that produces mysterious bug reports.

Design for terminal diversity. Claude Code’s keybinding system defines platform-specific alternatives at the binding level, not the handler level. Image paste is `ctrl+v` or `alt+v` depending on the OS. Mode cycling is `shift+tab` or `meta+m` depending on VT mode support. The handler for each action is the same regardless of which key triggers it. This means testing covers one code path per action, not one per platform-key combination. And when a new terminal quirk surfaces (Windows Terminal lacking VT mode before Node 24.2.0, for example), the fix is a single conditional in the binding definition, not a scattered set of `if (platform === 'windows')` checks in handler code.

Provide escape hatches. The null-action unbinding mechanism is small but important. A user who runs Claude Code inside a terminal multiplexer might find that `ctrl+t` (toggle todos) conflicts with their multiplexer’s tab-switching shortcut. By adding `{ "ctrl+t": null }` to their `keybindings.json`, they disable the binding entirely. The key press passes through to the multiplexer. Without null unbinding, the user’s only option would be to rebind `ctrl+t` to some other action they do not want, or to reconfigure their multiplexer – neither of which is a good experience.

The vim mode implementation adds one more lesson: **make the type system enforce your state machine**. The 12-variant `CommandState` union makes it impossible to forget a state in a switch statement. The `TransitionResult` type separates state changes from side effects, making the machine testable as a pure function. If your application has modal input, express the modes as a discriminated union and let the compiler verify exhaustiveness. The time spent defining the types pays for itself in eliminated runtime bugs.

Consider the alternative: a vim implementation using mutable state and imperative conditionals. The `fromOperator` handler would be a nest of `if (mode === 'operator' && pendingCount !== null && isDigit(key))` checks, with each branch mutating shared variables. Adding a new state (say, a macro-recording mode) would require auditing every branch to ensure the new state is handled. With a discriminated union, the compiler does the audit – the PR that adds the new variant will not build until every switch

statement handles it.

This is the deeper lesson of Claude Code’s input system: at every layer – tokenizer, parser, keybinding resolver, vim state machine – the architecture converts unstructured input into typed, exhaustively handled structures as early as possible. Raw bytes become `ParsedKey` at the parser boundary. `ParsedKey` becomes an action name at the keybinding boundary. The action name becomes a typed handler at the component boundary. Each conversion narrows the space of possible states, and each narrowing is enforced by TypeScript’s type system. By the time a keystroke reaches application logic, the ambiguity is gone. There is no “what if the key is undefined?” There is no “what if the modifier combination is impossible?” The types have already forbidden those states from existing.

The two chapters together tell one story. Chapter 13 showed how the rendering system eliminates unnecessary work – blitting unchanged regions, interning repeated values, diffing at the cell level, tracking damage bounds. Chapter 14 showed how the input system eliminates ambiguity – parsing five protocols into one type, resolving keys against contextual bindings, expressing modal state as exhaustive unions. The rendering system answers “how do you paint 24,000 cells 60 times per second?” The input system answers “how do you turn a byte stream into meaningful actions across a fragmented ecosystem?” Both answers follow the same principle: push complexity to the boundaries, where it can be handled once and correctly, so that everything downstream operates on clean, typed, well-bounded data. The terminal is chaos. The application is order. The boundary code does the hard work of converting one into the other.

14.9 Summary: Two Systems, One Design Philosophy

Chapters 13 and 14 covered the two halves of the terminal interface: output and input. Despite their different concerns, both systems follow the same architectural principles.

Interning and indirection. The rendering system interns characters, styles, and hyperlinks into pools, replacing string comparisons with integer comparisons throughout the hot path. The input system interns escape sequences into structured `ParsedKey` objects at the parser boundary, replacing

byte-level pattern matching with typed field access throughout the handler path.

Layered elimination of work. The rendering system stacks five optimizations (dirty flags, blit, damage rectangles, cell-level diff, patch optimization) that each eliminate a category of unnecessary computation. The input system stacks three (tokenizer, protocol parser, keybinding resolver) that each eliminate a category of ambiguity.

Pure functions and typed state machines. The vim mode is a pure state machine with typed transitions. The keybinding resolver is a pure function from (key, contexts, chord-state) to resolution-result. The rendering pipeline is a pure function from (DOM tree, previous screen) to (new screen, patches). Side effects happen at the boundaries – writing to stdout, dispatching to React – not in the core logic.

Graceful degradation across environments. The rendering system adapts to terminal size, alt-screen support, and synchronized-update protocol availability. The input system adapts to Kitty keyboard protocol, xterm modifyOtherKeys, legacy VT sequences, and multiplexer passthrough requirements. Neither system requires a specific terminal to function; both get better on more capable terminals.

These principles are not specific to terminal applications. They apply to any system that must process high-frequency input and produce low-latency output across a diverse set of runtime environments. The terminal just happens to be an environment where the constraints are sharp enough that violating these principles produces immediately visible degradation – a dropped frame, a swallowed keystroke, a flicker. That sharpness makes it an excellent teacher.

The next chapter moves from the UI layer to the protocol layer: how Claude Code implements MCP – the universal tool protocol that lets any external service become a first-class tool. The terminal UI handles the last mile of the user experience – converting data structures into pixels on a screen and keystrokes into application actions. MCP handles the first mile of extensibility – discovering, connecting, and executing tools that live outside the agent’s own codebase. Between them, the memory system (Chapter 11) and the skills/hooks system (Chapter 12) define the intelligence and control layers. The quality ceiling of the entire system depends on all four: no amount of model intelligence compensates for a laggy UI, and no amount of rendering performance compensates for a model that cannot reach the

tools it needs.

Part VI

Part VI: Connectivity

Chapter 15

Chapter 15: MCP – The Universal Tool Protocol

15.1 Why MCP Matters Beyond Claude Code

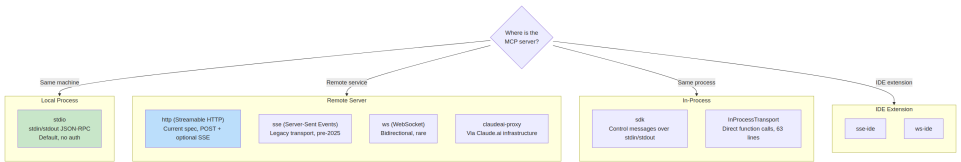
Every other chapter in this book is about Claude Code’s internals. This one is different. The Model Context Protocol is an open specification that any agent can implement, and Claude Code’s MCP subsystem is one of the most complete production clients in existence. If you are building an agent that needs to call external tools – any agent, in any language, on any model – the patterns in this chapter transfer directly.

The core proposition is straightforward: MCP defines a JSON-RPC 2.0 protocol for tool discovery and invocation between a client (the agent) and a server (the tool provider). The client sends `tools/list` to discover what a server offers, then `tools/call` to execute. The server describes each tool with a name, description, and JSON Schema for its inputs. That is the entire contract. Everything else – transport selection, authentication, config loading, tool name normalization – is the implementation work that turns a clean spec into something that survives contact with the real world.

Claude Code’s MCP implementation spans four core files: `types.ts`, `client.ts`, `auth.ts`, and `InProcessTransport.ts`. Together they support eight transport types, seven configuration scopes, OAuth discovery across two RFCs, and a tool wrapping layer that makes MCP tools indistinguishable from built-in ones – the same `Tool` interface covered in Chapter 6. This chapter walks through each layer.

15.2 Eight Transport Types

The first design decision in any MCP integration is how the client talks to the server. Claude Code supports eight transport configurations:



Three design choices are worth noting. First, `stdio` is the default – when `type` is omitted, the system assumes a local subprocess. This is backwards-compatible with the earliest MCP configs. Second, the fetch wrappers stack: timeout wrapping outside step-up detection, outside the base fetch. Each wrapper handles one concern. Third, the `ws-ide` branch has a Bun/Node runtime split – Bun’s `WebSocket` accepts proxy and TLS options natively, while Node requires the `ws` package.

When to use which. For local tools (filesystem, database, custom scripts), `stdio` – no network, no auth, just pipes. For remote services, `http` (Streamable HTTP) is the current spec recommendation. `sse` is legacy but widely deployed. The `sdk`, `IDE`, and `claudeai-proxy` types are internal to their respective ecosystems.

15.3 Configuration Loading and Scoping

MCP server configs load from seven scopes, merged and deduplicated:

Scope	Source	Trust
local	<code>.mcp.json</code> in working directory	Requires user approval
user	<code>~/.claude.json</code> <code>mcpServers</code> field	User-managed
project	Project-level config	Shared project settings
enterprise	Managed enterprise config	Pre-approved by org
managed	Plugin-provided servers	Auto-discovered
claudeai	Claude.ai web interface	Pre-authorized via web
dynamic	Runtime injection (SDK)	Programmatically added

Deduplication is content-based, not name-based. Two servers with different names but the same command or URL are recognized as the same server. The `getMcpServerSignature()` function computes a canonical key: `stdio:["command","arg1"]` for local servers, `url:https://example.com/mcp` for remote ones. Plugin-provided servers whose signature matches a manual config are suppressed.

15.4 Tool Wrapping: From MCP to Claude Code

When a connection succeeds, the client calls `tools/list`. Each tool definition is transformed into Claude Code's internal `Tool` interface – the same interface used by built-in tools. After wrapping, the model cannot distinguish between a built-in tool and an MCP tool.

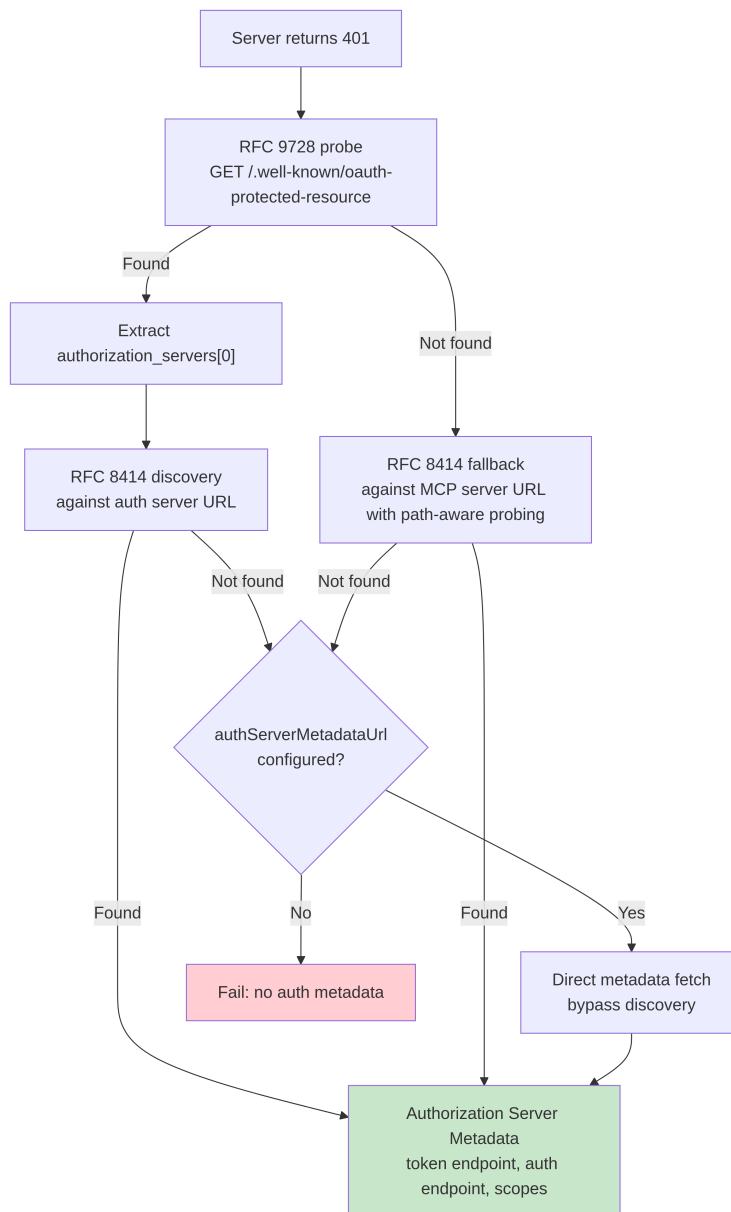
The wrapping process has four stages:

- 1. Name normalization.** `normalizeNameForMCP()` replaces invalid characters with underscores. The fully qualified name follows `mcp_{serverName}_{toolName}`.
 - 2. Description truncation.** Capped at 2,048 characters. OpenAPI-generated servers have been observed dumping 15-60KB into `tool.description` – roughly 15,000 tokens per turn for a single tool.
 - 3. Schema passthrough.** The tool's `inputSchema` passes directly to the API. No transformation, no validation at wrapping time. Schema errors surface at call time, not registration time.
 - 4. Annotation mapping.** MCP annotations map to behavior flags: `readOnlyHint` marks tools safe for concurrent execution (as discussed in Chapter 7's streaming executor), `destructiveHint` triggers extra permission scrutiny. These annotations come from the MCP server – a malicious server could mark a destructive tool as read-only. This is an accepted trust boundary, but one worth understanding: the user opted into the server, and a malicious server marking destructive tools as read-only is a real attack vector. The system accepts this tradeoff because the alternative – ignoring annotations entirely – would prevent legitimate servers from improving the user experience.
-

15.5 OAuth for MCP Servers

Remote MCP servers often require authentication. Claude Code implements the full OAuth 2.0 + PKCE flow with RFC-based discovery, Cross-App Access, and error body normalization.

15.5.1 Discovery Chain



The `authServerMetadataUrl` escape hatch exists because some OAuth servers implement neither RFC.

15.5.2 Cross-App Access (XAA)

When an MCP server config has `oauth.xaa: true`, the system performs federated token exchange through an Identity Provider – one IdP login unlocks multiple MCP servers.

15.5.3 Error Body Normalization

The `normalizeOAuthErrorBody()` function handles OAuth servers that violate the spec. Slack returns HTTP 200 for error responses with the error buried in the JSON body. The function peeks at 2xx POST response bodies, and when the body matches `OAuthErrorResponseSchema` but not `OAuthTokensSchema`, rewrites the response to HTTP 400. It also normalizes Slack-specific error codes (`invalid_refresh_token`, `expired_refresh_token`, `token_expired`) to the standard `invalid_grant`.

15.6 In-Process Transport

Not every MCP server needs to be a separate process. The `InProcessTransport` class enables running an MCP server and client in the same process:

```
class InProcessTransport implements Transport {
  async send(message: JSONRPCMessage): Promise<void> {
    if (this.closed) throw new Error('Transport is closed')
    queueMicrotask(() => { this.peer?.onmessage?.(message) })
  }
  async close(): Promise<void> {
    if (this.closed) return
    this.closed = true
    this.onclose?.()
    if (this.peer && !this.peer.closed) {
      this.peer.closed = true
      this.peer.onclose?.()
    }
  }
}
```

The entire file is 63 lines. Two design decisions deserve attention. First, `send()` delivers via `queueMicrotask()` to prevent stack depth issues in

synchronous request/response cycles. Second, `close()` cascades to the peer, preventing half-open states. The Chrome MCP server and Computer Use MCP server both use this pattern.

15.7 Connection Management

15.7.1 Connection States

Each MCP server connection exists in one of five states: `connected`, `failed`, `needs-auth` (with a 15-minute TTL cache to prevent 30 servers from independently discovering the same expired token), `pending`, or `disabled`.

15.7.2 Session Expiry Detection

MCP's Streamable HTTP transport uses session IDs. When a server restarts, requests return HTTP 404 with JSON-RPC error code -32001. The `isMcpSessionExpiredError()` function checks both signals – note that it uses string inclusion on the error message to detect the error code, which is pragmatic but fragile:

```
export function isMcpSessionExpiredError(error: Error): boolean {
  const httpStatus = 'code' in error ? (error as any).code : undefined
  if (httpStatus !== 404) return false
  return error.message.includes('"code":-32001') ||
    error.message.includes('"code": -32001')
}
```

On detection, the connection cache clears and the call retries once.

15.7.3 Batched Connections

Local servers connect in batches of 3 (spawning processes can exhaust file descriptors), remote servers in batches of 20. The React context provider `MCPConnectionManager.tsx` manages the lifecycle, diffing current connections against new configs.

15.8 Claude.ai Proxy Transport

The `claudeai-proxy` transport illustrates a common agent integration pattern: connecting through an intermediary. Claude.ai subscribers configure MCP “connectors” through the web interface, and the CLI routes through Claude.ai’s infrastructure which handles vendor-side OAuth.

The `createClaudeAiProxyFetch()` function captures the `sentToken` at request time, not re-read after a 401. Under concurrent 401s from multiple connectors, another connector’s retry might have already refreshed the token. The function also checks for concurrent refreshes even when the refresh handler returns false – the “ELOCKED contention” case where another connector won the lockfile race.

15.9 Timeout Architecture

MCP timeouts are layered, each protecting against a different failure mode:

Layer	Duration	Protects Against
Connection	30s	Unreachable or slow-starting servers
Per-request	60s (fresh per request)	Stale timeout signal bug
Tool call	~27.8 hours	Legitimately long operations
Auth	30s per OAuth request	Unreachable OAuth servers

The per-request timeout deserves emphasis. Early implementations created a single `AbortSignal.timeout(60000)` at connection time. After 60 seconds of idle time, the next request would abort immediately – the signal was already expired. The fix: `wrapFetchWithTimeout()` creates a fresh timeout signal for every request. It also normalizes the `Accept` header as a last-step defense against runtimes and proxies that drop it.

15.10 Apply This: Integrating MCP Into Your Own Agent

Start with `stdio`, add complexity later. `StdioClientTransport` handles everything: spawn, pipe, kill. One line of config, one transport class, and

you have MCP tools.

Normalize names and truncate descriptions. Names must match `^[a-zA-Z0-9_-]{1,64}$`. Prefix with `mcp_{serverName}_` to avoid collisions. Cap descriptions at 2,048 characters – OpenAPI-generated servers will waste context tokens otherwise.

Handle auth lazily. Do not attempt OAuth until a server returns 401. Most stdio servers need no auth.

Use in-process transport for built-in servers. `createLinkedTransportPair()` eliminates subprocess overhead for servers you control.

Respect tool annotations and sanitize output. `readOnlyHint` enables concurrent execution. Sanitize responses against malicious Unicode (bidirectional overrides, zero-width joiners) that could mislead the model.

The MCP protocol is deliberately minimal – two JSON-RPC methods. Everything between those methods and a production deployment is engineering: eight transports, seven config scopes, two OAuth RFCs, and timeout layering. Claude Code’s implementation shows what that engineering looks like at scale.

The next chapter examines what happens when the agent reaches beyond localhost: the remote execution protocols that let Claude Code run in cloud containers, accept instructions from web browsers, and tunnel API traffic through credential-injecting proxies.

Chapter 16

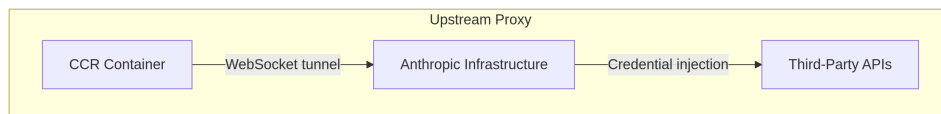
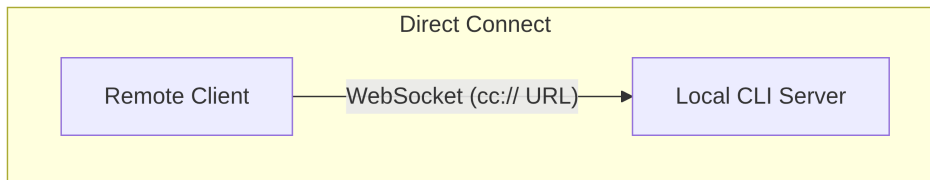
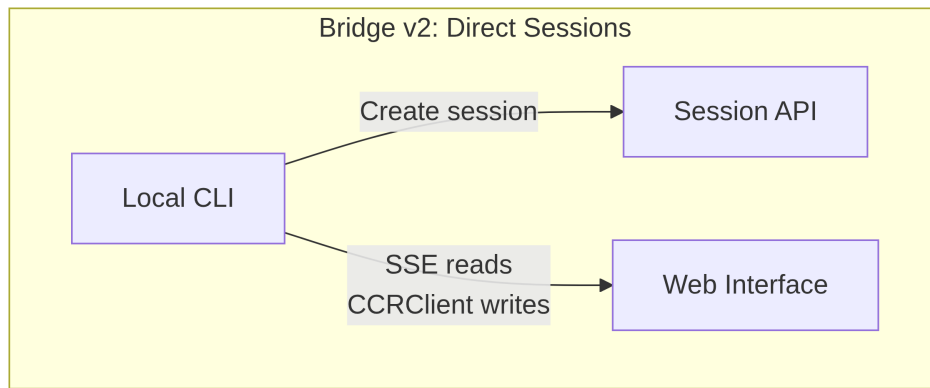
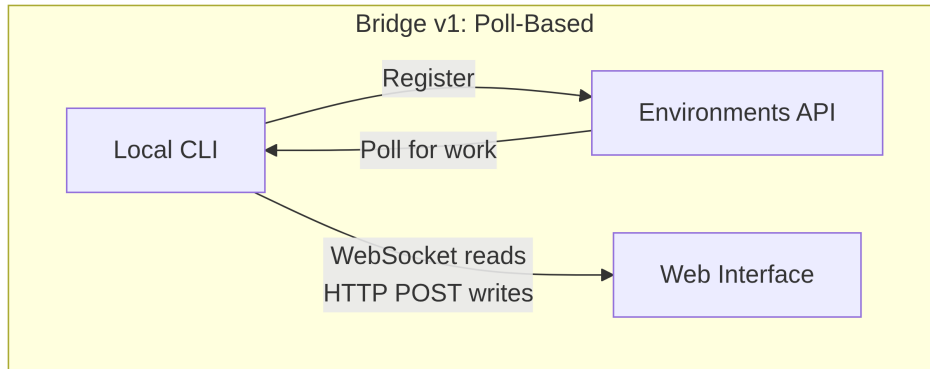
Chapter 16: Remote Control and Cloud Execution

16.1 The Agent Reaches Beyond Localhost

Every chapter so far has assumed that Claude Code runs on the same machine where the code lives. The terminal is local. The filesystem is local. The model responses stream back to a process that owns both the keyboard and the working directory.

That assumption breaks the moment you want to control Claude Code from a browser, run it inside a cloud container, or expose it as a service on your LAN. The agent needs a way to receive instructions from a web browser, a mobile app, or an automated pipeline – forward permission prompts to someone who is not sitting at the terminal, and tunnel its API traffic through infrastructure that might inject credentials or terminate TLS on the agent’s behalf.

Claude Code solves this with four systems, each addressing a different topology:



These systems share a common design philosophy: reads and writes are asymmetric, reconnection is automatic, and failures degrade gracefully.

16.2 Bridge v1: Poll, Dispatch, Spawn

The v1 bridge is the environment-based remote control system. When a developer runs `claude remote-control`, the CLI registers with the Environments API, polls for work, and spawns a child process per session.

Before registration, a gauntlet of pre-flight checks runs: runtime feature gate, OAuth token validation, organization policy check, dead token detection (a cross-process backoff after three consecutive failures with the same expired token), and proactive token refresh that eliminates roughly 9% of registrations that would otherwise fail on the first attempt.

Once registered, the bridge enters a long-poll loop. Work items arrive as sessions (with a `secret` field containing session tokens, API base URL, MCP configs, and environment variables) or healthchecks. The bridge throttles “no work” log messages to every 100 empty polls.

Each session spawns a child Claude Code process communicating via NDJSON on stdin/stdout. Permission requests flow through the bridge transport to the web interface where the user approves or denies. The round-trip must complete within roughly 10-14 seconds.

16.3 Bridge v2: Direct Sessions and SSE

The v2 bridge eliminates the entire Environments API layer – no registration, no polling, no acknowledgment, no heartbeat, no deregistration. The motivation: v1 required the server to know the machine’s capabilities before dispatching work. V2 collapses the lifecycle to three steps:

1. **Create session:** `POST /v1/code/sessions` with OAuth credentials.
2. **Connect bridge:** `POST /v1/code/sessions/{id}/bridge`. Returns a `worker_jwt`, `api_base_url`, and `worker_epoch`. Each `/bridge` call bumps the epoch – it IS the registration.
3. **Open transport:** SSE for reads, `CCRCClient` for writes.

The transport abstraction (`ReplBridgeTransport`) unifies v1 and v2 behind a common interface, so message handling does not need to know which generation it is talking to.

When the SSE connection drops due to a 401, the transport rebuilds with fresh credentials from a new `/bridge` call while preserving the sequence number

cursor – no messages are lost. The write path uses per-instance `getAuthToken` closures instead of process-wide environment variables, preventing JWT leakage across concurrent sessions.

16.3.1 The FlushGate

A subtle ordering problem: the bridge needs to send conversation history while accepting live writes from the web interface. If a live write arrives during the history flush, messages could be delivered out of order. The `FlushGate` queues live writes during the flush POST and drains them in order when it completes.

16.3.2 Token Refresh and Epoch Management

The v2 bridge proactively refreshes worker JWTs before expiry. A new epoch tells the server this is the same worker with fresh credentials. Epoch mismatches (409 responses) are handled aggressively: both connections close and an exception unwinds the caller, preventing split-brain scenarios.

16.4 Message Routing and Echo Deduplication

Both bridge generations share `handleIngressMessage()` as the central router:

1. Parse JSON, normalize control message keys.
2. Route `control_response` to permission handler, `control_request` to request handler.
3. Check UUID against `recentPostedUUIDs` (echo dedup) and `recentInboundUUIDs` (re-delivery dedup).
4. Forward validated user messages.

16.4.1 BoundedUUIDSet: O(1) Lookup, O(capacity) Memory

The bridge has an echo problem – messages may echo back on the read stream or be delivered twice during transport switches. `BoundedUUIDSet` is a FIFO-bounded set backed by a circular buffer:

```
class BoundedUUIDSet {
    private buffer: string[]
```



```

private set: Set<string>
private head = 0

add(uuid: string): void {
    if (this.set.size >= this.capacity) {
        this.set.delete(this.buffer[this.head])
    }
    this.buffer[this.head] = uuid
    this.set.add(uuid)
    this.head = (this.head + 1) % this.capacity
}

has(uuid: string): boolean { return this.set.has(uuid) }
}

```

Two instances run in parallel, each with capacity 2000. $O(1)$ lookup via the Set, $O(\text{capacity})$ memory via circular buffer eviction, no timers or TTLs. Unknown control request subtypes get an error response, not silence – preventing the server from waiting for a response that never comes.

16.5 The Asymmetric Design: Persistent Reads, HTTP POST Writes

The CCR protocol uses asymmetric transport: reads flow through a persistent connection (WebSocket or SSE), writes go through HTTP POST. This reflects a fundamental asymmetry in the communication pattern.

Reads are high-frequency, low-latency, server-initiated – hundreds of small messages per second during token streaming. A persistent connection is the only sensible choice. Writes are low-frequency, client-initiated, and require acknowledgment – messages per minute, not per second. HTTP POST provides reliable delivery, idempotency via UUIDs, and natural integration with load balancers.

Trying to unify them on a single WebSocket creates coupling: if the WebSocket drops during a write, you need retry logic and must distinguish “not sent” from “sent but acknowledgment lost.” Separate channels let each be optimized independently.

16.6 Remote Session Management

The `SessionsWebSocket` manages the client side of a CCR WebSocket connection. Its reconnection strategy discriminates between failure types:

Failure	Strategy
4003 (unauthorized)	Stop immediately, no retries
4001 (session not found)	Max 3 retries, linear backoff (transient during compaction)
Other transient	Exponential backoff, max 5 attempts

The `isSessionsMessage()` type guard accepts any object with a string `type` field – deliberately permissive. A hardcoded allowlist would silently drop new message types before the client is updated.

16.7 Direct Connect: The Local Server

Direct Connect is the simplest topology: Claude Code runs as a server and clients connect via WebSocket. No cloud intermediary, no OAuth tokens.

Sessions have five states: `starting`, `running`, `detached`, `stopping`, `stopped`. Metadata persists to `~/.claude/server-sessions.json` for resume across server restarts. The `cc://` URL scheme provides clean addressing for local connections.

16.8 Upstream Proxy: Credential Injection in Containers

The upstream proxy runs inside CCR containers and solves a specific problem: injecting organization credentials into outbound HTTPS traffic from a container where the agent might execute untrusted commands.

The setup sequence is carefully ordered:

1. Read the session token from `/run/ccr/session_token`.
2. Set `prctl(PR_SET_DUMPABLE, 0)` via Bun FFI – blocking same-UID ptrace of the process heap. Without this, a prompt-injected `gdb -p $PPID` could scrape the token from memory.
3. Download the upstream proxy CA certificate and concatenate with system CA bundle.
4. Start a local CONNECT-to-WebSocket relay on an ephemeral port.
5. Unlink the token file – the token now exists only on the heap.
6. Export environment variables for all subprocesses.

Every step fails open: errors disable the proxy rather than killing the session. The correct tradeoff – a failed proxy means some integrations will not work, but core functionality remains available.

16.8.1 Protobuf Hand-Encoding

Bytes through the tunnel are wrapped in `UpstreamProxyChunk` protobuf messages. The schema is trivial – message `UpstreamProxyChunk { bytes data = 1; }` – and Claude Code encodes it by hand in ten lines rather than pulling in a protobuf runtime:

```
export function encodeChunk(data: Uint8Array): Uint8Array {
  const varint: number[] = []
  let n = data.length
  while (n > 0x7f) { varint.push((n & 0x7f) | 0x80); n >>= 7 }
  varint.push(n)
  const out = new Uint8Array(1 + varint.length + data.length)
  out[0] = 0x0a // field 1, wire type 2
  out.set(varint, 1)
  out.set(data, 1 + varint.length)
  return out
}
```

Ten lines replace a full protobuf runtime. A single-field message does not justify a dependency – the maintenance burden of the bit manipulation is far lower than the supply chain risk.

16.9 Apply This: Designing Remote Agent Execution

Separate read and write channels. When reads are high-frequency streams and writes are low-frequency RPCs, unifying them creates unnecessary coupling. Let each channel fail and recover independently.

Bound your deduplication memory. The BoundedUUIDSet pattern provides fixed-memory deduplication. Any at-least-once delivery system needs a bounded dedup buffer, not an unbounded Set.

Make reconnection strategy proportional to the failure signal. Permanent failures should not retry. Transient failures should retry with backoff. Ambiguous failures should retry with a low cap.

Keep secrets heap-only in adversarial environments. Reading the token from a file, disabling ptrace, and unlinking the file eliminates both filesystem and memory-inspection attack vectors.

Fail open for auxiliary systems. The upstream proxy fails open because it provides enhanced functionality (credential injection), not core functionality (model inference).

The remote execution systems encode a deeper principle: the agent's core loop (Chapter 5) should be agnostic about where instructions come from and where results go. The bridge, Direct Connect, and upstream proxy are transport layers. The message handling, tool execution, and permission flows above them are identical regardless of whether the user is sitting at the terminal or on the other side of a WebSocket.

The next chapter examines the other operational concern: performance – how Claude Code makes every millisecond and token count across startup, rendering, search, and API costs.

Part VII

Part VII: Performance Engineering

Chapter 17

Chapter 17: Performance – Every Millisecond and Token Counts

17.1 The Senior Engineer’s Playbook

Performance optimization in an agentic system is not one problem. It is five:

1. **Startup latency** – the time from keystroke to first useful output. Users abandon tools that feel slow to launch.
2. **Token efficiency** – the fraction of the context window consumed by useful content versus overhead. The context window is the most constrained resource.
3. **API cost** – the dollar amount per turn. Prompt caching can reduce this by 90%, but only if the system preserves cache stability across turns.
4. **Rendering throughput** – the frames per second during streaming output. Chapter 13 covered the rendering architecture; this chapter covers the performance measurements and optimizations that keep it fast.
5. **Search speed** – the time to find a file in a 270,000-path codebase on every keystroke.

Claude Code attacks all five with techniques ranging from the obvious (memoization) to the subtle (26-bit bitmaps for pre-filtering fuzzy search). A

note on methodology: these are not theoretical optimizations. Claude Code ships with 50+ startup profiling checkpoints, sampled at 100% of internal users and 0.5% of external users. Every optimization below was motivated by data from this instrumentation, not by intuition.

17.2 Saving Milliseconds at Startup

17.2.1 Module-Level I/O Parallelism

The entry point `main.tsx` deliberately violates “no side effects at module scope”:

```
profileCheckpoint('main_tsx_entry');
startMdmRawRead();           // fires plutil/reg-query subprocesses
startKeychainPrefetch();    // fires both macOS keychain reads in parallel
```

Two macOS keychain entries would otherwise cost ~65ms of sequential synchronous spawns. By launching both as fire-and-forget promises at the module level, they execute in parallel with ~135ms of module loading during which the CPU would otherwise be idle.

17.2.2 API Preconnection

`apiPreconnect.ts` fires a HEAD request to the Anthropic API during initialization, overlapping the TCP+TLS handshake (100-200ms) with setup work. In interactive mode, the overlap is unbounded – the connection warms while the user types. The request fires after `applyExtraCACertsFromConfig()` and `configureGlobalAgents()` so the warmed connection uses the correct transport configuration.

17.2.3 Fast-Path Dispatch and Deferred Imports

The CLI entry point contains early-return paths for specialized subcommands – `claude mcp` never loads the React REPL, `claude daemon` never loads the tool system. Heavy modules are loaded via dynamic `import()` only when needed: OpenTelemetry (~400KB + ~700KB gRPC), event logging, error dialogs, upstream proxy. `LazySchema` defers Zod schema construction to first validation, pushing the cost past startup.

17.3 Saving Tokens in the Context Window

17.3.1 Slot Reservation: 8K Default, 64K Escalation

The most impactful single optimization:

The default output slot reservation is 8,000 tokens, escalating to 64,000 on truncation. The API reserves `max_output_tokens` of capacity for the model’s response. The default SDK value is 32K-64K, but production data shows p99 output length is 4,911 tokens. The default over-reserves by 8-16x, wasting 24,000-59,000 tokens per turn. Claude Code caps at 8K and retries at 64K on the rare truncation (<1% of requests). For a 200K window, this is a 12-28% improvement in usable context – for free.

17.3.2 Tool Result Budgeting

Limit	Value	Purpose
Per-tool characters	50,000	Results persisted to disk when exceeded
Per-tool tokens	100,000	~400KB text upper bound
Per-message aggregate	200,000 chars	Prevents N parallel tools from blowing the budget in one turn

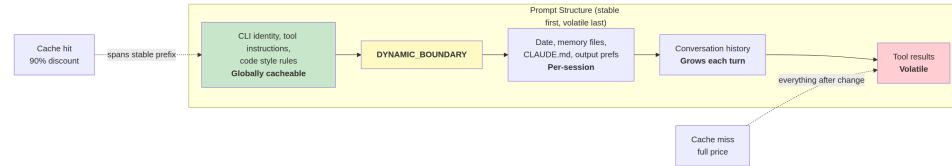
The per-message aggregate is the key insight. Without it, “read all files in src/” could produce 10 parallel reads each returning 40K characters.

17.3.3 Context Window Sizing

The default 200K-token window is expandable to 1M via the `[1m]` suffix on model names or experiment treatment. When usage approaches the limit, a 4-layer compaction system progressively summarizes older content. Token counting is anchored on the API’s actual `usage` field, not client-side estimation – accounting for prompt caching credits, thinking tokens, and server-side transformations.

17.4 Saving Money on API Calls

17.4.1 The Prompt Cache Architecture



Anthropic’s prompt cache operates on exact prefix matching. If a single token changes mid-prefix, everything after is a cache miss. Claude Code structures the entire prompt so stable parts come first and volatile parts come last.

When `shouldUseGlobalCacheScope()` returns true, system prompt entries before the dynamic boundary get `scope: 'global'` – two users running the same Claude Code version share the prefix cache. Global scope is disabled when MCP tools are present, since MCP schemas are per-user.

17.4.2 Sticky Latch Fields

Five boolean fields use a “sticky-on” pattern – once true, they remain true for the session:

Latch Field	What It Prevents
<code>promptCache1hEligible</code>	Mid-session overage flip changing cache TTL
<code>afkModeHeaderLatched</code>	Shift+Tab toggles busting cache
<code>fastModeHeaderLatched</code>	Cooldown enter/exit double-busting cache
<code>cacheEditingHeaderLatched</code>	Mid-session config toggles busting cache
<code>thinkingClearLatched</code>	Flipping thinking mode after confirmed cache miss

Each corresponds to a header or parameter that, if changed mid-session, would bust ~50,000-70,000 tokens of cached prompt. The latches sacrifice mid-session toggling to preserve the cache.

17.4.3 Memoized Session Date

```
const getSessionStartDate = memoize(getLocalISODate)
```

Without this, the date would change at midnight, busting the entire cached prefix. A stale date is cosmetic; a cache bust reprocesses the entire conversation.

17.4.4 Section Memoization

System prompt sections use a two-tier cache. Most content uses `systemPromptSection(name, compute)`, cached until `/clear` or `/compact`. The nuclear option `DANGEROUS_uncachedSystemPromptSection(name, compute, reason)` recomputes every turn – the naming convention forces developers to document WHY cache-breaking is necessary.

17.5 Saving CPU in Rendering

Chapter 13 covered the rendering architecture in depth – the packed typed arrays, pool-based interning, double buffering, and cell-level diffing. Here we focus on the performance measurements and adaptive behaviors that keep it fast.

The terminal renderer throttles at 60fps via `throttle(deferredRender, FRAME_INTERVAL_MS)`. When the terminal is blurred, the interval doubles to 30fps. Scroll drain frames run at quarter interval for maximum scroll speed. This adaptive throttling ensures rendering never consumes more CPU than necessary.

The React Compiler (`react/compiler-runtime`) auto-memoizes component renders throughout the codebase. Manual `useMemo` and `useCallback` are error-prone; the compiler gets it right by construction. Pre-allocated frozen objects (`Object.freeze()`) eliminate allocations for common render-path values – one allocation saved per frame in alt-screen mode compounds over thousands of frames.

For the full rendering pipeline details – the `CharPool/StylePool/HyperlinkPool` interning system, the blit optimization, the damage rectangle tracking, the `OffscreenFreeze` component – see Chapter 13.

17.6 Saving Memory and Time in Search

The fuzzy file search runs on every keystroke, searching 270,000+ paths. Three optimization layers keep it under a few milliseconds.

17.6.1 The Bitmap Pre-Filter

Every indexed path gets a 26-bit bitmap of which lowercase letters it contains:

```
// Pseudocode - illustrates the 26-bit bitmap concept
function buildCharBitmap(filepath: string): number {
  let mask = 0
  for (const ch of filepath.toLowerCase()) {
    const code = ch.charCodeAt(0)
    if (code >= 97 && code <= 122) mask |= 1 << (code - 97)
  }
  return mask // Each bit represents presence of a-z
}
```

At search time: `if ((charBits[i] & needleBitmap) !== needleBitmap) continue`. Any path missing a query letter fails instantly – one integer comparison, no string operations. Rejection rate: ~10% for broad queries like “test,” 90%+ for queries with rare letters. Cost: 4 bytes per path, ~1MB for 270,000 paths.

17.6.2 Score-Bound Rejection and Fused indexOf Scan

Paths surviving the bitmap face a score ceiling check before the expensive boundary/camelCase scoring. If the best-case score cannot beat the current top-K threshold, the path is skipped.

The actual matching fuses position finding with gap/consecutive bonus computation using `String.indexOf()`, which is SIMD-accelerated in both JSC (Bun) and V8 (Node). The engine’s optimized search is significantly faster than manual character loops.

17.6.3 Async Indexing with Partial Queryability

For large codebases, `loadFromFileListAsync()` yields to the event loop every ~4ms of work (time-based, not count-based – adapting to machine

speed). It returns two promises: `queryable` (resolves on first chunk, enabling immediate partial results) and `done` (full index complete). The user can start searching within 5-10ms of the file list becoming available.

The yield check uses `(i & 0xff) === 0xff` – a branchless modulo-256 to amortize the cost of `performance.now()`.

17.7 The Memory Relevance Side-Query

One optimization sits at the intersection of token efficiency and API cost. As described in Chapter 11, the memory system uses a lightweight Sonnet model call – not the main Opus model – to select which memory files to include. The cost (256 max output tokens on a fast model) is negligible compared to the tokens saved by not including irrelevant memory files. A single irrelevant 2,000-token memory costs more in wasted context than the side query costs in API calls.

17.8 Speculative Tool Execution

The `StreamingToolExecutor` begins executing tools as they stream in, before the full response completes. Read-only tools (Glob, Grep, Read) can execute in parallel; write tools require exclusive access. The `partitionToolCalls()` function groups consecutive safe tools into batches: [Read, Read, Grep, Edit, Read, Read] becomes three batches – [Read, Read, Grep] concurrent, [Edit] serial, [Read, Read] concurrent.

Results are always yielded in the original tool order for deterministic model reasoning. A sibling abort controller kills parallel subprocesses when a Bash tool errors, preventing resource waste.

17.9 Streaming and the Raw API

Claude Code uses the raw streaming API instead of the SDK's `BetaMessageStream` helper. The helper calls `partialParse()` on ev-

ery `input_json_delta` – $O(n^2)$ in tool input length. Claude Code accumulates raw strings and parses once when the block is complete.

A streaming watchdog (`CLAUDE_STREAM_IDLE_TIMEOUT_MS`, default 90 seconds) aborts and retries if no chunks arrive, with fallback to non-streaming `messages.create()` on proxy failure.

17.10 Apply This: Performance for Agentic Systems

Audit your context window budget. The gap between your `max_output_tokens` reservation and your actual p99 output length is wasted context. Set a tight default and escalate on truncation.

Design for cache stability. Every field in your prompt is stable or volatile. Put stable first, volatile last. Treat any mid-conversation change to the stable prefix as a bug with a dollar cost.

Parallelize startup I/O. Module loading is CPU-bound. Keychain reads and network handshakes are I/O-bound. Launch I/O before imports.

Use bitmap pre-filters for search. A cheap pre-filter rejecting 10-90% of candidates before expensive scoring is a significant win at 4 bytes per entry.

Measure where it matters. Claude Code has 50+ startup checkpoints, sampled at 100% internally and 0.5% externally. Performance work without measurement is guesswork.

A final observation: most of these optimizations are not algorithmically sophisticated. Bitmap pre-filters, circular buffers, memoization, interning – these are CS fundamentals. The sophistication is in knowing where to apply them. The startup profiler tells you where the milliseconds are. The API usage field tells you where the tokens are. The cache hit rate tells you where the money is. Measurement first, optimization second, always.

Chapter 18

Chapter 18: What We Learned

18.1 Five Architectural Bets

Claude Code is not the only agentic system. It is not the first. But it made five architectural bets that distinguish it from the landscape of agent frameworks, and after nearly two thousand files and seventeen chapters, those bets deserve examination.

18.1.1 Bet 1: The Generator Loop Over Callbacks

Most agent frameworks give you a pipeline: define tools, register handlers, let the framework orchestrate. The developer writes callbacks. The framework decides when to call them.

Claude Code does the opposite. The `query()` function is an async generator – the developer owns the loop. The model streams a response, the generator yields tool calls, the caller executes them, appends results, and the generator loops. There is one function, one data flow, one place where every interaction passes through. The 10 terminal states and 7 continuation states of the generator’s return type encode every possible outcome. The loop is the system.

The bet was that a single generator function, even one that grew to 1,700 lines, would be more comprehensible than a distributed callback graph. After studying the source, the bet paid off. When you want to understand why

a session ended, you look at one function. When you want to add a new terminal state, you add one variant to one discriminated union. The type system enforces exhaustive handling. A callback architecture would scatter this logic across dozens of files, and the interactions between callbacks would be implicit rather than visible in the control flow.

18.1.2 Bet 2: File-Based Memory Over Databases

Chapter 11 made the case in detail, but the architectural significance extends beyond memory. The decision to use plain Markdown files instead of SQLite, a vector database, or a cloud service was a bet on transparency over capability. A database would support richer queries, faster lookups, and transactional guarantees. Files provide none of that. What files provide is trust.

A user who opens `~/.claude/projects/myapp/memory/MEMORY.md` in vim and sees exactly what the agent remembers about them has a fundamentally different relationship with the system than a user who must ask the agent “what do you remember?” and hope the answer is complete. The file-based design makes the agent’s knowledge state externally observable, not just self-reported. This matters more than query performance. The LLM-powered recall system compensates for the storage simplicity with retrieval intelligence – a Sonnet side-query selecting five relevant memories from a manifest is more precise than embedding similarity and requires zero infrastructure.

18.1.3 Bet 3: Self-Describing Tools Over Central Orchestrators

Agent frameworks typically provide a tool registry: you describe your tools in a central configuration, and the framework presents them to the model. Claude Code’s tools describe themselves. Each `Tool` object carries its own name, description, input schema, prompt contribution, concurrency safety flag, and execution logic. The tool system’s job is not to describe tools to the model – it is to let tools describe themselves.

This bet pays off in extensibility. MCP tools (Chapter 15) become first-class citizens by implementing the same interface. A tool from an MCP server and a built-in tool are indistinguishable to the model. The system does not need a separate “MCP tool adapter” layer – the wrapping produces a standard `Tool` object, and from that point forward, the existing tool pipeline handles it: permission checking, concurrent execution, result budgeting, hook interception.

18.1.4 Bet 4: Fork Agents for Cache Sharing

Chapter 9 covered the fork mechanism: a sub-agent that starts with the parent's full conversation in its context window, sharing the parent's prompt cache. This is not a convenience optimization – it is an architectural bet that the cache sharing model is worth the complexity of fork lifecycle management.

The alternative – spawning a fresh agent with a summary of the conversation – is simpler but expensive. Every fresh agent pays the full cost of processing its context from scratch. A forked agent gets the parent's cached prefix for free (a 90% discount on input tokens), making it economical to spawn agents for small tasks: memory extraction, code review, verification passes. The background memory extraction agent (Chapter 11) runs after every query loop turn, and its cost is marginal precisely because it shares the parent's cache. Without fork-based cache sharing, that agent would be prohibitively expensive.

18.1.5 Bet 5: Hooks Over Plugins

Most extensibility systems use plugins – code that registers capabilities and runs within the host process. Claude Code uses hooks – external processes that run at lifecycle points and communicate through exit codes and JSON on stdin/stdout.

The bet is that process isolation is worth the overhead. A plugin can crash the host. A hook crashes its own process. A plugin can leak memory into the host's heap. A hook's memory dies with its process. A plugin requires an API surface that must be versioned and maintained. A hook requires stdin, stdout, and an exit code – a protocol that has been stable since 1971.

The overhead is real: spawning a process per hook invocation costs milliseconds that an in-process callback would not. The -70% fast path for internal callbacks (Chapter 12) shows that the system knows this cost matters. But for external hooks – user scripts, team linters, enterprise policy servers – the isolation guarantee makes the system safer to extend. An enterprise can deploy hook-based policy enforcement without worrying that a malformed hook script will crash their developers' sessions.

18.2 What Transfers, What Does Not

Not every pattern in Claude Code generalizes. Some are consequences of scale, resources, or specific constraints that other agent builders may not share.

18.2.1 Patterns That Transfer to Any Agent

The generator loop pattern. Any agent that needs to stream responses, handle tool calls, and manage multiple terminal states benefits from making the loop explicit rather than hiding it behind callbacks. The discriminated union return type – encoding exactly why the loop stopped – is a pattern that eliminates an entire class of “why did the agent stop?” debugging sessions.

File-based memory with LLM recall. The specific implementation details are Claude Code’s, but the principle – simple storage combined with intelligent retrieval – applies to any agent that needs to persist knowledge across sessions. The four-type taxonomy (user, feedback, project, reference) and the derivability test (“can this be re-derived from the current project state?”) are reusable design heuristics.

Asymmetric read/write channels for remote execution. When reads are high-frequency streams and writes are low-frequency RPCs, separating them is correct regardless of the specific transport protocol.

Bitmap pre-filters for search. Any agent searching a large file index benefits from a 26-bit letter bitmap as a pre-filter. Four bytes per entry, one integer comparison per candidate – the cost-to-benefit ratio is remarkable.

Prompt cache stability as an architectural concern. If your agent uses an API with prompt caching, structuring the prompt with stable content first and volatile content last is not an optimization – it is an architectural decision that determines your cost structure.

18.2.2 Patterns Specific to Claude Code’s Scale

The forked terminal renderer. Claude Code forked Ink and reimplemented the rendering pipeline with packed typed arrays, pool-based interning, and cell-level diffing because it needed 60fps streaming in a terminal. Most agents render to a web interface or a simple log output. The engineering investment only makes sense when terminal rendering is your primary UI and you are streaming at high frequency.

The 50+ startup profiling checkpoints. Meaningful when you have hundreds of thousands of users and 0.5% sampling produces statistically significant data. For a smaller agent, a simpler timing system suffices.

Eight MCP transport types. Claude Code supports stdio, SSE, HTTP, WebSocket, SDK, two IDE variants, and a Claude.ai proxy because it must integrate with every deployment topology. Most agents need stdio and HTTP.

The hooks snapshot security model. Freezing hook configuration at startup and never re-reading it implicitly is a defense against a specific threat: malicious repository code modifying hooks after the user accepts the trust dialog. This matters when your agent runs in arbitrary repositories with untrusted `.claude/` configurations. An agent that only runs in trusted environments can use simpler hook management.

18.3 The Cost of Complexity

Nearly two thousand files. What does that buy, and what does it cost?

The file count is misleading as a complexity metric. Much of it is test infrastructure, type definitions, configuration schemas, and the forked Ink renderer. The actual behavioral complexity concentrates in a small number of high-density files: `query.ts` (1,700 lines, the agent loop), `hooks.ts` (4,900 lines, the lifecycle interception system), `REPL.tsx` (5,000 lines, the interactive orchestrator), and the memory system's prompt building functions.

The complexity comes from three sources, each with a different character:

Protocol diversity. Supporting five terminal keyboard protocols, eight MCP transport types, four remote execution topologies, and seven configuration scopes is inherently complex. Each additional protocol is a linear addition to the codebase, not an exponential one – but the sum is large. This complexity is accidental in the Brooksonian sense: it comes from the environment (terminal fragmentation, MCP transport evolution, remote deployment topologies), not from the problem being solved.

Performance optimization. The pool-based rendering, bitmap search pre-filters, sticky cache latches, and speculative tool execution each add complexity in exchange for measurable performance gains. This complexity is justified by measurement – every optimization was preceded by profiling

data that identified the bottleneck. The risk is that optimizations accumulate and interact in ways that make the hot paths harder to modify.

Behavioral tuning. The memory system’s prompt instructions, the staleness warnings, the verification protocol, the “ignore memory” anti-pattern instruction – these are not code complexity. They are prompt complexity, and they carry a different maintenance burden. When the model’s behavior changes between versions, prompt instructions that were carefully tuned through evals may need re-tuning. The eval infrastructure (referenced throughout the codebase as case numbers and eval scores) is the defense against regression, but it requires ongoing investment.

The maintenance burden of this system is significant. A new engineer reading the codebase must understand not just the code paths but the eval outcomes that motivated specific prompt phrasings, the production incidents that motivated specific security checks, and the performance profiles that motivated specific optimizations. The code comments are thorough – many include eval case numbers and before/after measurements – but thorough comments in nearly two thousand files are themselves a reading burden.

18.4 Where Agentic Systems Are Heading

Four trends are visible from the patterns in Claude Code, and they point toward where the field is going.

18.4.1 MCP as the Universal Protocol

Chapter 15 described Claude Code as one of the most complete MCP clients. The significance is not Claude Code’s implementation – it is that MCP exists at all. A standardized protocol for tool discovery and invocation means that tools built for one agent work with any agent. The ecosystem effects are obvious: an MCP server for Postgres, once built, serves every agent that speaks MCP. The developer’s investment in tool integration is portable.

The implication for agent builders: if you are defining a custom tool protocol, you are probably making a mistake. MCP is good enough, it is getting better, and the ecosystem advantages of a standard protocol compound over time. Build an MCP client, contribute to the spec, and let the protocol evolve through community feedback.

18.4.2 Multi-Agent Coordination

Claude Code’s sub-agent system (Chapter 8), task coordination (Chapter 10), and fork mechanism (Chapter 9) are early implementations of multi-agent patterns. They solve specific problems – cache sharing, parallel exploration, structured verification – but they also reveal the fundamental challenge: coordination overhead.

Every message between agents consumes tokens. Every fork shares a cache but adds a conversation branch that the parent must eventually reconcile. The Task system’s state machine (queued, running, completed, failed, cancelled) is coordination machinery that adds complexity without adding capability. As agents become more capable, the pressure will shift from “how do we coordinate multiple agents?” to “how do we make one agent capable enough that coordination is unnecessary?”

The current evidence suggests both approaches will coexist. Simple tasks will use single agents. Complex tasks will use coordinated multi-agent systems. The engineering challenge is making the coordination overhead low enough that the crossover point favors multi-agent for genuinely parallel work, not just for tasks that are complex.

18.4.3 Persistent Memory

Claude Code’s memory system is version 1 of persistent agent memory. The file-based design, the four-type taxonomy, the LLM-powered recall, the staleness system, and the KAIROS mode for long-running sessions are all first-generation solutions to a problem that will evolve significantly.

Future memory systems will likely add structured retrieval (the current system retrieves whole files; future systems might retrieve specific facts), cross-project transfer learning (user preferences that apply everywhere, project conventions that do not), and collaborative memory (Chapter 11’s team memory is a first step, but the sync, conflict resolution, and access control are minimal).

The open question is whether the file-based approach scales. At 200 memories per project, it works. At 2,000 memories per project, the Sonnet side-query’s manifest becomes too large, the consolidation becomes too expensive, and the index exceeds its caps. The architectural bet on files-over-databases will face its hardest test as usage grows.

18.4.4 Autonomous Operation

The KAIROS mode, the background memory extraction agent, the auto-dream consolidation, the speculative tool execution – these are all steps toward autonomous operation. The agent does useful work without being asked: it remembers what you forgot to tell it to remember, it consolidates its own knowledge while you sleep, it starts executing the next tool before the current response is complete.

The trajectory is clear. Future agents will be less reactive and more proactive. They will notice patterns the user has not described, suggest corrections the user has not requested, and maintain their own knowledge without explicit `/remember` commands. Claude Code’s memory system, with its background extraction safety net and its prompt-engineered “what to save” heuristics, is the prototype for this future.

The constraint is trust. Autonomous operation requires the user to trust that the agent will do the right thing when unattended. The file-based memory, the observable hook system, the staleness warnings, the permission dialogs – all of these exist because trust must be earned, not assumed. The path to more autonomous agents runs through more transparent agents.

18.5 Closing

Seventeen chapters. Six core abstractions. A generator loop at the center, tools extending outward, memory reaching backward through time, hooks guarding the perimeter, a rendering engine translating it all into characters on a screen, and MCP connecting it to the world beyond the codebase.

The deepest pattern in Claude Code is not any single technique. It is the recurring decision to push complexity to the boundaries. The rendering system pushes complexity to the pools and the diff – inside the pipeline, everything is integer comparisons. The input system pushes complexity to the tokenizer and the keybinding resolver – inside the handlers, everything is typed actions. The memory system pushes complexity to the write protocol and the recall selector – inside the conversation, everything is context. The agent loop pushes complexity to the terminal states and the tool system – inside the loop, it is just: stream, collect, execute, append, repeat.

Each boundary absorbs chaos and exports order. Raw bytes become

ParsedKey. Markdown files become recalled memories. MCP JSON-RPC becomes **Tool** objects. Hook exit codes become permission decisions. On one side of each boundary, the world is messy – five keyboard protocols, fragile OAuth servers, stale memories, untrusted repository hooks. On the other side, the world is typed, bounded, and exhaustively handled.

If you are building an agentic system, this is the transferable lesson. Not the specific techniques – you may not need pool-based rendering or KAIROS mode or eight MCP transports. But the principle: define your boundaries, absorb complexity there, and keep everything between them clean. The boundaries are where the engineering is hard. The interior is where the engineering is pleasant. Design for pleasant interiors, and invest your complexity budget at the edges.

The source code is open. The crab has the map in its claw. Go read it.

